AccessMatrix Cloud Deployment Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - AccessMatrix Cloud Deployment Guide

AccessMatrix can be deployed as a Docker container. While containers can be deployed anywhere, to enjoy elasticity from cloud computing, containers should ideally be deployed on an orchestration platform.

- Kubernetes is an open-source container-orchestration platform for deployment automation, scaling and management. It is widely supported by major cloud service providers.
- Redhat OpenShift Container Platform is another container platform for developing and containerizing applications. It leverages Kubernetes as the primary infrastructure component. The OpenShift Container Platform offers powerful and flexible platform management tools and processes, essential benefits that Kubernetes does not provide.

This document provides detailed information on deploying AccessMatrix as a container on both the Kubernetes and OpenShift platforms.

Intended Reader

This guide is intended for system administrators or consultants to set up, operate and maintain an AccessMatrix system deployed on the Kubernetes or OpenShift orchestration platforms.

This document assumes that readers already understand basic concepts such as Docker images, containers, container orchestration, Kubernetes and OpenShift, as well as cloud computing in general.

Document Scope

The scope of this document covers the deployment of AccessMatrix on the Kubernetes and Openshift platforms. While both platforms are supported by various cloud service providers, this document only provides the necessary deployment steps for Microsoft Azure and AWS. It will not provide documentation on operating Azure or AWS, as comprehensive documentation on both products is readily available. However, where it is needed, this document will only provide:

- Links to Azure and AWS documentation to provision/manage supporting resources that must be prepared.
- Azure and AWS command-line instruction for resources that are directly used by AccessMatrix. While you can manage and configure Azure and AWS objects using various methods, this

documentation provides instructions using the Azure command line (`az`) and AWS command line (`aws`).

Since this guide is specifically about the deployment of AccessMatrix containers on the Kubernetes and OpenShift platforms, AccessMatrix configurations in the **amsystem.properties** file are provided as needed. Readers are encouraged to also refer to the *AccessMatrix Common Deployment Guide* for more detailed information on this.

Note: For advanced deployments where the AccessMatrix Audit Microservice is deployed separately, refer to the *Advanced Deployment with Audit Microservice* section of the respective guide (i.e. Kubernetes Deployment with Azure or Kubernetes Deployment with AWS). Complete all the necessary preparations in the preceding sections before attempting the steps in the Audit Microservice section.

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents:

- *AccessMatrix Common Deployment Guide*
- *AccessMatrix Common Administration Guide*
- Kubernetes Concepts

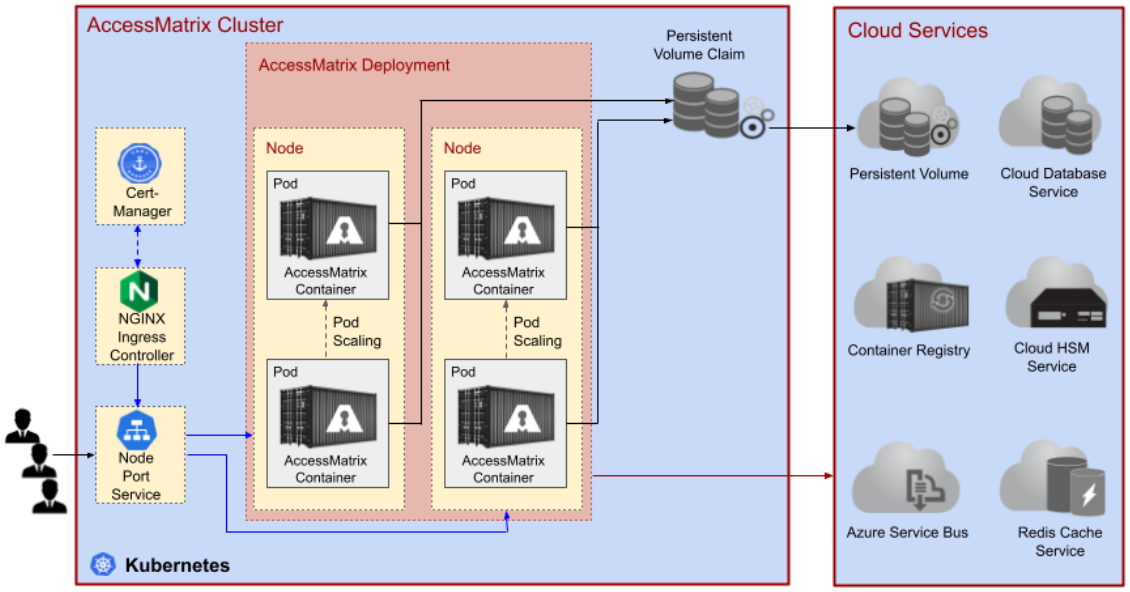
2. Introduction to AccessMatrix Cloud Deployment

This document guides you through deploying AccessMatrix containers on the Kubernetes and OpenShift platforms (as the container orchestration platform). While it specifically shows deployment steps on the Azure and AWS cloud platforms, the concepts are general for any cloud service providers.

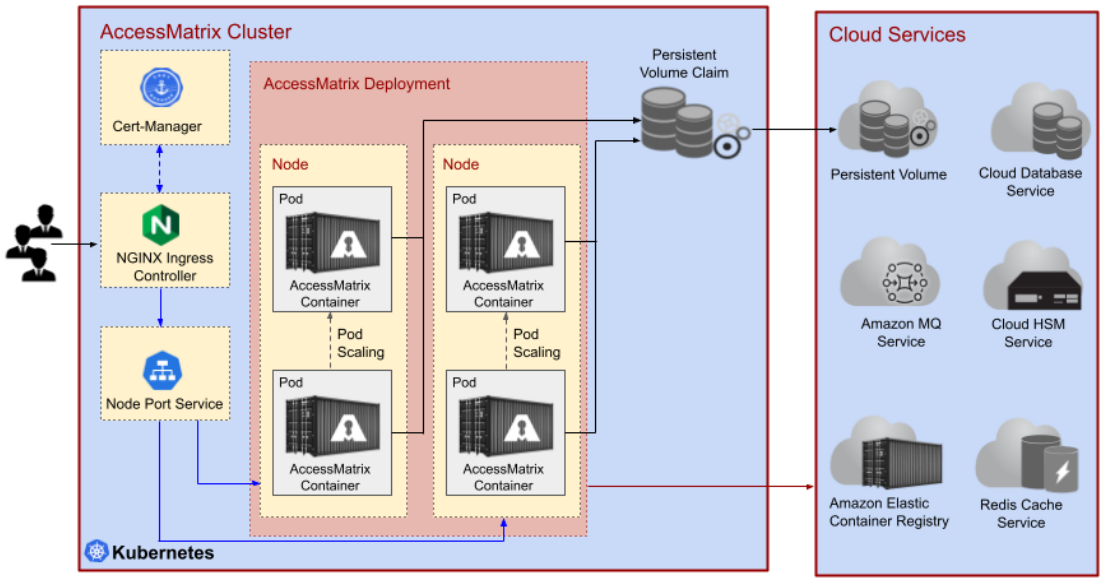
> **Note:** For a list of of common terms used within this guide, refer to Terminology.

AccessMatrix Deployment on Kubernetes and OpenShift Container Platforms

Kubernetes	Kubernetes is a powerful open-source system, initially developed by Google, for managing containerized applications in a clustered environment. It aims to provide better ways of managing related, distributed components and services across the varied infrastructure. Today, Kubernetes services are common in any Cloud Service Provider (CSP) like Azure, AWS, Google, Ali and others. The infrastructure and ecosystem of Kubernetes benefit AccessMatrix from various perspectives. For example: running AccessMatrix in different Kubernetes Pods replicas with the embedded Ingress load balancer improves the stability; The Rolling Update strategy on Kubernetes Pods improves AccessMatrix availability by allowing application receiving updates without disrupting service. The Kubernetes Horizontal Auto Scaler improves AccessMatrix on its stability by automatically adjusting the number of application replicas based on load conditions. The following are deployment architecture diagrams of a typical AccessMatrix deployment in the Kubernetes cluster with Azure or AWS. **AccessMatrix Kubernetes Deployment Architecture (Azure)**



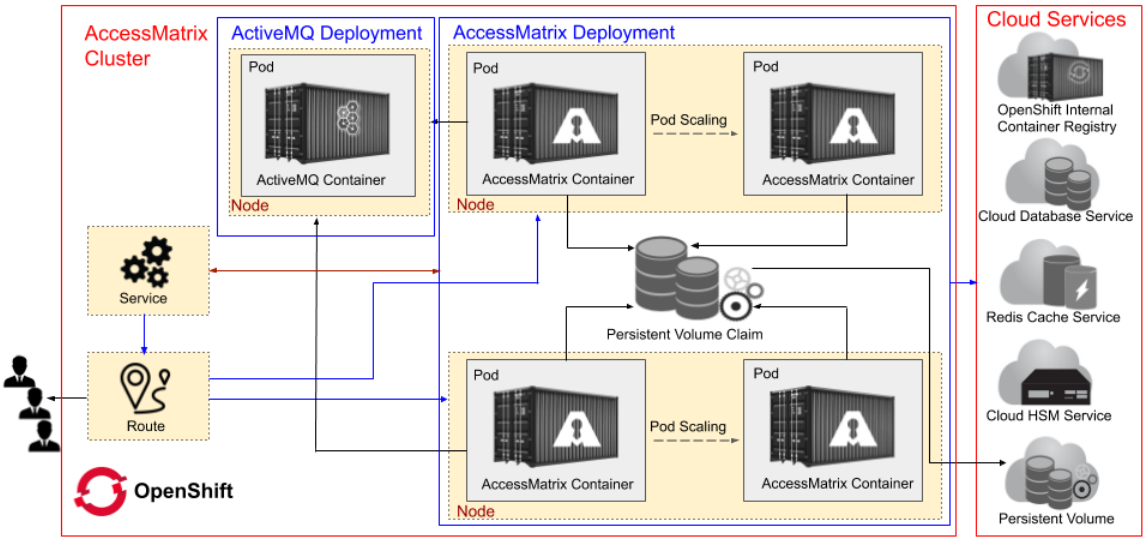
AccessMatrix Kubernetes Deployment Architecture (AWS)



The above diagrams each show a typical container orchestration on the Kubernetes service. AccessMatrix server is deployed in a Docker container. Kubernetes Pods manages these containers, the smallest deployable computing units you can create and manage in Kubernetes. In turn, a pod is run in a Kubernetes Node, which is the virtual machine.

A Pod is a group of one or more containers, shared storage/network resources, and a specification for running the containers. However, we recommend you run a single AccessMatrix container for each Pod.

	To deploy the AccessMatrix container, the Kubernetes cluster needs to have access to its Docker image. The AccessMatrix Docker images are stored in a Container Registry provided by Cloud Service Provider (CSP). The number of running AccessMatrix pods can vary that depends on the configuration and the loads. For example, you can set the minimum and the maximum number of pods from one (min) to 10 (max), and the CPU utilization threshold to 80%. This means if the load is low, Kubernetes will only run one pod/container. But if the load getting higher, and hits 80% of CPU utilization, then Kubernetes will spawn a new Pod (scaling-out horizontally) as necessary. When the load is getting lower again, Kubernetes will reduce the number of running Pods. Since any AccessMatrix instances can go up and down anytime, we cannot have any persistent information like log files inside the container. Thus, we employ Persistence Volume Claim (PVC) to store all the log files from all AccessMatrix Pods. PVC is a storage that is claimed by the Kubernetes cluster. PVC is provided by Persistence Volume service from the Cloud Service Provider (CSP).
OpenShift	Openshift is an open-source application container platform developed by RedHat for producing and hosting enterprise-grade applications. Openshift, as a Platform as a Service (PaaS) offering, aims to take care of managing the underlying infrastructure components. Openshift is a managed private cluster on any cloud platforms like Azure, AWS and Google. The infrastructure and ecosystem of OpenShift Container Platform benefit AccessMatrix. For example: running AccessMatrix in different replicas of OpenShift Pods with the route improves the stability. Routes map the Node IP and port to a domain name in OpenShift to be accessed publicly. In OpenShift service, you create your project and service and then expose the service to create a route. See below an Architecture diagram of a typical AccessMatrix deployment in Openshift Container Platform cluster. **AccessMatrix Openshift Deployment Architecture**



The above diagram shows a typical container platform on OpenShift service. AccessMatrix server is deployed in a Docker container. These containers are managed by OpenShift Pods, which are the smallest deployable computing units that you can create and manage in OpenShift. In turn, a pod is run in an OpenShift Node, which is the virtual machine.

A Pod is a group of one or more containers, shared storage/network resources, and a specification for running the containers. However, we recommend you to run a single AccessMatrix container for each Pod.

To deploy the AccessMatrix container, OpenShift cluster needs to have access to its Docker image. The AccessMatrix Docker images are stored in a Container Registry provided by Cloud Service Provider (CSP). In this case, you need to use the Openshift Container Registry.

The number of running AccessMatrix pods can vary that depends on the configuration and the loads. For example, you can set the minimum and a maximum number of pods from one (min) to 10 (max), and the CPU utilization threshold to 80%. Which means if the load is low, OpenShift will only run one pod/container. But if the load getting higher, and hits 80% of CPU utilization, then OpenShift will spawn a new Pod (scaling-out horizontally) as necessary. When the load is getting lower again, OpenShift will reduce the number of running Pods.

Since any AccessMatrix instances can go up and down anytime, we cannot have any persistent information inside the container, like log files. Thus, we employ Persistence Volume Claim (PVC) to store all the log files from all AccessMatrix Pods. PVC is storage that is claimed by the OpenShift cluster. PVC is provided by Persistence Volume service from the Cloud Service Provider (CSP).

For its operation, AccessMatrix has transient data that are frequently updated. For example, E2EE session, SMS-OTP tokens, SAML Response Id (to prevent replay attack), CLP login sessions (for AccessMatrix Applications, such as UAS TAP, USO SSF), and OIDC Relying Party Lookup Module nonces

(to prevent request forgery or replay). This information provided by the Cloud Service Provider (CSP) is stored in Redis Cache service.

AccessMatrix also has some common objects (such as policies, configurations and login modules) that are frequently accessed but are not frequently updated. For performance and efficiency, these objects are stored in Ehcache, an in-memory cache inside the AccessMatrix process. Ehcache contents are replicated to other AccessMatrix containers by using Java Message Service (JMS) that will require a message queue server. On AWS, AccessMatrix uses Amazon MQ Service as the message queue server.

The following table provides a brief overview of the various implementations available for each cache:

AccessMatrix Caches for Cloud Environment			
Types of Caches	**Disabled**	**Ehcache JMS**	**Redis**
Hibernate Cache	Disabled by default	Not available	Not available
DB-backed Object Cache	Cannot be disabled	Must be configured	Not available
Transient Data Cache	Cannot be disabled	Not available	Must be configured

The NGINX Ingress controller exposes HTTP and HTTPS routes from outside the cluster to services within the cluster. Traffic routing is controlled by rules defined on the Ingress resource. For TLS connection, you can choose whether you want to end it in the Ingress controller or in AccessMatrix containers (which will only pass through the Ingress controller).

Finally, the database service is used to store any persistent data.

The rest of this document will guide you on how to deploy AccessMatrix containers on the Kubernetes and OpenShift platforms.

Important Note:

- Follow all steps sequentially unless the steps are flagged as optional.
- If you want to use the deployment script samples (for example, YAML sample scripts), use a JSON formatter to tidy up the indentations after copying them.

3. AccessMatrix Kubernetes Deployment

The following sections provide detailed instructions on deploying AccessMatrix on the Kubernetes platform:

- Kubernetes Deployment with Azure
- Kubernetes Deployment with AWS

3.1. Kubernetes Deployment with Azure

The following pages provide detailed instructions on how to deploy AccessMatrix on the Kubernetes platform with Microsoft Azure:

- Prepare Kubernetes Deployment Prerequisites
- Prepare Database/Cache Services and Configure AccessMatrix
- Create and Deploy Kubernetes Objects
- Advanced Deployment with Audit Microservice

3.1.1. Prepare Kubernetes Deployment Prerequisites

You must already have a valid account with a proper subscription to the Azure cloud service provider. This section explains preparing all prerequisite items on the respective cloud service provider before deploying AccessMatrix on Kubernetes. Here, you learn how to:

1. Access to Cloud Service Providers. This section guides you through installing the command-line tool and creating a resource group to manage all the cloud objects used in this guide.
2. Prepare Container/Image Registry. An image registry is used to host the Docker images used by Kubernetes cluster. Though it is to register/store Docker image, some cloud service providers use container terminology.
3. Create Kubernetes Cluster. This section guides you through creating the Kubernetes cluster, where AccessMatrix containers will be run and orchestrated.

Access to Cloud Service Providers

To configure and run the Kubernetes service, Azure command-line tool must be installed.

1. Install the Azure CLI.
2. Install Azure Kubernetes Service CLI:

Command
`$ sudo az aks install-cli`
Response
`[sudo] password for haoz:` `Downloading client to "/usr/local/bin/kubectl" from` `"https://storage.googleapis.com/kubernetes-` `release/release/v1.18.2/bin/linux/amd64/kubectl"` `Please ensure that /usr/local/bin is in your search PATH, so the `kubectl`command` `can be found.`

   To use the services like Azure Container Registry (ACR), Redis Cache service and others, you must first create a resource group:

Command
`$ az group create --name AksGroup --location southeastasia`
Response

Command


```
{
  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AKSGroup",
  "location": "southeastasia",
  "managedBy": null,
  "name": "AKSGroup",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```


Create Container/Image Registry

In order to deploy AccessMatrix as containers, Kubernetes needs to pull the AccessMatrix Docker image from an image registry. Thus, you need to create a container/image registry, then build and push your AccessMatrix Docker image to it.

1. Use `az acr create` to create Azure Container Registry (ACR):

Command

```
$ az acr create --resource-group AksGroup --name AccessMatrix --sku Basic
```

Response


```
{
  "adminUserEnabled": false,
  "creationDate": "2020-04-19T09:22:43.915864+00:00",
  "dataEndpointEnabled": false,
  "dataEndpointHostNames": [],
  "encryption": {
    "keyVaultProperties": null,
    "status": "disabled"
  },
  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AksGroup/providers/Microsoft.ContainerRegistry/regist
ries/AccessMatrix",
  "identity": null,
  "location": "southeastasia",
  "loginServer": "accessmatrix.azurecr.io",
  "name": "AccessMatrix",
  "networkRuleSet": null,
  "policies": {
    "quarantinePolicy": {
      "status": "disabled"
    },
    "retentionPolicy": {
      "days": 7,
      "lastUpdatedTime": "2020-04-19T09:22:45.992243+00:00",
```

Command
`"status": "disabled" }, "trustPolicy": { "status": "disabled", "type": "Notary" } }, "privateEndpointConnections": [], "provisioningState": "Succeeded", "resourceGroup": "AksGroup", "sku": { "name": "Basic", "tier": "Basic" }, "status": null, "storageAccount": null, "tags": {}, "type": "Microsoft.ContainerRegistry/registries" }`

2. Use `az acr list` to query the registry server address:

Command
`$ az acr list --resource-group AksGroup --query "[].{acrLoginServer:loginServer}" --output table`
Response
`AcrLoginServer ---------------------- accessmatrix.azurecr.io`

Create Kubernetes Cluster

A Kubernetes cluster is a set of node machines for running containerized applications. At a minimum, a cluster contains a worker node and a master node. The master node is responsible for maintaining the desired state of the cluster, such as which applications are running and which container images they use. Worker nodes run the applications and workloads.

Cloud Service Providers usually provide API to create Kubernetes cluster by specifying the size, node VM specs, etc.

1. Use `az aks create` to create an Azure Kubernetes Service(AKS) Cluster. The example below creates the cluster:
    - Uses default Azure VM specs (Standard_DS2_v2).

- Contains 2 nodes.
- Attaches the AccessMatrix container registry created in the previous chapter.

Command

```
$ az aks create --resource-group AksGroup --name AMAksCluster --node-
count 2 --generate-ssh-keys --attach-acr accessmatrix
```

Response


```
{
  "aadProfile": null,
  "addonProfiles": null,
  "agentPoolProfiles": [
    {
      "availabilityZones": null,
      "count": 2,
      "enableAutoScaling": null,
      "enableNodePublicIp": null,
      "maxCount": null,
      "maxPods": 110,
      "minCount": null,
      "name": "nodepool1",
      "nodeLabels": null,
      "nodeTaints": null,
      "orchestratorVersion": "1.15.10",
      "osDiskSizeGb": 100,
      "osType": "Linux",
      "provisioningState": "Succeeded",
      "scaleSetEvictionPolicy": null,
      "scaleSetPriority": null,
      "tags": null,
      "type": "VirtualMachineScaleSets",
      "vmSize": "Standard_DS2_v2",
      "vnetSubnetId": null
    }
  ],
  "apiServerAccessProfile": null,
  "dnsPrefix": "AMAksClust-AksGroup-ff9f4e",
  "enablePodSecurityPolicy": null,
  "enableRbac": true,
  "fqdn": "amaksclust-aksgroup-ff9f4e-
8d234788.hcp.southeastasia.azmk8s.io",
  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourcegroups/AksGroup/providers/Microsoft.ContainerService/
managedClusters/AMAksCluster",
  "identity": null,
  "identityProfile": null,
  "kubernetesVersion": "1.15.10",
  "linuxProfile": {
    "adminUsername": "azureuser",
    "ssh": {
      "publicKeys": [
        {
          "keyData": "ssh-rsaxxxx"
        }
```

Command


```
      ]
    }
  },
  "location": "southeastasia",
  "maxAgentPools": 10,
  "name": "AMAksCluster",
  "networkProfile": {
    "dnsServiceIp": "10.0.0.10",
    "dockerBridgeCidr": "172.17.0.1/16",
    "loadBalancerProfile": {
      "allocatedOutboundPorts": null,
      "effectiveOutboundIps": [
        {
          "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/MC_AksGroup_AMAksCluster_southeastasia/provide
rs/Microsoft.Network/publicIPAddresses/01fd975d-dcf6-448a-800d-
5f9d879a49e3",
          "resourceGroup": "MC_AksGroup_AMAksCluster_southeastasia"
        }
      ],
      "idleTimeoutInMinutes": null,
      "managedOutboundIps": {
        "count": 1
      },
      "outboundIpPrefixes": null,
      "outboundIps": null
    },
    "loadBalancerSku": "Standard",
    "networkPlugin": "kubenet",
    "networkPolicy": null,
    "outboundType": "loadBalancer",
    "podCidr": "10.244.0.0/16",
    "serviceCidr": "10.0.0.0/16"
  },
  "nodeResourceGroup": "MC_AksGroup_AMAksCluster_southeastasia",
  "privateFqdn": null,
  "provisioningState": "Succeeded",
  "resourceGroup": "AksGroup",
  "servicePrincipalProfile": {
    "clientId": "cc27752d-7201-4648-93aa-757a4aca174e",
    "secret": null
  },
  "tags": null,
  "type": "Microsoft.ContainerService/ManagedClusters",
  "windowsProfile": null
}
```


2. Get access credentials for a managed Kubernetes cluster using the following command:

   `$az aks get-credentials --resource-group AksGroup --name AMAksCluster`

3. Use `kubectl get nodes` to verify the Kubernetes cluster is created by telling all nodes are ready:

Command
`$ kubectl get nodes`
Response
NAME STATUS ROLES AGE VERSION aks-nodepool1-16308542-vmss000000 Ready agent 19m v1.15.10 aks-nodepool1-16308542-vmss000001 Ready agent 18m v1.15.10

4. Optionally, we can use `az aks browse` to access the Kubernetes Dashboard.
    - Create a cluster role binding of the Kubernetes Dashboard:

Command
`$ kubectl create clusterrolebinding kubernetes-dashboard --clusterrole=cluster-admin --serviceaccount=kube-system:kubernetes-dashboard`
Response
`clusterrolebinding.rbac.authorization.k8s.io/kubernetes-dashboard created`

    - Start the Kubernetes Dashboard proxy locally:

Command
`$ az aks browse --resource-group AksGroup --name AMAksCluster`
Response
Merged "AMAksCluster" as current context in /tmp/tmpzpgm4e0q Proxy running on http://127.0.0.1:8001/ Press CTRL+C to close the tunnel...

3.1.2. Prepare Database/Cache Services and Configure AccessMatrix

The AccessMatrix Docker image is built based on **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz** package found in the CD image. For example, **AccessMatrix-5.7.5.7535-GA-tomcat-only.tar.gz.**

> **Important Note:**In some places, you are required to update AccessMatrix configuration files like **amsystem.properties**, **am5.xml**, **am5-ehcache.xml** and **server.xml**, which means you need to unzip the **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**, then rezip to build Docker image.

The following sections provide details on the preparation of services and containers used directly by AccessMatrix:

1. Database Service to store any persistence data of AccessMatrix.
2. Redis Cache Service to store any transient data that are frequently updated, e.g. E2EE session Id.
3. Azure Service Bus as message broker for Ehcache replication.
4. TLS configuration, either you want to have TLS connection ends in Ingress Controller or in AccessMatrix instance.
5. Configure AccessMatrix REST API URL for USO Client, if you are using the USO Client component.

Unpack AccessMatrix Base Package

To deploy AccessMatrix in a container, you need to take the package **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz** found in AccessMatrix CD Image, for example, **AccessMatrix-5.7.5.7535-GA-tomcat-only.tar.gz**. As explained above, you need to change some configurations in AccessMatrix; hence the file must be unpacked/unzipped first.

Database

There are various options for the database. You can choose to host a database server on VM or choose the database service offered by Cloud Service Provider. The database server must be accessible to both AccessMatrix containers in the Kubernetes Cluster and the administrator who would execute AccessMatrix database scripts. This document assumes that the Cloud Database Service is being used.

If Azure Kubernetes Service is selected for deployment, its Kubernetes Cluster has access to Azure SQL Database by default.

Depending on the JRE version used, an appropriate driver version must be used to connect Azure SQL. See the compatibility matrix in the Azure SQL documentation.

For the following examples, we will be using MSSQL JDBC version 8.4.1 for JRE 11.

1. Use `az sql server create` to create Azure SQL Server:

Command

```
$ az sql server create --name amdbaz --resource-group AksGroup --location
southeastasia --admin-user user1 --admin-password Password.1234
```

Response

```
{
  "administratorLogin": "user1",
  "administratorLoginPassword": null,
  "fullyQualifiedDomainName": "amdbaz.database.windows.net",
  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AksGroup/providers/Microsoft.Sql/servers/amdbaz",
  "identity": null,
  "kind": "v12.0",
  "location": "southeastasia",
  "minimalTlsVersion": null,
  "name": "amdbaz",
  "privateEndpointConnections": [],
  "publicNetworkAccess": "Enabled",
  "resourceGroup": "AksGroup",
  "state": "Ready",
  "tags": null,
  "type": "Microsoft.Sql/servers",
  "version": "12.0"
}
```

Important Note: It is crucial to use a strong and secure password in production.

2. Use `az sql server firewall-rule create` to create a firewall rule:

Note: To allow for Azure SQL connections, go to Azure Portal, enable "Allow Azure services and resources to access this server". According to Azure documentation, this option allows all Azure connections, including the connections from other customers' subscriptions. If this option is selected, ensure that login and user permissions limit access to authorized users only. For more information, refer to the Azure firewall documentation.

Command

```
$ az sql server firewall-rule create --resource-group AksGroup --server amdbaz
-n AllowYourIp --start-ip-address 192.168.1.1 --end-ip-address 192.168.1.1
```

Command

Response


```
{
  "endIpAddress": "0.0.0.0",
  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AksGroup/providers/Microsoft.Sql/servers/amdbaz/firew
allRules/AllowYourIp",
  "kind": "v12.0",
  "location": "Southeast Asia",
  "name": "AllowYourIp",
  "resourceGroup": "AksGroup",
  "startIpAddress": "0.0.0.0",
  "type": "Microsoft.Sql/servers/firewallRules"
}
```


3. Use `az sql db create` to create Azure SQL Database:

Command

```
$ az sql db create --resource-group AksGroup --server amdbaz --name amdb --
edition GeneralPurpose --family Gen5 --capacity 2
```

Response


```
{
  "autoPauseDelay": null,
  "catalogCollation": "SQL_Latin1_General_CP1_CI_AS",
  "collation": "SQL_Latin1_General_CP1_CI_AS",
  "createMode": null,
  "creationDate": "2020-04-20T10:34:38.343000+00:00",
  "currentServiceObjectiveName": "GP_Gen5_2",
  "currentSku": {
    "capacity": 2,
    "family": "Gen5",
    "name": "GP_Gen5",
    "size": null,
    "tier": "GeneralPurpose"
  },
  "databaseId": "6d18f9b1-5e21-4708-9da1-7c54a959f700",
  "defaultSecondaryLocation": "eastasia",
  "earliestRestoreDate": "2020-04-20T11:04:38.343000+00:00",
  "edition": "GeneralPurpose",
  "elasticPoolId": null,
  "elasticPoolName": null,
  "failoverGroupId": null,
  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AksGroup/providers/Microsoft.Sql/servers/amdbaz/datab
ases/amdb",
  "kind": "v12.0,user,vcore",
  "licenseType": "LicenseIncluded",
  "location": "southeastasia",
  "longTermRetentionBackupResourceId": null,
  "managedBy": null,
  "maxLogSizeBytes": 10307502080,
  "maxSizeBytes": 34359738368,
```

Command
<pre> "minCapacity": null,   "name": "amdb",   "pausedDate": null,   "readReplicaCount": 0,   "readScale": "Disabled",   "recoverableDatabaseId": null,   "recoveryServicesRecoveryPointId": null,   "requestedServiceObjectiveName": "GP_Gen5_2",   "resourceGroup": "AksGroup",   "restorableDroppedDatabaseId": null,   "restorePointInTime": null,   "resumedDate": null,   "sampleName": null,   "sku": {   "capacity": 2,   "family": "Gen5",   "name": "GP_Gen5",   "size": null,   "tier": "GeneralPurpose"   },   "sourceDatabaseDeletionDate": null,   "sourceDatabaseId": null,   "status": "Online",   "tags": null,   "type": "Microsoft.Sql/servers/databases",   "zoneRedundant": false }</pre>

4. Use `az sql db show-connection-string` to show jdbc connection String:

Command
`$ az sql db show-connection-string -s amdbaz -n amdb -c jdbc`
Response
"jdbc:sqlserver://amdbaz.database.windows.net:1433;database=amdb;user=<username>@amdbaz;password=<password>; encrypt=true;trustServerCertificate=false;hostNameInCertificate=*.database.windows.net;loginTimeout=30"

Configure Database Connection in AccessMatrix

After you set up the database schema properly and get the connection string, now it is time to update the database configuration in AccessMatrix to point to the database.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\conf\Catalina\localhost*.
2. Open the **am5.xml** file.
3. Comment out the following:

XML

```
    <Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"
        initialSize="10" maxTotal="80" maxIdle="20"
minIdle="10" maxWaitMillis="10000"
        validationQuery="SELECT COUNT(*) FROM am_vars
WHERE va_name IS NULL" testOnBorrow="true"
        username="user1" password="password"
driverClassName="org.apache.derby.jdbc.EmbeddedDriver"
                url="jdbc:derby:amdb"
   />
```

4. Uncomment the following and provide values for the properties as appropriate:

XML

```
<!--
<Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"
initialSize="10" maxWaitMillis="10000" maxTotal="80" maxIdle="20"
minIdle="10" validationQuery="select 1"
username="sa" password="password"
driverClassName="com.microsoft.sqlserver.jdbc.SQLServerDriver"

url="jdbc:sqlserver://192.168.1.205:1433;databaseName=amdb;sendStri
ngParametersAsUnicode=false"
   />
-->
```

5. Add the EncryptorUtil **factory** property to allow the decryption of a password that is encrypted using the EncryptorUtil:

XML

```
    <Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"

factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
        initialSize="10" maxWaitMillis="10000" maxTotal="80"
maxIdle="20" minIdle="10" validationQuery="select 1"
        username="sa" password="password"
driverClassName="com.microsoft.sqlserver.jdbc.SQLServerDriver"

url="jdbc:sqlserver://192.168.1.205:1433;databaseName=amdb;sendStri
ngParametersAsUnicode=false"
   />
```

Notes:

- Replace **username**, **password** and **url** to your credentials and database server URL, i.e. *jdbc:sqlserver://amdbaz.database.windows.net:1433;database=amdb;encrypt=true;trustServerCertificate=false;hostNameInCertificate=*.database.windows.net;loginTimeout=30*.

- The database administrator is required to use the EncryptorUtil to encrypt the database password, using the "Environment Variable as user salt" option. The encrypted password will look like the following:
  AesEncryptor::env=DBCRED:HtjCuzS34QxySZMsIav3umHgVFA5G5j5HCXoc7KQsUDDwyjcf/+65+756F4YJBGH,
  where env=DBCRED is the environment variable containing a user salt.
- For more information on using the EncryptorUtil, refer to *AccessMatrix Common Deployment Guide - Encryptor Utility*.

Enable AccessMatrix Container Mode

AccessMatrix itself needs a special configuration to set to run in a container. To set AccessMatrix to run in a container:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Uncomment the following am.container.enabled configuration and set it to "true":

   Properties (.properties files)

   ```
   # Enable this property if am run in container like
   docker
   #am.container.enabled=true
   ```

Make Configurations for Database Initialization

To allow for a pod to connect to initialize the database, the credentials for the database have to be configured. The password must be encrypted using the EncryptorUtil with an environment variable as the user salt.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\db-init*.
2. Open the **amsystem.properties** file.
3. Comment out the following:

   Properties (.properties files)

   ```
   #am.domain.store.datasource=java:/comp/env/jdbc/DSamdb
   ```
4. Uncomment the following and input the relevant information, for example:

   Properties (.properties files)

```
# JDBC
am.domain.store.driver_class=com.microsoft.sqlserver.jdbc.SQLServer
Driver
am.domain.store.url=jdbc:sqlserver://accessMatrix.database.windows.
net:1433;database=amdb;
am.domain.store.username=user1
am.domain.store.password=AesEncryptor::env=DBCRED:HtjCuzS34QxySZMsl
av3umHgVFA5G5j5HCXoc7KQsUDDwyjcf/+65+756F4YJBGH
```

5. Ensure container mode for AccessMatrix is enabled, as detailed in the previous section.

Redis Cache Server

For AccessMatrix, Redis Cache is treated as an in-memory database, used to store transient data that are frequently updated, for example, E2EE session, SMS-OTP tokens, SAML Response Id (to prevent replay attack), CLP login sessions (for AccessMatrix Applications, such as UAS TAP, USO SSF), and OIDC Relying Party Lookup Module nonces (to prevent request forgery or replay). For more information about Redis, refer to official Redis documentation.

Relevant certificates for SSL connectivity should be prepared. Mutual authentication depends on the capability of the Redis server in their respective clouds.

Azure as Redis Cache Server

Single Instance

On Azure, AccessMatrix will use Azure Cache for Redis as the Redis cache server. Azure only supports one way SSL.

The following creates a Basic C0 (250MB) Azure Cache for Redis instance:

Command
`$ az redis create --name accessmatrixCache --resource-group AksGroup --location southeastasia --sku Basic --vm-size C0`
Response
accessKeys: null enableNonSslPort: false hostName: accessmatrixCache.redis.cache.windows.net id: /subscriptions/ff9f4ef7-c33b-454c-9844-429361615308/resourceGroups/AksGroup/providers/Microsoft.Cache/Redis/accessmatrixCache instances: - isMaster: false

Command

```
  nonSslPort: null
  shardId: null
  sslPort: 15000
  zone: null
linkedServers: []
location: Southeast Asia
minimumTlsVersion: null
name: accessmatrixCache
port: 6379
provisioningState: Creating
redisConfiguration:
  maxclients: '256'
  maxfragmentationmemory-reserved: '12'
  maxmemory-delta: '2'
  maxmemory-reserved: '2'
redisVersion: 4.0.14
replicasPerMaster: null
resourceGroup: AksGroup
shardCount: null
sku:
  capacity: 0
  family: C
  name: Basic
sslPort: 6380
staticIp: null
subnetId: null
tags: {}
tenantSettings: {}
type: Microsoft.Cache/Redis
zones: null
```

Important Note: It may take 10-15 minutes to provision a Redis Cache.

To check the status of the Redis cache, enter the following:

Command

```
$az redis list
```

Response

```
- accessKeys: null
  enableNonSslPort: false
  hostName: accessmatrixCache.redis.cache.windows.net
  id: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AksGroup/providers/Microsoft.Cache/Redis/accessmatrixCache
  instances:
  - isMaster: true
    nonSslPort: null
    shardId: null
    sslPort: 15000
    zone: null
  linkedServers: []
  location: Southeast Asia
```

Command

```
minimumTlsVersion: null
name: accessmatrixCache
port: 6379
provisioningState: Succeeded
redisConfiguration:
  maxclients: '256'
  maxfragmentationmemory-reserved: '12'
  maxmemory-delta: '2'
  maxmemory-reserved: '2'
redisVersion: 4.0.14
replicasPerMaster: null
resourceGroup: AksGroup
shardCount: null
sku:
  capacity: 0
  family: C
  name: Basic
sslPort: 6380
staticIp: null
subnetId: null
tags: {}
tenantSettings: {}
type: Microsoft.Cache/Redis
zones: null
```

Clustered Instances

Azure Cache for Redis supports Redis Clusters, where data is automatically shared across multiple Redis nodes. This enables high availability through failovers. Azure cache handles the scaling and clustering internally and manages the shards and replication for the user, so no additional configuration is required on AccessMatrix. This feature is only available on the premium tier.

The following command creates a clustered Azure Cache for Redis with two shards:

Command

```
$az redis create --location southeastasia --name AccessMatrix --resource-group
AksGroup --sku Premium --vm-size p1 --shard-count 2
```

Response

JSON


```
{
  "accessKeys": null,
  "enableNonSslPort": false,
  "hostName": "AccessMatrix.redis.cache.windows.net",
  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AksGroup/providers/Microsoft.Cache/Redis/AccessMat
```

Command


```
rix",
  "identity": null,
  "instances": [
    {
      "isMaster": true,
      "isPrimary": true,
      "nonSslPort": null,
      "shardId": 0,
      "sslPort": 15000,
      "zone": null
    },
    {
      "isMaster": false,
      "isPrimary": false,
      "nonSslPort": null,
      "shardId": 0,
      "sslPort": 15001,
      "zone": null
    },
    {
      "isMaster": true,
      "isPrimary": true,
      "nonSslPort": null,
      "shardId": 1,
      "sslPort": 15002,
      "zone": null
    },
    {
      "isMaster": false,
      "isPrimary": false,
      "nonSslPort": null,
      "shardId": 1,
      "sslPort": 15003,
      "zone": null
    }
  ],
  "linkedServers": [],
  "location": "Southeast Asia",
  "minimumTlsVersion": null,
  "name": "AccessMatrix",
  "port": 6379,
  "privateEndpointConnections": null,
  "provisioningState": "Succeeded",
  "publicNetworkAccess": "Enabled",
  "redisConfiguration": {
    "additionalProperties": null,
    "aofStorageConnectionString0": null,
    "aofStorageConnectionString1": null,
    "maxclients": "7500",
    "maxfragmentationmemoryReserved": "642",
    "maxmemoryDelta": "642",
    "maxmemoryPolicy": null,
    "maxmemoryReserved": "642",
    "preferredDataArchiveAuthMethod": null,
    "preferredDataPersistenceAuthMethod": null,
    "rdbBackupEnabled": null,
    "rdbBackupFrequency": null,
    "rdbBackupMaxSnapshotCount": null,
```

Command


```
    "rdbStorageConnectionString": null
  },
  "redisVersion": "6.0.14",
  "replicasPerMaster": null,
  "replicasPerPrimary": null,
  "resourceGroup": "AksGroup",
  "shardCount": 2,
  "sku": {
    "capacity": 1,
    "family": "P",
    "name": "Premium"
  },
  "sslPort": 6380,
  "staticIp": null,
  "subnetId": null,
  "tags": {},
  "tenantSettings": {},
  "type": "Microsoft.Cache/Redis",
  "zones": null
}
```


The information required by AccessMatrix is listed below:

Key	Description
hostName	Host name of the Redis server, e.g. accessmatrixCache.redis.cache.windows.net.
port	Port of Redis defaults to 6379 for non-SSL and 6380 for SSL.
password	Access key for Azure Redis Cache. Access Keys can be obtained from the Azure Portal. For more information, check the official documentation about Azure Redis Access Key.

Configure Redis Connection in AccessMatrix

After you set up Redis service properly and get the connection information (hostname and port), it is time to update Redis configuration in AccessMatrix to point to the Redis service.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Configure the following items according to the information in the above table:
    - host
    - port

- am.redis.password

  For example, if using a non-secure port:

- host=accessmatrixCache.redis.cache.windows.net
- port=6379
- Obtain the access key and input it for am.redis.password

Some examples are shown below:

Azure Redis Non-SSL port

Properties (.properties files)

```
# == Cache Provider Configuration ==
#choose redis or ehcache, if not set, default is
ehcache
am.cache.vendor=redis
am.redis.host=accessmatrixCache.redis.cache.windows.net
am.redis.port=6379
#optional, don't set if no password required
am.redis.password={YOUR_AZURE_REDIS_ACCESS_KEY}
#optional, default is false
am.redis.ssl.enabled=false
```

Azure Redis SSL port

If the SSL port is enabled, make sure certificate files are prepared.

Properties (.properties files)

```
# == Cache Provider Configuration ==
#choose redis or ehcache, if not set, default is
ehcache
am.cache.vendor=redis
am.redis.host=accessmatrixCache.redis.cache.windows.net
am.redis.port=6380
#optional, don't set if no password required
am.redis.password={YOUR_AZURE_REDIS_ACCESS_KEY}
#optional, default is false
am.redis.ssl.enabled=true
#optional parameters for setting up redis ssl
connection, they will only be effective when
am.redis.ssl.enabled=true:
am.redis.ssl.keyStore=conf/teststore.jks
am.redis.ssl.keyStorePassword=test
am.redis.ssl.keyStoreAlias=Server (CA)
am.redis.ssl.keyStoreKeyPassword=test
```

Use Encryptor Util to Encrypt Redis Password

The property am.redis.password in the **amsystem.properties** file accepts Redis password as an encrypted text. The encrypted text must be generated from the EncryptorUtil.

For more information on using EncryptorUtil, refer to *AccessMatrix Common Deployment Guide - Encryptor Utility*.

Azure Cache for Redis Pub/Sub

Azure supports the Redis Publish/Subscribe paradigm, where messages are not sent to specific receivers (subscribers), but instead sent to channels without knowledge of the subscribers. This service can be used as an alternative to a JMS service like Azure Service Bus or ActiveMQ.

> **Note:** It is recommended that you use Redis Pub/Sub on an AccessMatrix cloud deployment instead of Java Messaging Service (JMS) services like ActiveMQ when deploying with Redis.

Configure Redis Pub/Sub in AccessMatrix

After you set up Redis service properly and get the connection information (hostname and port) in the previous section:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Configure the following items:

Properties (.properties files)

```
# == Ehcache ==
# EhCache Cache Replicator Factory: RMI(default) or JMS
or Redis
# If JMS is used, related properties(starting with
am.ehcache.jms) must be enabled.
#am.ehcache.replicator.factory=RMI
# If Redis is used
am.ehcache.replicator.factory = Redis
am.ehcache.replicator.redis.channel = ampubsub
am.ehcache.replicator.redis.reconnect.secs = 10
```

Azure Service Bus

Azure Service Bus is a fully managed enterprise message broker with message queues and publish-subscribe topics. For AccessMatrix deployment, we will use Azure Service Bus as the Ehcache JMS Cache Replicator.

Create Service Bus Namespace

Use `az servicebus namespace create` to create a Service Bus Namespace.

Command

```
$ az servicebus namespace create --name AM5ServiceBus --resource-group AKSGroup --sku standard --location southeastasia
```

Response


```
{
  "createdAt": "2021-07-24T13:28:29.523000+00:00",
  "encryption": null,
  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-429361615308/resourceGroups/AKSGroup/providers/Microsoft.ServiceBus/namespaces/AxMxServiceBus",
  "identity": null,
  "location": "Southeast Asia",
  "metricId": "ff9f4ef7-c33b-454c-9844-429361615308:axmxservicebus",
  "name": "AxMxServiceBus",
  "provisioningState": "Succeeded",
  "resourceGroup": "AKSGroup",
  "serviceBusEndpoint": "https://AxMxServiceBus.servicebus.windows.net:443/",
  "sku": {
    "capacity": null,
    "name": "Standard",
    "tier": "Standard"
  },
  "tags": {},
  "type": "Microsoft.ServiceBus/Namespaces",
  "updatedAt": "2021-07-24T13:29:15.253000+00:00",
  "zoneRedundant": false
}
```


Check Authorization Rule

We need an authorization rule (also called a "Shared Access Policy" in Azure Portal) to access the Service Bus; a default rule "RootManageSharedAccessKey" has been created when Service Bus is ready. Thus we can use `az servicebus namespace authorization-rule keys list` to get retrieve access keys of the default rule.

Command

```
$ az servicebus namespace authorization-rule keys list --name
RootManageSharedAccessKey --namespace-name AxMxServiceBus --resource-group AKSGroup
```

Response


```
{
  "aliasPrimaryConnectionString": null,
  "aliasSecondaryConnectionString": null,
  "keyName": "RootManageSharedAccessKey",
  "primaryConnectionString":
  "Endpoint=sb://axmxservicebus.servicebus.windows.net/;SharedAccessKeyName=RootManageS
haredAccessKey;SharedAccessKey=PEd1hApvyfJ+pawfV5rwT7mzMYMSqo2eWunk0Lv3UkI=",
  "primaryKey": "PEd1hApvyfJ+pawfV5rwT7mzMYMSqo2eWunk0Lv3UkI=",
  "secondaryConnectionString":
  "Endpoint=sb://axmxservicebus.servicebus.windows.net/;SharedAccessKeyName=RootManageS
haredAccessKey;SharedAccessKey=PRgPZqJLhEvhLWcwF37eUvHOGUEz/535+zadUkOy8+E=",
  "secondaryKey": "PRgPZqJLhEvhLWcwF37eUvHOGUEz/535+zadUkOy8+E="
}
```


Create Service Bus Topic

After the Namespace is ready, we will use `az servicebus topic create` to create a Service Bus Topic used by AccessMatrix's EhCache JMS cache replicator.

Command

```
$ az servicebus topic create --name amtopic --namespace-name AxMxServiceBus --
resource-group AKSGroup
```

Response


```
{
  "accessedAt": "0001-01-01T00:00:00",
  "autoDeleteOnIdle": "10675199 days, 2:48:05.477581",
  "countDetails": {
    "activeMessageCount": 0,
    "deadLetterMessageCount": 0,
    "scheduledMessageCount": 0,
    "transferDeadLetterMessageCount": 0,
    "transferMessageCount": 0
  },
  "createdAt": "2021-07-24T13:54:47.080000+00:00",
  "defaultMessageTimeToLive": "10675199 days, 2:48:05.477581",
  "duplicateDetectionHistoryTimeWindow": "0:10:00",
  "enableBatchedOperations": true,
  "enableExpress": false,
  "enablePartitioning": false,
  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AKSGroup/providers/Microsoft.ServiceBus/namespaces/AxMxServ
iceBus/topics/amtopic",
  "location": "Southeast Asia",
```

Command
<pre> "maxSizeInMegabytes": 1024,   "name": "amtopic",   "requiresDuplicateDetection": false,   "resourceGroup": "AKSGroup",   "sizeInBytes": 0,   "status": "Active",   "subscriptionCount": 0,   "supportOrdering": true,   "type": "Microsoft.ServiceBus/Namespaces/Topics",   "updatedAt": "2021-07-24T13:54:47.147000+00:00" }</pre>

Create Service Principle

The last step is to create an Azure Service Principle (also called App Registrations in Azure Portal) that will be used in AccessMatrix to communicate with Service Bus. We will use `az ad sp create-for-rbac` to create one.

Command
`$ az ad sp create-for-rbac --name amsb`
Response
<pre>{   "appId": "2c397cf6-cf47-44f5-b8d9-389820d3c588",   "displayName": "amsb",   "name": "2c397cf6-cf47-44f5-b8d9-389820d3c588",   "password": "C-O63M6-sMmE5xR4NfefkPWtIAPqlH80Tx",   "tenant": "e13fdd6a-4b47-4ce3-9bcd-771d54a3e68d" }</pre>

Please store the password clear text properly as it will not be obtainable from Azure again.

Configure Azure Service Bus Connection in AccessMatrix

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Change the replication factory from RMI to JMS:

   am.ehcache.replicator.factory=RMI

   to

   am.ehcache.replicator.factory=JMS
4. Uncomment and update the configuration for am.ehcache.jms.azuresb:

Properties (.properties files)

```
<!-- Uncomment the following if Azure ServiceBus is used -->
# EhCache JMS Cache Peer Provider (ONLY IF using Azure Service Bus)
# Other am.ehcache.jms properties must be disabled
#am.ehcache.jms.azuresb.providerURL=failover:(amqps://axmxservicebu
s.servicebus.windows.net?amqp.idleTimeout=120000&amp;amqp.traceFram
es=true)
#am.ehcache.jms.azuresb.securityPrincipalName=RootManageSharedAcces
sKey
#am.ehcache.jms.azuresb.securityCredentials=PEd1hApvyfJ+pawfV5rwT7m
zMYMSqo2eWunk0Lv3UkI=
#am.ehcache.jms.azuresb.tenantId=e13fdd6a-4b47-4ce3-9bcd-
771d54a3e68d
#am.ehcache.jms.azuresb.subscriptionId=ff9f4ef7-c33b-454c-9844-
429361615308
#am.ehcache.jms.azuresb.client_id=cc27752d-7201-4648-93aa-
757a4aca174e
#am.ehcache.jms.azuresb.client_secret=<your_client_secret>
#am.ehcache.jms.azuresb.namespaceName=AxMxServiceBus
#am.ehcache.jms.azuresb.resourceGroupName=AKSGroup
#am.ehcache.jms.azuresb.topicName=amtopic
# Time interval (seconds) of generating cache to send to Azure
Service Bus for heartbeat purpose. Default is 300.
#am.ehcache.jms.azuresb.heartbeat=300
```

Important Note: Update the configuration with the required information

5. Copy Apache Qpid JMS lilbraries from product release package *AccessMatrix-<version>-.zip/sdk/AzureServiceBus/qpid-libs/* into *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/lib/*.
6. Take out **javax-jms-1.1.jar** from *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/lib/*.

Domain name and TLS Connection

A domain name must be prepared (e.g. *www.example.com*), to be used in the TLS certificate and set in a proper DNS server. The Ingress load balancer will use it in Kubernetes.

The terminology TLS and SSL are used interchangeably. SSL is deprecated, replaced with TLS. However, SSL terminology is still being used by Kubernetes itself.

TLS Connection Ends in AccessMatrix

Refer to the architecture diagram. This configuration means a secure TLS connection from the user's endpoint (browser or mobile device) ends in AccessMatrix instances/pods. Ingress controller is configured as SSL-Passthrough, which means any TLS connection will only pass through Ingress Controller.

1. Navigate to the directory *<ACMATRIX_ROOT>/tomcat/conf/* and *<AMS_ROOT>/tomcat/conf/*.
2. Open the **server.xml** file.
3. Check the Connector element for HTTPS port (which defaults to 8443).
    a. Inject your TLS certificate with the proper domain name to a keystore file, for example, **testserver.keystore**.
    b. Enter the keystore file name in the keystoreFile parameter of the Connector element in **server.xml**.

Configure AccessMatrix REST API URL for USO Client

If you are using the USO Client component, you are required to configure the AccessMatrix REST API URL so the USO Client can communicate with the AccessMatrix server.

> **Note:** Refer to *USO Administration Guide - USO Client User Guide* for more information on the USO Client.

Typically, the USO Client and the SSO Portal back end use the same URL to communicate with the AccessMatrix server. However, in some cloud deployments (such as with Kubernetes/OpenShift), they must use different URLs: the SSO Portal back end uses the internal private Kubernetes service URL, while the USO Client must use AccessMatrix's public URL. This is because the USO Client is installed on the end user's desktop, outside of Kubernetes/OpenShift (where the AccessMatrix server is).

To configure this public REST API URL for the USO Client:

1. Navigate to *<ACMATRIX_ROOT>/tomcat/webapps/usoserver/WEB-INF/* and open the **web.xml** file.
2. Search for the commented USO Client URL section:

XML

```
<!-- [Optional] URL used by USO Client to connect to
AM5 server -->
<!-- If not specified, default to same value as
'restAPIURL' -->
<!-- For Kubernetes deployment, using this is mandatory
as the internal
     K8s services URL used by usoserver is different
from the external
     URL used by USO Client -->
<!--
<context-param>
    <param-name>restAPIURLPublic</param-name>
    <param-value>https://am5-public-
```

```
url.com:443/am5/rest</param-value>
</context-param>
-->
```

3. Uncomment the section and change the param-value to the public REST API URL of the AccessMatrix server. For example:

XML

```
<!-- [Optional] URL used by USO Client to connect to
AM5 server -->
<!-- If not specified, default to same value as
'restAPIURL' -->
<!-- For Kubernetes deployment, using this is mandatory
as the internal
     K8s services URL used by usoserver is different
from the external
     URL used by USO Client -->
<context-param>
    <param-name>restAPIURLPublic</param-name>
    <param-value>https://amserver.com/am5/rest</param-
value>
</context-param>
```

4. Save the changes.

3.1.3. Create AccessMatrix Docker Image

At this stage, all of the services that will be used by AccessMatrix are ready, and related AccessMatrix base package configurations have been updated. You can now proceed to build the AccessMatrix Docker image. An AccessMatrix Docker image must be built locally before being pushed to the registry.

Repack AccessMatrix Base Package

You have prepared the database service, Redis cache service, and Azure Service Bus. You have also updated the related configurations in AccessMatrix, you need to repack it back to its respective name (**AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**). For example: **AccessMatrix-5.7.5.7535-GA-tomcat-only.tar.gz**.

Pre-image Build Checklist

Before building the AccessMatrix Docker Container Image, ensure the following are done:

Checklist	Description
AccessMatrix container mode	AccessMatrix container mode has been enabled in **amsystem.properties**. Refer to Enable AM Container Mode for more information.
Database service	Database service is ready and its connection information is configured in **am5.xml**. Refer to Database Service for more information.
Redis Cache service	Redis Cache service is ready and its connection information is configured in **amsystem.properties**. Refer to Redis Cache Server for more information.
Azure Service Bus	Azure Service Bus is up and running and its connection is configured in **am5-ehcache.xml** and **am5.xml**. Refer to Configure Azure Service Bus Connection in AccessMatrix for more information.
TLS Connection Ends in AM	If the TLS connection were to end in AccessMatrix instance (Ingress controller passthrough), the certificate and port are configured in **server.xml**. Refer to TLS Connection Ends in AccessMatrix for more information.
AM Base Package	AM base package that was previously unpacked must be repacked back to its original name.
Dependent libraries	Any additional libraries or drivers such as JDBC, or any external drivers, must be placed in the lib folder in the same directory as the Dockerfile.

Build AccessMatrix Docker Image

A sample AccessMatrix Dockerfile is shown in the steps below. Arrange the files as follows:

1. Place the Dockerfile in any working directory that you choose. Change the environment variable AM_VERSION to the respective AccessMatrix version.

Dockerfile

Code

```
FROM adoptopenjdk/openjdk11:slim

ENV AM_VERSION "5.7.5.7535-GA"

RUN mkdir -p /opt/am/${AM_VERSION}
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}
RUN groupadd -g 2020 am \
    && useradd -r -g am -u 2021 -s /bin/bash am \
    && chown -R am:am /opt/am/${AM_VERSION} \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/db-init/*.sh \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:am /opt/am/am

COPY ["lib", "/opt/am/am/tomcat/lib"]

EXPOSE 8080 8443 8161 61616
USER am
CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
```

Note: The user and group are created with fixed uid and gid to access control and avoid deployment using root user.

2. Place the Dockerfile in any working directory that you choose. Change the environment variable AM_VERSION to the respective AccessMatrix version.
3. Create a subdirectory, *lib*, to place all dependent libraries, e.g. JDBC drivers.
4. Create a subdirectory, *repo*, to place AccessMatrix base package **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**.
5. Use `docker build` to build image:

Command

```
$ docker build . -f Dockerfile -t am5:5.7.5
```

Response

Command

```
Sending build context to Docker daemon 157.8MB
Step 1/9 : FROM adoptopenjdk/openjdk11:slim
  ---> a73e537ebcd4
Step 2/9 : ENV AM_VERSION "5.7.5.7535-GA"
  ---> Using cache
  ---> 6c209940bd1a
Step 3/9 : RUN mkdir -p /opt/am/${AM_VERSION}
  ---> Using cache
  ---> 9141c3c1a1f0
Step 4/9 : ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}
  ---> Using cache
  ---> ba4e81749f03
Step 5/9 : RUN groupadd am && useradd -r -g am -s /bin/bash am && chown -R am:am
/opt/am/${AM_VERSION}&& chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh && ln -s
/opt/am/${AM_VERSION} /opt/am/am && chown -R am:am /opt/am/am
  ---> Using cache
  ---> 212ce4fbbbf1
Step 6/9 : COPY ["lib", "/opt/am/am/tomcat/lib"]
  ---> d2a8441678c6
Step 7/9 : EXPOSE 8080 8443 8161 61616
  ---> Running in cfba31838a66
Removing intermediate container cfba31838a66
  ---> 642affec76dd
Step 8/9 : USER am
  ---> Running in ec4e05c7b34d
Removing intermediate container ec4e05c7b34d
  ---> 43fcb8a5dc33
Step 9/9 : CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
  ---> Running in aab7293a547b
Removing intermediate container aab7293a547b
  ---> afc1178a5a33
Successfully built afc1178a5a33
Successfully tagged am5:5.7.5
```

Build AccessMatrix Docker Image in RedHat Universal Base Image 8 (UBI 8)

The following is a sample AccessMatrix Dockerfile build using Red Hat Enterprise Linux (RHEL) Universal Base Image 8 (UBI 8) with OpenJDK-11.

1. Place the Dockerfile in any working directory that you choose. Then, change the environment variable AM_VERSION to the respective AccessMatrix version.

Dockerfile

Code

```
FROM redhat/ubi8
```

Dockerfile

```
ENV AM_VERSION "5.7.5.7535-GA"
ENV JAVA_VERSION "11.0.13_linux-x64_bin"
ENV JDK_VERSION "jdk-11.0.13"

ENV JAVA_HOME "/opt/jdk/${JDK_VERSION}"
ENV PATH "$PATH:/opt/jdk/${JDK_VERSION}/bin"

USER 0

RUN mkdir -p /opt/jdk/
RUN mkdir -p /opt/am/${AM_VERSION}

ADD repo/jdk-${JAVA_VERSION}.tar.gz /opt/jdk/
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}


RUN groupadd -g 2020 am \
    && useradd -r -g am -u 2021 -s /bin/bash am \
    && chown -R am:am /opt/am/${AM_VERSION} \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:am /opt/am/am


COPY ["lib", "/opt/am/am/tomcat/lib"]

EXPOSE 8080 8443 8161 61616
USER am
CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
```

2. Create a subdirectory, *lib*, to place all dependent libraries, e.g. JDBC drivers.
3. Create a subdirectory, *repo*, to place AccessMatrix base package **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**.
4. Place the Java JDK archive **jdk-${JAVA_VERSION}.tar.gz** in *repo.*
5. Use `docker build` to build image:

Command

```
$ docker build . -f Dockerfile -t am5:5.7.5
```

Response

```
Sending build context to Docker daemon 156MB
Step 1/16 : FROM redhat/ubi8
  ---> ad42391b9b46
Step 2/16 : ENV AM_VERSION "5.7.5.7535-GA"
  ---> Using cache
  ---> 673f17444946
Step 3/16 : ENV JAVA_VERSION "11.0.13-linux-x64"
```

Command

```
  ---> Running in ef2b9da9dadb
Removing intermediate container ef2b9da9dadb
  ---> 6b3860c722eb
Step 4/16 : ENV JDK_VERSION "jdk-11.0.13"
  ---> Running in 9ed6bb0200a1
Removing intermediate container 9ed6bb0200a1
  ---> 658c9387f3e8
Step 5/16 : ENV JAVA_HOME "/opt/jdk/${JDK_VERSION}"
  ---> Running in e7741a2c3596
Removing intermediate container e7741a2c3596
  ---> 7720fc2f91ff
  ---> Running in b15fae561b21
Removing intermediate container b15fae561b21
  ---> 7a3e8e58fb9f
Step 7/16 : USER 0
  ---> Running in 2f8ef45ca034
Removing intermediate container 2f8ef45ca034
  ---> 9b33d21a4c5f
Step 8/16 : RUN mkdir -p /opt/jdk/
  ---> Running in fe4fb69a97f7
Removing intermediate container fe4fb69a97f7
  ---> 3912c0b890af
Step 9/16 : RUN mkdir -p /opt/am/${AM_VERSION}
  ---> Running in ae3bf08ce982
Removing intermediate container ae3bf08ce982
  ---> 3496e2c69c46
Step 10/16 : ADD repo/jdk-${JAVA_VERSION}.tar.gz /opt/jdk/
Step 11/16 : ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}
  ---> b792174dac85
Step 12/16 : RUN groupadd -g 2020 am && useradd -r -g am -u 2021 -s /bin/bash am
&& chown -R am:am /opt/am/${AM_VERSION}&& chmod 754
/opt/am/${AM_VERSION}/tomcat/bin/*.sh && ln -s /opt/am/${AM_VERSION} /opt/am/am
&& chown -R am:am /opt/am/am
  ---> Running in 5e9cc425d11a
Removing intermediate container 5e9cc425d11a
  ---> 94168b6a6e8f
Step 13/16 : COPY ["lib", "/opt/am/am/tomcat/lib"]
  ---> 0d63a6514333
Step 14/16 : EXPOSE 8080 8443 8161 61616
  ---> Running in 5ab68b7615a8
Removing intermediate container 5ab68b7615a8
  ---> 06c911148d4c
Step 15/16 : USER am
  ---> Running in d99a5b893972
Removing intermediate container d99a5b893972
  ---> f44b70dbfed5
Step 16/16 : CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
  ---> Running in 8d4e07008898
Removing intermediate container 8d4e07008898
  ---> bc9fe96fc176
Successfully built bc9fe96fc176
Successfully tagged am:5.7.5
```

Build AccessMatrix Docker Image in RedHat Universal Base Image 8 (UBI 8) with OpenJDK

The AccessMatrix Dockerfile can also be built using OpenJDK, at risk to the overall build time for installation of OpenJDK.

Dockerfile

Code

```
FROM redhat/ubi8

ENV AM_VERSION "5.7.5.7535-GA"

USER 0

RUN mkdir -p /opt/am/${AM_VERSION}
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}

RUN yum update \
    && yum install java-11-openjdk.x86_64 -y \
    && yum clean all

RUN groupadd -g 2020 am \
    && useradd -r -g am -u 2021 -s /bin/bash am \
    && chown -R am:am /opt/am/${AM_VERSION} \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:am /opt/am/am


COPY ["lib", "/opt/am/am/tomcat/lib"]

EXPOSE 8080 8443 8161 61616
USER am
CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
```

Note: Image build time will increase due to the installation of OpenJDK. Alternatively, the builder can opt for using the official ubi8 openjdk-11 container image.

Push AccessMatrix Docker Image to Registry

1. Use `az acr login` to log in before pushing the image:

Command

```
$ az acr login --name accessmatrix
```

Command
Response
`Login Succeeded`

2. Use `docker tag` and `docker push` to tag the docker image then push to the registry (e.g. Azure Container Registry):

Command
`$ docker tag am5:5.7.5 accessmatrix.azurecr.io/am5:5.7.5` `$ docker image ls`
Response
`REPOSITORY TAG IMAGE ID CREATED SIZE` `accessmatrix.azurecr.io/am5 5.7.5 afc1178a5a33 10 minutes ago 758MB` `am5 5.7.5 afc1178a5a33 10 minutes ago 758MB`

Command
`$ docker image push accessmatrix.azurecr.io/am5:5.7.5`
Response
`The push refers to repository [accessmatrix.azurecr.io/am5:5.7.5]` `247946ea3c85: Pushed` `047dc610d4f8: Pushed` `f1289005a4b3: Pushed` `66ff9452d4c6: Pushed` `5d0f1dae109c: Pushed` `1b853245b739: Pushed` `f801f021a84a: Pushed` `789c82f5f442: Pushed` `3e2a3ebdb62c: Pushed` `e8243c609e31: Pushed` `cfb7bbd3b9bc: Pushed` `51a0cde65eeb: Pushed` `16542a8fc3be: Pushed` `6597da2e2e52: Pushed` `977183d4e999: Pushed` `c8be1b8f4d60: Pushed` `5.7.5: digest:` `sha256:a977f44de80368bd83511456f8facd2375da70691ac62dd23d49afade326ac2f size:` `3672`

!	**Important Note**: Version tagging of images is important as tagging with the "latest" tag can lead to problems in a managed environment.

	According to Kubernetes docs, labelling all images with the latest makes it difficult to track the current version of the image running and makes it difficult to roll back when errors occur.

3.1.4. Create and Deploy Kubernetes Objects

At this stage, AccessMatrix Docker image has been configured properly and pushed to the container registry. Now you are ready to create/deploy Kubernetes objects.

Kubernetes objects are persistent entities in the Kubernetes system. Kubernetes uses these entities to represent the state of your cluster. We describe the Kubernetes object using YAML files. For any YAML files listed in this chapter, use command `kubectl apply -f <file-name>.yaml` to create Kubernetes objects in the cluster. For example:

Command
`$ kubectl apply -f am-k8s.yaml`
Response
`deployment.apps/am-k8s created`

Persistent Volumes and Claims

A Persistent Volume (PV) is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes. It is a resource in the cluster, just like a node is a cluster resource. PVs are volume plugins similar to Docker Volumes but have a lifecycle independent of any individual pod that uses the PV. This API object captures the details of the storage implementation, be that NFS, iSCSI, or a cloud-provider-specific storage system.

A Persistent Volume Claim (PVC) is a request for storage by a user (refer to architecture diagram). It is similar to a Pod so that Pods consume node resources, and PVCs consume PV resources. Pods can request specific levels of resources (CPU and Memory). PVC can request specific size and access modes (e.g., they can be mounted once read/write or many times read-only).

Usage of PV (Persistent Volume) and PVC (Persistent Volume Claim) for AccessMatrix deployed in Kubernetes is to have a persistent centralized storage of log files generated in multiple AccessMatrix instances/Pods.

Azure has various storage class options. The PV and PVC yaml below shows an example of Azure File to be claimed as:

am-pv-azure.yaml

YAML

```yaml
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: azurefile
provisioner: kubernetes.io/azure-file
mountOptions:
  - dir_mode=0755
  - file_mode=0644
  - uid=2021
  - gid=2020
  - mfsymlinks
  - cache=strict
parameters:
  skuName: Standard_LRS
```

Important Notes:

- While there are multiple storage access modes, note that when using ReadWriteMany, the volume can be mounted as read-write by many nodes.
- The default reclaim policy is "Delete", if an old Persistent Volume is deleted and then recreated with the same Persistent Volume Claim, the old contents will be overwritten.
- The uid and gid value must be the same as the uid and gid of am user in the Dockerfile as described in Build AccessMatrix Docker Image.

Temporary Report Directory

When the AccessMatrix system receives a request to generate a report, the system will generate a report resulting in any of these report files: CSV, PDF, or HTML. The generated report file(s) will be stored temporarily in a reports folder under this directory: *ACMATRIX_HOME/work/*. Example: *opt/am/am/tomcat/work/reports/*.

This PVC needs to be created and mounted as part of the deployment to allow report generation and download requests to be served by different pods.

Below is a sample YAML file for creating PVC for the reports:

am-report-pvc.yaml

YAML

am-report-pvc.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: amreportspvc
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: azurefile
  resources:
    requests:
      storage: 1Gi
```

Note: Refer to TLS Connection Ends in AccessMatrix section for the sample Deployment YAMLs.

Log Files

Log files may be backed by PVC so that the log files are not deleted when the pods are deleted.

am-deployment-logs-pvc.yaml

YAML

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: amlogspvc
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: azurefile
  resources:
    requests:
      storage: 1Gi
```

Note: The name of the PVCs will be referenced later in the deployment configuration.

There might be a need to register the storage provider if provisioning a PVC fails. To do so, register the storage provider with the following:

Command

```
$ az provider register --namespace Microsoft.Storage
```

For more information about the registration of providers, refer to the official Azure documentation.

AccessMatrix Deployment

Deployment is the main Kubernetes Object that represents which and how an application will run in the cluster.

Each Kubernetes Pod is meant to run a single instance of the application; thus, we deploy only one AccessMatrix container per Pod. In this case, Pod is a wrapper around a single container while Kubernetes manages the Pods rather than the containers directly.

As explained in Domain name and TLS Connection section, the deployment will have a TLS connection that ends in AM and AMS. The sample deployment for each AM and AMS will have:

- Two Pod replicas.
- Request for 100m CPU, which is equivalent to 10% of 1vCPU. For further information, read Kubernetes Resource Units.

Kubernetes Secrets

Secrets can be used like ConfigMap and are used to store sensitive information such as passwords. Pods in a deployment can refer to the secret as environment variables as well. For example, use Kubernetes Secrets to reference credentials:

Command
`kubectl create secret generic am-credentials \ --from-file=DBCRED`
Response
`secret/am-credentials created`

By default, Kubernetes uses the filename of the text file as the key. In this case, DBCRED is the key, and the value will be the file's contents, the user salt used to encrypt the database passphrase.

Database Initialization

To avoid the concurrent initialization of the AccessMatrix database because of multiple replicas, a database administrator must initialize AccessMatrix database with a pod. The pod will run a script to initialize the server and close, and it must be running before the deployment of AccessMatrix.

Below is a sample YAML for a Database Initialization pod:

am-dbinit.yaml

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: dbinit
spec:
  containers:
  - image: <container-registry-server>/am5:<version>
    name: dbinit
    env:
    - name: DBCRED
      valueFrom:
        secretKeyRef:
          key: DBCRED
          name: am-credentials
    command: [ "/bin/sh", "-c"]
    args: ["cd /opt/am/am/tomcat/db-init; ./dbUtil.sh;"]
    volumeMounts:
    - mountPath: /opt/am/am/tomcat/logs
      name: amlogspvc
  volumes:
  - name: amlogspvc
    persistentVolumeClaim:
      claimName: amlogspvc
  restartPolicy: Never
```

Note: The PVC and claim name is defined in the previous section in Persistent Volumes and Claims.

Deployment Where TLS Connection Ends in AccessMatrix

Below is a sample Deployment YAML for AccessMatrix where the TLS connection ends in AccessMatrix Pods (which means, from Ingress controller to AccessMatrix Pods will still be TLS encrypted):

am-k8s-ssl.yaml

YAML

```yaml
apiVersion: "apps/v1"
kind: "Deployment"
metadata:
  name: "am-k8s"
  namespace: "default"
  labels:
    app: "am-k8s"
```

am-k8s-ssl.yaml

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: "am-k8s"
  template:
    metadata:
      labels:
        app: "am-k8s"
    spec:
      containers:
      - name: am5
        image: <container-registry-server>/am5:<version>
        env:
        - name: DBCRED
          valueFrom:
            secretKeyRef:
              key: DBCRED
              name: am-credentials
        readinessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        livenessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        ports:
        - containerPort: 8080
          protocol: TCP
        - containerPort: 8443
          protocol: TCP
        lifecycle:
          preStop:
            exec:
              command: ["/bin/sh", "-c", "echo Stopping
server; sh /opt/am/am/tomcat/bin/catalina.sh stop"]
        volumeMounts:
        - mountPath: /opt/am/am/tomcat/work/reports
          name: amreportspvc
        - mountPath: /opt/am/am/tomcat/logs
          name: amlogspvc
        securityContext:
          privileged: true
        resources:
          requests:
            cpu: 100m
```

am-k8s-ssl.yaml

```
      volumes:
      - name: amreportspvc
        persistentVolumeClaim:
          claimName: amreportspvc
      - name: amdeploymentlogs
        persistentVolumeClaim:
          claimName: amlogspvc
```

Note: The PVC and claim name is defined in the previous section in Persistent Volumes and Claims.

Horizontal Pod Autoscaler

The Horizontal Pod Autoscaler automatically scales the number of pods in a replication controller, deployment, replica set or stateful set based on observed CPU utilization (or, with custom metrics support, on some other application-provided metrics).

The sample **am-hpa.yaml** below defines an Horizontal Autoscaler that will scale to four Pod replicas when CPU utilization hits 80% on current Pods:

am-hpa.yaml

YAML

```
apiVersion: "autoscaling/v2beta1"
kind: "HorizontalPodAutoscaler"
metadata:
  name: "am-k8s-hpa"
  namespace: "default"
  labels:
    app: "am-k8s"
spec:
  scaleTargetRef:
    kind: "Deployment"
    name: "am-k8s"
    apiVersion: "apps/v1"
  minReplicas: 1
  maxReplicas: 4
  metrics:
  - type: "Resource"
    resource:
      name: "cpu"
      targetAverageUtilization: 80
```

Use `kubectl get hpa` to view current status of the scaler:

Command
`$ kubectl get hpa`
Response

```
NAME          REFERENCE             TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
am-k8s-hpa    Deployment/am-k8s     8%/80%    1         4         1          14d
```

NGINX Ingress Controller

Ingress exposes HTTP and HTTPS routes from outside the cluster to services within the cluster. Traffic routing is controlled by rules defined on the Ingress resource.

For TLS Connection Ends in AccessMatrix and Audit Microservice, enable ssl-pass-through in the ingress controller. All SSL/TLS requests will end in AccessMatrix and Audit Microservice instances/Pods.

Preparation

Helm is the package manager for Kubernetes. We use helm to install Ingress in the Kubernetes cluster.

Install Helm

Helm provides a script that will automatically download and install the latest Helm release on your local machine.

Command
`$ curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/master/scripts/get-helm-3` `$ chmod 700 get_helm.sh` `$ ./get_helm.sh`

Create namespace and add Helm Repo

Command
`# Create a namespace for your ingress resources` `kubectl create namespace ingress-basic` `# Add the ingress-nginx repository` `helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx`
Response

Command
`"ingress-nginx" has been added to your repositories`

Install Ingress and Create Routes

As mentioned in the previous chapter, we will configure the TLS connection ends AccessMatrix, which means it just passes through the Ingress Controller.

TLS certificate and corresponding **server.xml** must be updated correctly in AccessMatrix and Audit Microservice package. Refer to:

- TLS Connection Ends in AccessMatrix section to update **server.xml**.
- **am-k8s-ssl.yaml** for sample of Deployment YAML.

Install Ingress Controller with SSL-Passthrough Enabled

1. For SSL-Passthrough, `enable-ssl-passthrough` must be set when creating the Ingress controller:

Command
`helm install nginx-ingress ingress-nginx/ingress-nginx` `  --namespace ingress-basic` `  --set controller.replicaCount=1` `  --set controller.nodeSelector."beta\.kubernetes\.io/os"=linux` `  --set defaultBackend.nodeSelector."beta\.kubernetes\.io/os"=linux` `  --set controller.admissionWebhooks.patch.nodeSelector."beta\.kubernetes\.io/os"=linux` `  --set "controller.extraArgs.enable-ssl-passthrough="`
Response
`REVISION: 1` `TEST SUITE: None` `NOTES:` `The ingress-nginx controller has been installed.` `It may take a few minutes for the LoadBalancer IP to be available.` `You can watch the status by running 'kubectl --namespace ingress-basic get services -o wide -w nginx-ingress-ingress-nginx-controller'.` `An example Ingress that makes use of the controller:` `  apiVersion: networking.k8s.io/v1beta1` `  kind: Ingress` `  metadata:` `    annotations:` `      kubernetes.io/ingress.class: nginx`

Command

```
    name: example
    namespace: foo
  spec:
    rules:
      - host: www.example.com
        http:
          paths:
            - backend:
              serviceName: exampleService
              servicePort: 80
            path: /
    # This section is only required if TLS is to be enabled for the Ingress
    tls:
        - hosts:
          - www.example.com
        secretName: example-tls

If TLS is enabled for the Ingress, a Secret containing the certificate and key
must also be provided:

  apiVersion: v1
  kind: Secret
  metadata:
    name: example-tls
    namespace: foo
  data:
    tls.crt: <base64 encoded cert>
    tls.key: <base64 encoded key>
  type: kubernetes.io/tls
```

2. Upon successful installation, you will be able to see the external IP of your Ingress:

Command

```
$ kubectl get service --namespace ingress-basic
```

Response

```
NAME                                                TYPE           CLUSTER-IP
   EXTERNAL-IP      PORT(S)                      AGE
nginx-ingress-ingress-nginx-controller              LoadBalancer   10.0.173.51
  20.198.200.134   80:30443/TCP,443:31285/TCP  9d
nginx-ingress-ingress-nginx-controller-admission    ClusterIP      10.0.234.0
   <none>           443/TCP                      9d
```

3. Add a DNS record from your domain name (e.g. *am-k8s.i-sprint.com* and *ams-k8s.i-sprint.com*) of the SSL certificate to <External-IP>.

Create Ingress Routes

1. Create **am-ingress-ssl.yaml**; this will create a ClusterIP to expose port 8443 internally and an ingress route for *am-k8s.i-sprint.com*:

am-ingress-ssl.yaml

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: am-service
spec:
  ports:
- name: am-port-ssl
    port:443
    protocol: TCP
    targetPort:8443
  selector:
    app: am-k8s
  type: ClusterIP
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: am-ingress-ssl
  annotations:
    kubernetes.io/ingress.class: nginx
    nginx.ingress.kubernetes.io/backend-protocol:"HTTPS"
    nginx.ingress.kubernetes.io/force-ssl-redirect:"true"
    nginx.ingress.kubernetes.io/ssl-passthrough:"true"

spec:
  rules:
- host: am-k8s.i-sprint.com
    http:
      paths:
- path:/
        pathType: Prefix
        backend:
          service:
            name: am-service
            port:
              name: am-port-ssl
```

2. Once the yaml files have been applied, we can use the following command to verify the ingress route:

Command

```
kubectl get ingress
```

Response

```
NAME              CLASS    HOSTS                 ADDRESS          PORTS   AGE
am-ingress-ssl    <none>   am-k8s.i-sprint.com   20.198.200.134   80      8d
```

Verify Kubernetes Deployment

- Use `kubectl get pods` to verify all the Pods are running.
- Visit AccessMatrix Admin Console via your domain name (e.g. *k8s.i-sprint.com*) to verify it is successfully deployed.

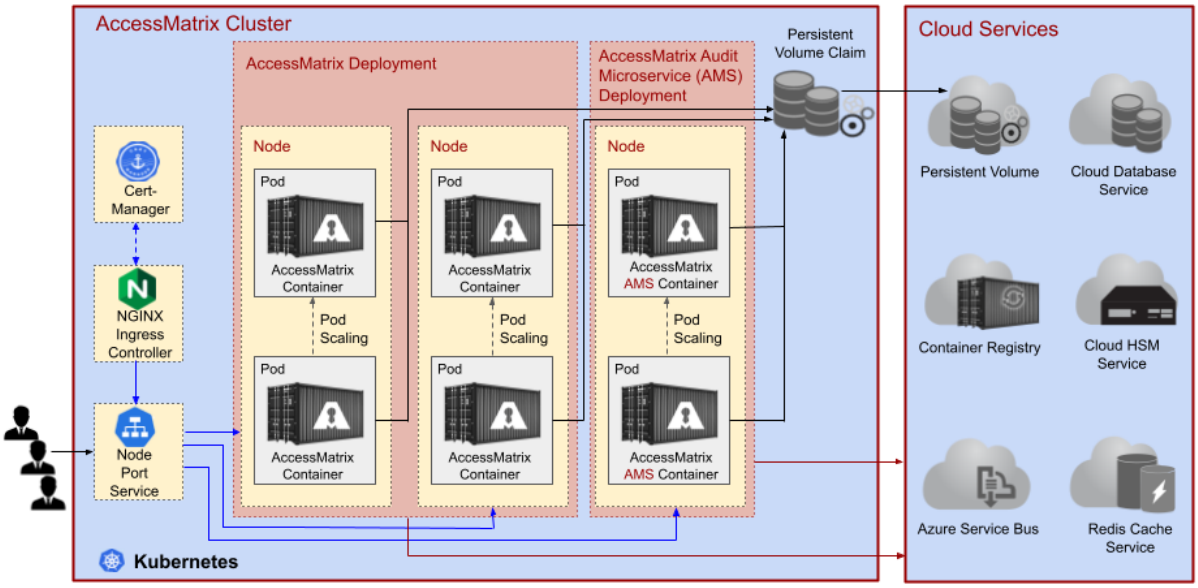
3.1.5. Advanced Deployment with Audit Microservice

Audit Micro Service (AMS) is a web application developed by i-Sprint independent from AccessMatrix. It is used to greatly improve AccessMatrix's performance during authentication by separating out audits as different microservice. When AMS is deployed, instead of directly writing into the database, AccessMatrix will send audit write requests to AMS via REST API. It is also possible to integrate AMS with the Message Queue (MQ) service to further improve the load distribution on multiple AMS instances.

As an advanced deployment option, AMS can be deployed with AccessMatrix in the same Kubernetes Cluster.

Below is the architecture diagram of a typical AccessMatrix advanced deployment with Audit Microservice in a Kubernetes cluster.

AccessMatrix Kubernetes with Audit Microservice Deployment Architecture



AccessMatrix Audit Microservice provides REST API to handle audit requests from AccessMatrix.

mTLS Connection between AccessMatrix and Audit Microservice

Mutual authentication is required for AccessMatrix instance connecting to Audit Microservice.

1. Ensure Audit Microservice's CA certificate has been trusted by AccessMatrix's truststore.

2. Prepare and place the client keystore of Audit Microservice in AccessMatrix directory. e.g. *<ACMATRIX_ROOT>/conf/testagent.keystore*.
3. Go to the directory *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes*.
4. Open the **amsystem.properties** file.
5. Make sure am.microservice.audit.ssl.client.keystore and am.microservice.audit.ssl.client.keystore.passphrase are set accordingly.

Prepare Audit Microservice Docker Image

An AccessMatrix Audit Microservice Docker image must be built locally before being pushed to the registry.

Configuration for Database

To allow for a pod to connect to initialize the database, the credentials for the database has to be configured. The password must be encrypted using the EncryptorUtil with an environment variable as the user salt.

1. Open and edit *<ACMATRIX_AMS_ROOT>/tomcat/webapps/am-audit/WEB-INF/classes/ams.properties* with the relevant information.

Properties (.properties files)

```
#activemq.url=tcp://<activemq-clusterIP>:<activemq-port>
#hibernate.dialect=org.hibernate.dialect.DerbyDialect
hibernate.connection.datasource=java:/comp/env/jdbc/DSam-audit
#hibernate.connection.driver_class=org.apache.derby.jdbc.ClientDriv
er
#hibernate.connection.url=jdbc:derby://localhost:1527/amdb;create=f
alse
#hibernate.connection.username=user1
#hibernate.connection.password=passwordYJBGH
```

2. Open and edit *<ACMATRIX_AMS_ROOT>/tomcat/conf/Catalina/localhost/am-audit.xml* with the following uncommented:

XML

```
<Resource name="jdbc/DSam-audit" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"
   factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
   initialSize="10" maxWaitMillis="10000"
   maxTotal="80" maxIdle="20" minIdle="10"
```

```
   validationQuery="select 1" username="user1"

password="Aes256Encryptor::env=DBCRED:HhfyR5EFe7a6sfpo9sxa8FoRV04uX
Iam7HCouuhwpwnqkbsHAaVlcAp/8SiLNrJ8;"
   driverClassName="com.microsoft.sqlserver.jdbc.SQLServerDriver"
   url="jdbc:sqlserver://<database-host>:<database-
port>;database=<database-
name>;encrypt=true;trustServerCertificate=false;hostNameInCertifica
te=*.database.windows.net;loginTimeout=30;"
/>
```

Repack AccessMatrix Audit Microservice Base Package

So you have prepared database service and ActiveMQ, you also have updated related configurations in AccessMatrix, now you need to repack it back to its respective name (**AccessMatrix-${AM_VERSION}-tomcat-only-am-audit.tar.gz**), for example, **AccessMatrix-5.7.5.7535-GA-tomcat-only-am-audit.tar.gz**.

Pre-image Build Checklist

Before building the AccessMatrix Audit Microservice Docker Container Image, ensure the following are done:

Checklist	Description
Database service	Database service is ready, and its connection information is configured in **am-audit.xml**.
TLS Connection Ends in AMS	If the TLS connection were to end in AccessMatrix AMS instance (Ingress controller passthrough), the certificate and port are configured in **server.xml**. The configuration should be identical to AccessMatrix instance. Refer to TLS Connection Ends in AccessMatrix for more information.
AMS Base Package	AMS base package that was previously unpacked must be repacked back to its original name.
Dependent libraries	Any additional libraries or drivers such as JDBC, or any external drivers, must be placed in the lib folder in the same directory as the Dockerfile.

Build AccessMatrix Audit Microservice Docker Image

A sample AccessMatrix Dockerfile looks like below. Arrange the files as follows,

1. Place the Dockerfile in any working directory that you choose. Change the environment variable AM_VERSION to the respective AccessMatrix version.
2. Create a subdirectory lib to place all dependent libraries, e.g. JDBC drivers.
3. Create a subdirectory repo to place AccessMatrix base package **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**.

Dockerfile

Code

```
FROM adoptopenjdk/openjdk11:slim

ENV AM_VERSION "5.7.5.7535-GA"

RUN mkdir -p /opt/am/${AM_VERSION}
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only-am-
audit.tar.gz /opt/am/${AM_VERSION}
RUN groupadd -g 2020 am \
    && useradd -r -g am -u 2021 -s /bin/bash am \
    && chown -R am:am /opt/am/${AM_VERSION} \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/db-init/*.sh \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:am /opt/am/am

COPY ["lib", "/opt/am/am/tomcat/lib"]

EXPOSE 8080 8443 8161 61616
USER am
CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
```

Note: The user and group are created with fixed uid and gid to access control and avoid deployment using root user.

4. Use `docker build` to build image:

Command

```
$ docker build . -f Dockerfile -t ams:5.7.5
```

Response

```
Sending build context to Docker daemon 157.8MB
Step 1/9 : FROM adoptopenjdk/openjdk11:slim
  ---> a73e537ebcd4
Step 2/9 : ENV AM_VERSION "5.7.5.7535-GA"
  ---> Using cache
  ---> 6c209940bd1a
Step 3/9 : RUN mkdir -p /opt/am/${AM_VERSION}
  ---> Using cache
  ---> 9141c3c1a1f0
Step 4/9 : ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only-am-audit.tar.gz
```

Command
/opt/am/${AM_VERSION}   ---> Using cache   ---> ba4e81749f03 Step 5/9 : RUN groupadd am && useradd -r -g am -s /bin/bash am && chown -R am:am /opt/am/${AM_VERSION}&& chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh && ln -s /opt/am/${AM_VERSION} /opt/am/am && chown -R am:am /opt/am/am   ---> Using cache   ---> 212ce4fbbbf1 Step 6/9 : COPY ["lib", "/opt/am/am/tomcat/lib"]   ---> d2a8441678c6 Step 7/9 : EXPOSE 8080 8443 8161 61616   ---> Running in cfba31838a66 Removing intermediate container cfba31838a66   ---> 642affec76dd Step 8/9 : USER am   ---> Running in ec4e05c7b34d Removing intermediate container ec4e05c7b34d   ---> 43fcb8a5dc33 Step 9/9 : CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]   ---> Running in aab7293a547b Removing intermediate container aab7293a547b   ---> afc1178a5a33 Successfully built afc1178a5a33 Successfully tagged am5:5.7.5

Push AccessMatrix Audit Microservice Docker Image to Registry

1. Use `az acr login` to log in before pushing the image:

Command
`$ az acr login --name accessmatrix`
Response
`Login Succeeded`

2. Use `docker tag` and `docker push` to tag the docker image then push to the registry (e.g. Azure Container Registry):

Command
`$ docker tag ams:5.7.5 accessmatrix.azurecr.io/ams:5.7.5` $ docker image ls
Response
REPOSITORY TAG IMAGE ID CREATED SIZE accessmatrix.azurecr.io/ams 5.7.5 afc1178a5a33 10 minutes ago 758MB

Command

```
ams                                 5.7.5               afc1178a5a33        10 minutes ago
        758MB
```

Command

```
$ docker image push accessmatrix.azurecr.io/ams:5.7.5
```

Response

```
The push refers to repository [accessmatrix.azurecr.io/ams:5.7.5]
247946ea3c85: Pushed
047dc610d4f8: Pushed
f1289005a4b3: Pushed
66ff9452d4c6: Pushed
5d0f1dae109c: Pushed
1b853245b739: Pushed
f801f021a84a: Pushed
789c82f5f442: Pushed
3e2a3ebdb62c: Pushed
e8243c609e31: Pushed
cfb7bbd3b9bc: Pushed
51a0cde65eeb: Pushed
16542a8fc3be: Pushed
6597da2e2e52: Pushed
977183d4e999: Pushed
c8be1b8f4d60: Pushed
5.7.5: digest:
sha256:a977f44de80368bd83511456f8facd2375da70691ac62dd23d49afade326ac2f size:
3672
```

Important Note: Version tagging of images is important as tagging with the "latest" tag can lead to problems in a managed environment. According to Kubernetes docs, labelling all images with the latest makes it difficult to track the current version of the image running and makes it difficult to roll back when errors occur.

Deployment Where TLS Connection Ends in Audit Microservice

Since AccessMatrix (AM) relies on Audit Microservice (AMS) to write audit records, you must deploy Audit Microservice before AccessMatrix.

Below is a sample Deployment YAML for AccessMatrix where the TLS connection ends in Audit Microservice Pods (which means, from Ingress controller to AMS Pods will still be TLS encrypted):

ams-k8s-ssl.yaml

ams-k8s-ssl.yaml

YAML

```yaml
apiVersion: "apps/v1"
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ams-k8s
  namespace: default
  labels:
    app: ams-k8s
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ams-k8s
  template:
    metadata:
      labels:
        app: ams-k8s
    spec:
      containers:
      - name: ams
        image: <container-registry-name>/ams:<version>
        env:
        - name: DBCRED
          valueFrom:
            secretKeyRef:
              key: DBCRED
              name: am-credentials
        readinessProbe:
          exec:
            command:
            - "curl"
            - "-X POST"
            - "http://localhost:8080/am-
audit/rest/audit/?version"
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        livenessProbe:
          exec:
            command:
            - "curl"
            - "-X POST"
            - "http://localhost:8080/am-
audit/rest/audit/?version"
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        ports:
        - containerPort: 8080
          protocol: TCP
```

ams-k8s-ssl.yaml

```
        volumeMounts:
        - mountPath: /opt/am/am/tomcat/logs
          name: amlogspvc
        securityContext:
          privileged: true
        resources:
          requests:
            cpu: 100m
      volumes:
      - name: amlogspvc
        persistentVolumeClaim:
          claimName: amlogspvc
```

Note: The PVC and claim name is defined in the previous section in Persistent Volumes and Claims.

Horizontal Pod Autoscaler for Audit Microservice

The sample **ams-hpa.yaml** below defines an Horizontal Autoscaler that will scale to four Pod replicas when CPU utilization hits 80% on current Pods:

ams-hpa.yaml

YAML

```yaml
apiVersion: "autoscaling/v2beta1"
kind: "HorizontalPodAutoscaler"
metadata:
  name: "ams-k8s-hpa"
  namespace: "default"
  labels:
    app: "ams-k8s"
spec:
  scaleTargetRef:
    kind: "Deployment"
    name: "ams-k8s"
    apiVersion: "apps/v1"
  minReplicas: 1
  maxReplicas: 4
  metrics:
  - type: "Resource"
    resource:
      name: "cpu"
      targetAverageUtilization: 80
```

Use `kubectl get hpa` to view current status of the scaler:

Command

```
$ kubectl get hpa
```

Response

```
NAME          REFERENCE              TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
ams-k8s-hpa   Deployment/ams-k8s     8%/80%    1         4         1          14d
```

Ingress Route for Audit Microservice

1. Create **ams-ingress-ssl.yaml**; this will create a ClusterIP to expose port 8443 internally and an ingress route for *ams-k8s.i-sprint.com*:

ams-ingress-ssl.yaml

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: ams-service
spec:
  ports:
- name: ams-port-ssl
    port:443
    protocol: TCP
    targetPort:8443
  selector:
    app: ams-k8s
  type: ClusterIP
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ams-ingress-ssl
  annotations:
    kubernetes.io/ingress.class: nginx
    nginx.ingress.kubernetes.io/backend-protocol:"HTTPS"
    nginx.ingress.kubernetes.io/force-ssl-redirect:"true"
    nginx.ingress.kubernetes.io/ssl-passthrough:"true"

spec:
  rules:
- host: ams-k8s.i-sprint.com
    http:
      paths:
- path:/
        pathType: Prefix
        backend:
          service:
            name: ams-service
            port:
              name: ams-port-ssl
```

2. Once the yaml files have been applied, we can use the following command to verify the ingress route:

Command

```
kubectl get ingress
```

Response

```
NAME              CLASS    HOSTS                   ADDRESS          PORTS   AGE
am-ingress-ssl    <none>   am-k8s.i-sprint.com     20.198.200.134   80      8d
ams-ingress-ssl   <none>   ams-k8s.i-sprint.com    20.198.200.134   80      8d
```

Configuration Changes in AccessMatrix

Once Audit Microservice has been deployed, we need to change some configurations in AccessMatrix, mainly for setting the Audit Microservice URL. You are required to have a copy of the **amsystem.properties** file kept in AccessMatrix Docker image or use the `kubectl cp` command to copy from existing AccessMatrix Pods directly.

Update amsystem.properties

Some properties from **amsystem.properties** file have to be set accordingly:

Portion of amsystem.properties

Properties (.properties files)

```
...
am.microservice.audit.url = https://ams-k8s.i-sprint.com/am-audit/rest
am.microservice.audit.ssl.client.keystore =<path-to-AMS-client-keystore>
am.microservice.audit.ssl.client.keystore.passphrase =<passphrase-of-AMS-cllient-keystore>
...
```

Create Secret from amsystem.properties

Command

```
kubectl create secret generic amsystems --from-file=<path-to-amsystem.properties>
```

Command
Response
`secret/amsystems created`

Mount amsystem.properties from Secret and Re-Deploy AccessMatrix

am-k8s-ssl.yaml

YAML

```yaml
apiVersion: "apps/v1"
kind: "Deployment"
metadata:
  name: "am-k8s"
  namespace: "default"
  labels:
    app: "am-k8s"
spec:
  replicas: 2
  selector:
    matchLabels:
      app: "am-k8s"
  template:
    metadata:
      labels:
        app: "am-k8s"
    spec:
      containers:
      - name: am5
        image: <container-registry-server>/am5:<version>
        env:
        - name: DBCRED
          valueFrom:
            secretKeyRef:
              key: DBCRED
              name: am-credentials
        readinessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        livenessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
```

am-k8s-ssl.yaml

```
          failureThreshold: 3
          initialDelaySeconds: 70
        ports:
        - containerPort: 8080
          protocol: TCP
        - containerPort: 8443
          protocol: TCP
        volumeMounts:
        - mountPath: /opt/am/am/tomcat/work/reports
          name: amreportspvc
        - mountPath: /opt/am/am/tomcat/logs
          name: amlogspvc
        - mountPath: /opt/am/am/tomcat/webapps/am5/WEB-
INF/classes/amsystem.properties
          name: amsystems
        securityContext:
          privileged: true
        resources:
          requests:
            cpu: 100m
      volumes:
      - name: amreportspvc
        persistentVolumeClaim:
          claimName: amreportspvc
      - name: amdeploymentlogs
        persistentVolumeClaim:
          claimName: amlogspvc
      - name: amsystems
        secret:
          secretName: amsystems
```

Use the `kubectl apply` command to apply the changes and re-deploy AccessMatrix:

Command
`$ kubectl apply -f am-k8s-ssl.yaml`
Response
`deployment.apps/am-k8s configured`

3.2. Kubernetes Deployment with Azure and On-Premise HSM

The pages below provide detailed instructions on the deploying AccessMatrix, connected to an on-premise HSM, on the Kubernetes platform with Microsoft Azure. This deployment enables AccessMatrix to connect to an external HSM for key management purposes (e.g. creating wrapping keys).

- Prepare Kubernetes Deployment with HSM Prerequisites
- Configure HSM
- Create AccessMatrix Docker Image
- Deploy Kubernetes

Notes:

- The instructions in this guide describe the deployment of the Luna Network HSM in particular.
- The process of setting up the HSM is beyond the scope of this guide.

3.2.1. Prepare Kubernetes Deployment with HSM Prerequisites

This section lists all of the prerequisites that must be met before proceeding with the configuration and deployment of the AccessMatrix Server and HSM on Kubernetes.

The following table details the Kubernetes and AccessMatrix-related configurations that should first be made:

Step	Reference
Enable Access to Cloud Service Providers	Prepare Kubernetes Deployment Prerequisites (Azure)
Prepare Container/Image Registry	
Create Kubernetes Cluster	
Unpack AccessMatrix Base Package	Prepare Database/Cache Services and Configure AccessMatrix (Azure)
Prepare Database	
Prepare Redis Cache Service	
Configure Azure Service Bus	
Prepare Domain Name and TLS Connection	

Additionally, please ensure that the following HSM-related prerequisites are also met:

> **Note**: Instructions for the steps listed below are beyond the scope of this guide.

- The full version (i.e. non-minimal version) of the Luna HSM client software is installed on the host machine
- *$PATH* should contain the folder in which the full Luna client software is installed. In this document, the folder containing the binaries is */usr/safenet/lunaclient/bin/*.
- The environment contains the variable `ChrystokiConfigurationPath`, and *$ChrystokiConfigurationPath/Chrystoki.conf* exists; In this document, `ChrystokiConfigurationPath=/etc`.
- The HSM is set up, initialized, provisioned, and ready for deployment - refer to the Luna HSM Product Documentation for more information.

3.2.2. Configure HSM

This section covers the deployment of AccessMatrix in Kubernetes connecting an on-premise Luna Network HSM. The process of setting up the HSM is beyond the scope of this guide.

Ensure that you are logged in to an account with the necessary privileges (i.e. `sudo` privileges).

Deployment Plan

In this document, the following folder structure is adopted; ensure that these folders and all subfolders along the paths are created:

Folder	Description
$HOME/luna-docker	This folder in the host machine contains the files that go into the Docker image for the Luna client. The name “luna-docker” is arbitrary. This is also the working directory for this setup; `$PWD = $HOME/luna-docker`.
$PWD/config	This folder contains configuration necessary for establishing a connection with the HSM.
$PWD/config/certs/client	This folder contains the certificates generated by the Luna client.
$PWD/config/certs/server	This folder contains the certificates held by the HSM.
$PWD/config/stc/token/001	This folder is necessary only if you are configuring the Secure Trusted Channel (STC).
$PWD/lib	This folder contains libraries to be used by Tomcat in the AM Docker image - for instance, database connector .jar files.

Create NTLS Connection to HSM

The steps below open an NTLS connection between the HSM and the bearer of the client certificates - in this case, the Docker container hosting a Luna client. The following IP addresses are used in this guide:

- **Internal IP address** - 192.168.1.207
- **Public IP address** - 100.100.100.100

1. Create a client certificate and its corresponding private key for the certificate bearer (e.g "clientDocker")

```
# vtl createCert -n clientDocker
```

The created material can be found in */usr/safenet/lunaclient/cert/client/*.

2. Copy the created certificates to config, then copy the certificate **clientDocker.pem** to the HSM:

```
cp /usr/safenet/lunaclient/cert/client/*.pem ./config/certs/client/
scp ./config/certs/client/clientDocker.pem admin@192.168.1.207:
```

> **Note**: Ensure that the trailing colon (:) at the end of of the command is not omitted. Omitting this will cause the copy operation to silently fail.

3. Fetch the appliance server certificate from the HSM:

```
scp admin@192.168.1.207:server.pem ./config/certs/server/100.100.100.100server.pem
```

> **Note**: Prefixing the public IP address to the certificate name is optional.

4. Register the appliance server certificate with the client:

```
vtl addServer -c ./config/certs/server/server.pem -n 100.100.100.100
```

> **Note**: Using the public IP address as the server name is optional but recommended for clarity.

5. Connect to the HSM via SSH (you should log in directly to LunaSH):

```
ssh admin@192.168.1.207
```

6. Register the full Luna client with the appliance, and assign a partition (e.g partition1) to the client:

```
lunash:> client register -client clientDocker -hostname clientDocker
lunash:> client assignpartition -client clientDocker -partition partition1
```

7. Once done, disable IP address checking (if it has not already been done) and exit the HSM:

```
lunash:> ntls ipcheck disable
lunash:> exit
```

8. On the client workstation, run the `lunacm` utility, then set the active slot (e.g slot 0) to the registered partition:

```
lunacm
lunacm:> slot set -slot 0
```

9. Verify that the partition is successfully registered and configured:

Command
`lunacm:>slot list`
Response

Command
Slot Id -> 0 Label -> partition1 Serial Number -> 1280822085360 Model -> LunaSA 7.4.0 Firmware Version -> 7.4.0 Configuration -> Luna User Partition With SO (PED) Key Export With Cloning Mode Slot Description -> Net Token Slot FM HW Status -> FM Current Slot Id: 0 Command Result : No Error

10. Copy *$ChrystokiConfigurationPath/Chrystoki.conf* to *config*:

    ```
    cp $ChrystokiConfigurationPath/Chrystoki.conf ./config/
    ```

11. Create **pkcs11.config** in the *config* folder with the following contents:

    ```
    cat config/pkcs11.config
    library=/usr/local/luna/libs/64/libCryptoki2.so
    slot=0
    name=LunaSA
    ```

Once the steps above have been completed, verify that the folder contents have been properly configured. To do so, enter the following commands and ensure that the responses match the sample responses:

- *$PWD* folder:

Command
`pwd`
Response
/home/hsmuser/luna-docker

- *$PWD/config* folder:

Command
`ls -1 ./config`
Response
certs Chrystoki.conf pkcs11.config

Command
`stc`

- *$PWD/config/certs/client/* folder:

Command
`ls -1 ./config/certs/client/`
Response
`clientDockerKey.pem` `clientDocker.pem`

- *$PWD/config/certs/server/* folder:

Command
`ls -1 ./config/certs/server/`
Response
`100.100.100.100Cert.pem` `CAFile.pem`

Configure Luna Minimal Client

The following steps detail the creation of the folder structure to be set up in the Docker container:

1. If you are configuring Secure Trusted Channel (STC), create the empty file *$HOME/luna-docker/config/stc/token/001/token.db*; otherwise, you may skip this step.
2. Copy the Luna Minimal Client tarball (e.g **LunaClient-Minimal-v7.4.0-226.x86_64.tar**) to *$HOME/luna-docker/*.
3. Untar the Luna Minimal Client tarball:
   `tar xf LunaClient-Minimal-v7.4.0-226.x86_64.tar`
4. Edit the following sections of */config/Chrystoki.conf* to use the Luna Minimal Client (paths are determined by how they are mounted during deployment, as explained below):

Code

```
Chrystoki2 = {
   LibUNIX = /usr/local/luna/libs/64/libCryptoki2.so;
   LibUNIX64 = /usr/local/luna/libs/64/libCryptoki2.so;
}

Misc = {
```

```
   ProtectedAuthenticationPathFlagStatus = 1;
   LoginAllowedOnFMEnabledHSMs = 1;
}

LunaSA Client = {
   ClientPrivKeyFile =
/usr/local/luna/config/certs/client/clientDockerKey.pem;
   ClientCertFile =
/usr/local/luna/config/certs/client/clientDocker.pem;
   ServerCAFile =
/usr/local/luna/config/certs/server/CAFile.pem;
   ServerName00 = 100.100.100.100;
}

Secure Trusted Channel = {
   ClientTokenLib =
/usr/local/luna/libs/64/libSoftToken.so;
}
```

> **Note**: The initial value of ServerName00 depends on the IP address used when creating the NTLS connection to the HSM (as detailed in the previous section); update it to use the public IP address of the HSM.

5. Navigate to *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes* and open **amsystem.properties**.
6. Search for the Luna section and uncomment and edit the following setting:

Properties (.properties files)

```
# == Luna ==
am.crypto.luna.lunakeyprov.userpin=password
```

Replace the following values as appropriate:

a. lunakeyprov - The name of the Luna key provider.

b. password - The password to access the HSM.

3.2.3. Create AccessMatrix Docker Image

At this stage, all of the services that will be used by AccessMatrix are ready, and related AccessMatrix base package configurations have been updated. You can now proceed to build the AccessMatrix Docker image. An AccessMatrix Docker image must be built locally before being pushed to the registry.

Before doing so, ensure that the following steps have been completed:

1. Enable AccessMatrix Container Mode
2. Repack AccessMatrix Base Package

Build AccessMatrix Docker Image

The sample Dockerfile below is used to construct the image. Arrange the files as follows:

1. Place the Dockerfile in any working directory that you choose.

Dockerfile

YAML

```
FROM adoptopenjdk/openjdk11:slim
ARG MIN_CLIENT=placeholder, AM_VERSION=placeholder
COPY ${MIN_CLIENT}.tar /tmp
RUN mkdir -p /usr/local/luna
RUN tar xvf /tmp/${MIN_CLIENT}.tar --strip 1 -C /usr/local/luna

ENV ChrystokiConfigurationPath=/usr/local/luna/config
ENV CATALINA_OPTS="-
Djava.library.path=/usr/local/luna/jsp/64:/usr/local/luna/libs/64"
COPY lunacm /usr/local/bin
COPY vtl /usr/local/bin

LABEL version="${AM_VERSION}"
RUN mkdir -p /opt/am/${AM_VERSION}
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}
RUN groupadd -g 2020 am \
    && useradd -r -g am -u 2021 -s /bin/bash am \
    && chown -R am:am /opt/am/${AM_VERSION} \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/lib/*.jar \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/db-init/*.sh \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:am /opt/am/am
    && cp /usr/local/luna/jsp/LunaProvider.jar
/opt/am/am/tomcat/webapps/am5/WEB-INF/lib/

COPY ["lib", "/opt/am/am/tomcat/lib"]
```

Dockerfile

```
EXPOSE 8080 8443
USER am
CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
```

2. Create a subdirectory, *lib*, to place all dependent libraries, e.g. JDBC drivers.
3. Create a subdirectory, *repo*, to place the AccessMatrix base package **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**.

Once the necessary modifications to the build file have been made:

1. Create a Docker build using this build file and the build arguments in the following command:

   `docker build . --build-arg MIN_CLIENT=LunaClient-Minimal-v7.4.0-226.x86_64 --build-arg AM_VERSION=5.7.5.7535-GA -t am5luna`
2. Verify that the image is created:

Command

```
docker images
```

Response

```
REPOSITORY   TAG      IMAGE ID       CREATED         SIZE
am5luna      latest   d2b8147b36ac   25 hours ago    1.08GB
```

Push AccessMatrix Docker Image to Registry

1. Log in to the Azure Container Registry (ACR):

   `az acr login -n peregistry`
2. Tag the image with the ACR (e.g `am5:5.7.5-luna`):

   `docker tag am5luna peregistry.azurecr.io/am5:5.7.5-luna`
3. Push the image to Docker Hub:

   `docker push peregistry.azurecr.io/am5:5.7.5-luna`

3.2.4. Deploy Kubernetes

At this stage, AccessMatrix Docker image has been configured properly and pushed to the container registry.

Before deploying AccessMatrix, you may need to create and deploy various Kubernetes objects. Kubernetes objects are persistent entities in the Kubernetes system. Kubernetes uses these entities to represent the state of your cluster. For detailed instructions on this, refer to the following sections:

1. Persistent Volumes and Claims
2. Temporary Report Directory
3. Log Files

When ready, proceed to the next section.

AccessMatrix Deployment

Before deploying to the Kubernetes cluster, create the required Kubernetes secrets with the following commands:

- `kubectl create secret generic server-auth --from-file=./config/certs/server/100.100.100.100Cert.pem --from-file=./config/certs/server/CAFile.pem`
- `kubectl create secret generic client-auth --from-file=./config/certs/client/clientDocker.pem --from-file=./config/certs/client/clientDockerKey.pem`
- `kubectl create secret generic configs --from-file=./config/Chrystoki.conf --from-file=./config/pkcs11.config`
- `kubectl create secret generic amluna-creds --from-file=DBCRED`
- `kubectl create secret generic amsystem-properties --from-file=amsystem.properties`

Verify that the secrets exist with the command `kubectl get secrets`; compare the results with the sample response provided below:

Command
`kubectl get secrets`

Command

Response

```
NAME                          TYPE     DATA   AGE
amluna-creds                  Opaque   1      5d22h
amsystem-properties           Opaque   1      174m
client-auth                   Opaque   2      6d4h
configs                       Opaque   1      6d4h
server-auth                   Opaque   2      6d4h
```

Once this is done, create a deployment YAML named **amluna-k8s-ssl.yaml** and apply it using `kubectl apply -f amluna-k8s-ssl.yaml`. The following is a sample deployment YAML file:

amluna-k8s-ssl.yaml

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: amluna
  namespace: default
  labels:
    app: amluna
spec:
  replicas: 1
  selector:
    matchLabels:
      app: amluna
  template:
    metadata:
      labels:
        app: amluna
    spec:
      containers:
      - name: am5
        image: peregistry.azurecr.io/am5:5.7.5-luna
        imagePullPolicy: Always
        ports:
        - containerPort: 8080
          protocol: TCP
        env:
        - name: DBCRED
          valueFrom:
            secretKeyRef:
              key: DBCRED
              name: amluna-creds
        readinessProbe:
          httpGet:
            path: /am5/
            port: 8080
```

amluna-k8s-ssl.yaml

```
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        livenessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        lifecycle:
          preStop:
            exec:
              command: ["/bin/sh", "-c", "echo Stopping
server; sh /opt/am/am/tomcat/bin/catalina.sh stop"]
        volumeMounts:
        - name: lunalogspvc
          mountPath: /opt/am/am/tomcat/logs
        - name: lunareportspvc
          mountPath: /opt/am/am/tomcat/work/reports
        - name: amsystemprops
          mountPath: /opt/am/am/tomcat/webapps/am5/WEB-
INF/classes/amsystem.properties
          subPath: amsystem.properties
        - name: lunaconf
          mountPath: /usr/local/luna/config
          readOnly: true
        - name: servercerts
          mountPath: /usr/local/luna/config/certs/server
          readOnly: true
        - name: clientcerts
          mountPath: /usr/local/luna/config/certs/client
          readOnly: true
        securityContext:
          privileged: true
        resources:
          requests:
            cpu: 100m
      volumes:
      - name: lunalogspvc
        persistentVolumeClaim:
          claimName: lunalogspvc
      - name: lunareportspvc
        persistentVolumeClaim:
          claimName: lunareportspvc
      - name: lunaconf
        secret:
          secretName: configs
      - name: servercerts
        secret:
          secretName: server-auth
```

amluna-k8s-ssl.yaml

```
      - name: clientcerts
        secret:
          secretName: client-auth
      - name: amsystemprops
        secret:
          secretName: amsystem-properties
```

Verify HSM Connection

To test if the AM instance can connect to the HSM, you should perform the following test, depending on which key provider you intend to use:

PKCS#11 Key Provider	1. In the Admin Console, navigate to **Administration Dashboard > Security Policy > Key Management > Key Manager > Key Provider**. 2. Click the **Create** button and select "PKCS11 Key Provider" from the **Implementation** drop-down list. 3. Supply the following values to these attributes: a. **Key Provider Id** and **Key Provider Name** - Enter a suitable key provider name (e.g. "pkcskeyprov"). b. **PKCS11 configuration file** - Enter the path to the **pkcs11.config** file (e.g. "/usr/local/luna/config/pkcs11.config"). c. **HSM User PIN** - Enter the HSM password (e.g "password"). 4. Verify that the process to select an encryption key template is not interrupted by error messages (any connection error throws an error message, and no options appear in the encryption key template list). To do so: a. Navigate to **Administration Dashboard > Security Policy > Key Management > Encryption Key** and click **Create**. b. Select "pkcskeyprov" from the **Key Provider** drop-down list depending on which provider you configured, then select an **Encryption key template** option. c. Click **Next**. The **Create Encryption Key** page is displayed. d. Configure the encryption key with the relevant attributes and click **Save**. The encryption key is created. e. At the top of the **View Encryption Key** page, click the **Test KeyInfo** and **View KCV** buttons and ensure they successfully return values. f. Repeat this for all keys as needed.
Gemalto Luna Key Provider	1. In the Admin Console, navigate to **Administration Dashboard > Security Policy > Key Management > Key Manager > Key Provider**. 2. Click the **Create** button and select "Gemalto Luna Key Provider" from the **Implementation** drop-down list.

	3. Supply the following values to these attributes: a. **Key Provider Id** and **Key Provider Name** - Enter the Luna key provider name configured in the Configure Luna Minimal Client section (e.g. "lunakeyprov"). b. **Slot Label** - Enter the partition configured in the Create NTLS Connection to HSM section (e.g. "partition1"). c. **Wrapping Key Label** - Enter the key label configured in the HSM (e.g. "AES01"). 4. Verify that the process to select an encryption key template is not interrupted by error messages (any connection error throws an error message, and no options appear in the encryption key template list). To do so: a. Navigate to **Administration Dashboard > Security Policy > Key Management > Encryption Key** and click **Create**. b. Select "lunakeyprov" from the **Key Provider** drop-down list depending on which provider you configured, then select an **Encryption key template** option. c. Click **Next**. The **Create Encryption Key** page is displayed. d. Configure the encryption key with the relevant attributes and click **Save**. The encryption key is created. e. At the top of the **View Encryption Key** page, click the **Test KeyInfo** and **View KCV** buttons and ensure they successfully return values. f. Repeat this for all keys as needed.

3.3. Kubernetes Deployment with AWS

The following pages provide detailed instructions on how to deploy AccessMatrix on the Kubernetes platform with Amazon Web Services (AWS):

- Prepare Kubernetes Deployment Prerequisites
- Prepare Database/Cache Services and Configure AccessMatrix
- Create and Deploy Kubernetes Objects
- Advanced Deployment with Audit Microservice

3.3.1. Prepare Database/Cache Services and Configure AccessMatrix

AccessMatrix Docker image is built based on **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz** package found in the CD image. For example, **AccessMatrix-5.7.5.7535-GA-tomcat-only.tar.gz**.

> **Important Note**: In some places, you must update AccessMatrix configuration files like **amsystem.properties**, **am5.xml**, **am5-ehcache.xml** and **server.xml**, which means you need to unzip the **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**, then rezip to build Docker image.

The next sections explain preparing services and containers used directly by AccessMatrix, requiring some configurations to be done in some AccessMatrix config files.

1. Database Service to store any persistence data of AccessMatrix.
2. Redis Cache Service to store any transient data that are frequently updated, e.g. E2EE session id.
3. Amazon MQ Service as message queue server for Ehcache replication.
4. SSH Local Port Forwarding to access cloud services in a private subnet.
5. TLS configuration, to configure TLS connection ends in AccessMatrix instance.

Unpack AccessMatrix Base Package

To deploy AccessMatrix in a container, you need to take the package **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz** found in AccessMatrix CD Image, for example, **AccessMatrix-5.7.5.7535-GA-tomcat-only.tar.gz.** As explained above, you need to change some configurations in AccessMatrix. Hence the file must be unpacked/unzipped first.

Database

There are various options for the database. You can choose to host a database server on Virtual Machine (VM) or choose a database service offered by Cloud Service Provider (CSP). The database server must be accessible to both AccessMatrix containers in the Kubernetes cluster and the administrator who would execute AccessMatrix database scripts. This document assumes that the Aurora MySQL database service is being used.

Depending on the JRE version used, an appropriate MySQL JDBC driver version must be used for connections to Aurora MySQL database.

For the following examples, we will be using MySQL JDBC version 5.1.40 compiled with JRE version 1.5.

1. Use `aws rds create-db-subnet-group` to create subnet group for Aurora MySQL database service:

Parameters

```
# Set subnet group name and store in an environment variable for ease of
reference.
$ DatabaseSubnetGroupName=amdb-subnet-group

# Set subnet group description and store in an environment variable for ease of
reference.
$ DatabaseSubnetGroupDesc="AccessMatrix Database Subnet Group"

# Set subnet name in ap-southeast-1a and store in an environment variable for
ease of reference.
$ DatabaseSubnetName1A=AMDB-Private-Sub-1A

# Set subnet name in ap-southeast-1b and store in an environment variable for
ease of reference.
$ DatabaseSubnetName1B=AMDB-Private-Sub-1B

# Get subnet ids by name and store in an environment variable for ease of
reference.
$ DatabaseServiceSubnetIDs=$(aws ec2 describe-subnets --filter
Name=tag:Name,Values=$DatabaseSubnetName1A,$DatabaseSubnetName1B
    --query Subnets[*].SubnetId --output text)
```

Command

```
$ aws rds create-db-subnet-group
    --db-subnet-group-name $DatabaseSubnetGroupName
    --db-subnet-group-description "$DatabaseSubnetGroupDesc"
    --subnet-ids $DatabaseServiceSubnetIDs
```

Response


```
{
  "DBSubnetGroup": {
    "DBSubnetGroupName": "amdb-subnet-group",
    "DBSubnetGroupDescription": "AccessMatrix Database Subnet Group",
    "VpcId": "vpc-013315608e0b403a3",
    "SubnetGroupStatus": "Complete",
    "Subnets": [
      {
        "SubnetIdentifier": "subnet-0894f6d1aee4ab57e",
        "SubnetAvailabilityZone": {
          "Name": "ap-southeast-1a"
        },
        "SubnetOutpost": {},
        "SubnetStatus": "Active"
      },
      {
        "SubnetIdentifier": "subnet-084873d974ecdda70",
```

Parameters

```
        "SubnetAvailabilityZone": {
          "Name": "ap-southeast-1b"
        },
        "SubnetOutpost": {},
        "SubnetStatus": "Active"
      }
    ],
    "DBSubnetGroupArn": "arn:aws:rds:ap-southeast-1:021215294315:subgrp:amdb-
subnet-group"
  }
}
```

2. Use `aws ec2 create-security-group` to create security group for Aurora MySQL database service:

Parameters

```
# Set cloud service VPC name and store in an environment variable for ease of
reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Set security group name and store in an environment variable for ease of
reference.
$ DatabaseSecurityGroupName=amdb-security-group

# Set security group description and store in an environment variable for ease
of reference.
$ DatabaseSecurityGroupDesc="AccessMatrix Database Security Group"

# Get cloud service VPC ID by name and store in an environment variable for
ease of reference.
$ CloudServiceVPCID=$(aws ec2 describe-vpcs --filter
Name=tag:Name,Values=$CloudServiceVPCName --query Vpcs[0].VpcId --output text)
```

Command

```
$ aws ec2 create-security-group
    --group-name $DatabaseSecurityGroupName
    --description "$DatabaseSecurityGroupDesc"
    --vpc-id $CloudServiceVPCID
```

Response

```
{
  "GroupId": "sg-023c4faeb3504aa84"
}
```

3. Use `aws ec2 authorize-security-group-ingress` to allow AccessMatrix to access the database:

Parameters

```
# Set security group name and store in an environment variable for ease of
reference.
$ DatabaseSecurityGroupName=amdb-security-group

# Get security group id by name and store in an environment variable for ease
of reference.
$ DatabaseSecurityGroupID=$(aws ec2 describe-security-groups --filter
Name=group-name,Values=$DatabaseSecurityGroupName
    --query SecurityGroups[0].GroupId --output text)

# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster

# Get AccessMatrix EKS VPC ID by name and store in an environment variable for
ease of reference.
$ AccessMatrixVPCID=$(aws eks describe-cluster --name $EKSClusterName --query
cluster.resourcesVpcConfig.vpcId --output text)

# Get AccessMatrix CIDR by VPC ID and store in an environment variable for ease
of reference.
$ AccessMatrixCIDR=$(aws ec2 describe-vpcs --vpc-id $AccessMatrixVPCID --query
Vpcs[0].CidrBlock --output text)
```

Command

```
$ aws ec2 authorize-security-group-ingress
    --group-id $DatabaseSecurityGroupID
    --protocol tcp
    --port 3306
    --cidr $AccessMatrixCIDR
```

4. Use `aws rds create-db-cluster` to create Amazon Aurora database cluster:

Parameters

```
# Set database cluster identifier and store in an environment variable for ease
of reference.
$ DatabaseClusterID=amdb-cluster

# Set security group name and store in an environment variable for ease of
reference.
$ DatabaseSecurityGroupName=amdb-security-group

# Set subnet group name and store in an environment variable for ease of
reference.
$ DatabaseSubnetGroupName=amdb-subnet-group

# Get database security group id by name and store in an environment variable
for ease of reference.
$ DatabaseSecurityGroupID=$(aws ec2 describe-security-groups --filter
Name=group-name,Values=$DatabaseSecurityGroupName
    --query SecurityGroups[0].GroupId --output text)
```

Parameters

Command

```
$ aws rds create-db-cluster
    --db-cluster-identifier $DatabaseClusterID
    --database-name amdb
    --engine aurora-mysql
    --engine-version 5.7.12
    --master-username user1
    --master-user-password Password.1234
    --vpc-security-group-ids $DatabaseSecurityGroupID
    --db-subnet-group-name $DatabaseSubnetGroupName
    --port 3306
```

Response


```
{
  "DBCluster": {
    "AllocatedStorage": 1,
    "AvailabilityZones": [
      "ap-southeast-1c",
      "ap-southeast-1b",
      "ap-southeast-1a"
    ],
    "BackupRetentionPeriod": 1,
    "DatabaseName": "amdb",
    "DBClusterIdentifier": "amdb-cluster",
    "DBClusterParameterGroup": "default.aurora-mysql5.7",
    "DBSubnetGroup": "amdb-subnet-group",
    "Status": "creating",
    "Endpoint": "amdb-cluster.cluster-cfgu2tppti3v.ap-southeast-
1.rds.amazonaws.com",
    "ReaderEndpoint": "amdb-cluster.cluster-ro-cfgu2tppti3v.ap-southeast-
1.rds.amazonaws.com",
    "MultiAZ": false,
    "Engine": "aurora-mysql",
    "EngineVersion": "5.7.12",
    "Port": 3306,
    "MasterUsername": "user1",
    "PreferredBackupWindow": "18:36-19:06",
    "PreferredMaintenanceWindow": "fri:14:42-fri:15:12",
    "ReadReplicaIdentifiers": [],
    "DBClusterMembers": [],
    "VpcSecurityGroups": [
      {
        "VpcSecurityGroupId": "sg-023c4faeb3504aa84",
        "Status": "active"
      }
    ],
    "HostedZoneId": "Z2G0U3KFCY8NZ5",
    "StorageEncrypted": false,
    "DbClusterResourceId": "cluster-L454DCGWVUVO6LGGFT3LMDJJA4",
    "DBClusterArn": "arn:aws:rds:ap-southeast-1:021215294315:cluster:amdb-
cluster",
    "AssociatedRoles": [],
    "IAMDatabaseAuthenticationEnabled": false,
```

Parameters

```
    "ClusterCreateTime": "2021-01-15T04:56:52.643000+00:00",
    "EngineMode": "provisioned",
    "DeletionProtection": false,
    "HttpEndpointEnabled": false,
    "CopyTagsToSnapshot": false,
    "CrossAccountClone": false,
    "DomainMemberships": [],
    "TagList": []
  }
}
```

Important Note: It is crucial to use a strong and secure password in production.

5. Use `aws rds create-db-instance` to create database instance:

Parameters

```
# Set database instance identifier and store in an environment variable for
ease of reference.
$ DatabaseInstanceID=amdb-instance

# Set database cluster identifier and store in an environment variable for ease
of reference.
$ DatabaseClusterID=amdb-cluster
```

Command

```
$ aws rds create-db-instance
    --db-instance-identifier $DatabaseInstanceID
    --db-cluster-identifier $DatabaseClusterID
    --no-publicly-accessible
    --engine aurora-mysql
    --db-instance-class db.r5.large
```

Response

```
{
  "DBInstance": {
    "DBInstanceIdentifier": "amdb-instance",
    "DBInstanceClass": "db.r5.large",
    "Engine": "aurora-mysql",
    "DBInstanceStatus": "creating",
    "MasterUsername": "user1",
    "DBName": "amdb",
    "AllocatedStorage": 1,
    "PreferredBackupWindow": "18:36-19:06",
    "BackupRetentionPeriod": 1,
    "DBSecurityGroups": [],
    "VpcSecurityGroups": [
      {
        "VpcSecurityGroupId": "sg-023c4faeb3504aa84",
        "Status": "active"
      }
```

Parameters


```
    ],
    "DBParameterGroups": [
      {
        "DBParameterGroupName": "default.aurora-mysql5.7",
        "ParameterApplyStatus": "in-sync"
      }
    ],
    "DBSubnetGroup": {
      "DBSubnetGroupName": "amdb-subnetgroup",
      "DBSubnetGroupDescription": "AccessMatrix Database Subnet Group",
      "VpcId": "vpc-013315608e0b403a3",
      "SubnetGroupStatus": "Complete",
      "Subnets": [
        {
          "SubnetIdentifier": "subnet-0894f6d1aee4ab57e",
          "SubnetAvailabilityZone": {
            "Name": "ap-southeast-1a"
          },
          "SubnetOutpost": {},
          "SubnetStatus": "Active"
        },
        {
          "SubnetIdentifier": "subnet-084873d974ecdda70",
          "SubnetAvailabilityZone": {
            "Name": "ap-southeast-1b"
          },
          "SubnetOutpost": {},
          "SubnetStatus": "Active"
        }
      ]
    },
    "PreferredMaintenanceWindow": "fri:21:30-fri:22:00",
    "PendingModifiedValues": {},
    "MultiAZ": false,
    "EngineVersion": "5.7.12",
    "AutoMinorVersionUpgrade": true,
    "ReadReplicaDBInstanceIdentifiers": [],
    "LicenseModel": "general-public-license",
    "OptionGroupMemberships": [
      {
        "OptionGroupName": "default:aurora-mysql-5-7",
        "Status": "in-sync"
      }
    ],
    "PubliclyAccessible": false,
    "StorageType": "aurora",
    "DbInstancePort": 0,
    "DBClusterIdentifier": "amdb-cluster",
    "StorageEncrypted": false,
    "DbiResourceId": "db-3MYEALNQD2YOWTK7BHBITWSH5E",
    "CACertificateIdentifier": "rds-ca-2019",
    "DomainMemberships": [],
    "CopyTagsToSnapshot": false,
    "MonitoringInterval": 0,
    "PromotionTier": 1,
    "DBInstanceArn": "arn:aws:rds:ap-southeast-1:021215294315:db:amdb-instance",
```

Parameters
"IAMDatabaseAuthenticationEnabled": false, "PerformanceInsightsEnabled": false, "DeletionProtection": false, "AssociatedRoles": [], "TagList": [], "CustomerOwnedIpEnabled": false } }

6. Use `aws rds describe-db-instances` to show the JDBC endpoint url:

Parameters
`# Set database instance identifier and store in an environment variable for ease of reference.` `$ DatabaseInstanceID=amdb-instance`
Command
`$ aws rds describe-db-instances --query 'DBInstances[?DBInstanceIdentifier==`'$DatabaseInstanceID'`].Endpoint'`
Response
[{ "Address": "amdb-instance.cfgu2tppti3v.ap-southeast-1.rds.amazonaws.com", "Port": 3306, "HostedZoneId": "Z2G0U3KFCY8NZ5" }]

Configure Database Connection in AccessMatrix

After you set up the database schema properly and get the connection string, update the database configuration in AccessMatrix to point to the database.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\conf\Catalina\localhost*.
2. Open the **am5.xml** file.
3. Comment out the following:

XML

```
    <Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"
        initialSize="10" maxTotal="80" maxIdle="20"
minIdle="10" maxWaitMillis="10000"
        validationQuery="SELECT COUNT(*) FROM am_vars
```

```
WHERE va_name IS NULL" testOnBorrow="true"
        username="user1" password="password"
driverClassName="org.apache.derby.jdbc.EmbeddedDriver"
               url="jdbc:derby:amdb"
   />
```

4. Uncomment the following and provide values for the properties as appropriate:

XML

```
<!--
    <Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"
        initialSize="10" maxWaitMillis="10000" maxTotal="80"
maxIdle="20" minIdle="10" validationQuery="select 1"
        username="changeToDbUserId" password="changeToDBPassword"
driverClassName="com.mysql.jdbc.Driver"

url="jdbc:mysql://10.1.11.85:3306/amdb?useSSL=false&amp;useUnicode=
true&amp;characterEncoding=utf8"
   />
-->
```

5. Add the EncryptorUtil factory property to allow the decryption of a password that is encrypted using the EncryptorUtil:

XML

```
    <Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"

factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
        initialSize="10" maxWaitMillis="10000" maxTotal="80"
maxIdle="20" minIdle="10" validationQuery="select 1"
        username="changeToDbUserId" password="changeToDBPassword"
driverClassName="com.mysql.jdbc.Driver"

url="jdbc:mysql://10.1.11.85:3306/amdb?useSSL=false&amp;useUnicode=
true&amp;characterEncoding=utf8"
   />
```

Notes:

- Replace username, password and URL with your credentials and database server URL, i.e. jdbc:mysql://amdb-instance.cfgu2tppti3v.ap-southeast-1.rds.amazonaws.com/amdb?useSSL=false&useUnicode=true&characterEncoding=utf8.
- The JDBC parameters useSSL=false is added to the URL. If you want to enable SSL, follow the instruction Using SSL with a MySQL instance.
- The database administrator is required to use the EncryptorUtil to encrypt the database password, using the "Environment Variable as user salt" option. The encrypted password will look like the following:

AesEncryptor::env=DBCRED:HtjCuzS34QxySZMslav3umHgVFA5G5j5HCXoc7KQsUDDwyjcf/+65+756F4YJBGH,
where env=DBCRED is the environment variable containing a user salt.

- For more information on using the EncryptorUtil, refer to the *AccessMatrix Common Deployment Guide - Encryptor Utility*.

Enable AccessMatrix Container Mode

AccessMatrix itself needs a special configuration to set to run in a container. To set AccessMatrix to run in a container,

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Uncomment the following am.container.enabled configuration and set it to "true":

```
Properties (.properties files)

# Enable this property if am run in container like
docker
#am.container.enabled=true
```

Make Configuration for Database Initialization

The database's credentials have to be configured to allow a pod to connect to initialize the database. The password must be encrypted using the EncryptorUtil with an environment variable as the user salt.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\db-init*.
2. Open the **amsystem.properties** file.
3. Comment out the following:

```
Properties (.properties files)

#am.domain.store.datasource=java:/comp/env/jdbc/DSamdb
```

4. Uncomment the following and input the relevant information, for example:

```
Properties (.properties files)

# JDBC
am.domain.store.driver_class=com.microsoft.sqlserver.jdbc.SQLServer
Driver
am.domain.store.url=jdbc:sqlserver://amdb-instance.cfgu2tppti3v.ap-
```

```
southeast-
1.rds.amazonaws.com/amdb?useSSL=false&amp;useUnicode=true&amp;chara
cterEncoding=utf8
am.domain.store.username=user1
am.domain.store.password=AesEncryptor::env=DBCRED:HtjCuzS34QxySZMsl
av3umHgVFA5G5j5HCXoc7KQsUDDwyjcf/+65+756F4YJBGH
```

5. Ensure container mode for AccessMatrix is enabled, as detailed in the previous section.

Redis Cache Server

For AccessMatrix, Redis Cache is treated as an in-memory database, used to store transient data that are frequently updated, for example, E2EE session, SMS-OTP tokens, SAML Response Id (to prevent replay attack), Common Login Page (CLP) login sessions (for AccessMatrix Applications, such as UAS TAP, USO SSF), and OIDC Relying Party lookup module nonces (to prevent request forgery or replay). For more information about Redis, refer to the official Redis documentation.

Relevant certificates for SSL connectivity should be prepared. Mutual authentication depends on the capability of the Redis server in their respective clouds.

AWS as Redis Cache Server

Single Instance

On AWS, Access Matrix will use Amazon ElastiCache for Redis as the Redis cache server.

The following creates a cache.t3.micro (500MB) ElastiCache for Redis instance:

1. Use `aws elasticache create-cache-subnet-group` to create subnet group for ElastiCache Redis service:

Parameters
`# Set subnet group name and store in an environment variable for ease of reference.` `$ RedisSubnetGroupName=amredis-subnet-group` `# Set subnet group description and store in an environment variable for ease of reference.` `$ RedisSubnetGroupDesc="AccessMatrix Redis Subnet Group"` `# Set subnet name in ap-southeast-1a and store in an environment variable for ease of reference.` `$ RedisSubnetName1A=AMREDIS-Private-Sub-1A`

Parameters

```
# Set subnet name in ap-southeast-1b and store in an environment variable for
ease of reference.
$ RedisSubnetName1B=AMREDIS-Private-Sub-1B

# Get subnet ids by name and store in an environment variable for ease of
reference.
$ RedisServiceSubnetIDs=$(aws ec2 describe-subnets --filter
Name=tag:Name,Values=$RedisSubnetName1A,$RedisSubnetName1B
    --query Subnets[*].SubnetId --output text)
```

Command

```
$ aws elasticache create-cache-subnet-group
    --cache-subnet-group-name $RedisSubnetGroupName
    --cache-subnet-group-description "$RedisSubnetGroupDesc"
    --subnet-ids $RedisServiceSubnetIDs
```

Response


```
{
  "CacheSubnetGroup": {
    "CacheSubnetGroupName": "amredis-subnet-group",
    "CacheSubnetGroupDescription": "AccessMatrix Redis Subnet Group",
    "VpcId": "vpc-013315608e0b403a3",
    "Subnets": [
      {
          "SubnetIdentifier": "subnet-0040150eafa7e6af2",
          "SubnetAvailabilityZone": {
            "Name": "ap-southeast-1a"
        }
      },
      {
        "SubnetIdentifier": "subnet-06adaef03545d85af",
        "SubnetAvailabilityZone": {
          "Name": "ap-southeast-1b"
      }
    }
    ],
    "ARN": "arn:aws:elasticache:ap-southeast-1:021215294315:subnetgroup:amredis-
subnet-group"
  }
}
```


2. Use `aws ec2 create-security-group` to create security group for ElastiCache redis service:

Parameters

```
# Set cloud service VPC name and store in an environment variable for ease of
reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Set security group name and store in an environment variable for ease of
reference.
$ RedisSecurityGroupName=amredis-security-group
```

Parameters

```
# Set security group description and store in an environment variable for ease
of reference.
$ RedisSecurityGroupDesc="AccessMatrix Redis Security Group"

# Get cloud service VPC ID by name and store in an environment variable for
ease of reference.
$ CloudServiceVPCID=$(aws ec2 describe-vpcs --filter
Name=tag:Name,Values=$CloudServiceVPCName
    --query Vpcs[0].VpcId --output text)
```

Command

```
$ aws ec2 create-security-group
    --group-name $RedisSecurityGroupName
    --description "$RedisSecurityGroupDesc"
    --vpc-id $CloudServiceVPCID
```

Response

```
{
  "GroupId": "sg-07e9906fb82b57b42"
}
```

3. Use `aws ec2 authorize-security-group-ingress` to allow AccessMatrix to access the Redis service:

Parameters

```
# Set security group name and store in an environment variable for ease of
reference.
$ RedisSecurityGroupName=amredis-security-group

# Get security group id by name and store in an environment variable for ease
of reference.
$ RedisSecurityGroupID=$(aws ec2 describe-security-groups --filter Name=group-
name,Values=$RedisSecurityGroupName
    --query SecurityGroups[0].GroupId --output text)

# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster

# Get AccessMatrix EKS VPC ID by name and store in an environment variable for
ease of reference.
$ AccessMatrixVPCID=$(aws eks describe-cluster --name $EKSClusterName --query
cluster.resourcesVpcConfig.vpcId --output text)

# Get AccessMatrix CIDR by VPC ID and store in an environment variable for ease
of reference.
$ AccessMatrixCIDR=$(aws ec2 describe-vpcs --vpc-id $AccessMatrixVPCID --query
Vpcs[0].CidrBlock --output text)
```

Command

Parameters

```
$ aws ec2 authorize-security-group-ingress
    --group-id $RedisSecurityGroupID
    --protocol tcp
    --port 6379
    --cidr $AccessMatrixCIDR
```

4. Use `aws elasticache create-replication-group` to create ElastiCache cluster:

Parameters

```
# Set replication group identifier and store in an environment variable for
ease of reference.
$ RedisReplicationGroupID=am-redis

# Set replication group description and store in an environment variable for
ease of reference.
$ RedisReplicationGroupDesc="AccessMatrix Redis Replication Group"

# Set subnet group name and store in an environment variable for ease of
reference.
$ RedisSubnetGroupName=amredis-subnet-group

# Set security group name and store in an environment variable for ease of
reference.
$ RedisSecurityGroupName=amredis-security-group

# Get security group id by name and store in an environment variable for ease
of reference.
$ RedisSecurityGroupID=$(aws ec2 describe-security-groups --filter Name=group-
name,Values=$RedisSecurityGroupName
    --query SecurityGroups[0].GroupId --output text)
```

Command

```
$ aws elasticache create-replication-group
    --replication-group-id $RedisReplicationGroupID
    --replication-group-description "$RedisReplicationGroupDesc"
    --cache-node-type cache.t3.micro
    --engine redis --multi-az-enabled
    --engine-version 3.2.6
    --num-cache-clusters 2
    --transit-encryption-enabled
    --auth-token This-is-the-auth-token
    --cache-parameter-group-name default.redis3.2
    --cache-subnet-group-name $RedisSubnetGroupName
    --security-group-ids $RedisSecurityGroupID
```

Response

```
{
  "ReplicationGroup": {
    "ReplicationGroupId": "am-redis",
    "Description": "AccessMatrix Redis Replication Group",
    "GlobalReplicationGroupInfo": {},
```

Parameters


```
    "Status": "creating",
    "PendingModifiedValues": {},
    "MemberClusters": [
      "am-redis-001"
    ],
    "AutomaticFailover": "disabled",
    "MultiAZ": "disabled",
    "SnapshotRetentionLimit": 0,
    "SnapshotWindow": "21:00-22:00",
    "ClusterEnabled": false,
    "CacheNodeType": "cache.t3.micro",
    "TransitEncryptionEnabled": true,
    "AtRestEncryptionEnabled": false,
    "ARN": "arn:aws:elasticache:ap-southeast-1:021215294315:replicationgroup:am-
redis"
  }
}
```


Important Note:

- It is crucial to use a strong and secure auth-token (password) in production.
- It may take 10-15 minutes to provision a Redis Cache.

5. To check the status of the Redis cache, enter the following:

Parameters

```
# Set replication group identifier and store in an environment variable for
ease of reference.
$ RedisReplicationGroupID=am-redis
```

Command

```
$ aws elasticache describe-replication-groups --replication-group-id
$RedisReplicationGroupID
```

Response


```
{
  "ReplicationGroups": [
    {
      "ReplicationGroupId": "am-redis",
      "Description": "AccessMatrix Redis Replication Group",
      "GlobalReplicationGroupInfo": {},
      "Status": "available",
      "PendingModifiedValues": {},
      "MemberClusters": [
        "am-redis-001"
      ],
      "NodeGroups": [
        {
          "NodeGroupId": "0001",
```

Parameters


```
          "Status": "available",
          "PrimaryEndpoint": {
            "Address": "master.am-redis.ixmzf1.apse1.cache.amazonaws.com",
            "Port": 6379
        },
        "ReaderEndpoint": {
          "Address": "replica.am-redis.ixmzf1.apse1.cache.amazonaws.com",
          "Port": 6379
        },
        "NodeGroupMembers": [
          {
            "CacheClusterId": "am-redis-001",
            "CacheNodeId": "0001",
            "ReadEndpoint": {
              "Address": "am-redis-001.am-
redis.ixmzf1.apse1.cache.amazonaws.com",
              "Port": 6379
            },
            "PreferredAvailabilityZone": "ap-southeast-1b",
            "CurrentRole": "primary"
            }
          ]
        }
      ],
      "AutomaticFailover": "disabled",
      "MultiAZ": "disabled",
      "SnapshotRetentionLimit": 0,
      "SnapshotWindow": "21:00-22:00",
      "ClusterEnabled": false,
      "CacheNodeType": "cache.t3.micro",
      "AuthTokenEnabled": true,
      "AuthTokenLastModifiedDate": "2021-01-15T10:13:00.381000+00:00",
      "TransitEncryptionEnabled": true,
      "AtRestEncryptionEnabled": false,
      "ARN": "arn:aws:elasticache:ap-southeast-
1:021215294315:replicationgroup:am-redis"
    }
  ]
}
```


The information required by AccessMatrix is listed below:

Key	Description
PrimaryEndpoint Address	Host name of the Redis Server, e.g. master.am-redis.ixmzf1.apse1.cache.amazonaws.com
PrimaryEndpoint Port	Port of Redis with TLS connection

After you set up the Redis service properly and get the connection information (hostname and port), update the Redis configuration in AccessMatrix to point to the Redis service.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Configure the following items according to the information above, for example:
    a. host=master.am-redis.ixmzf1.apse1.cache.amazonaws.com
    b. port=6379

Properties (.properties files)

```
# == Cache Provider Configuration ==
#choose redis or ehcache, if not set, default is
ehcache
am.cache.vendor=redis
#set the below to true if cluster is enabled
#am.redis.cluster.enabled=false
#once cluster is enabled, host and port accepts
multiple values(comma separated) when necessary
am.redis.host=master.am-
redis.ixmzf1.apse1.cache.amazonaws.com
am.redis.port=6379
```

Clustered Instances

AWS Cache supports Redis (Cluster Mode Enabled), where data is automatically distributed across multiple nodes. ElastiCache automatically shards the data across multiple Redis Nodes. This enables high availability through failovers and fault tolerance.

To use AWS Redis, during step 4 of the previous section, configure the replication group:

4. Use `aws elasticache create-replication-group` to create ElastiCache cluster:

Parameters

```
# Set replication group identifier and store in an environment variable for ease of
reference.
$ RedisReplicationGroupID=am-redis

# Set replication group description and store in an environment variable for ease of
reference.
$ RedisReplicationGroupDesc="AccessMatrix Redis Replication Group"

# Set subnet group name and store in an environment variable for ease of reference.
$ RedisSubnetGroupName=amredis-subnet-group

# Set security group name and store in an environment variable for ease of reference.
$ RedisSecurityGroupName=amredis-security-group
```

Parameters

```
# Get security group id by name and store in an environment variable for ease of
reference.
$ RedisSecurityGroupID=$(aws ec2 describe-security-groups --filter Name=group-
name,Values=$RedisSecurityGroupName
    --query SecurityGroups[0].GroupId --output text)
```

Command

```
$aws elasticache create-replication-group \
    --replication-group-id $RedisReplicationGroupID\
    --replication-group-description "$RedisReplicationGroupDesc"\
    --cache-node-type cache.t3.micro \
    --engine redis --multi-az-enabled \
    --engine-version 7.0 \
    --num-node-groups 2 \
    --transit-encryption-enabled \
    --auth-token This-is-the-auth-token \
    --cache-parameter-group-name default.redis7.0.cluster.on \
    --cache-subnet-group-name $RedisSubnetGroupName\
    --security-group-ids $RedisSecurityGroupID \
    --node-group-configuration \
    "ReplicaCount=1,Slots=0-8999,PrimaryAvailabilityZone'='ap-southeast-
1a',ReplicaAvailabilityZones='ap-southeast-1b'" \
    "ReplicaCount=2,Slots=9000-16383,PrimaryAvailabilityZone='ap-southeast-
1b',ReplicaAvailabilityZones='ap-southeast-1a','ap-southeast-1b'"
```

Response


```
{
  "ReplicationGroup": {
     "ReplicationGroupId": "new-group",
     "Description": "Sharded replication group",
     "GlobalReplicationGroupInfo": {},
     "Status": "creating",
     "PendingModifiedValues": {},
     "MemberClusters": [
         "am-redis-0001-001",
         "am-redis-0001-002",
         "am-redis-0002-001",
         "am-redis-0002-002",
         "am-redis-0002-003"
      ],
     "AutomaticFailover": "enabled",
     "MultiAZ": "enabled",
     "SnapshotRetentionLimit": 0,
     "SnapshotWindow": "15:30-16:30",
     "ClusterEnabled": true,
     "CacheNodeType": "cache.t3.micro",
     "TransitEncryptionEnabled": true,
     "AtRestEncryptionEnabled": false,
```

Parameters

```
    "ARN": "arn:aws:elasticache:ap-southeast-1:021215294315:replicationgroup:am-
redis",
    "LogDeliveryConfigurations": [],
    "ReplicationGroupCreateTime": "2024-04-02T06:47:13.601000+00:00",
    "DataTiering": "disabled",
    "AutoMinorVersionUpgrade": true
   }
}
```

Important Note:

- It is crucial to use a strong and secure auth-token (password) in production.
- It may take 10-15 minutes to provision a Redis Cache.

5. To check the status of the Redis cache, enter the following:

Parameters

```
# Set replication group identifier and store in an environment variable for ease of
reference.
$ RedisReplicationGroupID=am-redis
```

Command

```
$ aws elasticache describe-replication-groups --replication-group-id
$RedisReplicationGroupID
```

Response

```
{
  "ReplicationGroups": [
    {
      "ReplicationGroupId": "am-redis",
      "Description": "Sharded replication group",
      "GlobalReplicationGroupInfo": {},
      "Status": "available",
      "PendingModifiedValues": {},
      "MemberClusters": [
        "am-redis-0001-001",
        "am-redis-0001-002",
        "am-redis-0002-001",
        "am-redis-0002-002",
        "am-redis-0002-003"
      ],
      "NodeGroups": [
        {
          "NodeGroupId": "0001",
          "Status": "available",
          "Slots": "0-8999",
          "NodeGroupMembers": [
            {
```

Parameters


```
              "CacheClusterId": "am-redis-0001-001",
              "CacheNodeId": "0001",
              "PreferredAvailabilityZone": "ap-southeast-1a"
            },
            {
              "CacheClusterId": "am-redis-0001-002",
              "CacheNodeId": "0001",
              "PreferredAvailabilityZone": "ap-southeast-1b"
            }
          ]
        },
        {
          "NodeGroupId": "0002",
          "Status": "available",
          "Slots": "9000-16383",
          "NodeGroupMembers": [
            {
              "CacheClusterId": "am-redis-0002-001",
              "CacheNodeId": "0001",
              "PreferredAvailabilityZone": "ap-southeast-1b"
            },
            {
              "CacheClusterId": "am-redis-0002-002",
              "CacheNodeId": "0001",
              "PreferredAvailabilityZone": "ap-southeast-1a"
            },
            {
              "CacheClusterId": "am-redis-0002-003",
              "CacheNodeId": "0001",
              "PreferredAvailabilityZone": "ap-southeast-1b"
            }
          ]
        }
      ],
      "AutomaticFailover": "enabled",
      "MultiAZ": "enabled",
      "ConfigurationEndpoint": {
        "Address": "clustercfg.am-redis.ixmzf1.apse1.cache.amazonaws.com",
        "Port": 6379
      },
      "SnapshotRetentionLimit": 0,
      "SnapshotWindow": "17:30-18:30",
      "ClusterEnabled": true,
      "CacheNodeType": "cache.t3.micro",
      "AuthTokenEnabled": false,
      "TransitEncryptionEnabled": true,
      "AtRestEncryptionEnabled": false,
      "ARN": "arn:aws:elasticache:ap-southeast-1:021215294315:replicationgroup:am-
redis",
      "LogDeliveryConfigurations": [],
      "ReplicationGroupCreateTime": "2024-04-02T07:58:22.071000+00:00",
      "DataTiering": "disabled",
      "AutoMinorVersionUpgrade": true
    }
  ]
}
```

The information required by AccessMatrix is listed below:

Key	Description
`ConfigurationEndPoint`	The DNS record that is valid for the lifetime of the cluster. Elasticache will ensure that this endpoint will always point to a target node.
Address	Endpoint name of the Redis Server, e.g. clustercfg.am-redis.ixmzf1.apse1.cache.amazonaws.com
Port	Port of Redis with TLS connection

After you have set up the Redis service and obtained the connection information (cluster URL and port), update the Redis configuration in AccessMatrix to point to the Redis service.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Configure the following items according to the information abovee. For example:
    - Set am.redis.cluster.enabled=true
    - host=clustercfg.am-redis.ixmzf1.apse1.cache.amazonaws.com
    - port=6379

Properties (.properties files)

```
# == Cache Provider Configuration ==
#choose redis or ehcache, if not set, default is
ehcache
am.cache.vendor=redis
#set the below to true if cluster is enabled
am.redis.cluster.enabled=true
#once cluster is enabled, host and port accepts
multiple values(comma separated) when necessary
am.redis.host=clustercfg.am-
redis.ixmzf1.apse1.cache.amazonaws.com
am.redis.port=6379
```

Amazon ElastiCache Redis TLS Enabled

amsystem.properties
Properties (.properties files)

amsystem.properties

```
# == Cache Provider Configuration ==
#choose redis or ehcache, if not set, default is ehcache
am.cache.vendor=redis
#set the below to true if cluster is enabled
#am.redis.cluster.enabled=false
#once cluster is enabled, host and port accepts multiple
values(comma separated) when necessary
am.redis.host=master.am-
redis.ixmzf1.apse1.cache.amazonaws.com
am.redis.port=6379
#optional, don't set if no password required
am.redis.password=This-is-the-auth-token
#optional, default is false
am.redis.ssl.enabled=true
```

Use Encryptor Util to Encrypt Redis Password

The property am.redis.password in the **amsystem.properties** file accepts the Redis password as an encrypted text. The encrypted text must be generated from the EncryptorUtil.

For more information on using EncryptorUtil, refer to *AccessMatrix Common Deployment Guide - Encryptor Utility*.

ElastiCache Pub/Sub

AWS Elasticache supports the Redis Publish/Subscribe paradigm, where messages are not sent to specific receivers (subscribers), but instead sent to channels (publishers), without requiring any knowledge of their subscribers.

> **Note:** It is recommended that you use Redis Pub/Sub on AccessMatrix cloud deployments instead of Java Messaging Service (JMS) services like ActiveMQ when deploying with Redis.

Configure Redis Pub/Sub in AccessMatrix

After you set up Redis service properly and get the connection information (hostname and port) in the previous section:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Configure the following items:

Properties (.properties files)

```
# == Ehcache ==
# EhCache Cache Replicator Factory: RMI(default) or JMS
or Redis
# If JMS is used, related properties(starting with
am.ehcache.jms) must be enabled.
#am.ehcache.replicator.factory=RMI
# If Redis is used
am.ehcache.replicator.factory = Redis
am.ehcache.replicator.redis.channel = ampubsub
am.ehcache.replicator.redis.reconnect.secs = 10
```

Amazon MQ Server

Some common objects in AccessMatrix server such as policies, configurations and login modules that are often accessed but are not constantly updated are stored in Ehcache (in-memory cache inside AccessMatrix server) for performance and efficiency reasons. AccessMatrix server uses Apache ActiveMQ for Ehcache replication using Java Message Service (JMS).

> **Important Note**: AccessMatrix currently supports ActiveMQ Classic 5 release.

Amazon MQ as Message Broker Service for Active MQ

On AWS, AccessMatrix will use Amazon MQ as a message broker service for Active MQ.

The following creates an mq.t3.micro (1 GB) broker instance:

1. Use `aws ec2 create-security-group` to create a security group for Amazon MQ service:

Parameters

```
# Set security group name and store in an environment variable for ease of
reference.
$ AMQSecurityGroupName=amq-security-group

# Set security group description and store in an environment variable for ease
of reference.
$ AMQSecurityGroupDesc="AccessMatrix Amazon MQ Security Group."

# Set cloud service VPC name and store in an environment variable for ease of
reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Get cloud service VPC ID by name and store in an environment variable for
ease of reference.
```

Parameters

```
$ CloudServiceVPCID=$(aws ec2 describe-vpcs --filter
Name=tag:Name,Values=$CloudServiceVPCName
    --query Vpcs[0].VpcId --output text)
```

Command

```
$ aws ec2 create-security-group
    --group-name $AMQSecurityGroupName
    --description "$AMQSecurityGroupDesc"
    --vpc-id $CloudServiceVPCID
```

Response

```
{
  "GroupId": "sg-00d8448b91ec19f28"
}
```

2. Use `aws ec2 authorize-security-group-ingress` to allow AccessMatrix to access the Amazon MQ OpenWire port:

Parameters

```
# Set security group name and store in an environment variable for ease of
reference.
$ AMQSecurityGroupName=amq-security-group

# Get security group id by name and store in an environment variable for ease
of reference.
$ AMQSecurityGroupID=$(aws ec2 describe-security-groups --filter Name=group-
name,Values=$AMQSecurityGroupName
    --query SecurityGroups[0].GroupId --output text)

# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster

# Get AccessMatrix EKS VPC ID by name and store in an environment variable for
ease of reference.
$ AccessMatrixVPCID=$(aws eks describe-cluster --name $EKSClusterName --query
cluster.resourcesVpcConfig.vpcId --output text)

# Get AccessMatrix CIDR by VPC ID and store in an environment variable for ease
of reference.
$ AccessMatrixCIDR=$(aws ec2 describe-vpcs --vpc-id $AccessMatrixVPCID --query
Vpcs[0].CidrBlock --output text)
```

Command

```
$ aws ec2 authorize-security-group-ingress
    --group-id $AMQSecurityGroupID
    --protocol tcp
    --port 61617
    --cidr $AccessMatrixCIDR
```

3. Use `aws ec2 authorize-security-group-ingress` to allow AccessMatrix to access the Amazon MQ WebConsole port:

Parameters

```
# Set security group name and store in an environment variable for ease of
reference.
$ AMQSecurityGroupName=amq-security-group

# Get security group id by name and store in an environment variable for ease
of reference.
$ AMQSecurityGroupID=$(aws ec2 describe-security-groups --filter Name=group-
name,Values=$AMQSecurityGroupName
    --query SecurityGroups[0].GroupId --output text)

# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster

# Get AccessMatrix EKS VPC ID by name and store in an environment variable for
ease of reference.
$ AccessMatrixVPCID=$(aws eks describe-cluster --name $EKSClusterName
    --query cluster.resourcesVpcConfig.vpcId --output text)

# Get AccessMatrix CIDR by VPC ID and store in an environment variable for ease
of reference.
$ AccessMatrixCIDR=$(aws ec2 describe-vpcs --vpc-id $AccessMatrixVPCID --query
Vpcs[0].CidrBlock --output text)
```

Command

```
$ aws ec2 authorize-security-group-ingress
    --group-id $AMQSecurityGroupID
    --protocol tcp
    --port 8162
    --cidr $AccessMatrixCIDR
```

4. Use `aws mq create-broker` to create Amazon MQ broker:

Parameters

```
# Set broker name and store in an environment variable for ease of reference.
$ AMQBrokerName=am-amazonmq-broker

# Set security group name and store in an environment variable for ease of
reference.
$ AMQSecurityGroupName=amq-security-group

# Get security group id by name and store in an environment variable for ease
of reference.
$ AMQSecurityGroupID=$(aws ec2 describe-security-groups --filter Name=group-
name,Values=$AMQSecurityGroupName
    --query SecurityGroups[0].GroupId --output text)

# Set subnet name in ap-southeast-1a and store in an environment variable for
ease of reference.
```

Parameters

```
$ AMQSubnetName1A=AMQ-Private-Sub-1A

# Set subnet name in ap-southeast-1b and store in an environment variable for
ease of reference.
$ AMQSubnetName1B=AMQ-Private-Sub-1B

# Get subnet ids by name and store in an environment variable for ease of
reference.
$ AMQServiceSubnetIDs=$(aws ec2 describe-subnets --filter
Name=tag:Name,Values=$AMQSubnetName1A,$AMQSubnetName1B
    --query Subnets[*].SubnetId --output text)
```

Command

```
$ aws mq create-broker
    --broker-name $AMQBrokerName
    --deployment-mode ACTIVE_STANDBY_MULTI_AZ
    --engine-type ACTIVEMQ
    --engine-version 5.15.14
    --host-instance-type mq.t3.micro
    --subnet-ids $AMQServiceSubnetIDs
    --security-groups $AMQSecurityGroupID
    --users ConsoleAccess=true,Password=Password.1234,Username=amquser1
    --no-auto-minor-version-upgrade
    --no-publicly-accessible
```

Response

```
{
  "BrokerArn": "arn:aws:mq:ap-southeast-1:021215294315:broker:am-amazonmq-
broker:b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2",
  "BrokerId": "b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2"
}
```

Important Note: It may take 10-15 minutes to provision an Amazon MQ broker.

5. To check the status of the Amazon MQ broker, enter the following:

Parameters

```
# Set broker name and store in an environment variable for ease of reference.
$ AMQBrokerName=am-amazonmq-broker

# Get broker id by name and store in an environment variable for ease of
reference.
$ AMQBrokerId=$(aws mq list-brokers --query
'BrokerSummaries[?BrokerName==`'$AMQBrokerName'`].BrokerId' --output text)
```

Command

```
$ aws mq describe-broker --broker-id $AMQBrokerId --query BrokerInstances
```

Parameters
Response


```
[
    {
      "ConsoleURL": "https://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-
southeast-1.amazonaws.com:8162",
      "Endpoints": [
        "ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-
1.amazonaws.com:61617",
        "amqp+ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-
1.amazonaws.com:5671",
        "stomp+ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-
1.amazonaws.com:61614",
        "mqtt+ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-
1.amazonaws.com:8883",
        "wss://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-
1.amazonaws.com:61619"
      ],
      "IpAddress": "172.50.2.13"
    },
    {
      "ConsoleURL": "https://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-
southeast-1.amazonaws.com:8162",
      "Endpoints": [
        "ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-southeast-
1.amazonaws.com:61617",
        "amqp+ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-southeast-
1.amazonaws.com:5671",
        "stomp+ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-southeast-
1.amazonaws.com:61614",
        "mqtt+ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-southeast-
1.amazonaws.com:8883",
        "wss://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-southeast-
1.amazonaws.com:61619"
      ],
      "IpAddress": "172.50.2.169"
    }
  ]
```


The information required by AccessMatrix is listed below:

Key	Description
OpenWire Endpoints	Failover endpoint URLs of the Amazon Message Queue (MQ) server, example: *ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-1.amazonaws.com:61617* and *ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-southeast-1.amazonaws.com:61617*

Configure Amazon Message Queue (MQ) Connection in AccessMatrix

After setting up the ActiveMQ container correctly and getting the IP information (in the sample above are 172.50.2.13 and 172.50.2.169, on port 8161/61616), you update related configurations in AccessMatrix to point to the ActiveMQ container.

The IP address can be used directly in the configuration below, or you can use a domain name, e.g. *b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-1.amazonaws.com* and *b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-southeast-1.amazonaws.com*.

You need to configure in AM system properties file (**amsystem.properties**) by carrying out the following steps.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Change am.ehcache.replicator.factory to JMS and uncomment related JMS properties:

Properties (.properties files)

```
# Cache Replicator Implementation: RMI,JMS
am.ehcache.replicator.factory=JMS
...
# EhCache JMS Cache Peer Provider
am.ehcache.jms.initialContextFactoryName=com.isprint.am.core.cache.
ehcache.AtiveMQInitialContextFactory
am.ehcache.jms.providerURL=failover:tcp://localhost:61616
am.ehcache.jms.userName=amquser1
am.ehcache.jms.password=Password.1234
am.ehcache.jms.replicationTopicBindingName=amtopic
am.ehcache.jms.getQueueBindingName=amqueue
#am.ehcache.jms.acknowledgementMode
#       : (Optional) The mode used to acknowledge messages after
they are consumed.
#    Configure with one of the following values:
#        - AUTO_ACKNOWLEDGE
#        - DUPS_OK_ACKNOWLEDGE (Default)
#        - SESSION_TRANSACTED
#am.ehcache.jms.acknowledgementMode=AUTO_ACKNOWLEDGE
```

Notes:

- Replace am.ehcache.jms.providerURL=failover:tcp://localhost:61616 with your domain name or IP. For example: failover:(ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-1.amazonaws.com:61617,ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-southeast-1.amazonaws.com:61617).

> - The "Ative" in am.ehcache.jms.initialContextFactoryName=com.isprint.am.core.cache.ehcache.AtiveMQInitialContextFactory is not a misspelling.

4. Save the changes.

Next, configure the ActiveMQ trusted package:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\bin*.
2. Open the **setenv.sh** file.
3. Search for JAVA_OPTS and add -Dorg.apache.activemq.SERIALIZABLE_PACKAGES=* under it:

```
Bash (Unix Shell)

JAVA_OPTS="$JAVA_OPTS $JAVA_GC_OPTS -Xms400m -Xmx1g -
Dderby.system.home=db
-Dam.ehcache.set_rmi_ip_addr=true -
XX:MaxMetaspaceSize=256M -XX:NewSize=128M
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=logs -
Dhost.name=$(cat /etc/hostname)
-Djavax.net.ssl.trustStore=conf/amtrust.keystore -
Djavax.net.ssl.trustStorePassword=passphrase
-Djava.awt.headless=true" -
Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*
```

4. Save the changes.

Lastly, add the ActiveMQ libraries in the *WEB-INF\lib* folder:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\tomcat\am5\WEB-INF\lib*.
2. Obtain and copy the ActiveMQ libraries listed below into the directory:
   - **activemq-broker-5.17.6.jar**
   - **activemq-client-5.17.6.jar**
   - **hawtbuf-1.11.jar**

SSH Local Port Forwarding

The cloud services such as Aurora Database, Elasticache Redis, and Amazon MQ are deployed in the private network which not accessible from the internet. Some use cases where the administrator needs to access the service, such as creating the database schema in Aurora MySQL Database for the first time or opening the ActiveMQ web console to check the Broker status. The administrator can use SSH local port forwarding to access the service from the local PC. Ensure you have the private key file **am-key-**

pair.pem from Create Keypair For Connection To Linux Instances, if not please create a key pair and add to your EKS node instance.

Create AccessMatrix Database Schema in Aurora MySQL Database

The following example, we use SSH local port forwarding to create AccessMatrix database schema in Aurora MySQL Database for the first time.

1. Use SSH local port forwarding to create a local SSH tunnel to your Kubernetes node and don't close the terminal/console window to keep the tunnel open.

Parameters

```
# Set database instance identifier and store in an environment variable for
ease of reference.
$ DatabaseInstanceID=amdb-instance

# Set local PC port to open the tunnel and store in an environment variable for
ease of reference.
$ LocalDatabasePort=3307

# Get the database instance endpoint address and store in an environment
variable for ease of reference.
$ DatabaseEndpointAddress=$(aws rds describe-db-instances
    --query
'DBInstances[?DBInstanceIdentifier==`'$DatabaseInstanceID'`].Endpoint[0].Addres
s' --output text)

# Get the database instance endpoint port and store in an environment variable
for ease of reference.
$ DatabaseEndpointPort=$(aws rds describe-db-instances
    --query
'DBInstances[?DBInstanceIdentifier==`'$DatabaseInstanceID'`].Endpoint[0].Port'
--output text)

# Get the first external IP node and store in an environment variable for ease
of reference.
$ NodeExternalIP=$(kubectl get nodes -
o=jsonpath='{.items[0].status.addresses[?(@.type=="ExternalIP")].address}')
```

Command

```
$ ssh -i am-key-pair.pem -L
$LocalDatabasePort:$DatabaseEndpointAddress:$DatabaseEndpointPort ec2-
user@$NodeExternalIP
```

Response

```
The authenticity of host '18.139.255.32 (18.139.255.32)' can't be established.
ECDSA key fingerprint is SHA256:XAol9AT315ygbTFZlGQvBU3lFz7iDkmFqvNSwWotHLo.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.139.255.32'(ECDSA) to the list of known hosts.
```

Parameters

```
Last login: Mon Jan 25 17:52:37 2021 from 205.251.233.50

       __|  __|_  )
       _|  (     /   Amazon Linux 2 AMI
      ___|\___|___|

https://aws.amazon.com/amazon-linux-2/
1 package(s) needed for security, out of 3 available
Run "sudo yum update" to apply all updates.
```

2. You can now access the database instance using your local IP and port on your PC. The following creates database schema using MySQL client:

Parameters

```
# Set the location of your database script file and store in an environment
variable for ease of reference.
$ DatabaseScriptFile=/opt/dbscript/mysql-create.sql

# Set database name and store in an environment variable for ease of reference.
$ DatabaseName=amdb

# Set aurora database user id and store in an environment variable for ease of
reference.
$ DatabaseUserId=user1

# Set local PC port to open the tunnel and store in an environment variable for
ease of reference.
$ LocalDatabasePort=3307
```

Command

```
$ mysql -u $DatabaseUserId -P $LocalDatabasePort -h 127.0.0.1 -p $DatabaseName
< $DatabaseScriptFile
```

Response

```
Enter password:
$
```

Important Note: Don't forget to close the opened SSH tunnel by typing "exit" on terminal/console windows in step 1 once you have finished using it.

Access Active MQ Dashboard From Web Browser

The following example, we use SSH local port forwarding to access the ActiveMQ dashboard or web console from your web browser.

1. Use SSH local port forwarding to create a local SSH tunnel to your Kubernetes node and don't close the terminal/console window to keep the tunnel open.

Parameters

```
# Set ActiveMQ dashboard address and store in an environment variable for ease of reference.
$ DashboardAddress=b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-1.amazonaws.com

# Set ActiveMQ dashboard port and store in an environment variable for ease of reference.
$ DashboardPort=8162

# Set local PC port to open the tunnel and store in an environment variable for ease of reference.
$ LocalWebConsolePort=8162

# Get the first external IP node and store in an environment variable for ease of reference.
$ NodeExternalIP=$(kubectl get nodes -o=jsonpath='{.items[0].status.addresses[?(@.type=="ExternalIP")].address}')
```

Command

```
$ ssh -i am-key-pair.pem -L $LocalWebConsolePort:$DashboardAddress:$DashboardPort ec2-user@$NodeExternalIP
```

Response

```
The authenticity of host '18.139.255.32 (18.139.255.32)' can't be established.
ECDSA key fingerprint is SHA256:XAol9AT315ygbTFZlGQvBU3lFz7iDkmFqvNSwWotHLo.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.139.255.32'(ECDSA) to the list of known hosts.
Last login: Mon Jan 25 17:52:37 2021 from 205.251.233.50

       __|  __|_  )
       _|  (     /   Amazon Linux 2 AMI
      ___|\___|___|

https://aws.amazon.com/amazon-linux-2/
1 package(s) needed for security, out of 3 available
Run "sudo yum update" to apply all updates.
```

2. You can access the ActiveMQ dashboard via the browser at *https://<your_ip_address>:8162*.

> **Important Note**: Don't forget to close the opened SSH tunnel by typing `exit` on terminal/console windows in step 1 once you have finished using it.

Domain name and TLS Connection

A domain name must be prepared (e.g. *www.example.com*), to be used in the TLS certificate and set in a proper DNS server.

The terminology TLS and SSL are used interchangeably. SSL is deprecated, replaced with TLS. However, SSL terminology is still being used by Kubernetes itself.

TLS Connection Ends in AccessMatrix

Refer to the architecture diagram. This configuration means secure TLS connection from the user's endpoint (browser or mobile device) ends in AccessMatrix instances/pods. Ingress controller is configured as SSL-Passthrough, which means, any TLS connection will only pass through Ingress Controller.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\conf*.
2. Open the **server.xml** file.
3. Check Connector element for HTTPS port (which default to 8443).
    a. Insert your TLS certificate with the proper domain name to a keystore file, for example, **testserver.keystore**.
    b. Set the keystore file name in the keystoreFile parameter of the Connector element in **server.xml**.

Configure AccessMatrix REST API URL for USO Client

If you are using the USO Client component, you are required to configure the AccessMatrix REST API URL so the USO Client can communicate with the AccessMatrix server.

> **Note:** Refer to *USO Administration Guide - USO Client User Guide* for more information on the USO Client.

Typically, the USO Client and the SSO Portal back end use the same URL to communicate with the AccessMatrix server. However, in some cloud deployments (such as with Kubernetes/OpenShift), they must use different URLs: the SSO Portal back end uses the internal private Kubernetes service URL, while the USO Client must use AccessMatrix's public URL. This is because the USO Client is installed on the end user's desktop, outside of Kubernetes/OpenShift (where the AccessMatrix server is).

To configure this public REST API URL for the USO Client:

1. Navigate to *<ACMATRIX_ROOT>/tomcat/webapps/usoserver/WEB-INF/* and open the **web.xml** file.
2. Search for the commented USO Client URL section:

```
XML

<!-- [Optional] URL used by USO Client to connect to
AM5 server -->
<!-- If not specified, default to same value as
'restAPIURL' -->
<!-- For Kubernetes deployment, using this is mandatory
as the internal
     K8s services URL used by usoserver is different
from the external
     URL used by USO Client -->
<!--
<context-param>
    <param-name>restAPIURLPublic</param-name>
    <param-value>https://am5-public-
url.com:443/am5/rest</param-value>
</context-param>
-->
```

3. Uncomment the section and change the param-value to the public REST API URL of the AccessMatrix server. For example:

```
XML

<!-- [Optional] URL used by USO Client to connect to
AM5 server -->
<!-- If not specified, default to same value as
'restAPIURL' -->
<!-- For Kubernetes deployment, using this is mandatory
as the internal
     K8s services URL used by usoserver is different
from the external
     URL used by USO Client -->
<context-param>
    <param-name>restAPIURLPublic</param-name>
    <param-value>https://amserver.com/am5/rest</param-
value>
</context-param>
```

4. Save the changes.

3.3.2. Prepare Kubernetes Deployment Prerequisites

You must already have a valid account with proper subscription in an AWS cloud service provider. This section explains preparing all the prerequisite items on the respective cloud service provider prior to deploying AccessMatrix on Kubernetes. Here, you learn how to:

1. Access to Cloud Service Providers. This section guides you through installing command-line tool and creating a virtual private cloud to manage all the cloud objects used in this guide.
2. Prepare Container/Image Registry. An image registry is used to host the Docker images that will be used by Kubernetes cluster. Though it is to register/store Docker image, some cloud service providers use container terminology.
3. Create a Kubernetes Cluster. This section guides you through creating the Kubernetes cluster, where AccessMatrix containers will be run and orchestrated.
4. Create an AWS VPC Peering Connection. A VPC peering connection is a networking connection between two VPCs that enables you to route traffic between them using private IPv4 addresses or IPv6 addresses.

Access to Cloud Service Providers

To configure and run the Kubernetes service, AWS command-line tool must be installed.

1. Install the AWS CLI
2. Install the kubectl CLI for managing kubernetes
3. Install the Elastic Kubernetes Service CLI

Configure Virtual Private Cloud

To use the services like Amazon Aurora Database Service, Redis Cache service and others, a Virtual Private Cloud (VPC) and a route table must first be created and subnet IP range must be allocated.

Create VPC and Route Table

1. Use `aws ec2 create-vpc` to create VPC for cloud services:

Parameters
`# Set cloud service VPC name and store in an environment variable for ease of reference.`

Parameters

```
$ CloudServiceVPCName=cloud_svc_vpc

# Set CIDR block for cloud services VPC and store in an environment variable
for ease of reference.
$ CloudServiceCIDR=172.50.0.0/16
```

Command

```
$ aws ec2 create-vpc \
  --cidr-block $CloudServiceCIDR\
  --tag-specifications
'ResourceType=vpc,Tags=[{Key=Name,Value='$CloudServiceVPCName'}]'
```

Response


```
{
    "Vpc": {
      "CidrBlock": "172.50.0.0/16",
      "DhcpOptionsId": "dopt-87736de0",
      "State": "pending",
      "VpcId": "vpc-0513a47f574fa419d",
      "OwnerId": "021215294315",
      "InstanceTenancy": "default",
      "Ipv6CidrBlockAssociationSet": [],
      "CidrBlockAssociationSet": [
        {
          "AssociationId": "vpc-cidr-assoc-0c3d8c6e3cee0bf3c",
          "CidrBlock": "172.50.0.0/16",
          "CidrBlockState": {
            "State": "associated"
          }
        }
     ],
    "IsDefault": false,
    "Tags": [
      {
        "Key": "Name",
        "Value": "cloud_svc_vpc"
      }
    ]
  }
}
```


2. Use `aws ec2 create-route-table` to create a route table of the VPC cloud services:

Parameters

```
# Set cloud service VPC name and store in an environment variable for ease of
reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Set cloud service route table name and store in an environment variable for
ease of reference.
$ CloudServiceRouteName=CLOUD-SVC-PRV-RT
```

Parameters

```
# Get cloud service VPC ID by name and store in an environment variable for
ease of reference.
$ CloudServiceVPCID=$(aws ec2 describe-vpcs --filter
Name=tag:Name,Values=$CloudServiceVPCName\
    --query Vpcs[0].VpcId --output text)
```

Command

```
$ aws ec2 create-route-table \
    --vpc-id $CloudServiceVPCID\
    --tag-specifications 'ResourceType=route-
table,Tags=[{Key=Name,Value='$CloudServiceRouteName'}]'
```

Response


```
{
    "RouteTable": {
      "Associations": [],
      "PropagatingVgws": [],
      "RouteTableId": "rtb-02b68f9eedbe516b4",
      "Routes": [
        {
          "DestinationCidrBlock": "172.50.0.0/16",
          "GatewayId": "local",
          "Origin": "CreateRouteTable",
          "State": "active"
        }
      ],
      "Tags": [
        {
          "Key": "Name",
          "Value": "CLOUD-SVC-PRV-RT"
        }
    ],
    "VpcId": "vpc-0ac8dc0905f41a9c9",
    "OwnerId": "021215294315"
  }
}
```


3. After the VPC and route table are created, allocate IP range in a private subnet for each service created such as:

Service	Subnet Range (CIDR)	Availability Zone	Tag/Name
Aurora Database	172.50.0.0/25	ap-southeast-1a	AMDB-Private-Sub-1A
	172.50.0.128/25	ap-southeast-1b	AMDB-Private-Sub-1B
ElastiCache Redis	172.50.1.0/25	ap-southeast-1a	AMREDIS-Private-Sub-1A
	172.50.1.128/25	ap-southeast-1b	AMREDIS-Private-Sub-1B

Service	Subnet Range (CIDR)	Availability Zone	Tag/Name
Amazon MQ	172.50.2.0/25	ap-southeast-1a	AMQ-Private-Sub-1A
	172.50.2.128/25	ap-southeast-1b	AMQ-Private-Sub-1B

You can create your own IP allocation, use your choice of availability zones and set your own tags/names. The table above is just an example. However, in the subsequent subsections, this document will use information from that table to create subnets for Aurora Database, ElastiCache Redis and Amazon MQ.

Each step will always involve creating the subnet and associating it to the route table.

Create Subnets for Aurora Database

The following creates two private subnets for the aurora database in the ap-southeast-1 region and associate it with a route table.

1. Use `aws ec2 create-subnet` to create a subnets in ap-southeast-1a zone:

Parameters

```
# Set cloud service VPC name and store in an environment variable for ease of reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Set subnet name in ap-southeast-1a and store in an environment variable for ease of reference.
$ DatabaseSubnetName1A=AMDB-Private-Sub-1A

# Set CIDR block for Aurora Database Service in ap-southeast-1a zone and store in an environment variable for ease of reference.
$ DatabaseServiceCIDR1A=172.50.0.0/25

# Get cloud service VPC id and store in an environment variable for ease of reference.
$ CloudServiceVPCID=$(aws ec2 describe-vpcs --filter Name=tag:Name,Values=$CloudServiceVPCName --query Vpcs[0].VpcId --output text)
```

Command

```
$ aws ec2 create-subnet \
    --availability-zone "ap-southeast-1a"\
    --vpc-id $CloudServiceVPCID\
    --cidr-block $DatabaseServiceCIDR1A\
    --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value='$DatabaseSubnetName1A'}]'
```

Response

Parameters


```
{
  "Subnet": {
    "AvailabilityZone": "ap-southeast-1a",
    "AvailabilityZoneId": "apse1-az2",
    "AvailableIpAddressCount": 123,
    "CidrBlock": "172.50.0.0/25",
    "DefaultForAz": false,
    "MapPublicIpOnLaunch": false,
    "MapCustomerOwnedIpOnLaunch": false,
    "State": "available",
    "SubnetId": "subnet-0894f6d1aee4ab57e",
    "VpcId": "vpc-013315608e0b403a3",
    "OwnerId": "021215294315",
    "AssignIpv6AddressOnCreation": false,
    "Ipv6CidrBlockAssociationSet": [],
    "Tags": [
    {
      "Key": "Name",
      "Value": "AMDB-Private-Sub-1A"
    }],
    "SubnetArn": "arn:aws:ec2:ap-southeast-1:021215294315:subnet/subnet-
0894f6d1aee4ab57e"
  }
}
```


2. Use `aws ec2 associate-route-table` to associate subnet in ap-southeast-1a with a default route table:

Parameters

```
# Set subnet name in ap-southeast-1a and store in an environment variable for
ease of reference.
$ DatabaseSubnetName1A=AMDB-Private-Sub-1A

# Get aurora database service subnet id by name in ap-southeast-1a and store in
an environment variable for ease of reference.
$ DatabaseSubnetID1A=$(aws ec2 describe-subnets --filter
Name=tag:Name,Values=$DatabaseSubnetName1A\
    --query Subnets[0].SubnetId --output text)

# Get cloud service route table id by name and store in an environment variable
for ease of reference.
$ CloudServiceRouteID=$(aws ec2 describe-route-tables --filter
Name=tag:Name,Values=$CloudServiceRouteName\
    --query RouteTables[0].RouteTableId --output text)
```

Command

```
$ aws ec2 associate-route-table \
    --route-table-id $CloudServiceRouteID\
    --subnet-id $DatabaseSubnetID1A
```

Response

Parameters

```
{
  "AssociationId": "rtbassoc-0a472094ce84380f9",
  "AssociationState": {
    "State": "associated"
  }
}
```

3. Use `aws ec2 create-subnet` to create a subnets in ap-southeast-1b zone:

Parameters

```
# Set cloud service VPC name and store in an environment variable for ease of
reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Set subnet name in ap-southeast-1b and store in an environment variable for
ease of reference.
$ DatabaseSubnetName1B=AMDB-Private-Sub-1B

# Set CIDR Block for Aurora Database Service in ap-southeast-1b zone and store
in an environment variable for ease of reference.
$ DatabaseServiceCIDR1B=172.50.0.128/25

# Get cloud service VPC id and store in an environment variable for ease of
reference.
$ CloudServiceVPCID=$(aws ec2 describe-vpcs --filter
Name=tag:Name,Values=$CloudServiceVPCName --query Vpcs[0].VpcId --output text)
```

Command

```
$ aws ec2 create-subnet \
    --availability-zone "ap-southeast-1b"\
    --vpc-id $CloudServiceVPCID\
    --cidr-block $DatabaseServiceCIDR1B\
    --tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value='$DatabaseSubnetName1B'}]'
```

Response

```
{
  "Subnet": {
    "AvailabilityZone": "ap-southeast-1b",
    "AvailabilityZoneId": "apse1-az1",
    "AvailableIpAddressCount": 123,
    "CidrBlock": "172.50.0.128/25",
    "DefaultForAz": false,
    "MapPublicIpOnLaunch": false,
    "State": "available",
    "SubnetId": "subnet-084873d974ecdda70",
    "VpcId": "vpc-013315608e0b403a3",
    "OwnerId": "021215294315",
    "AssignIpv6AddressOnCreation": false,
    "Ipv6CidrBlockAssociationSet": [],
    "Tags": [
```

Parameters

```
      {
        "Key": "Name",
        "Value": "AMDB-Private-Sub-1B"
      }
    ],
    "SubnetArn": "arn:aws:ec2:ap-southeast-1:021215294315:subnet/subnet-
084873d974ecdda70"
  }
}
```

4. Use `aws ec2 associate-route-table` to associate subnet in ap-southeast-1b with a default route table:

Parameters

```
# Set subnet name in ap-southeast-1b and store in an environment variable for
ease of reference.
$ DatabaseSubnetName1B=AMDB-Private-Sub-1B

# Set cloud service route table name and store in an environment variable for
ease of reference.
$ CloudServiceRouteName=CLOUD-SVC-PRV-RT

# Get aurora database service subnet id by name in ap-southeast-1b and store in
an environment variable for ease of reference.
$ DatabaseSubnetID1B=$(aws ec2 describe-subnets --filter
Name=tag:Name,Values=$DatabaseSubnetName1B\
    --query Subnets[0].SubnetId --output text)

# Get cloud service route table id by name and store in an environment variable
for ease of reference.
$ CloudServiceRouteID=$(aws ec2 describe-route-tables --filter
Name=tag:Name,Values=$CloudServiceRouteName\
    --query RouteTables[0].RouteTableId --output text)
```

Command

```
$ aws ec2 associate-route-table \
    --route-table-id $CloudServiceRouteID\
    --subnet-id $DatabaseSubnetID1B
```

Response

```
{
  "AssociationId": "rtbassoc-042942581fd89248a",
  "AssociationState": {
    "State": "associated"
  }
}
```

Create Subnets for ElastiCache Redis

The following creates two private subnets for ElastiCache Redis in the ap-southeast-1 region and associate them with a route table.

1. Use `aws ec2 create-subnet` to create a subnets in ap-southeast-1a zone:

Parameters

```
# Set cloud service VPC name and store in an environment variable for ease of reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Set subnet name in ap-southeast-1a and store in an environment variable for ease of reference.
$ RedisSubnetName1A=AMREDIS-Private-Sub-1A

# Set CIDR block for ElastiCache Redis Service in ap-southeast-1a zone and store in an environment variable for ease of reference.
$ RedisServiceCIDR1A=172.50.1.0/25

# Get cloud service VPC id and store in an environment variable for ease of reference.
$ CloudServiceVPCID=$(aws ec2 describe-vpcs --filter Name=tag:Name,Values=$CloudServiceVPCName --query Vpcs[0].VpcId --output text)
```

Command

```
$ aws ec2 create-subnet \
    --availability-zone "ap-southeast-1a"\
    --vpc-id $CloudServiceVPCID\
    --cidr-block $RedisServiceCIDR1A\
    --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value='$RedisSubnetName1A'}]'
```

Response


```
{
  "Subnet": {
    "AvailabilityZone": "ap-southeast-1a",
    "AvailabilityZoneId": "apse1-az2",
    "AvailableIpAddressCount": 123,
    "CidrBlock": "172.50.1.0/25",
    "DefaultForAz": false,
    "MapPublicIpOnLaunch": false,
    "MapCustomerOwnedIpOnLaunch": false,
    "State": "available",
    "SubnetId": "subnet-0040150eafa7e6af2",
    "VpcId": "vpc-013315608e0b403a3",
    "OwnerId": "021215294315",
    "AssignIpv6AddressOnCreation": false,
    "Ipv6CidrBlockAssociationSet": [],
    "Tags": [
    {
```

Parameters

```
      "Key": "Name",
      "Value": "AMREDIS-Private-Sub-1A"
    }],
    "SubnetArn": "arn:aws:ec2:ap-southeast-1:021215294315:subnet/subnet-
0040150eafa7e6af2"
  }
}
```

2. Use `aws ec2 associate-route-table` to associate subnet in ap-southeast-1a with a default route table:

Parameters

```
# Set subnet name in ap-southeast-1a and store in an environment variable for
ease of reference.
$ RedisSubnetName1A=AMREDIS-Private-Sub-1A

# Get ElastiCache Redis Service subnet id by name in ap-southeast-1a and store
in an environment variable for ease of reference.
$ RedisSubnetID1A=$(aws ec2 describe-subnets --filter
Name=tag:Name,Values=$RedisSubnetName1A\
    --query Subnets[0].SubnetId --output text)

# Get cloud service route table id by name and store in an environment variable
for ease of reference.
$ CloudServiceRouteID=$(aws ec2 describe-route-tables --filter
Name=tag:Name,Values=$CloudServiceRouteName\
    --query RouteTables[0].RouteTableId --output text)
```

Command

```
$ aws ec2 associate-route-table \
    --route-table-id $CloudServiceRouteID\
    --subnet-id $RedisSubnetID1A
```

Response

```
{
  "AssociationId": "rtbassoc-0c783716894eccda9",
  "AssociationState": {
    "State": "associated"
  }
}
```

3. Use `aws ec2 create-subnet` to create a subnets in ap-southeast-1b zone:

Parameters

```
# Set cloud service VPC name and store in an environment variable for ease of
reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Set subnet name in ap-southeast-1b and store in an environment variable for
```

Parameters

```
ease of reference.
$ RedisSubnetName1B=AMREDIS-Private-Sub-1B

# Set CIDR Block for ElastiCache Redis Service in ap-southeast-1b zone and
store in an environment variable for ease of reference.
$ RedisServiceCIDR1B=172.50.1.128/25

# Get cloud service VPC id and store in an environment variable for ease of
reference.
$ CloudServiceVPCID=$(aws ec2 describe-vpcs --filter
Name=tag:Name,Values=$CloudServiceVPCName --query Vpcs[0].VpcId --output text)
```

Command

```
$ aws ec2 create-subnet \
    --availability-zone "ap-southeast-1b"\
    --vpc-id $CloudServiceVPCID\
    --cidr-block $RedisServiceCIDR1B\
    --tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value='$RedisSubnetName1B'}]'
```

Response


```
{
  "Subnet": {
    "AvailabilityZone": "ap-southeast-1b",
    "AvailabilityZoneId": "apse1-az1",
    "AvailableIpAddressCount": 123,
    "CidrBlock": "172.50.1.128/25",
    "DefaultForAz": false,
    "MapPublicIpOnLaunch": false,
    "State": "available",
    "SubnetId": "subnet-06adaef03545d85af",
    "VpcId": "vpc-013315608e0b403a3",
    "OwnerId": "021215294315",
    "AssignIpv6AddressOnCreation": false,
    "Ipv6CidrBlockAssociationSet": [],
    "Tags": [
      {
        "Key": "Name",
        "Value": "AMREDIS-Private-Sub-1B"
      }
    ],
    "SubnetArn": "arn:aws:ec2:ap-southeast-1:021215294315:subnet/subnet-
06adaef03545d85af"
  }
}
```


4. Use `aws ec2 associate-route-table` to associate subnet in ap-southeast-1b with a default route table:

Parameters

```
# Set subnet name in ap-southeast-1b and store in an environment variable for
ease of reference.
$ RedisSubnetName1B=AMREDIS-Private-Sub-1B

# Set cloud service route table name and store in an environment variable for
ease of reference.
$ CloudServiceRouteName=CLOUD-SVC-PRV-RT

# Get ElastiCache Redis service subnet id by name in ap-southeast-1b and store
in an environment variable for ease of reference.
$ RedisSubnetID1B=$(aws ec2 describe-subnets --filter
Name=tag:Name,Values=$RedisSubnetName1B\
    --query Subnets[0].SubnetId --output text)

# Get cloud service route table id by name and store in an environment variable
for ease of reference.
$ CloudServiceRouteID=$(aws ec2 describe-route-tables --filter
Name=tag:Name,Values=$CloudServiceRouteName\
    --query RouteTables[0].RouteTableId --output text)
```

Command

```
$ aws ec2 associate-route-table \
    --route-table-id $CloudServiceRouteID\
    --subnet-id $RedisSubnetID1B
```

Response

```
{
  "AssociationId": "rtbassoc-0cf4870abcdd7a38c",
  "AssociationState": {
    "State": "associated"
  }
}
```

Create Subnets for Amazon MQ

The following creates two private subnets for Amazon MQ Service in the ap-southeast-1 region and associate them with a Route table.

1. Use `aws ec2 create-subnet` to create a subnets in ap-southeast-1a zone:

Parameters

```
# Set cloud service VPC name and store in an environment variable for ease of
reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Set subnet name in ap-southeast-1a and store in an environment variable for
ease of reference.
$ AMQSubnetName1A=AMQ-Private-Sub-1A
```

Parameters

```
# Set CIDR block for Amazon MQ Service in ap-southeast-1a zone and store in an
environment variable for ease of reference.
$ AMQServiceCIDR1A=172.50.2.0/25

# Get cloud service VPC id and store in an environment variable for ease of
reference.
$ CloudServiceVPCID=$(aws ec2 describe-vpcs --filter
Name=tag:Name,Values=$CloudServiceVPCName --query Vpcs[0].VpcId --output text)
```

Command

```
$ aws ec2 create-subnet \
    --availability-zone "ap-southeast-1a"\
    --vpc-id $CloudServiceVPCID\
    --cidr-block $AMQServiceCIDR1A\
    --tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value='$AMQSubnetName1A'}]'
```

Response


```
{
  "Subnet": {
    "AvailabilityZone": "ap-southeast-1a",
    "AvailabilityZoneId": "apse1-az2",
    "AvailableIpAddressCount": 123,
    "CidrBlock": "172.50.2.0/25",
    "DefaultForAz": false,
    "MapPublicIpOnLaunch": false,
    "MapCustomerOwnedIpOnLaunch": false,
    "State": "available",
    "SubnetId": "subnet-024e4b4fd3f552cdf",
    "VpcId": "vpc-013315608e0b403a3",
    "OwnerId": "021215294315",
    "AssignIpv6AddressOnCreation": false,
    "Ipv6CidrBlockAssociationSet": [],
    "Tags": [
    {
      "Key": "Name",
      "Value": "AMQ-Private-Sub-1A"
    }],
    "SubnetArn": "arn:aws:ec2:ap-southeast-1:021215294315:subnet/subnet-
024e4b4fd3f552cdf"
  }
}
```


2. Use `aws ec2 associate-route-table` to associate subnet in ap-southeast-1a with a default route table:

Parameters

```
# Set subnet name in ap-southeast-1a and store in an environment variable for
ease of reference.
```

Parameters

```
$ AMQSubnetName1A=AMQ-Private-Sub-1A

# Get Amazon MQ Service subnet id by name in ap-southeast-1a and store in an
environment variable for ease of reference.
$ AMQSubnetID1A=$(aws ec2 describe-subnets --filter
Name=tag:Name,Values=$AMQSubnetName1A\
    --query Subnets[0].SubnetId --output text)

# Get cloud service route table id by name and store in an environment variable
for ease of reference.
$ CloudServiceRouteID=$(aws ec2 describe-route-tables --filter
Name=tag:Name,Values=$CloudServiceRouteName\
    --query RouteTables[0].RouteTableId --output text)
```

Command

```
$ aws ec2 associate-route-table \
    --route-table-id $CloudServiceRouteID\
    --subnet-id $AMQSubnetID1A
```

Response

```
{
  "AssociationId": "rtbassoc-0b00c40713f79f5dd",
  "AssociationState": {
    "State": "associated"
  }
}
```

3. Use `aws ec2 create-subnet` to create a subnets in ap-southeast-1b zone:

Parameters

```
# Set cloud service VPC name and store in an environment variable for ease of
reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Set subnet name in ap-southeast-1b and store in an environment variable for
ease of reference.
$ AMQSubnetName1B=AMQ-Private-Sub-1B

# Set CIDR Block for Amazon MQ Service in ap-southeast-1b zone and store in an
environment variable for ease of reference.
$ AMQServiceCIDR1B=172.50.2.128/25

# Get cloud service VPC id and store in an environment variable for ease of
reference.
$ CloudServiceVPCID=$(aws ec2 describe-vpcs --filter
Name=tag:Name,Values=$CloudServiceVPCName --query Vpcs[0].VpcId --output text)
```

Command

Parameters

```
$ aws ec2 create-subnet \
    --availability-zone "ap-southeast-1b"\
    --vpc-id $CloudServiceVPCID\
    --cidr-block $AMQServiceCIDR1B\
    --tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value='$AMQSubnetName1B'}]'
```

Response

```
{
  "Subnet": {
    "AvailabilityZone": "ap-southeast-1b",
    "AvailabilityZoneId": "apse1-az1",
    "AvailableIpAddressCount": 123,
    "CidrBlock": "172.50.2.128/25",
    "DefaultForAz": false,
    "MapPublicIpOnLaunch": false,
    "State": "available",
    "SubnetId": "subnet-0da49349e28ff6adf",
    "VpcId": "vpc-013315608e0b403a3",
    "OwnerId": "021215294315",
    "AssignIpv6AddressOnCreation": false,
    "Ipv6CidrBlockAssociationSet": [],
    "Tags": [
      {
        "Key": "Name",
        "Value": "AMQ-Private-Sub-1B"
      }
    ],
    "SubnetArn": "arn:aws:ec2:ap-southeast-1:021215294315:subnet/subnet-
0da49349e28ff6adf"
  }
}
```

4. Use `aws ec2 associate-route-table` to associate subnet in ap-southeast-1b with a default route table:

Parameters

```
# Set subnet name in ap-southeast-1b and store in an environment variable for
ease of reference.
$ AMQSubnetName1B=AMQ-Private-Sub-1B

# Set cloud service route table name and store in an environment variable for
ease of reference.
$ CloudServiceRouteName=CLOUD-SVC-PRV-RT

# Get Amazon MQ service subnet id by name in ap-southeast-1b and store in an
environment variable for ease of reference.
$ AMQSubnetID1B=$(aws ec2 describe-subnets --filter
Name=tag:Name,Values=$AMQSubnetName1B\
    --query Subnets[0].SubnetId --output text)

# Get cloud service route table id by name and store in an environment variable
```

Parameters

```
for ease of reference.
$ CloudServiceRouteID=$(aws ec2 describe-route-tables --filter
Name=tag:Name,Values=$CloudServiceRouteName\
    --query RouteTables[0].RouteTableId --output text)
```

Command

```
$ aws ec2 associate-route-table \
    --route-table-id $CloudServiceRouteID\
    --subnet-id $AMQSubnetID1B
```

Response

```
{
  "AssociationId": "rtbassoc-0a7651759a390f8fa",
  "AssociationState": {
    "State": "associated"
  }
}
```

Create Keypair for Connection to Linux Instances

An AWS key pair is required to prove your identity when connecting to an instance. Amazon EC2 stores the public key, and you store the private key. You use the private key instead of a password to access your instances securely. Beware that anyone who possesses your private keys can connect to your instances, so you must store your private keys in a secure place.

1. Use `aws ec2 create-key-pair` to create a key pair:

Parameters

```
# Set Key-Pair Name and store in an environment variable for ease of reference.
$ AMKeyPair=am5-key
```

Command

```
$ aws ec2 create-key-pair --key-name $AMKeyPair
```

Response

```
{
  "KeyFingerprint":
"27:b0:40:f6:0e:b0:5d:88:3a:4b:5d:d2:f4:09:6b:c7:d3:aa:97:42",
  "KeyMaterial": "-----BEGIN RSA PRIVATE KEY-----\nMIIEpAIB...Lm7TrrUg==\n-----
END RSA PRIVATE KEY-----",
  "KeyName": "am5-key",
  "KeyPairId": "key-0d7497ca8bbaf8eab"
}
```

The `KeyMaterial` in the response output is an unencrypted PEM encoded RSA private key.

2. Copy the `KeyMaterial` value to a file. For example:

am-key-pair.pem

Code

```
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAieBbI8wlqau57oXV2AYUBQz3ThZhoOJxXvWplc+JpMCaslGk
Pr+jFM9cltQaycPyXy2GqeUrISsyY0sc0CigGpXjavrsMeP7SriDR0aTKmP9mfJ1
Jq42m76AISlje2xMGoriheqCIFPl8DxC9cif4sD/D8CcIzhQaY+lJLBjchri82qf
ajNBOZbgx+0bQSEWLETww9qcZVa8cLtklJhm99RZ9dBF6ELVsY5BzO+C0OXJ44zI
60AeLEVg4w4rOhrgsRIan2xEuw0iWfVaZvv/J11NMjfwCgeiKWuXcODWOozSRtVo
5hVN9uXYNvLimSe/OeMKPzakoJn8BrJ02KUI1wIDAQABAoIBAFRJU24O3vdag8/N
AW2OSa7MvmJ+TDCOPcBPbdyngv20wN1CC6ZQgOG34mJSSA4QO5rzud8iubsNLUCG
KpcWPsoB14UO4FWfiX+kTJzGCDFh7yatkfCTj86mJq4fHzHUgrG3NcOoGOSqZWb3
b/fRLezxDvwSQ2FEi+P8bHiBXEnkO5x8aw7y37W9qsSEudzV3ukMgeDjiMErfBF0
xmzmKQQ6vM3dDw2qvNU9N3oXDJk9orJkviotsZF0zzGF8Ni7ZoykibAS2vZGvWJO
Ddt2DGOG6gUEwzjsowgaUo2eCTwz7xV++VJ7SJHC8QUZjXfhr0Z+kbyOp3N9UHF5
wu9V6AECgYEA5Rg2BtNzIbgzM0InYL+QzuIwIJlFmZ58+eDHzqYBVMIC/gDej4Ms
CDdMEbaFaJKv8WJ+NhqCLy//o0uluYaHM/61a8IxJUb2S+ijb07FCUYnGeA0IddU
008p+vcxShvSKRf2nvVM9R3R8TuasjS+guENQJpMN3g7J7iC7c+MWW8CgYEAmhGl
arbdrY8bFjLco8y1hQwNljcFNG8AoKOtgNxPxkjPTtDKrYi0/158/qkiFdhRRWK/
VjIzH6fIzbRO3rGoiREqpiUF47Bl19DwsMJPpsov+s9DI662T1VdqH0ORtcuhlzJ
bdD1KIJOxc5K+++PbSnsH/Z/diQLychlLGVcAxkCgYAjY9YUv91tPBNMu89xdqZc
Dzz4DDVgQcd0onC7T5GtEjt8GY5IBS9sJX6uzc9WFDLxyxFMMlQiDfwjsa/sNQf1
1EcaWmlVJN0vZMS6kSSHBjtMGAJ1p9oGupf1mPIYTKvuhTEn5ST6SJ0B8v82wEcd
2L4/WJaliwE4ZZx0ODeqKQKBgQCAT3nn6eusg6YXUryWXgI+j88w7xmSlW98WVRC
ITj+9dxQ02o4I8Y5d0O6Arm0X7cvhhG7g4s4Lc/6INQeeu7VWEqgm+zPtzTaC9So
IOudCH3073AG45ZJc9rQW3XsYYEJvXSz1HaYCg34htnD/7r/Ow6/f9afWV3H6vD3
jGKY4QKBgQCH61mfJwxOadFJCpUi6pTnsfAVAtyfuvQuFcjqO+tRmm+o9DAdjpht
crVnqwu1cZKuigtfE1nae47QZ97NM5/I+hJoxQohW1IcoCu8vwz56amSreA3tOBV
QqyeruFQKm2lo2i1kzb4CIgewsqGESuoD0Afk5K6e+6DPCLm7TrrUg==
-----END RSA PRIVATE KEY-----
```

Note: When copying the KeyMaterial value, ensure that you replace any "\n" characters with line breaks. For an example of this, refer to the above sample code.

3. Store private key file **am-key-pair.pem** securely.

Create Container/Image Registry

To deploy AccessMatrix as containers, Kubernetes needs to pull the AccessMatrix Docker image from an image registry. Thus, you need to create a container/image registry, then build and push your AccessMatrix Docker image to it.

1. Use `aws ecr create-repository` to create a docker repository on AWS Elastic Container Registry (ECR):

Parameters

```
# Set image registry name and store in an environment variable for ease of
reference.
$ AMImageRegistryName=am5
```

Command

```
$ aws ecr create-repository \
    --repository-name $AMImageRegistryName\
    --image-scanning-configuration scanOnPush=true\
    --region ap-southeast-1
```

Response

```
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:ap-southeast-1:021215294315:repository/am5",
    "registryId": "021215294315",
    "repositoryName": "accessmatrix",
    "repositoryUri": "021215294315.dkr.ecr.ap-southeast-1.amazonaws.com/am5",
    "createdAt": "2021-01-18T08:07:58+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": true
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

2. Use `aws ecr describe-repositories` to query the repository URL:

Parameters

```
# Set image registry name and store in an environment variable for ease of
reference.
$ AMImageRegistryName=am5
```

Command

```
$ aws ecr describe-repositories \
    --repository-names $AMImageRegistryName\
    --query repositories[0].repositoryUri
```

Response

```
"<aws_account_id>.dkr.ecr.<region>.amazonaws.com/am5"
```

Create Kubernetes Cluster

A Kubernetes cluster is a set of node machines for running containerized applications. At a minimum, a cluster contains a worker node and a master node. The master node is responsible for maintaining the cluster's desired state, such as which applications are running and which container images they use. Worker nodes run the applications and workloads.

Cloud Service Providers usually provide API to create Kubernetes cluster by specifying the size, node VM specs, etc.

1. Use `eksctl create cluster` to create an AWS Elastic Kubernetes Service (EKS) Cluster. The example below creates the cluster:
    - Uses default EC2 instance type (m5.large).
    - Contains 2 nodes which can be scale between 1 and 4 nodes.

Parameters

```
# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster

# Set Key-Pair Name and store in an environment variable for ease of reference.
$ AMKeyPair=am5-key
```

Command

```
$ eksctl create cluster \
    --name $EKSClusterName\
    --version 1.25 \
    --region ap-southeast-1 \
    --nodegroup-name AMEksNodeGroup \
    --nodes 2 \
    --nodes-min 1 \
    --nodes-max 4 \
    --with-oidc \
    --ssh-access \
    --ssh-public-key $AMKeyPair\
    --managed
```

Response

```
...
[✓] EKS cluster "AMEksCluster" in "ap-southeast-1" region is ready
```

2. Optional, if you already have kubectl installed, update your kubeconfig to use AWS cluster.

Parameters
`# Set Cluster Name and store in an environment variable for ease of reference.` `$ EKSClusterName=AMEksCluster`
Command
`$ aws eks --region ap-southeast-1 update-kubeconfig --name $EKSClusterName`

3. Use `kubectl get nodes` to verify the Kubernetes cluster is created by telling all nodes are ready:

Command
`$ kubectl get nodes`
Response

```
NAME                                                STATUS   ROLES    AGE
   VERSION
ip-192-168-16-21.ap-southeast-1.compute.internal    Ready    <none>   135m
  v1.18.9-eks-d1db3c
ip-192-168-89-187.ap-southeast-1.compute.internal   Ready    <none>   136m
  v1.18.9-eks-d1db3c
```

Create AWS VPC Peering Connection

Amazon Elastic Kubernetes Service (Amazon EKS) cluster and cloud services such as Aurora Database, Amazon ElastiCache for Redis, and Amazon Message Queue (MQ) are deployed in different Virtual Private Cloud (VPC). One VPC is isolated from other VPC, which means AccessMatrix deployed in Amazon EKS cluster cannot communicate with Aurora Database, Amazon ElastiCache for Redis or Amazon MQ because these cloud services are deployed in different VPCs. A VPC Peering Connection is required to route the traffic from one VPC to other VPC.

1. Use `aws ec2 create-vpc-peering-connection` to create VPC Peering Connection between Amazon EKS and cloud services:

Parameters

```
# Set cloud service VPC name and store in an environment variable for ease of
reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Set VPC Peering Name and store in an environment variable for ease of
reference.
$ AMVpcPeerName=am-vpc-peering

# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster
```

Parameters

```
# Get AccessMatrix VPC id by cluster name and store in an environment variable
for ease of reference.
$ AccessMatrixVPCID=$(aws eks describe-cluster --name $EKSClusterName --query
"cluster.resourcesVpcConfig.vpcId" --output text)

# Get cloud service VPC id by name and store in an environment variable for
ease of reference.
$ CloudServiceVPCID=$(aws ec2 describe-vpcs --filter
Name=tag:Name,Values=$CloudServiceVPCName\
    --query Vpcs[0].VpcId --output text)
```

Command

```
$ aws ec2 create-vpc-peering-connection \
    --vpc-id $AccessMatrixVPCID\
    --peer-vpc-id $CloudServiceVPCID\
    --tag-specifications 'ResourceType=vpc-peering-
connection,Tags=[{Key=Name,Value='$AMVpcPeerName'}]'
```

Response

```
{
  "VpcPeeringConnection": {
    "Status": {
      "Message": "Initiating Request to 021215294315",
      "Code": "initiating-request"
    },
    "Tags": [
      {
        "Key": "Name",
        "Value": "am-vpc-peering"
      }
    ],
    "RequesterVpcInfo": {
      "OwnerId": "021215294315",
      "VpcId": "vpc-05982544206122bba",
      "CidrBlock": "192.168.0.0/16"
    },
    "VpcPeeringConnectionId": "pcx-03c7e063111966a46",
    "ExpirationTime": "2020-12-30T16:13:36.000Z",
    "AccepterVpcInfo": {
      "OwnerId": "021215294315",
      "VpcId": "vpc-013315608e0b403a3"
    }
  }
}
```

2. Use `aws ec2 accept-vpc-peering-connection` to accept the VPC peering connection:

Parameters

```
# Set VPC Peering Name and store in an environment variable for ease of
reference.
```

Parameters

```
$ AMVpcPeerName=am-vpc-peering

# Get VPC Peering Connection ID by name and store in an environment variable
for ease of reference.
$ VPCPeeringID=$(aws ec2 describe-vpc-peering-connections --filter
Name=tag:Name,Values=$AMVpcPeerName\
    --query VpcPeeringConnections[0].VpcPeeringConnectionId --output text)
```

Command

```
$ aws ec2 accept-vpc-peering-connection --vpc-peering-connection-id
$VPCPeeringID
```

Response


```
{
  "VpcPeeringConnection": {
    "Status": {
      "Message": "Provisioning",
      "Code": "provisioning"
    },
    "Tags": [],
    "RequesterVpcInfo": {
      "OwnerId": "021215294315",
      "VpcId": "vpc-05982544206122bba",
      "CidrBlock": "192.168.0.0/16"
    },
    "VpcPeeringConnectionId": "pcx-03c7e063111966a46",
    "ExpirationTime": "2020-12-30T16:13:36.000Z",
    "AccepterVpcInfo": {
      "OwnerId": "021215294315",
      "VpcId": "vpc-013315608e0b403a3",
      "CidrBlock": "172.50.0.0/16"
    }
  }
}
```


3. Use `aws ec2 create-route` to create a route to cloud service IP via peering connection:

Parameters

```
# Set cloud service VPC name and store in an environment variable for ease of
reference.
$ CloudServiceVPCName=cloud_svc_vpc

# Set VPC Peering Name and store in an environment variable for ease of
reference.
$ AMVpcPeerName=am-vpc-peering

# Get Amazon EKS Cluster's public route table and store in an environment
variable for ease of reference.
$ AccessMatrixRouteID=$(aws ec2 describe-route-tables --filters
Name="tag:aws:cloudformation:logical-id",Values="PublicRouteTable"\
    --query RouteTables[0].RouteTableId --output text)
```

Parameters

```
# Get cloud service CIDR and store in an environment variable for ease of
reference.
$ CloudServiceCIDR=$(aws ec2 describe-vpcs --filter
Name=tag:Name,Values=$CloudServiceVPCName\
    --query Vpcs[0].CidrBlock --output text)

# Get VPC Peering Connection ID by name and store in an environment variable
for ease of reference.
$ VPCPeeringID=$(aws ec2 describe-vpc-peering-connections --filter
Name=tag:Name,Values=$AMVpcPeerName\
    --query VpcPeeringConnections[0].VpcPeeringConnectionId --output text)
```

Command

```
$ aws ec2 create-route \
    --route-table-id $AccessMatrixRouteID\
    --destination-cidr-block $CloudServiceCIDR\
    --vpc-peering-connection-id $VPCPeeringID
```

Response

```
{
  "Return": true
}
```

4. Use `aws ec2 create-route` to create a route to Amazon EKS Cluster IP via peering connection:

Parameters

```
# Set cloud service route table name and store in an environment variable for
ease of reference.
$ CloudServiceRouteName=CLOUD-SVC-PRV-RT

# Set VPC Peering Name and store in an environment variable for ease of
reference.
$ AMVpcPeerName=am-vpc-peering

# Get cloud service route table by name and store in an environment variable
for ease of reference.
$ CloudServiceRouteID=$(aws ec2 describe-route-tables --filters
Name=tag:Name,Values=$CloudServiceRouteName\
    --query RouteTables[0].RouteTableId --output text)

# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster

# Get AccessMatrix VPC id by cluster name and store in an environment variable
for ease of reference.
$ AccessMatrixVPCID=$(aws eks describe-cluster --name $EKSClusterName --query
"cluster.resourcesVpcConfig.vpcId" --output text)

# Get AccessMatrix CIDR by vpc-id and store in an environment variable for ease
of reference.
```

Parameters

```
$ AccessMatrixCIDR=$(aws ec2 describe-vpcs --vpc-id $AccessMatrixVPCID --query
Vpcs[0].CidrBlock --output text)

# Get VPC Peering Connection id by tag name and store in an environment
variable for ease of reference.
$ VPCPeeringID=$(aws ec2 describe-vpc-peering-connections --filter
Name=tag:Name,Values=$AMVpcPeerName\
    --query VpcPeeringConnections[0].VpcPeeringConnectionId --output text)
```

Command

```
$ aws ec2 create-route \
    --route-table-id $CloudServiceRouteID\
    --destination-cidr-block $AccessMatrixCIDR\
    --vpc-peering-connection-id $VPCPeeringID
```

Response

```
{
  "Return": true
}
```

3.3.3. Create AccessMatrix Docker Image

At this stage, all of the services that will be used by AccessMatrix are ready, and related AccessMatrix base package configurations have been updated. You can now proceed to build the AccessMatrix Docker image. An AccessMatrix Docker image must be built locally before being pushed to the registry.

Repack AccessMatrix Base Package

You have prepared the database service, Redis cache service, and ActiveMQ. You also have updated related configurations in AccessMatrix. Now, you need to repack it back to its respective name (**AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**), for example, **AccessMatrix-5.7.5.7535-GA-tomcat-only.tar.gz**.

Pre-image Build Checklist

Before building the AccessMatrix Docker Container Image, ensure the following is done:

Checklist	Description
AccessMatrix container mode	AccessMatrix container mode has been enabled in **amsystem.properties**. Refer to Enable AccessMatrix Container Mode for more information.
Database service	Database service is ready and its connection information is configured in **am5.xml**. Refer to Database Service for more information.
Redis Cache service	Redis Cache service is ready and its connection information is configured in **amsystem.properties**. Refer to Redis Cache Server for more information.
Apache ActiveMQ service	Amazon MQ service is ready and its connection information is configured in **am5-ehcache.xml** and **am5.xml**. Refer to ActiveMQ am5-ehcache.xml configuration for more information.
TLS Connection Ends in AccessMatrix	If the TLS connection were to end in AccessMatrix instance (Ingress controller passthrough), the certificate and port are configured in **server.xml**. Refer to TLS Connection Ends in AccessMatrix for more information.
AM Base Package	AM base package that was previously unpacked must be repacked back to its original name.
Dependent libraries	Any additional libraries or drivers such as JDBC, or any external drivers, must be placed in the lib folder in the same directory as the Dockerfile.

Build AccessMatrix Docker Image

A sample AccessMatrix Dockerfile is shown in the steps below. Arrange the files as follows:

1. Place the Dockerfile in any working directory that you choose. Change the environment variable AM_VERSION to respective AccessMatrix version.

Dockerfile

Code

```
FROM adoptopenjdk/openjdk11:slim

ENV AM_VERSION "5.7.5.7535-GA."

RUN mkdir -p /opt/am/${AM_VERSION}
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}
RUN groupadd am \
    && useradd -r -g am -s /bin/bash am \
    && chown -R am:am /opt/am/${AM_VERSION} \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:am /opt/am/am

COPY ["lib", "/opt/am/am/tomcat/lib"]

EXPOSE 8080 8443 8162 61617
USER am
CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
```

2. Create a subdirectory, *lib*, to place all dependent libraries, e.g. JDBC drivers.
3. Create a sub directory, *repo*, to place AccessMatrix base package **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**.
4. Use `docker build` to build image:

Command

```
$ docker build . -f Dockerfile -t am5:5.7.5
```

Response

```
Sending build context to Docker daemon 487MB
Step 1/9: FROM adoptopenjdk/openjdk11:slim
  ---> 929ec6e49e18
Step 2/9 : ENV AM_VERSION "5.7.5.7535-GA"
  ---> Using cache
  ---> b0d0c2c01f60
Step 3/9 : RUN mkdir -p /opt/am/${AM_VERSION}
  ---> Using cache
```

Command

```
  ---> de8869ca3cdb
Step 4/9 : ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}
  ---> Using cache
  ---> 8af986fad67c
Step 5/9 : RUN groupadd am && useradd -r -g am -s /bin/bash am && chown -R am:am
/opt/am/${AM_VERSION}&& chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh && ln -s
/opt/am/${AM_VERSION} /opt/am/am && chown -R am:am /opt/am/am
  ---> Using cache
  ---> 7db6babe5785
Step 6/9 : COPY ["lib", "/opt/am/am/tomcat/lib"]
  ---> 37d2ed664232
Step 7/9 : EXPOSE 8080 8443 8162 61617
  ---> Running in fb9131b193eb
Removing intermediate container fb9131b193eb
  ---> c2569dc2d46c
Step 8/9 : USER am
  ---> Running in 1325411889c0
Removing intermediate container 1325411889c0
  ---> cebaba38368f
Step 9/9 : CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
  ---> Running in 94fc2ed17e1a
Removing intermediate container 94fc2ed17e1a
  ---> 915b854f2836
Successfully built 915b854f2836
Successfully tagged am:5.7.5
```

Build AccessMatrix Docker Image in RedHat Universal Base Image 8 (UBI 8)

The following is a sample AccessMatrix Dockerfile build using Red Hat Enterprise Linux (RHEL) Universal Base Image 8 (UBI 8), with manual installation of Java Environment.

1. Place the Dockerfile in any working directory that you choose. Then, change the environment variable AM_VERSION to the respective AccessMatrix version.

Dockerfile

Code

```
FROM redhat/ubi8

ENV AM_VERSION "5.7.5.7535-GA"
ENV JAVA_VERSION "8u301-linux-x64"
ENV JDK_VERSION "jdk1.8.0_301"

ENV JAVA_HOME "/opt/jdk/${JDK_VERSION}"
ENV PATH "$PATH:/opt/jdk/${JDK_VERSION}/bin"

USER 0
```

Dockerfile

```
RUN mkdir -p /opt/jdk/
RUN mkdir -p /opt/am/${AM_VERSION}

ADD repo/jdk-${JAVA_VERSION}.tar.gz /opt/jdk/
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}


RUN groupadd -g 2020 am \
    && useradd -r -g am -u 2021 -s /bin/bash am \
    && chown -R am:am /opt/am/${AM_VERSION} \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:am /opt/am/am


COPY ["lib", "/opt/am/am/tomcat/lib"]

EXPOSE 8080 8443 8161 61616
USER am
CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
```

2. Create a subdirectory, *lib*, to place all dependent libraries, e.g. JDBC drivers.
3. Create a subdirectory, *repo*, to place AccessMatrix base package **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**.
4. Place the Java JDK archive **jdk-${JAVA_VERSION}.tar.gz** in *repo*.
5. Use `docker build` to build image:

Command

```
$ docker build . -f Dockerfile -t am5:5.7.5
```

Response

```
Sending build context to Docker daemon 156MB
Step 1/16 : FROM redhat/ubi8
  ---> ad42391b9b46
Step 2/16 : ENV AM_VERSION "5.7.5.7535-GA"
  ---> Using cache
  ---> 673f17444946
Step 3/16 : ENV JAVA_VERSION "8u301-linux-x64"
  ---> Running in ef2b9da9dadb
Removing intermediate container ef2b9da9dadb
  ---> 6b3860c722eb
Step 4/16 : ENV JDK_VERSION "jdk1.8.0_301"
  ---> Running in 9ed6bb0200a1
Removing intermediate container 9ed6bb0200a1
  ---> 658c9387f3e8
Step 5/16 : ENV JAVA_HOME "/opt/jdk/${JDK_VERSION}"
  ---> Running in e7741a2c3596
```

Command
Removing intermediate container e7741a2c3596   ---> 7720fc2f91ff   ---> Running in b15fae561b21 Removing intermediate container b15fae561b21   ---> 7a3e8e58fb9f Step 7/16 : USER 0   ---> Running in 2f8ef45ca034 Removing intermediate container 2f8ef45ca034   ---> 9b33d21a4c5f Step 8/16 : RUN mkdir -p /opt/jdk/   ---> Running in fe4fb69a97f7 Removing intermediate container fe4fb69a97f7   ---> 3912c0b890af Step 9/16 : RUN mkdir -p /opt/am/${AM_VERSION}   ---> Running in ae3bf08ce982 Removing intermediate container ae3bf08ce982   ---> 3496e2c69c46 Step 10/16 : ADD repo/jdk-${JAVA_VERSION}.tar.gz /opt/jdk/ Step 11/16 : ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz /opt/am/${AM_VERSION}   ---> b792174dac85 Step 12/16 : RUN groupadd -g 2020 am && useradd -r -g am -u 2021 -s /bin/bash am && chown -R am:am /opt/am/${AM_VERSION}&& chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh && ln -s /opt/am/${AM_VERSION} /opt/am/am && chown -R am:am /opt/am/am   ---> Running in 5e9cc425d11a Removing intermediate container 5e9cc425d11a   ---> 94168b6a6e8f Step 13/16 : COPY ["lib", "/opt/am/am/tomcat/lib"]   ---> 0d63a6514333 Step 14/16 : EXPOSE 8080 8443 8161 61616   ---> Running in 5ab68b7615a8 Removing intermediate container 5ab68b7615a8   ---> 06c911148d4c Step 15/16 : USER am   ---> Running in d99a5b893972 Removing intermediate container d99a5b893972   ---> f44b70dbfed5 Step 16/16 : CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]   ---> Running in 8d4e07008898 Removing intermediate container 8d4e07008898   ---> bc9fe96fc176 Successfully built bc9fe96fc176 Successfully tagged am:5.7.5

Build AccessMatrix Docker Image in RedHat Universal Base Image 8 (UBI 8) with OpenJDK

The AccessMatrix Dockerfile can also be built using OpenJDK, at risk to the overall build time for installation of OpenJDK.

Dockerfile

Code

```
FROM redhat/ubi8

ENV AM_VERSION "5.7.5.7535-GA"

USER 0

RUN mkdir -p /opt/am/${AM_VERSION}
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}

RUN yum update \
    && yum install java-11-openjdk.x86_64 -y \
    && yum clean all

RUN groupadd -g 2020 am \
    && useradd -r -g am -u 2021 -s /bin/bash am \
    && chown -R am:am /opt/am/${AM_VERSION} \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:am /opt/am/am


COPY ["lib", "/opt/am/am/tomcat/lib"]

EXPOSE 8080 8443 8161 61616
USER am
CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
```

Note: Image build time will increase due to the installation of OpenJDK. Alternatively, the builder can opt for using the official ubi8 openjdk-11 container image.

Push AccessMatrix Docker Image to Registry

1. Use `aws ecr get-login-password` to retrieve the auth token then use docker login to log in before pushing the image:

Parameters

```
# Get container registry id and store in an environment variable for ease of
reference.
$ AMRegistryId=$(aws ecr describe-registry --query registryId --output text)
```

Command

```
$ aws ecr get-login-password --region ap-southeast-1 | \
    docker login --username AWS --password-stdin $AMRegistryId.dkr.ecr.ap-
southeast-1.amazonaws.com
```

Parameters
Response
WARNING! Your password will be stored unencrypted in /home/ismail/.docker/config.json. Configure a credential helper to remove this warning. See https://docs.docker.com/engine/reference/commandline/login/#credentials-store Login Succeeded

2. Use `docker tag` and `docker push` to tag the docker image then push to the registry (e.g. Amazon Elastic Container Registry):

Parameters
`# Set image registry name and store in an environment variable for ease of reference.` `$ AMImageRegistryName=am5` `# Get container registry id.` `$ AMImageRegistryUri=$(aws ecr describe-repositories --repository-name $AMImageRegistryName --query repositories[0].repositoryUri --output text)`
Command
`$ docker tag am5:5.7.5 $AMImageRegistryUri:5.7.5` `$ docker image ls`
Response
REPOSITORY TAG IMAGE ID CREATED SIZE 021215294315.dkr.ecr.ap-southeast-1.amazonaws.com/am5 5.7.5 915b854f2836 17 minutes ago 758MB am5 5.7.5 915b854f2836 17 minutes ago 758MB

Parameters
`# Set image registry name and store in an environment variable for ease of reference.` `$ AMImageRegistryName=am5` `# Get container registry id.` `$ AMImageRegistryUri=$(aws ecr describe-repositories --repository-name $AMImageRegistryName --query repositories[0].repositoryUri --output text)`
Command
`$ docker image push $AMImageRegistryUri:5.7.5`

Parameters
Response
The push refers to repository [021215294315.dkr.ecr.ap-southeast-1.amazonaws.com/am5] 247946ea3c85: Pushed 047dc610d4f8: Pushed f1289005a4b3: Pushed 66ff9452d4c6: Pushed 5d0f1dae109c: Pushed 1b853245b739: Pushed f801f021a84a: Pushed 789c82f5f442: Pushed 3e2a3ebdb62c: Pushed e8243c609e31: Pushed cfb7bbd3b9bc: Pushed 51a0cde65eeb: Pushed 16542a8fc3be: Pushed 6597da2e2e52: Pushed 977183d4e999: Pushed c8be1b8f4d60: Pushed 5.7.5: digest: sha256:a977f44de80368bd83511456f8facd2375da70691ac62dd23d49afade326ac2f size: 3672

Important Note: Version tagging of images is important as tagging with the "latest" tag can lead to problems in a managed environment. According to Kubernetes docs, labelling all images with the latest makes it difficult to track the current version of the image running and makes it difficult to roll back when errors occur.

3.3.4. Create and Deploy Kubernetes Objects

At this stage, the AccessMatrix Docker image has been configured properly and pushed to the container registry. Now you are ready to create/deploy Kubernetes objects.

Kubernetes objects are persistent entities in the Kubernetes system. Kubernetes uses these entities to represent the state of your cluster. We describe the Kubernetes object using YAML files. For any YAML files listed in this chapter, use command `kubectl apply -f <file-name>.yaml` to create Kubernetes objects in the cluster. For example:

Command
`$ kubectl apply -f am-k8s.yaml`
Response
`deployment.apps/am-k8s created`

Persistent Volumes and Claims

A Persistent Volume (PV) is a piece of storage in the cluster that an administrator has provisioned or dynamically provisioned using Storage Classes. It is a resource in the cluster, just like a node is a cluster resource. PVs are volume plugins similar to Docker Volumes but have a lifecycle independent of any individual pod that uses the PV. This API object captures the details of the storage implementation, be that NFS, iSCSI, or a cloud-provider-specific storage system.

A Persistent Volume Claim (PVC) is a request for storage by a user (refer to architecture diagram). It is similar to a Pod so that Pods consume node resources, and PVCs consume PV resources. Pods can request specific levels of resources (CPU and Memory). PVC can request specific size and access modes (e.g., they can be mounted once read/write or many times read-only).

Usage of PV (Persistent Volume) and PVC (Persistent Volume Claim) for AccessMatrix deployed in Kubernetes is to have persistent centralized storage of log files generated in multiple AccessMatrix instances/Pods.

AWS has various storage class options. The PV and PVC YAML below shows an example of Amazon Elastic File System that can be mounted as read-write by many nodes to be claimed as the logs folder:

1. Use `kubectl apply -k` to deploy Amazon EFS CSI Driver in the EKS cluster.

Parameters

```
# Set aws csi driver uri and store in an environment variable for ease of
reference.
$ AWSCSIDriverUri="github.com/kubernetes-sigs/aws-efs-csi-
driver/deploy/kubernetes/overlays/stable/ecr/?ref=release-1.3"
```

Command

```
$ kubectl apply -k "$AWSCSIDriverUri"
```

Response

```
daemonset.apps/efs-csi-node created
csidriver.storage.k8s.io/efs.csi.aws.com configured
```

2. Use `aws ec2 create-security-group` to create security group for Amazon EFS storage:

Parameters

```
# Set security group name and store in an environment variable for ease of
reference.
$ EFSSecurityGroupName=am-efs-security-group

# Set security group description and store in an environment variable for ease
of reference.
$ EFSSecurityGroupDesc="AccessMatrix EFS Security Group"

# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster

# Get AccessMatrix VPC id by cluster name and store in an environment variable
for ease of reference.
$ AccessMatrixVPCID=$(aws eks describe-cluster --name $EKSClusterName --query
"cluster.resourcesVpcConfig.vpcId" --output text)
```

Command

```
$ aws ec2 create-security-group \
    --group-name $EFSSecurityGroupName\
    --description "$EFSSecurityGroupDesc"\
    --vpc-id $AccessMatrixVPCID
```

Response

```
{
  "GroupId": "sg-09029e58cdac6f43f"
}
```

3. Use `aws ec2 authorize-security-group-ingress` to allow AccessMatrix to access the Amazon EFS storage using TCP and UDP:

Parameters

```
# Set security group name and store in an environment variable for ease of
reference.
$ EFSSecurityGroupName=am-efs-security-group

# Get security group id by name and store in an environment variable for ease
of reference.
$ EFSSecurityGroupID=$(aws ec2 describe-security-groups --filter Name=group-
name,Values=$EFSSecurityGroupName\
    --query SecurityGroups[0].GroupId --output text)

# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster

# Get AccessMatrix VPC id by cluster name and store in an environment variable
for ease of reference.
$ AccessMatrixVPCID=$(aws eks describe-cluster --name $EKSClusterName --query
"cluster.resourcesVpcConfig.vpcId" --output text)

# Get AccessMatrix CIDR by vpc-id and store in an environment variable for ease
of reference.
$ AccessMatrixCIDR=$(aws ec2 describe-vpcs --vpc-id $AccessMatrixVPCID --query
Vpcs[0].CidrBlock --output text)
```

Command

```
$ aws ec2 authorize-security-group-ingress \
    --group-id $EFSSecurityGroupID\
    --protocol tcp \
    --port 2049 \
    --cidr $AccessMatrixCIDR
```

Command

```
$ aws ec2 authorize-security-group-ingress \
    --group-id $EFSSecurityGroupID\
    --protocol udp \
    --port 2049 \
    --cidr $AccessMatrixCIDR
```

4. Use `aws efs create-file-system` to create empty file system:

Parameters

```
# Set EFS filesystem name and store in an environment variable for ease of
reference.
$ EFSName=am-file-system
```

Command

```
$ aws efs create-file-system \
    --performance-mode generalPurpose \
    --throughput-mode bursting \
```

Parameters

```
    --encrypted \
    --tags Key=Name,Value=$EFSName
```

Response


```
{
  "OwnerId": "021215294315",
  "CreationToken": "ab669f0e-3214-444b-8bb4-ce95d25a0938",
  "FileSystemId": "fs-6f13fa2f",
  "FileSystemArn": "arn:aws:elasticfilesystem:ap-southeast-1:021215294315:file-
system/fs-6f13fa2f",
  "CreationTime": "2021-01-18T16:26:34+07:00",
  "LifeCycleState": "creating",
  "Name": "am-file-system",
  "NumberOfMountTargets": 0,
  "SizeInBytes": {
  "Value": 0,
    "ValueInIA": 0,
    "ValueInStandard": 0
  },
  "PerformanceMode": "generalPurpose",
  "Encrypted": true,
  "KmsKeyId": "arn:aws:kms:ap-southeast-1:021215294315:key/eb03301f-0890-4812-
a46e-0479da5c8130",
  "ThroughputMode": "bursting",
  "Tags": [
    {
    "Key": "Name",
    "Value": "am-file-system"
    }
  ]
}
```


Note: Copy the value of `FileSystemId` and set to `EFSID` parameter in the following steps.

5. Use `aws efs create-mount-target` to create mount target on each subnet:

Parameters

```
# Set EFS filesystem name and store in an environment variable for ease of
reference.
$ EFSName=am-file-system

# Set security group name and store in an environment variable for ease of
reference.
$ EFSSecurityGroupName=am-efs-security-group

# Get file system id and store in an environment variable for ease of
reference.
$ EFSID=$(aws efs describe-file-systems --query
'FileSystems[?Name==`'$EFSName'`].FileSystemId' --output text)

# Get security group id by name and store in an environment variable for ease
```

Parameters

```
of reference.
$ EFSSecurityGroupID=$(aws ec2 describe-security-groups --filter Name=group-
name,Values=$EFSSecurityGroupName\
    --query SecurityGroups[0].GroupId --output text)

# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster

# Get AccessMatrix VPC id by cluster name and store in an environment variable
for ease of reference.
$ AccessMatrixVPCID=$(aws eks describe-cluster --name $EKSClusterName --query
"cluster.resourcesVpcConfig.vpcId" --output text)

# Get private subnet of AccessMatrix VPC in ap-southeast-1a and store in an
environment variable for ease of reference.
$ AccessMatrixPrivateSubnet1A=$(aws ec2 describe-subnets \
    --filter Name=vpc-id,Values=$AccessMatrixVPCIDName=availability-
zone,Values=ap-southeast-1a \
    --query 'Subnets[?MapPublicIpOnLaunch==`false`].SubnetId' --output text)
```

Command

```
$ aws efs create-mount-target \
    --file-system-id $EFSID\
    --security-groups $EFSSecurityGroupID\
    --subnet-id $AccessMatrixPrivateSubnet1A
```

Response

```
{
  "OwnerId": "021215294315",
  "MountTargetId": "fsmt-1745d756",
  "FileSystemId": "fs-6f13fa2f",
  "SubnetId": "subnet-02d27bca4c1104611",
  "LifeCycleState": "creating",
  "IpAddress": "192.168.130.148",
  "NetworkInterfaceId": "eni-00c233404d945d21a",
  "AvailabilityZoneId": "apse1-az2",
  "AvailabilityZoneName": "ap-southeast-1a",
  "VpcId": "vpc-05982544206122bba"
}
```

Parameters

```
# Set EFS filesystem name and store in an environment variable for ease of
reference.
$ EFSName=am-file-system

# Set security group name and store in an environment variable for ease of
reference.
$ EFSSecurityGroupName=am-efs-security-group
```

Parameters

```
# Get file system id and store in an environment variable for ease of
reference.
$ EFSID=$(aws efs describe-file-systems --query
'FileSystems[?Name==`'$EFSName'`].FileSystemId' --output text)

# Get security group id by name and store in an environment variable for ease
of reference.
$ EFSSecurityGroupID=$(aws ec2 describe-security-groups --filter Name=group-
name,Values=$EFSSecurityGroupName\
    --query SecurityGroups[0].GroupId --output text)

# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster

# Get AccessMatrix VPC id by cluster name and store in an environment variable
for ease of reference.
$ AccessMatrixVPCID=$(aws eks describe-cluster --name $EKSClusterName --query
"cluster.resourcesVpcConfig.vpcId" --output text)

# Get private subnet of AccessMatrix VPC in ap-southeast-1b and store in an
environment variable for ease of reference.
$ AccessMatrixPrivateSubnet1B=$(aws ec2 describe-subnets \
    --filter Name=vpc-id,Values=$AccessMatrixVPCIDName=availability-
zone,Values=ap-southeast-1b \
    --query 'Subnets[?MapPublicIpOnLaunch==`false`].SubnetId' --output text)
```

Command

```
$ aws efs create-mount-target \
    --file-system-id $EFSID\
    --security-groups $EFSSecurityGroupID\
    --subnet-id $AccessMatrixPrivateSubnet1B
```

Response


```
{
  "OwnerId": "021215294315",
  "MountTargetId": "fsmt-2e45d76f",
  "FileSystemId": "fs-6f13fa2f",
  "SubnetId": "subnet-0411bcda1cd1fc6e6",
  "LifeCycleState": "creating",
  "IpAddress": "192.168.172.156",
  "NetworkInterfaceId": "eni-0b56305b5f460ef86",
  "AvailabilityZoneId": "apse1-az1",
  "AvailabilityZoneName": "ap-southeast-1b",
  "VpcId": "vpc-05982544206122bba"
}
```


Parameters

```
# Set EFS filesystem name and store in an environment variable for ease of
reference.
$ EFSName=am-file-system
```

Parameters

```
# Set security group name and store in an environment variable for ease of
reference.
$ EFSSecurityGroupName=am-efs-security-group

# Get file system id and store in an environment variable for ease of
reference.
$ EFSID=$(aws efs describe-file-systems --query
'FileSystems[?Name==`'$EFSName'`].FileSystemId' --output text)

# Get security group id by name and store in an environment variable for ease
of reference.
$ EFSSecurityGroupID=$(aws ec2 describe-security-groups --filter Name=group-
name,Values=$EFSSecurityGroupName\
  --query SecurityGroups[0].GroupId --output text)

# Set Cluster Name and store in an environment variable for ease of reference.
$ EKSClusterName=AMEksCluster

# Get AccessMatrix VPC id by cluster name and store in an environment variable
for ease of reference.
$ AccessMatrixVPCID=$(aws eks describe-cluster --name $EKSClusterName --query
"cluster.resourcesVpcConfig.vpcId" --output text)

# Get private subnet of AccessMatrix VPC in ap-southeast-1c and store in an
environment variable for ease of reference.
$ AccessMatrixPrivateSubnet1C=$(aws ec2 describe-subnets \
  --filter Name=vpc-id,Values=$AccessMatrixVPCIDName=availability-
zone,Values=ap-southeast-1c \
  --query 'Subnets[?MapPublicIpOnLaunch==`false`].SubnetId' --output text)
```

Command

```
$ aws efs create-mount-target \
  --file-system-id $EFSID\
  --security-groups $EFSSecurityGroupID\
  --subnet-id $AccessMatrixPrivateSubnet1C
```

Response

```
{
  "OwnerId": "021215294315",
  "MountTargetId": "fsmt-0545d744",
  "FileSystemId": "fs-6f13fa2f",
  "SubnetId": "subnet-00d1c1477de545222",
  "LifeCycleState": "creating",
  "IpAddress": "192.168.115.151",
  "NetworkInterfaceId": "eni-04f92b40b1ca34c1f",
  "AvailabilityZoneId": "apse1-az3",
  "AvailabilityZoneName": "ap-southeast-1c",
  "VpcId": "vpc-05982544206122bba"
}
```

6. Prepare the following yaml file to deploy persistent volume and storage:

am-pv-aws.yaml

YAML

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: efs-pv
spec:
  capacity:
    storage: 1Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
  storageClassName: efs-sc
  mountOptions:
    - rsize=1048576
    - wsize=1048576
    - hard
    - timeo=600
    - retrans=2
    - noresvport
    - _netdev
  csi:
    driver: efs.csi.aws.com
    volumeHandle: <EFS File System ID>
---
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: efs-sc
provisioner: efs.csi.aws.com
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: logs
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: efs-sc
  resources:
    requests:
      storage: 1Gi
```

Note: The mount options are based on AWS Recommended NFS Mount Options.

7. Use `kubectl apply -f` to deploy the PV and PVC.

Command

```
$ kubectl apply -f am-pv-aws.yaml
```

Response

Command

```
persistentvolume/efs-pv created
storageclass.storage.k8s.io/efs-sc created
persistentvolumeclaim/logs created
```

Important Note: While there are multiple storage access modes, note that when using ReadWriteMany means the volume can be mounted as read-write by many nodes.

If there are multiple deployments, it might not be desirable for all the deployments to share the same volume. To separate the logs based on deployment, repeat steps 4 and 5 to create new EFS File System in AWS with a different name (EFS Name) then create a Persistent Volume with volumeHandle set to the new EFS file system id such as:

am-deployment-blabla-pvc.yaml

YAML

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: efs-pv-blabla
spec:
  capacity:
    storage: 1Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
  storageClassName: efs-sc-blabla
  mountOptions:
    - rsize=1048576
    - wsize=1048576
    - hard
    - timeo=600
    - retrans=2
    - noresvport
    - _netdev
  csi:
    driver: efs.csi.aws.com
    volumeHandle: <New EFS File System ID>
---
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: efs-sc-blabla
provisioner: efs.csi.aws.com
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: logs-blabla
```

am-deployment-blabla-pvc.yaml

```
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: efs-sc-blabla
  resources:
    requests:
      storage: 1Gi
```

Temporary Report Directory

When the AccessMatrix system receives a request to generate a report, the system will generate a report resulting in any of these report files: CSV, PDF, or HTML. The generated report file(s) will be stored temporarily in a reports folder under this directory: *<ACMATRIX_HOME>/tomcat/work/*. Example: *opt/am/am/tomcat/work/reports*.

This PVC needs to be created and mounted as part of the deployment to allow report generation and download requests to be served by different pods. Repeat steps 4 and 5 of the Persistent Volumes and Claims section to create a new EFS File System in AWS with a different name (EFS Name). Then, create a Persistent Volume with the volumeHandle set to the new EFS file system ID.

Below is a sample YAML file for creating PVC for the reports:

am-pv-aws-reports.yaml

YAML

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: efs-pv-report
spec:
  capacity:
    storage: 1Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
  storageClassName: efs-sc-report
  mountOptions:
    - rsize=1048576
    - wsize=1048576
    - hard
    - timeo=600
    - retrans=2
    - noresvport
```

am-pv-aws-reports.yaml

```
    - _netdev
  csi:
    driver: efs.csi.aws.com
    volumeHandle: <new EFS File System ID>
---
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: efs-sc-report
provisioner: efs.csi.aws.com
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: amreports
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: efs-sc-report
  resources:
    requests:
      storage: 1Gi
```

Use `kubectl apply -f` to deploy the PV and PVC:

Command

```
$ kubectl apply -f am-pv-aws-reports.yaml
```

Response

```
persistentvolume/efs-pv-report created
storageclass.storage.k8s.io/efs-sc-report created
persistentvolumeclaim/amreports created
```

Install NGINX Ingress Controller

Ingress exposes HTTP and HTTPS routes from outside the cluster to services within the cluster. Traffic routing is controlled by rules defined on the Ingress resource.

Referring to Domain name and TLS Connection section again, in a deployment where TLS connection ends in AccessMatrix Pods, valid and signed certificates must be configured in **server.xml**.

For TLS Connection that ends in pods, you need to enable ssl-pass-through in the ingress controller. All SSL/TLS requests will end in instances/Pods.

Install Helm

Helm is the package manager for Kubernetes. We use helm to install Ingress in the Kubernetes cluster.

Helm provides a script that will automatically download and install the latest Helm release in your local machine.

Parameters

```
# Set helm package URL and store in an environment variable for ease of reference.
$ HelmPackageUri=https://raw.githubusercontent.com/helm/helm/master/scripts/get-helm-3
```

Command

```
$ curl -fsSL -o get_helm.sh $HelmPackageUri
$ chmod 700 get_helm.sh
$ ./get_helm.sh
```

Response

```
Downloading https://get.helm.sh/helm-v3.5.1-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
```

Create namespace and add Helm Repo

Command

```
# Create a namespace for your ingress resources
kubectl create namespace ingress-basic
# Add the official stable repo
helm repo add stable https://charts.helm.sh/stable
```

Install Ingress Controller with SSL-Passthrough Enabled

1. For SSL-Passthrough, `enable-ssl-passthrough` must be set when creating the Ingress controller:

   Command

   ```
   $ helm install nginx-ingress stable/nginx-ingress \
       --namespace ingress-basic \
       --set controller.replicaCount=2 \
       --set controller.nodeSelector."beta\.kubernetes\.io/os"=linux \
       --set defaultBackend.nodeSelector."beta\.kubernetes\.io/os"=linux \
       --set controller.extraArgs.enable-ssl-passthrough=""
   ```

Command
Response
NAME: nginx-ingress LAST DEPLOYED: Mon May 11 11:04:14 2020 NAMESPACE: ingress-basic STATUS: deployed REVISION: 1 TEST SUITE: None NOTES: The nginx-ingress controller has been installed. It may take a few minutes for the LoadBalancer IP to be available. You can watch the status by running 'kubectl ~-~-namespace ingress-basic get services -o wide -w nginx-ingress-controller'

2. Upon successful installation, you will be able to see the external IP of your Ingress:

Command
`$ kubectl get service -l app=nginx-ingress --namespace ingress-basic`
Response
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE nginx-ingress-controller LoadBalancer 10.0.182.99 <External-IP> 80:30896/TCP,443:31201/TCP 24s nginx-ingress-default-backend ClusterIP 10.0.30.173 <none> 80/TCP 24s

3. Add a DNS record from your domain name(e.g. *ams-k8s-ssl.i-sprint.com*) of the SSL certificate to `<External-IP>`.

AccessMatrix Deployment

We deploy only one AccessMatrix container per Pod. In this case, Pod is a wrapper around a single container while Kubernetes manages the Pods rather than the containers directly.

As explained in Domain name and TLS Connection section, the TLS Connection will end in AccessMatrix.

At this stage AccessMatrix must have the valid signed certificate configured in server.xml. In the next subsection, the Deployment YAML file for this TLS scenario is shown. The AccessMatrix deployment will have:

- Two Pod replicas.
- Request for 100m CPU, which is equivalent to 10% of 1vCPU. For further information, read Kubernetes Resource Units.

Kubernetes Secrets

Secrets can be used like ConfigMap and are used to store sensitive information such as passwords. Pods in a deployment can refer to the secret as environment variables as well. For example, use Kubernetes Secrets to reference credentials:

Command
`kubectl create secret generic am-credentials --from-file=DBCRED`
Response
`secret/am-credentials created`

By default, Kubernetes uses the filename of the text file as the key. In this case, DBCRED is the key, and the value will be the file's contents, the user salt used to encrypt the database passphrase.

Database Initialization

To avoid the concurrent initialization of the AccessMatrix database because of multiple replicas, a database administrator must initialize the AccessMatrix database with a pod. The pod will run a script to initialize the server and close, and it must be run before deployment of AccessMatrix.

Below is a sample YAML for a Database Initialization pod.

am-dbinit.yaml

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: dbinit
spec:
  securityContext:
    runAsUser: 0
  initContainers:
  - name: logs-init
    image: <container-registry-server>/am5:<version>
    securityContext:
     runAsUser: 0
    command: ['sh', '-c', 'chown -R am:am
opt/am/am/tomcat/logs']
    volumeMounts:
    - mountPath: opt/am/am/tomcat/logs
```

am-dbinit.yaml

```
      name: logs
  containers:
  - image: <container-registry-server>/am5:<version>
    name: dbinit
    env:
    - name: DBCRED
      valueFrom:
        secretKeyRef:
          key: DBCRED
          name: am-credentials
    command: [ "/bin/sh", "-c"]
    args: ["cd opt/am/am/tomcat/db-init; ./dbUtil.sh;"]
    volumeMounts:
    - mountPath: opt/am/am/tomcat/logs
      name: logs
  volumes:
  - name: logs
    persistentVolumeClaim:
      claimName: "logs"
  restartPolicy: Never
```

Deployment Where TLS Connection Ends in AccessMatrix

Below is a sample Deployment YAML for AccessMatrix where the TLS connection ends in AccessMatrix Pods (which means the connection from the Ingress controller to the AccessMatrix Pods will still be TLS encrypted):

am-k8s-ssl.yaml

YAML

```
apiVersion: "apps/v1"
kind: "Deployment"
metadata:
  name: "am-k8s"
  namespace: "default"
  labels:
    app: "am-k8s"
spec:
  replicas: 2
  selector:
    matchLabels:
      app: "am-k8s"
  template:
    metadata:
      labels:
        app: "am-k8s"
    spec:
      initContainers:
```

am-k8s-ssl.yaml

```
      - name: logs-init
        image: <container-registry-server>/am5:<version>
        securityContext:
          runAsUser: 0
        command: ['sh', '-c', 'chown -R am:am
opt/am/am/tomcat/logs']
        volumeMounts:
        - mountPath: opt/am/am/tomcat/logs
          name: logs
      - name: reports-init
        image: <container-registry-server>/am5:<version>
        securityContext:
          runAsUser: 0
        command: ['sh', '-c', 'chown -R am:am
opt/am/am/tomcat/work/reports']
        volumeMounts:
        - mountPath: opt/am/am/tomcat/work/reports
          name: amreports
      containers:
      - name: am5
        image: <container-registry-server>/am5:<version>
        env:
        - name: DBCRED
          valueFrom:
            secretKeyRef:
              key: DBCRED
              name: am-credentials
        readinessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        livenessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        ports:
        - containerPort: 8080
          protocol: TCP
        - containerPort: 8443
          protocol: TCP
        volumeMounts:
        - mountPath: opt/am/am/tomcat/logs
          name: logs
        - mountPath: opt/am/am/tomcat/work/reports
          name: amreports
        securityContext:
          privileged: true
```

am-k8s-ssl.yaml

```
        resources:
          requests:
            cpu: 100m
      volumes:
      - name: logs
        persistentVolumeClaim:
          claimName: logs
      - name: amreports
        persistentVolumeClaim:
          claimName: amreports
```

Ingress Service Where TLS Connection Ends in AccessMatrix

In this configuration, the TLS connection ends in AccessMatrix instance/pods, which means it just passes through the Ingress Controller.

TLS certificate and corresponding **server.xml** must be updated correctly in AccessMatrix package. Refer to:

- TLS Connection Ends in AccessMatrix section to update **server.xml**
- **am-k8s-ssl.yaml** for sample of Deployment YAML.

Add a DNS record from your domain name(e.g. *k8s-ssl.i-sprint.com*) of the SSL certificate to `<External-IP>`.

Create an Ingress Route

Create **am-ingress-ssl.yaml**; this will create node service to expose port 8443 internally and an Ingress route for *k8s.i-sprint.com*:

am-ingress-ssl.yaml

YAML

```
apiVersion: v1
kind: Service
metadata:
  name: am-node-service
spec:
  ports:
- name: am-port-ssl
    port:443
```

am-ingress-ssl.yaml

```
    protocol: TCP
    targetPort:8443
  selector:
    app: am-k8s
  type: ClusterIP
---
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: am-ingress-ssl
  annotations:
    kubernetes.io/ingress.class: nginx
    nginx.ingress.kubernetes.io/app-root: /am5
    nginx.ingress.kubernetes.io/backend-protocol:"HTTPS"
    nginx.ingress.kubernetes.io/ssl-redirect:"true"
    nginx.ingress.kubernetes.io/ssl-passthrough:"true"
spec:
  rules:
- host: k8s-ssl.i-sprint.com
    http:
      paths:
- path:/
        backend:
          serviceName: am-node-service
          servicePort: am-port-ssl
```

Horizontal Pod Autoscaler - AccessMatrix

We will use the Horizontal Pod Autoscaler to automatically scales the number of AccessMatrix pods in a replication controller, deployment, replica set or stateful set based on observed CPU utilization (or, with custom metrics support, on some other application-provided metrics)

The sample **am-hpa.yaml** below defines an Horizontal Autoscaler that will scale to four Pod replicas when CPU utilization hits 80% on current Pods:

am-hpa.yaml

YAML

```
apiVersion: "autoscaling/v2beta1"
kind: "HorizontalPodAutoscaler"
metadata:
  name: "am-k8s-hpa"
  namespace: "default"
  labels:
    app: "am-k8s"
spec:
```

am-hpa.yaml

```
  scaleTargetRef:
    kind: "Deployment"
    name: "am-k8s"
    apiVersion: "apps/v1"
  minReplicas: 1
  maxReplicas: 4
  metrics:
  - type: "Resource"
    resource:
      name: "cpu"
      targetAverageUtilization: 80
```

Use `kubectl get hpa` to view current status of the scaler:

Command

```
$ kubectl get hpa
```

Response

```
NAME         REFERENCE           TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
am-k8s-hpa   Deployment/am-k8s   8%/80%    1         4         1          14d
```

Verify Kubernetes Deployment

- Use `kubectl get pods` to verify all the Pods are running.
- Visit AccessMatrix Admin Console via your domain name (e.g. *k8s-ssl.i-sprint.com*) to verify it is successfully deployed.

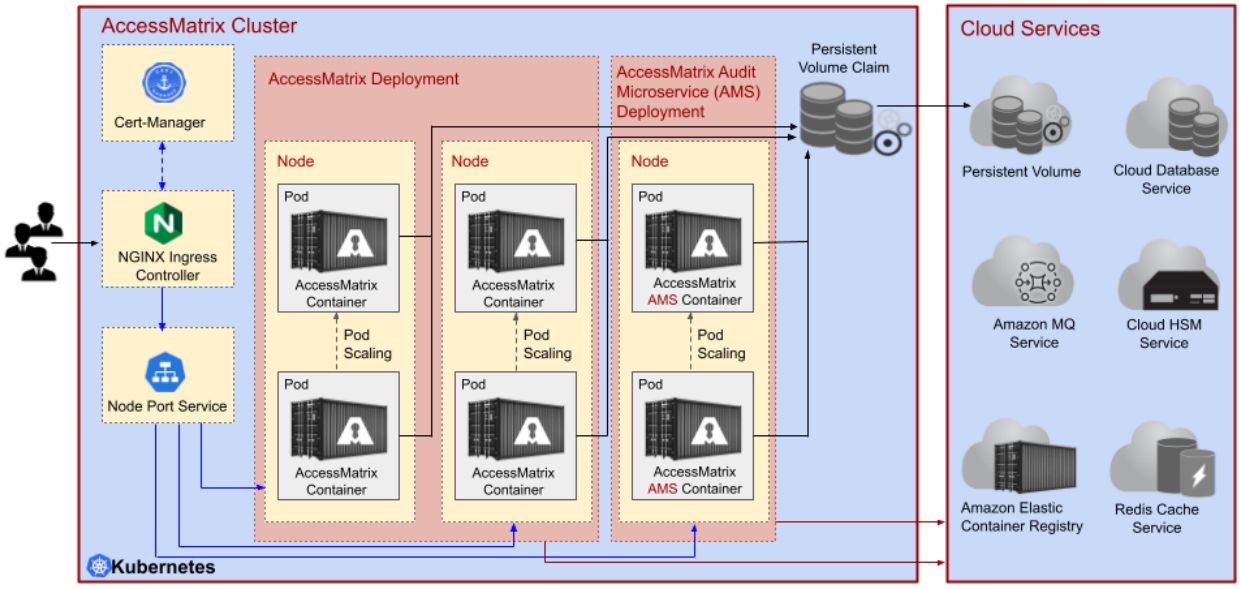
3.3.5. Advanced Deployment with Audit Microservice

Audit Micro Service (AMS) is a web application developed by i-Sprint independent from AccessMatrix. It is used to greatly improve AccessMatrix's performance during authentication by separating out audits as different microservice. When AMS is deployed, instead of directly writing into the database, AccessMatrix will send audit write requests to AMS via REST API. It is also possible to integrate AMS with the Message Queue (MQ) service to further improve the load distribution on multiple AMS instances.

As an advanced deployment option, AMS can be deployed with AccessMatrix in the same Kubernetes Cluster.

Below is an architecture diagram of a typical AccessMatrix advanced deployment with Audit Microservice in a Kubernetes cluster.

AccessMatrix Kubernetes with Audit Microservice Deployment Architecture



AccessMatrix Audit Microservice provides REST API to handle audit requests from AccessMatrix.

mTLS Connection between AccessMatrix and Audit Microservice

Mutual authentication is required for AccessMatrix instance connecting to Audit Microservice.

1. Ensure Audit Microservice's CA certificate has been trusted by AccessMatrix's truststore.

2. Prepare and place the client keystore of Audit Microservice in AccessMatrix directory. e.g. *<ACMATRIX_ROOT>/conf/testagent.keystore.*
3. Navigate to the directory *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes.*
4. Open the **amsystem.properties** file.
5. Make sure am.microservice.audit.ssl.client.keystore and am.microservice.audit.ssl.client.keystore.passphrase are set accordingly.

Create Audit Microservice Container Registry

AccessMatrix needs to call audit microservice (AMS) to write audit. A separate container/image registry need to be created as follows.

1. Use `aws ecr create-repository` to create a docker repository on AWS Elastic Container Registry (ECR):

Parameters

```
# Set image registry name and store in an environment variable for ease of reference.
$ AMAuditImageRegistryName=ams
```

Command

```
$ aws ecr create-repository \
    --repository-name $AMAuditImageRegistryName\
    --image-scanning-configuration scanOnPush=true\
    --region ap-southeast-1
```

Response

```
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:ap-southeast-1:021215294315:repository/ams",
    "registryId": "021215294315",
    "repositoryName": "ams",
    "repositoryUri": "021215294315.dkr.ecr.ap-southeast-1.amazonaws.com/ams",
    "createdAt": "2021-05-07T03:53:54+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": true
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

2. Use `aws ecr describe-repositories` to query the repository URL:

Parameters
`# Set image registry name and store in an environment variable for ease of reference.` `$ AMAuditImageRegistryName=ams`
Command
`$ aws ecr describe-repositories \` `  --repository-names $AMAuditImageRegistryName\` `  --query repositories[0].repositoryUri`
Response
`"<aws_account_id>.dkr.ecr.<region>.amazonaws.com/ams"`

Audit Microservice (AMS) Configuration

Audit Microservice (AMS) Docker image is built based on **AccessMatrix-${AM_VERSION}-tomcat-only-am-audit.tar.gz** package found in the CD image. For example, **AccessMatrix-5.7.5.7535-GA-tomcat-only-am-audit.tar.gz**.

Important: In some places, you must update AMS configuration files like **ams.properties** and **am-audit.xml**, you need to unzip the **AccessMatrix-${AM_VERSION}-tomcat-only-am-audit.tar.gz**, then rezip to build Docker image.

The next sections explain some configurations to be done in some AMS config files.

1. Configure Database Connection in Audit Microservice (AMS) to configure database connection in AMS to store audit data.
2. Configure Amazon Message Queue (MQ) Connection in Audit Microservice to configure message queue server in AMS.

Unpack Audit Microservice (AMS) Base Package

To deploy AMS in a container, you need to take the package **AccessMatrix-${AM_VERSION}-tomcat-only-am-audit.tar.gz** found in AccessMatrix CD Image, for example, **AccessMatrix-5.7.5.7535-GA-tomcat-only-am-audit.tar.gz**. As explained above, you need to change some configurations in AMS. Hence the file must be unpacked/unzipped first.

Configure Database Connection in Audit Microservice (AMS)

Audit Microservice will use a database to write audit records and update database configuration in Audit Microservice to point to the database.

The following describes few steps to configure the database in Audit Microservice, we will use the same database service as explained in Database section.

1. Navigate to the directory *<ACMATRIX_AMS_ROOT>\tomcat\conf\Catalina\localhost*.
2. Open the **am-audit.xml** file.
3. Comment out the following:

```
XML

    <Resource name="jdbc/DSam-audit" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"
        initialSize="10" maxTotal="80" maxIdle="20"
minIdle="10" maxWaitMillis="10000"
        validationQuery="SELECT COUNT(*) FROM am_vars
WHERE va_name IS NULL" testOnBorrow="true"
        username="user1" password="password"
driverClassName="org.apache.derby.jdbc.EmbeddedDriver"
                url="jdbc:derby:amdb"
   />
```

4. Uncomment the following and provide values for the properties as appropriate:

```
XML

<!-- Resource name="jdbc/DSam-audit" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"
        initialSize="10" maxWaitMillis="10000" maxTotal="80"
maxIdle="20" minIdle="10" validationQuery="select 1"
        username="changeToDbUserId" password="changeToDBPassword"
driverClassName="com.mysql.jdbc.Driver"

url="jdbc:mysql://10.1.11.85:3306/amdb?useSSL=false&amp;useUnicode=
true&amp;characterEncoding=utf8"
   />
-->
```

5. Add the EncryptorUtil factory property to allow decryption of password that is encrypted using the EncryptorUtil:

```
XML
```

```
    <Resource name="jdbc/DSam-audit" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"

factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
        initialSize="10" maxWaitMillis="10000" maxTotal="80"
maxIdle="20" minIdle="10" validationQuery="select 1"
        username="changeToDbUserId" password="changeToDBPassword"
driverClassName="com.mysql.jdbc.Driver"

url="jdbc:mysql://10.1.11.85:3306/amdb?useSSL=false&amp;useUnicode=
true&amp;characterEncoding=utf8"
   />
```

Notes:

- Replace username, password and URL with your credentials and database server URL, i.e. jdbc:mysql://amdb-instance.cfgu2tppti3v.ap-southeast-1.rds.amazonaws.com/amdb?useSSL=false&useUnicode=true&characterEncoding=utf8.
- The JDBC parameters useSSL=false is added to the URL. If you want to enable SSL, follow the instruction Using SSL with a MySQL instance.
- The database administrator is required to use the EncryptorUtil to encrypt the database password, using the "Environment Variable as user salt" option. The encrypted password will look like the following:
  AesEncryptor::env=DBCRED:HtjCuzS34QxySZMslav3umHgVFA5G5j5HCXoc7KQsUDDwyjcf/+65+756F4YJBGH,
  where env=DBCRED is the environment variable containing a user salt.
- For more information on how to use the EncryptorUtil, refer to the *AccessMatrix Common Deployment Guide - Encryptor Utility*

Update Hibernate Connection in Audit Microservice

After the database connection is configured, update the hibernate connection in the AMS properties file **ams.properties** to use the datasource.

1. Navigate to the directory *<ACMATRIX_AMS_ROOT>\tomcat\webapps\am-audit\WEB-INF\classes*.
2. Open the **ams.properties** file.
3. Comment out the following:

Properties (.properties files)

```
hibernate.connection.driver_class=org.apache.derby.jdbc.ClientDrive
r
hibernate.connection.url=jdbc:derby://localhost:1527/amdb;create=fa
lse
hibernate.connection.username=user1
hibernate.connection.password=password
```

4. Uncomment and add the following:

Properties (.properties files)

```
hibernate.dialect=com.isprint.am.hibernate.AmMySQL57InnoDBDialect
hibernate.connection.datasource=java:/comp/env/jdbc/DSam-audit
```

Note: Hibernate dialect must be set to com.isprint.am.hibernate.AmMySQL57InnoDBDialect if you are using MySQL 5.7

Configure Amazon Message Queue (MQ) Connection in Audit Microservice

You need to update the AMS properties file (ams.properties) to configure the ActiveMQ connection.

The following describes few steps to configure ActiveMQ in Audit Microservice, we will use the same AmazonMQ service as explained in Amazon MQ as Message Broker Service for Active MQ section.

1. Navigate to the directory *<ACMATRIX_AMS_ROOT>\tomcat\webapps\ams\WEB-INF\classes*.
2. Open the **ams.properties** file.
3. Uncomment and add the following:

Properties (.properties files)

```
activemq.url=tcp://192.168.50.51:61617
#-if user authentication is required, uncomment
username, password (password can be encrypted with
EncryptorUtil)
activemq.username=
activemq.password=
```

Notes:

- Replace activemq.url=tcp://192.168.50.51:61617 to your ActiveMQ domain name or IP, for example: failover:(ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-1.amazonaws.com:61617,ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-southeast-1.amazonaws.com:61617).
- Replace activemq.username and activemq.password values with the relevant credentials used when creating AMQ in the previous section for example: activemq.username=amquser1 and activemq.password=Password.1234.

Use Encryptor Util to ActiveMQ Password

The property activemq.password in the **ams.properties** file accepts AmazonMQ password as an encrypted text. The encrypted text must be generated from the EncryptorUtil.

Prepare Audit Microservice Docker Image

Audit microservice instances must be ready before any deployment of AccessMatrix instances. An Audit microservice Docker image must be built locally before being pushed to the registry.

Repack Audit Microservice Base Package

So you have prepared a database service and ActiveMQ. You also have updated related configurations in Audit Microservice. Now you need to repack it back to its respective name (**AccessMatrix-${AM_VERSION}-tomcat-only-am-audit.tar.gz**), for example, **AccessMatrix-5.7.5.7535-GA-tomcat-only-am-audit.tar.gz**.

Pre-image Build Checklist

Before building the Audit Microservice Docker Container Image, ensure the following are done:

Checklist	Description
Database service	Database service is ready and its connection information is configured in **am-audit.xml**. Refer to Configure Database Connection in Audit Microservice (AMS) for more information.
Apache ActiveMQ service	Amazon MQ service is ready and its connection information is configured in **ams.properties**. Refer to Configure Amazon Message Queue (MQ) Connection in Audit Microservice for more information.
TLS Connection Ends in AMS	The certificate and port are configured in **server.xml**. Refer to mTLS Connection between AccessMatrix and Audit Microservice for more information.
Audit Microservice Base Package	Audit Microservice base package that was previously unpacked must be repacked back to its original name.
Dependent libraries	Any additional libraries or drivers such as JDBC, or any external drivers, must be placed in the lib folder in the same directory as the Dockerfile.

Build Audit Microservice Docker Image

A sample Audit Microservice Dockerfile looks like below. Arrange the files as follows,

1. Place the Dockerfile in any working directory that you choose. Change the environment variable `AM_VERSION` to the respective AccessMatrix version.

2. Create a subdirectory lib to place all dependent libraries, e.g. JDBC drivers.

3. Create a subdirectory repo to place Audit Microservice base package **AccessMatrix-${AM_VERSION}-tomcat-only-am-audit.tar.gz**.

Dockerfile

Code

```
FROM adoptopenjdk/openjdk11:slim

ENV AM_VERSION "5.7.5.7535-GA"

RUN mkdir -p /opt/am/${AM_VERSION}
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only-am-
audit.tar.gz /opt/am/${AM_VERSION}
RUN groupadd am \
    && useradd -r -g am -s /bin/bash am \
    && chown -R am:am /opt/am/${AM_VERSION} \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:am /opt/am/am

COPY ["lib", "/opt/am/am/tomcat/lib"]

EXPOSE 8080 8443
USER am
CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]
```

4. Use `docker build` to build image:

Command

```
$ docker build . -f Dockerfile -t ams:5.7.5
```

Response

```
Sending build context to Docker daemon 315MB
Step 1/9 : FROM adoptopenjdk/openjdk11:slim
  ---> 929ec6e49e18
Step 2/9 : ENV AM_VERSION "5.7.5.7535-GA"
  ---> Running in e730b044f978
Removing intermediate container e730b044f978
  ---> 5fba0ee3ad9b
Step 3/9 : RUN mkdir -p /opt/am/${AM_VERSION}
  ---> Running in fa8d92caa7a5
Removing intermediate container fa8d92caa7a5
  ---> 6376118e9ef2
Step 4/9 : ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only-am-audit.tar.gz
/opt/am/${AM_VERSION}
  ---> f1a55ef69072
Step 5/9 : RUN groupadd am && useradd -r -g am -s /bin/bash am && chown -R am:am
/opt/am/${AM_VERSION}&& chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh && ln -s
/opt/am/${AM_VERSION} /opt/am/am && chown -R am:am /opt/am/am
  ---> Running in 9225d4cf6447
```

Command
Removing intermediate container 9225d4cf6447   ---> 6035ba26ffe0 Step 6/9 : EXPOSE 8080 8443   ---> Running in 898523ec60a8 Removing intermediate container 898523ec60a8   ---> ec687d074967 Step 7/9 : COPY ["lib", "/opt/am/am/tomcat/lib"]   ---> a14892a80162 Step 8/9 : USER am   ---> Running in a5d7a4d0d264 Removing intermediate container a5d7a4d0d264   ---> 28d5f8f1777f Step 9/9 : CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]   ---> Running in 5d3ce99839b7 Removing intermediate container 5d3ce99839b7   ---> 3e6f63a4dd8a Successfully built 3e6f63a4dd8a Successfully tagged ams:5.7.5

Push Audit Microservice Docker Image to Registry

1. Use `aws ecr get-login-password` to retrieve the auth token then use docker login to log in before pushing the image:

Parameters
`# Get container registry id and store in an environment variable for ease of reference.` `$ AMRegistryId=$(aws ecr describe-registry --query registryId --output text)`
Command
`$ aws ecr get-login-password --region ap-southeast-1 | \` `  docker login --username AWS --password-stdin $AMRegistryId.dkr.ecr.ap-southeast-1.amazonaws.com`
Response
WARNING! Your password will be stored unencrypted in /home/ismail/.docker/config.json. Configure a credential helper to remove this warning. See https://docs.docker.com/engine/reference/commandline/login/#credentials-store Login Succeeded

2. Use `docker tag` and `docker push` to tag the docker image then push to the registry (e.g. Amazon Elastic Container Registry):

Parameters

```
# Set image registry name and store in an environment variable for ease of
reference.
$ AMAuditImageRegistryName=ams

# Get container registry id.
$ AMAuditImageRegistryUri=$(aws ecr describe-repositories --repository-name
$AMAuditImageRegistryName --query repositories[0].repositoryUri --output text)
```

Command

```
$ docker tag ams:5.7.5 $AMAuditImageRegistryUri:5.7.5
$ docker image ls
```

Response

```
REPOSITORY                                                      TAG
 IMAGE ID       CREATED          SIZE
021215294315.dkr.ecr.ap-southeast-1.amazonaws.com/ams           5.7.5
 3e6f63a4dd8a   10 minutes ago   528MB
ams                                                             5.7.5
 3e6f63a4dd8a   10 minutes ago   528MB
```

Parameters

```
# Set image registry name and store in an environment variable for ease of
reference.
$ AMAuditImageRegistryName=ams

# Get container registry id.
$ AMAuditImageRegistryUri=$(aws ecr describe-repositories --repository-name
$AMAuditImageRegistryName --query repositories[0].repositoryUri --output text)
```

Command

```
$ docker image push $AMAuditImageRegistryUri:5.7.5
```

Response

```
The push refers to repository [021215294315.dkr.ecr.ap-southeast-
1.amazonaws.com/ams]
575b026926e9: Pushed
71ca247c7409: Pushed
b1f5d5607f40: Pushed
b0a5e549f2c2: Pushed
a018730cef21: Pushed
12e59530bb2a: Pushed
0f978f9f8690: Pushed
f6253634dc78: Pushed
9069f84dbbe9: Pushed
bacd3af13903: Pushed
5.7.5: digest:
```

Parameters
`sha256:09594815c15309b797d506a7d6416a8952469c5bda3c37865bbf44ccb2c9d919 size: 2417`

> **Important Note**: Version tagging of images is important as tagging with the "latest" tag can lead to problems in a managed environment. According to Kubernetes docs, labelling all images with the latest makes it difficult to track the current version of the image running and makes it difficult to roll back when errors occur.

Audit Microservice (AMS) Deployment

Deployment is the main Kubernetes Object that represents which and how an application will run in the cluster. AccessMatrix requires an audit microservice to write an Audit. Thus Audit Microservice will be deployed first before AccessMatrix.

Each Kubernetes Pod is meant to run a single instance of the application; thus, we deploy only one Audit Microservice container per Pod. In this case, Pod is a wrapper around a single container while Kubernetes manages the Pods rather than the containers directly.

As explained in Domain name and TLS Connection section, TLS Connection will be ends in Audit Microservice.

At this stage, Audit Microservice must have the valid signed certificate configured in server.xml. In the next subsection, the Deployment YAML file for this TLS scenario is shown. The Audit Microservice deployment will have:

- Two Pod replicas.
- Request for 100m CPU, which is equivalent to 10% of 1vCPU. For further information, read Kubernetes Resource Units.

Kubernetes Secrets

AMS and AccessMatrix will share the same Kubernetes Secrets. Secrets can be used like ConfigMap and are used to store sensitive information such as passwords. Pods in a deployment can refer to the secret as environment variables as well. For example, use Kubernetes Secrets to reference credentials:

Command
`kubectl create secret generic ams-credentials --from-file=DBCRED`

Command
Response
`secret/am-credentials created`

By default, Kubernetes uses the filename of the text file as the key. In this case, DBCRED the key, and the value will be the file's contents, the user salt used to encrypt the passphrase.

Deployment Where TLS Connection Ends in AMS

Below is a sample Deployment YAML for AMS where the TLS connection ends in AMS Pods (which means, from Ingress controller to AMS Pods will still be TLS encrypted):

ams-k8s-ssl.yaml

YAML

```yaml
apiVersion: "apps/v1"
kind: "Deployment"
metadata:
  name: "ams-k8s"
  namespace: "default"
  labels:
    app: "ams-k8s"
spec:
  replicas: 2
  selector:
    matchLabels:
      app: "ams-k8s"
  template:
    metadata:
      labels:
        app: "ams-k8s"
    spec:
      initContainers:
      - name: logs-init
        image: <container-registry-server>/ams:<version>
        securityContext:
          runAsUser: 0
        command: ['sh', '-c', 'chown -R am:am
/opt/am/am/tomcat/logs']
        volumeMounts:
        - mountPath: opt/am/am/tomcat/logs
          name: logs
      containers:
      - name: am5
        image: <container-registry-server>/ams:<version>
        env:
        - name: DBCRED
          valueFrom:
```

ams-k8s-ssl.yaml

```
            secretKeyRef:
              key: DBCRED
              name: am-credentials
        readinessProbe:
          exec:
            command:
            - "curl"
            - "-X POST"
            - "http://localhost:8080/am-
audit/rest/audit/?version"
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        livenessProbe:
          exec:
            command:
            - "curl"
            - "-X POST"
            - "http://localhost:8080/am-
audit/rest/audit/?version"
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        ports:
        - containerPort: 8080
          protocol: TCP
        - containerPort: 8443
          protocol: TCP
        volumeMounts:
        - mountPath: /opt/am/am/tomcat/logs
          name: logs
        - mountPath: opt/am/am/tomcat/work/reports
          name: amreports
        securityContext:
          privileged: true
        resources:
          requests:
            cpu: 100m
      volumes:
      - name: logs
        persistentVolumeClaim:
          claimName: logs
      - name: amreports
        persistentVolumeClaim:
          claimName: amreports
```

Ingress Service Where TLS Connection Ends in AMS

Refer to the architecture diagram. This configuration means a secure TLS connection from the user's endpoint (browser or mobile device) ends in Audit Microservice instances/pods. Ingress controller is

configured as SSL-Passthrough, which means any TLS connection will only pass through Ingress Controller.

TLS certificate and corresponding **server.xml** must be updated correctly in the AMS package as follow:

1. Navigate to the directory *<ACMATRIX_AMS_ROOT>\tomcat\conf\.*
2. Open the **server.xml** file.
3. Check Connector element for HTTPS port (which default to 8443).
    a. Insert your TLS certificate with the proper domain name to a keystore file, for example, **ams-k8s-ssl.keystore**.
    b. Set the keystore file name to keystoreFile parameter of Connector element in **server.xml**.

Create an Ingress Route

Create **ams-ingress-ssl.yaml**; this will create node service to expose port 8443 internally and an Ingress route for *ams-k8s-ssl.i-sprint.com*:

am-ingress-ssl.yaml

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: ams-node-service
spec:
  ports:
- name: ams-port-ssl
    port:443
    protocol: TCP
    targetPort:8443
  selector:
    app: ams-k8s
  type: ClusterIP
---
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: ams-ingress-ssl
  annotations:
    kubernetes.io/ingress.class: nginx
    nginx.ingress.kubernetes.io/app-root: /am-audit
    nginx.ingress.kubernetes.io/backend-protocol:"HTTPS"
    nginx.ingress.kubernetes.io/ssl-redirect:"true"
    nginx.ingress.kubernetes.io/ssl-passthrough:"true"
spec:
```

am-ingress-ssl.yaml

```
  rules:
- host: ams-k8s-ssl.i-sprint.com
    http:
      paths:
- path:/
        backend:
          serviceName: ams-node-service
          servicePort: ams-port-ssl
```

Horizontal Pod Autoscaler - Audit Microservice (AMS)

All audit records will be written to the database via Audit Microservice, thus the pod number must be able to autoscale based on certain CPU utilization.

The Horizontal Pod Autoscaler automatically scales the number of pods in a replication controller, deployment, replica set or stateful set based on observed CPU utilization (or, with custom metrics support, on some other application-provided metrics)

The metrics server is not deployed by default in Amazon EKS clusters. You can check for the metrics server with the following command:

Command

```
$ kubectl -n kube-system get deployment/metrics-server
```

If this command returns a NotFound error, then you must deploy the metrics server to your Amazon EKS cluster:

Parameters

```
# Set metric server deployment URL and store in an environment variable for ease of
reference.
$ AWSMetricServerUri=https://github.com/kubernetes-sigs/metrics-
server/releases/download/v0.3.7/components.yaml
```

Command

```
$ kubectl apply -f $AWSMetricServerUri
```

Response

```
clusterrole.rbac.authorization.k8s.io/system:aggregated-metrics-reader created
clusterrolebinding.rbac.authorization.k8s.io/metrics-server:system:auth-delegator
```

Parameters

```
created
rolebinding.rbac.authorization.k8s.io/metrics-server-auth-reader created
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created
serviceaccount/metrics-server created
deployment.apps/metrics-server created
service/metrics-server created
clusterrole.rbac.authorization.k8s.io/system:metrics-server created
clusterrolebinding.rbac.authorization.k8s.io/system:metrics-server created
```

The sample **am-hpa.yaml** below defines an Horizontal Autoscaler that will scale to four Pod replicas when CPU utilization hits 80% on current Pods:

am-hpa.yaml

YAML

```yaml
apiVersion: "autoscaling/v2beta1"
kind: "HorizontalPodAutoscaler"
metadata:
  name: "ams-k8s-hpa"
  namespace: "default"
  labels:
    app: "ams-k8s"
spec:
  scaleTargetRef:
    kind: "Deployment"
    name: "ams-k8s"
    apiVersion: "apps/v1"
  minReplicas: 1
  maxReplicas: 4
  metrics:
  - type: "Resource"
    resource:
      name: "cpu"
      targetAverageUtilization: 80
```

Use `kubectl get hpa` to view current status of the scaler:

Command

```
$ kubectl get hpa
```

Response

```
NAME         REFERENCE            TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
ams-k8s-hpa  Deployment/ams-k8s    8%/80%      1         4         1        14d
```

Configure Audit Microservice Connection

AccessMatrix requires writing audits to AMS, thus you need to configure the connection between AccessMatrix and AMS.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Uncomment and add following configurations:

Properties (.properties files)

```
# == Audit Microservice Configuration ==
am.microservice.audit.url = https://ams-k8s-ssl.i-sprint.com/am-audit/rest
am.microservice.audit.ssl.client.keystore = webapps/am5/WEB-INF/classes/testagent.keystore
am.microservice.audit.ssl.client.keystore.passphrase = passphrase
```

Note: In this example, AccessMatrix and AMS communication is secured by mutual authentication using the test client certificate, you may use your own client certificate in the production environment.

Rebuild and Redeploy AccessMatrix

AccessMatrix configuration has been changed at this stage, thus the AccessMatrix image must be rebuild and redeployed in kubernetes.

Redo the steps below in sequence to rebuild and redeploy AccessMatrix.

- Pre-image Build Checklist
- Build AccessMatrix Docker Image
- Push AccessMatrix Docker Image to Registry
- Deployment Where TLS Connection Ends in AccessMatrix

4. AccessMatrix OpenShift Deployment

The following pages provides detailed instructions on deploying AccessMatrix on the OpenShift platform:

- OpenShift Deployment with Azure
- On-Premise OpenShift Deployment

4.1. OpenShift Deployment with Azure

The following pages provide detailed instructions on how to deploy AccessMatrix on the OpenShift platform with Microsoft Azure:

- Prepare OpenShift Deployment Prerequisites
- Prepare Database/Cache Services and Configure AccessMatrix
- Create AccessMatrix Docker Image
- Create and Deploy Kubernetes Objects

4.1.1. Prepare OpenShift Deployment Prerequisites

You must already have a valid account with proper subscription in the Azure cloud service provider. This section explains preparing all prerequisite items on the respective cloud service provider before deploying AccessMatrix on OpenShift. Here you learn how to:

1. Access to Cloud Service Providers. This guides you through installing the command-line tool and creating a resource group to manage all the cloud objects used in this guide.
2. Create an OpenShift Cluster. This guides you through creating the OpenShift cluster, where AccessMatrix containers will be run and orchestrated.
3. Prepare Container/Image Registry. An image registry used to host the Docker images used by OpenShift cluster. Though it is to register/store Docker image, some cloud service providers use Container Terminology.

Access to Cloud Service Providers

To configure and run the OpenShift service, a command-line tool must be installed to access the cloud service provider and corresponding service accounts.

1. Install the Azure CLI.
2. Install Azure Kubernetes Service CLI:

Command
`$ sudo az aks install-cli`
Response
`[sudo] password for haoz:` `Downloading client to "/usr/local/bin/kubectl" from` `"https://storage.googleapis.com/kubernetes-` `release/release/v1.18.2/bin/linux/amd64/kubectl"` `Please ensure that /usr/local/bin is in your search PATH, so the `kubectl`command` `can be found.`

   To use the services like Azure Container Registry (ACR), Redis Cache service and others, you must first create a resource group:

Command
`$ az group create --name AksGroup \` `--location southeastasia`

Command
Response
<pre>{   "id": "/subscriptions/ff9f4ef7-c33b-454c-9844- 429361615308/resourceGroups/AKSGroup",   "location": "southeastasia",   "managedBy": null,   "name": "AKSGroup",   "properties": {   "provisioningState": "Succeeded"   },   "tags": null,   "type": "Microsoft.Resources/resourceGroups" }</pre>

3. Install the Openshift CLI.

   You will use the oc command to interact with the OpenShift cluster, and some OpenShift commands. Download the binary and unpack the archive in a directory desired. Alternatively, the Openshift CLI can be downloaded from the public link here.

Command
`$ tar xvf <file>`

   Place the oc binary on the PATH so you can access it anywhere.

> **Note**: The commands `kubectl` and `oc` are interchangeable, but oc is built on Openshift and has its own functionalities. For the purposes of this documentation, you will run all the `kubectl` commands with `oc` instead.

Create OpenShift Cluster

An OpenShift cluster is a set of node machines for running containerized applications. At a minimum, a cluster contains a worker node and a master node. The master node is responsible for maintaining the cluster's desired state, such as which applications are running and which container images they use. The worker nodes run the applications and workloads.

Cloud Service Providers usually provide API to create OpenShift cluster by specifying the size, node VM specs, etc.

Azure Red Hat Openshift Container Platform needs at least 40 vCPU cores to be successfully deployed, broken down as such:

- 1 Bootstrap VM (Standard_D4s_v3) (4 vCPU)
- 3 Control Plane VMs (Standard_D8s_v3) (8 vCPU each)
- 3 Worker Machine VMs (Standard_D4s_v3) (4 vCPU each)

For full details about the requirements, refer to the official Openshift Documentation.

To change the quota limitations of the Azure Subscription, check the official Azure Documentation on the resource limits.

1. Register resource providers.

Command
`$ az provider register -n Microsoft.RedHatOpenShift \ --wait`

Command
`$ az provider register -n Microsoft.Compute \ --wait`

Command
`$ az provider register -n Microsoft.Storage \ --wait`

2. Use `az aro create` to create a resource group for Openshift.

Parameters
`#Azure Resource Group` `$ RESOURCEGROUP=AMAROAksGroup` `#Azure Kubernetes Cluster name` `$ CLUSTER=AKsCluster` `#Location of cluster` `$ LOCATION=southeastasia`

Command
`$ az group create --name $RESOURCEGROUP\ --location $LOCATION`

Response
`{` `  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-429361615308/resourceGroups/AMAROAksGroup",` `  "location": "southeastasia",`

Parameters

```
  "managedBy": null,
  "name": "AMAROAksGroup",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```

3. Create Virtual Network and subnets.

   Before deploying the Openshift Cluster, there are some prerequisites to be done. Openshift 4.x requires a virtual network with two empty subnets for the master and worker nodes. To create a virtual network, run the following:

Parameters

```
#Azure Resource Group
$ RESOURCEGROUP=AMAROAksGroup

#Azure Kubernetes Cluster name
$ CLUSTER=AKsCluster
```

Command

```
$ az network vnet create --resource-group $RESOURCEGROUP\
--name aro-vnet \
--address-prefixes 10.0.0.0/22
```

Response

```
{
newVNet:
  addressSpace:
    addressPrefixes:
    - 10.0.0.0/22
  bgpCommunities: null
  ddosProtectionPlan: null
  dhcpOptions:
    dnsServers: []
  enableDdosProtection: false
  enableVmProtection: false
  etag: W/"ba8e0b34-b859-4d38-99fe-6c0bd0dc2585"
  id: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Network/virtualNetw
orks/aro-vnet
  ipAllocations: null
  location: southeastasia
  name: aro-vnet
  provisioningState: Succeeded
  resourceGroup: AMAROAksGroup
  resourceGuid: 2e2afecf-d7da-4acd-b773-fcee7bafa0d6
  subnets: []
```

Parameters

```
  tags: {}
  type: Microsoft.Network/virtualNetworks
  virtualNetworkPeerings: []
}
```

a. Add an empty subnet for the master node:

Parameters

```
#Azure Resource Group
$ RESOURCEGROUP=AMAROAksGroup

#Azure Kubernetes Cluster name
$ CLUSTER=AKsCluster
```

Command

```
$az network vnet subnet create --resource-group $RESOURCEGROUP\
--vnet-name aro-vnet \
--name master-subnet \
--address-prefixes 10.0.2.0/23 \
--service-endpoints Microsoft.ContainerRegistry
```

Response

```
{
addressPrefix: 10.0.0.0/23
addressPrefixes: null
delegations: []
etag: W/"2a3b584e-1f56-4d30-b50b-685b59390403"
id: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Network/virt
ualNetworks/aro-vnet/subnets/master-subnet
ipAllocations: null
ipConfigurationProfiles: null
ipConfigurations: null
name: master-subnet
natGateway: null
networkSecurityGroup: null
privateEndpointNetworkPolicies: Enabled
privateEndpoints: null
privateLinkServiceNetworkPolicies: Enabled
provisioningState: Succeeded
purpose: null
resourceGroup: AMAROAksGroup
resourceNavigationLinks: null
routeTable: null
serviceAssociationLinks: null
serviceEndpointPolicies: null
serviceEndpoints:
- locations:
  - '*'
  provisioningState: Succeeded
```

Parameters
service: Microsoft.ContainerRegistry type: Microsoft.Network/virtualNetworks/subnets

b. Add an empty subnet for the worker nodes:

Parameters
#Azure Resource Group $ RESOURCEGROUP=AMAROAksGroup #Azure Kubernetes Cluster name $ CLUSTER=AKsCluster
Command
$ az network vnet subnet create --resource-group $RESOURCEGROUP\ --vnet-name aro-vnet \ --name worker-subnet \ --address-prefixes 10.0.2.0/23 \ --service-endpoints Microsoft.ContainerRegistry
Response
{ addressPrefix: 10.0.2.0/23 addressPrefixes: null delegations: [] etag: W/"2a3b584e-1f56-4d30-b50b-685b59390403" id: /subscriptions/ff9f4ef7-c33b-454c-9844- 429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Network/virt ualNetworks/aro-vnet/subnets/worker-subnet ipAllocations: null ipConfigurationProfiles: null ipConfigurations: null name: worker-subnet natGateway: null networkSecurityGroup: null privateEndpointNetworkPolicies: Enabled privateEndpoints: null privateLinkServiceNetworkPolicies: Enabled provisioningState: Succeeded purpose: null resourceGroup: AMAROAksGroup resourceNavigationLinks: null routeTable: null serviceAssociationLinks: null serviceEndpointPolicies: null serviceEndpoints: - locations: - '*'   provisioningState: Succeeded   service: Microsoft.ContainerRegistry type: Microsoft.Network/virtualNetworks/subnets

c. Disable the subnet private endpoint policies on the master subnet. This is required for Openshift to be able to connect to and manage the cluster:

Parameters

```
#Azure Resource Group
$ RESOURCEGROUP=AMAROAksGroup

#Azure Kubernetes Cluster name
$ CLUSTER=AKsCluster
```

Command

```
az network vnet subnet update --name master-subnet \
--resource-group $RESOURCEGROUP\
--vnet-name aro-vnet \
--disable-private-link-service-network-policies true
```

Response

```
addressPrefix: 10.0.0.0/23
addressPrefixes: null
delegations: []
etag: W/"0a2c2524-903e-4e90-a024-b7cfd1c7d1b2"
id: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Network/virt
ualNetworks/aro-vnet/subnets/master-subnet
ipAllocations: null
ipConfigurationProfiles: null
ipConfigurations:
name: master-subnet
natGateway: null
networkSecurityGroup:
  defaultSecurityRules: null
  etag: null
  flowLogs: null
  id: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/aro-
  vmc1bjob/providers/Microsoft.Network/networkSecurityGroups/AKsCluster-
dxk8f-nsg
  location: null
  name: null
  networkInterfaces: null
  provisioningState: null
  resourceGroup: aro-vmc1bjob
  resourceGuid: null
  securityRules: null
  subnets: null
  tags: null
  type: null
privateEndpointNetworkPolicies: Enabled
privateEndpoints: null
privateLinkServiceNetworkPolicies: Disabled
provisioningState: Succeeded
purpose: null
resourceGroup: AMAROAksGroup
```

Parameters

```
resourceNavigationLinks: null
routeTable: null
serviceAssociationLinks: null
serviceEndpointPolicies: null
serviceEndpoints:
- locations:
  - '*'
  provisioningState: Succeeded
  service: Microsoft.ContainerRegistry
type: Microsoft.Network/virtualNetworks/subnets
```

4. Retrieve a Red Hat pull secret (Optional).

   To access Red Hat container registries and additional Openshift specific API, you can obtain a pull secret.

   a. Login to the Red Hat Openshift cluster manager portal.

   b. Click **Download pull secret** and download a pull secret to be used with your ARO cluster.

   c. Save the **pull-secret.txt** file in a safe place.

   d. The pull-secret can be referenced later during the creation of the cluster with `pull-secret @pull-secret.txt`.

5. Use `az aro create` to create the OpenShift cluster:

> **Note**: It will take approximately 35 minutes to create an Openshift Cluster.

Parameters

```
#Azure Resource Group
$ RESOURCEGROUP=AMAROAksGroup

#Azure Kubernetes Cluster name
$ CLUSTER=AKsCluster
```

Command

```
$ az aro create --resource-group $RESOURCEGROUP\
--name $CLUSTER\
--vnet aro-vnet \
--master-subnet master-subnet \
--worker-subnet worker-subnet \
--pull-secret @pull-secret.txt
```

Response

```
apiserverProfile:
  ip: 20.198.136.98
  url: https://api.vmc1bjob.southeastasia.aroapp.io:6443/
```

Parameters


```
  visibility: Public
clusterProfile:
  domain: vmc1bjob
  pullSecret: null
  resourceGroupId: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourcegroups/aro-vmc1bjob
  version: 4.5.16
consoleProfile:
  url: https://console-openshift-console.apps.vmc1bjob.southeastasia.aroapp.io/
id: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.RedHatOpenShift/ope
nShiftClusters/AKsCluster
ingressProfiles:
- ip: 20.197.106.111
  name: default
  visibility: Public
location: southeastasia
masterProfile:
  subnetId: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Network/virtualNetw
orks/aro-vnet/subnets/master-subnet
  vmSize: Standard_D8s_v3
name: AKsCluster
networkProfile:
  podCidr: 10.128.0.0/14
  serviceCidr: 172.30.0.0/16
provisioningState: Succeeded
resourceGroup: AMAROAksGroup
servicePrincipalProfile:
  clientId: e5480b33-8b53-44cf-a03d-bfb3e434f66b
  clientSecret: null
tags: null
type: Microsoft.RedHatOpenShift/openShiftClusters
workerProfiles:
- count: 1
  diskSizeGb: 128
  name: AKsCluster-dxk8f-worker-southeastasia1
  subnetId: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Network/virtualNetw
orks/aro-vnet/subnets/worker-subnet
  vmSize: Standard_D4s_v3
- count: 1
  diskSizeGb: 128
  name: AKsCluster-dxk8f-worker-southeastasia2
  subnetId: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Network/virtualNetw
orks/aro-vnet/subnets/worker-subnet
  vmSize: Standard_D4s_v3
- count: 1
  diskSizeGb: 128
  name: AKsCluster-dxk8f-worker-southeastasia3
  subnetId: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Network/virtualNetw
orks/aro-vnet/subnets/worker-subnet
  vmSize: Standard_D4s_v3
```

6. Obtain credentials to log in to Openshift.

   Before executing `oc` commands, the user needs to be authenticated first.

   a. To obtain the credentials for the default kubeadmin profile:

Parameters
`#Azure Resource Group` `$ RESOURCEGROUP=AMAROAksGroup` `#Azure Kubernetes Cluster name` `$ CLUSTER=AKsCluster`
Command
`$az aro list-credentials \` `--name $CLUSTER\` `--resource-group $RESOURCEGROUP`
Response
`kubeadminPassword: xxxxx-xxxxx-xxxxx-xxxxx` `kubeadminUsername: kubeadmin`

   b. Obtain the API Server URL:

Parameters
`#Azure Resource Group` `$ RESOURCEGROUP=AMAROAksGroup` `#Azure Kubernetes Cluster name` `$ CLUSTER=AKsCluster`
Command
`$ az aro show -g $RESOURCEGROUP\` `-n $CLUSTER\` `--query apiserverProfile.url -o tsv`
Response
`> https://api.vmc1bjob.southeastasia.aroapp.io:6443/`

   > **Note**: You can store this as an environment variable, e.g. `$ APISERVER=https://api.vmc1bjob.southeastasia.aroapp.io:6443/`.

   c. Log into the API Server:

Parameters
`#Azure Resource Group` `$ RESOURCEGROUP=AMAROAksGroup`

Parameters

```
#Azure Kubernetes Cluster name
$ CLUSTER=AKsCluster

#OpenShift API Server query
$ APISERVER=$(az aro show -g $RESOURCEGROUP -n $CLUSTER --query
apiserverProfile.url -o tsv)
```

Command

```
$ oc login $APISERVER -u kubeadmin -p xxxxx-xxxxx-xxxxx-xxxxx
```

Response

```
Login successful.

You have access to 60 projects, and the list has been suppressed. You can
list all projects with ' projects.'

Using project "default".
```

7. To verify that the OpenShift cluster is created by telling all nodes are ready:

Command

```
$ oc get nodes
```

Response

```
NAME                                        STATUS                     ROLES
   AGE   VERSION
AKsCluster-dxk8f-master-0                   Ready                      master
  34m   v1.18.3+2fbd7c7
AKsCluster-dxk8f-master-1                   Ready                      master
  34m   v1.18.3+2fbd7c7
AKsCluster-dxk8f-master-2                   Ready                      master
  34m   v1.18.3+2fbd7c7
AKsCluster-dxk8f-worker-southeastasia1-x29ph   Ready,SchedulingDisabled   worker
  20m   v1.18.3+2fbd7c7
AKsCluster-dxk8f-worker-southeastasia2-sbh8x   Ready                      worker
  21m   v1.18.3+2fbd7c7
AKsCluster-dxk8f-worker-southeastasia3-vrgln   Ready                      worker
  20m   v1.18.3+2fbd7c7
```

8. Obtain the URL for the Openshift web-console dashboard:

Parameters

```
#Azure Resource Group
$ RESOURCEGROUP=AMAROAksGroup

#Azure Kubernetes Cluster name
$ CLUSTER=AKsCluster
```

Parameters
Command
`$ az aro show --name $CLUSTER --resource-group $RESOURCEGROUP --query "consoleProfile.url" -o tsv`
Response
`https://console-openshift-console.apps.vmc1bjob.southeastasia.aroapp.io/`

Note: You can log in to the web console using the kubeadmin credentials above.

Prepare Container/Image Registry

To deploy AccessMatrix as containers, OpenShift needs to pull the AccessMatrix Docker image from the OpenShift Container Registry. Thus, you need to expose the container/image registry and then build and push your AccessMatrix Docker image.

From Openshift Cluster v4.x onwards, it does not allow connection to Azure Container Registry (ACR). OpenShift comes packaged with an internal container registry that can push, pull and manage images as any registry would. The default kubeadmin credentials generated in the previous section will not be granted access to the registry. Additional steps of letting Microsoft Entra ID users authenticate to it must be done as a prerequisite for connecting to the container registry.

Important Note: Refer to the Azure Documentation (CLI, Azure Portal (updated recently, preferred)) on setting up an Microsoft Entra ID Application and using it to authenticate against OpenShift Container Platform. In the next sections in this chapter, we will use the AAD user "aad_user001@i-sprint.com" as managing user of the image registry.

OpenShift Container Registry

Once you can authenticate the AAD user against OpenShift, several permissions need to be granted to the user before managing the images. To do so, log in to the OpenShift CLI using kubeadmin credentials with the following:

Parameters
`#Azure Resource Group` `$ RESOURCEGROUP=AMAROAksGroup`

Parameters
`#Azure Kubernetes Cluster name` `$ CLUSTER=AKsCluster` `#OpenShift API Server query` `$ APISERVER=$(az aro show -g $RESOURCEGROUP -n $CLUSTER --query apiserverProfile.url -o tsv)`
Command
`$ oc login $APISERVER\` `-u kubeadmin \` `-p xxxxx-xxxxx-xxxxx-xxxxx`
Response
`Login successful.` `You have access to 60 projects, and the list has been suppressed. You can list all projects with 'projects'`

1. Switch the project from default to OpenShift image registry.

Command
`$ oc project openshift-image-registry`
Response
`Now using project "openshift-image-registry" on server "https://api.vmc1bjob.southeastasia.aroapp.io:6443".`

2. Patch the registry by exposing the DefaultRoute.

   By default, the OpenShift Container Platform registry is secured during the cluster installation. It is not exposed outside of cluster the time of installation. Exposing the DefaultRoute allows users to log in to the registry from outside the cluster using the route address.

Command
`$oc patch configs.imageregistry.operator.openshift.io/cluster \` `--patch '{"spec":{"defaultRoute":true}}'\` `--type=merge` `$oc patch config.imageregistry.operator.openshift.io/cluster \` `--patch='[{"op": "add", "path": "/spec/disableRedirect", "value": true}]'\` `--type=json`
Response
`config.imageregistry.operator.openshift.io/cluster patched`

3. Add the following policies to the AAD-user:

Command
`$ oc policy add-role-to-user registry-viewer aad_user001@i-sprint.com` `$ oc policy add-role-to-user registry-editor aad_user001@i-sprint.com`
Response
clusterrole.rbac.authorization.k8s.io/registry-viewer added: "aad_user001@i-sprint.com" clusterrole.rbac.authorization.k8s.io/registry-editor added: "aad_user001@i-sprint.com"

4. Obtain the container registry URL:

Command
`$ oc get route default-route --template='{{ .spec.host }}'`
Response
default-route-openshift-image-registry.apps.vmc1bjob.southeastasia.aroapp.io

Create OpenShift Projects/Namespaces

A Kubernetes namespace provides a mechanism to scope resources in a cluster. OpenShift Projects are an extension of namespaces, with additional annotations. For more information about projects, refer to the official OpenShift documentation on Projects.

To create a project:

Command
`$oc new-project am`
Response
Now using project "am" on server "https://api.vmc1bjob.southeastasia.aroapp.io:6443". You can add applications to this project with the 'new-app' command. For example, try: oc new-app rails-postgresql-example to build a new example application in Ruby. Or use kubectl to deploy a simple Kubernetes application: kubectl create deployment hello-node --image=k8s.gcr.io/serve_hostname

To grant permissions and access to an authenticated AAD user, we need to grant the user "edit" permissions on the project:

> **Important Note**: Make sure the current project is the project that is to be granted. E.g., the current project is "am".

Command
`$ oc adm policy add-role-to-user edit aad_user001@i-sprint.com`
Response
`clusterrole.rbac.authorization.k8s.io/edit added: "aad_user001@i-sprint.com"`

Managing Security Context Constraints

AccessMatrix requires root privileges to start. Thus we need to create another service account with the "runasanyuid" Security Context Constraint (SCC). For more information, refer to the official OpenShift documentation.

Command
`$` `oc project am` `oc create serviceaccount runasanyuid`
Response
`serviceaccount "runasanyuid" created`

Associate the created service account with the correct scc:

Command
`$ oc adm policy add-scc-to-user anyuid \` `-z runasanyuid \` `--as system:admin`

To view the created service account and view details:

Command
`$ oc get sa` `$ oc describe sa runasanyuid`

Command
Response

```
$ oc get sa
NAME          SECRETS   AGE
builder       2         2d
default       2         2d
deployer      2         2d
runasanyuid   2         11m

Name:           runasanyuid
Namespace:      am
Labels:         <none>
Annotations:    <none>

Image pull secrets:     runasanyuid-dockercfg-rpx0z

Mountable secrets:      runasanyuid-token-3sm2b
                        runasanyuid-dockercfg-rpx0z

Tokens:                 runasanyuid-token-3sm2b
                        runasanyuid-token-bx6jc
```

Note: Take note of the Image pull secret, it needs to be configured in the latter section's deployment.

4.1.2. Prepare Database/Cache Services and Configure AccessMatrix

AccessMatrix Docker image is built based on **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz** package found in the CD image. For example, **AccessMatrix-5.7.5.7535-GA-tomcat-only.tar.gz**.

> **Important Note**: In some places, you must update AccessMatrix configuration files like **amsystem.properties**, **am5.xml**, **am5-ehcache.xml** and **server.xml** you need to unzip the **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**, then rezip to build Docker image.

The next sections explain preparing services and containers used directly by AccessMatrix, requiring some configurations to be done in some AccessMatrix config files.

1. Database Service to store any persistence data of AccessMatrix.
2. Redis Cache Service to store any transient data that are frequently updated, e.g. E2EE session id.
3. Apache ActiveMQ Server as message queue server for Ehcache replication.
4. TLS configuration, either you want to have TLS connection ends in Ingress Controller or in AccessMatrix instance.

Unpack AccessMatrix Base Package

To deploy AccessMatrix in a container, you need to take the package **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz** found in AccessMatrix CD Image, for example, **AccessMatrix-5.7.5.7535-GA-tomcat-only.tar.gz**. As explained above, you need to change some configurations in AccessMatrix. Hence the file must be unpacked/unzipped first.

Database

There are various options for the database. You can choose to host database server on VM or choose database service offered by Cloud Service Provider. The database server must be accessible to both AccessMatrix containers in the OpenShift Cluster and the administrator who would execute AccessMatrix database scripts. This document assumes that the Cloud Database Service is being used.

If Azure Kubernetes Service is selected for deployment, its Kubernetes Cluster has access to Azure SQL Database by default.

Depending on the JRE version used, an appropriate driver version must be used to connect Azure SQL. See the compatibility matrix under Azure SQL.

For the following examples, we will be using MSSQL JDBC version 8.4.1 for JRE 11.

1. Use `az sql server create` to create Azure SQL Server:

Parameters
`#Azure Resource Group` `$RESOURCEGROUP=AMAROAksGroup` `#Location of the cluster` `$LOCATION=southeastasia`
Command
`$ az sql server create --name amdbaz \` `--resource-group $RESOURCEGROUP\` `--location $LOCATION\` `--admin-user user1 \` `--admin-password Password.1234`
Response
`{` `  "administratorLogin": "user1",` `  "administratorLoginPassword": null,` `  "fullyQualifiedDomainName": "amdbaz.database.windows.net",` `  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Sql/servers/amdbaz",` `  "identity": null,` `  "kind": "v12.0",` `  "location": "southeastasia",` `  "minimalTlsVersion": null,` `  "name": "amdbaz",` `  "privateEndpointConnections": [],` `  "publicNetworkAccess": "Enabled",` `  "resourceGroup": "AMAROAksGroup",` `  "state": "Ready",` `  "tags": null,` `  "type": "Microsoft.Sql/servers",` `  "version": "12.0"` `}`

> ! **Important Note**: It is crucial to use a strong and secure password in production.

2. Use `az sql server firewall-rule create` to create a firewall rule:

   To allow for Azure SQL connections, go to Azure Portal, enable "Allow Azure services and resources to access this server". According to Azure documentation, this option allows all Azure connections, including the connections from other customers' subscriptions. If this option is

selected, ensure that login and user permissions limit access to authorized users only. For more information, refer to the Azure firewall documentation.

Parameters
`#Azure Resource Group` `$RESOURCEGROUP=AMAROAksGroup`
Command
`$ az sql server firewall-rule create --resource-group $RESOURCEGROUP\` `--server amdbaz \` `-n AllowYourIp \` `--start-ip-address 192.168.1.1 \` `--end-ip-address 192.168.1.1`
Response
`{` `  "endIpAddress": "0.0.0.0",` `  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-` `429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Sql/servers/amdbaz/` `firewallRules/AllowYourIp",` `  "kind": "v12.0",` `  "location": "Southeast Asia",` `  "name": "AllowYourIp",` `  "resourceGroup": "AMAROAksGroup",` `  "startIpAddress": "0.0.0.0",` `  "type": "Microsoft.Sql/servers/firewallRules"` `}`

3. Use `az sql db create` to create Azure SQL Database:

Parameters
`#Azure Resource Group` `$RESOURCEGROUP=AMAROAksGroup`
Command
`$ az sql db create --resource-group $RESOURCEGROUP\` `--server amdbaz \` `--name amdb \` `--edition GeneralPurpose \` `--family Gen5 \` `--capacity 2`
Response
`{` `  "autoPauseDelay": null,` `  "catalogCollation": "SQL_Latin1_General_CP1_CI_AS",` `  "collation": "SQL_Latin1_General_CP1_CI_AS",` `  "createMode": null,`

Parameters


```
  "creationDate": "2020-04-20T10:34:38.343000+00:00",
  "currentServiceObjectiveName": "GP_Gen5_2",
  "currentSku": {
    "capacity": 2,
    "family": "Gen5",
    "name": "GP_Gen5",
    "size": null,
    "tier": "GeneralPurpose"
  },
  "databaseId": "6d18f9b1-5e21-4708-9da1-7c54a959f700",
  "defaultSecondaryLocation": "eastasia",
  "earliestRestoreDate": "2020-04-20T11:04:38.343000+00:00",
  "edition": "GeneralPurpose",
  "elasticPoolId": null,
  "elasticPoolName": null,
  "failoverGroupId": null,
  "id": "/subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Sql/servers/amdbaz/
databases/amdb",
  "kind": "v12.0,user,vcore",
  "licenseType": "LicenseIncluded",
  "location": "southeastasia",
  "longTermRetentionBackupResourceId": null,
  "managedBy": null,
  "maxLogSizeBytes": 10307502080,
  "maxSizeBytes": 34359738368,
  "minCapacity": null,
  "name": "amdb",
  "pausedDate": null,
  "readReplicaCount": 0,
  "readScale": "Disabled",
  "recoverableDatabaseId": null,
  "recoveryServicesRecoveryPointId": null,
  "requestedServiceObjectiveName": "GP_Gen5_2",
  "resourceGroup": "AksGroup",
  "restorableDroppedDatabaseId": null,
  "restorePointInTime": null,
  "resumedDate": null,
  "sampleName": null,
  "sku": {
    "capacity": 2,
    "family": "Gen5",
    "name": "GP_Gen5",
    "size": null,
    "tier": "GeneralPurpose"
},
  "sourceDatabaseDeletionDate": null,
  "sourceDatabaseId": null,
  "status": "Online",
  "tags": null,
  "type": "Microsoft.Sql/servers/databases",
  "zoneRedundant": false
}
```


4. Use `az sql db show-connection-string` to show the JDBC connection String:

Command
`$ az sql db show-connection-string -s amdbaz \` `-n amdb \` `-c jdbc`
Response
"jdbc:sqlserver://amdbaz.database.windows.net:1433;database=amdb;user=<username>@amdbaz;password=<password>; encrypt=true;trustServerCertificate=false;hostNameInCertificate=*.database.windows.net;loginTimeout=30"

Configure Database Connection in AccessMatrix

After you set up database schema properly and get the connection string, now it is time to update database configuration in AccessMatrix to point to the database.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\conf\Catalina\localhost*.
2. Open the **am5.xml** file.
3. Comment the following:

XML

```
    <Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"
        initialSize="10" maxTotal="80" maxIdle="20"
minIdle="10" maxWaitMillis="10000"
        validationQuery="SELECT COUNT(*) FROM am_vars
WHERE va_name IS NULL" testOnBorrow="true"
        username="user1" password="password"
driverClassName="org.apache.derby.jdbc.EmbeddedDriver"
                url="jdbc:derby:amdb"
   />
```

4. Uncomment the following and provide values for the properties as appropriate:

XML

```
<!--
<Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"
        initialSize="10" maxWaitMillis="10000" maxTotal="80"
maxIdle="20" minIdle="10" validationQuery="select 1"
        username="sa" password="password"
driverClassName="com.microsoft.sqlserver.jdbc.SQLServerDriver"

url="jdbc:sqlserver://192.168.1.205:1433;databaseName=amdb;sendStri
ngParametersAsUnicode=false"
```

```
    />
-->
```

5. Add the EncryptorUtil factory property to allow the decryption of a password that is encrypted using the EncryptorUtil:

XML

```
    <Resource name="jdbc/DSamdb" auth="Container"
type="javax.sql.DataSource" description="AM5 DB"

factory="com.isprint.amutil.tomcat.EncryptedDataSourceFactory"
        initialSize="10" maxWaitMillis="10000" maxTotal="80"
maxIdle="20" minIdle="10" validationQuery="select 1"
        username="sa" password="password"
driverClassName="com.microsoft.sqlserver.jdbc.SQLServerDriver"

url="jdbc:sqlserver://192.168.1.205:1433;databaseName=amdb;sendStri
ngParametersAsUnicode=false"
    />
```

Notes:

- Replace username, password and url to your credentials and database server URL, i.e. jdbc:sqlserver://amdbaz.database.windows.net:1433;database=amdb;encrypt=true;trustServerCertificate=false;hostNameInCertificate=*.database.windows.net;loginTimeout=30.
- The database administrator is required to use the EncryptorUtil to encrypt the database password, using the "Environment Variable as user salt" option. The encrypted password will look like the following: AesEncryptor::env=DBCRED:HtjCuzS34QxySZMslav3umHgVFA5G5j5HCXoc7KQsUDDwyjcf/+65+756F4YJBGH, where env=DBCRED is the environment variable containing a user salt.
- For more information on how to use the EncryptorUtil, refer to *AccessMatrix Deployment Guide - Encryptor Utility Tool*.

Enable AccessMatrix Container Mode

AccessMatrix itself needs a special configuration to set to run in a container. To set AccessMatrix to run in a container:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Uncomment the following am.container.enabled configuration and set it to "true":

Properties (.properties files)

```
# Enable this property if am run in container like
docker
#am.container.enabled=true
```

Make Configuration for Database Initialization

To allow a pod to connect to initialize the database, the database's credentials have to be configured. The password must be encrypted using the EncryptorUtil with an environment variable as the user salt.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\db-init*.
2. Open the **amsystem.properties** file.
3. Comment out the following:

   Properties (.properties files)

   ```
   #am.domain.store.datasource=java:/comp/env/jdbc/DSamdb
   ```

4. Uncomment the following and input the relevant information, for example:

   Properties (.properties files)

   ```
   # JDBC
   am.domain.store.driver_class=com.microsoft.sqlserver.jdbc.SQLServer
   Driver
   am.domain.store.url=jdbc:sqlserver://accessMatrix.database.windows.
   net:1433;database=amdb;
   am.domain.store.username=user1
   am.domain.store.password=AesEncryptor::env=DBCRED:HtjCuzS34QxySZMsl
   av3umHgVFA5G5j5HCXoc7KQsUDDwyjcf/+65+756F4YJBGH
   ```

5. Ensure container mode for AccessMatrix is enabled, as detailed in the previous section.

Redis Cache Server

For AccessMatrix, Redis Cache is treated as an in-memory database, used to store transient data that are frequently updated, for example, E2EE session, SMS-OTP tokens, SAML Response Id (to prevent replay attack), CLP login sessions (for AccessMatrix Applications, such as UAS TAP, USO SSF), and OIDC Relying Party Lookup Module nonces (to prevent request forgery or replay). For more information about Redis, refer to the official Redis documentation.

Relevant certificates for SSL connectivity should be prepared. Mutual authentication depends on the capability of the Redis server in their respective clouds.

On Azure, Access Matrix will use Azure Cache for Redis as the Redis cache server. Azure only supports one way SSL.

The following creates a Basic C0 (250MB) Azure Cache for Redis instance:

Parameters

```
#Azure Resource Group
$RESOURCEGROUP=AMAROAksGroup

#Azure Kubernetes Cluster name
$CLUSTER=AKsCluster

#Location of cluster
$LOCATION=southeastasia
```

Command

```
$ az redis create --name accessmatrixCache \
--resource-group $RESOURCEGROUP\
--location $LOCATION\
--sku Basic \
--vm-size C0
```

Response

```
accessKeys: null
enableNonSslPort: false
hostName: accessmatrixCache.redis.cache.windows.net
id: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AMAROAksGroup/providers/Microsoft.Cache/Redis/accessmatrixC
ache
instances:
- isMaster: false
  nonSslPort: null
  shardId: null
  sslPort: 15000
  zone: null
linkedServers: []
location: Southeast Asia
minimumTlsVersion: null
name: accessmatrixCache
port: 6379
provisioningState: Creating
redisConfiguration:
  maxclients: '256'
  maxfragmentationmemory-reserved: '12'
  maxmemory-delta: '2'
  maxmemory-reserved: '2'
redisVersion: 4.0.14
replicasPerMaster: null
resourceGroup: AMAROAksGroup
shardCount: null
sku:
```

Parameters

```
  capacity: 0
  family: C
  name: Basic
sslPort: 6380
staticIp: null
subnetId: null
tags: {}
tenantSettings: {}
type: Microsoft.Cache/Redis
zones: null
```

Important Note: It may take 10-15 minutes to provision a Redis Cache.

To check the status of the Redis cache, enter the following:

Command

```
$az redis list
```

Response

```
- accessKeys: null
  enableNonSslPort: false
  hostName: accessmatrixCache.redis.cache.windows.net
  id: /subscriptions/ff9f4ef7-c33b-454c-9844-
429361615308/resourceGroups/AksGroup/providers/Microsoft.Cache/Redis/accessmatrixCache
  instances:
  - isMaster: true
    nonSslPort: null
    shardId: null
    sslPort: 15000
    zone: null
  linkedServers: []
  location: Southeast Asia
  minimumTlsVersion: null
  name: accessmatrixCache
  port: 6379
  provisioningState: Succeeded
  redisConfiguration:
    maxclients: '256'
    maxfragmentationmemory-reserved: '12'
    maxmemory-delta: '2'
    maxmemory-reserved: '2'
  redisVersion: 4.0.14
  replicasPerMaster: null
  resourceGroup: AksGroup
  shardCount: null
  sku:
    capacity: 0
    family: C
    name: Basic
  sslPort: 6380
  staticIp: null
  subnetId: null
```

Command
tags: {} tenantSettings: {} type: Microsoft.Cache/Redis zones: null

The information required by AccessMatrix is listed below:

Key	Description
hostName	Host name of the Redis Server, e.g. *accessmatrixCache.redis.cache.windows.net*
port	Port of Redis, defaults to 6379 for non-SSL and 6380 for SSL
password	Access key for Azure Redis Cache. Access Keys can be obtained from the Azure Portal. For more information, check the official documentation about Azure Redis Access Key

Configure Redis Connection in AccessMatrix

After you set up the Redis service properly and get the connection information (hostname and port), now it is time to update the Redis configuration in AccessMatrix to point to the Redis service.

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Configure the following items according to the information above, for example, if using a non-secure port:
    a. host=accessmatrixCache.redis.cache.windows.net
    b. port=6379
    c. Obtain the access key and input it for am.redis.password

Azure Redis Non-SSL port

amsystem.properties
Properties (.properties files)
== Cache Provider Configuration == #choose redis or ehcache, if not set, default is ehcache am.cache.vendor=redis am.redis.host=accessmatrixCache.redis.cache.windows.net

amsystem.properties

```
am.redis.port=6379
#optional, don't set if no password required
am.redis.password={YOUR_AZURE_REDIS_ACCESS_KEY}
#optional, default is false
am.redis.ssl.enabled=false
```

Azure Redis SSL port

If SSL port is enabled, make sure certificate files are prepared.

amsystem.properties

Properties (.properties files)

```
# == Cache Provider Configuration ==
#choose redis or ehcache, if not set, default is ehcache
am.cache.vendor=redis
am.redis.host=accessmatrixCache.redis.cache.windows.net
am.redis.port=6380
#optional, don't set if no password required
am.redis.password={YOUR_AZURE_REDIS_ACCESS_KEY}
#optional, default is false
am.redis.ssl.enabled=true
#optional parameters for setting up redis ssl connection,
they will only be effective when am.redis.ssl.enabled=true:
am.redis.ssl.keyStore=conf/teststore.jks
am.redis.ssl.keyStorePassword=test
am.redis.ssl.keyStoreAlias=Server (CA)
am.redis.ssl.keyStoreKeyPassword=test
am.redis.ssl.trustStore=trust.jks
am.redis.ssl.trustStorePassword=test
```

Use Encryptor Util to Encrypt Redis Password

The property am.redis.password in the **amsystem.properties** file accepts Redis password as an encrypted text. The encrypted text must be generated from the EncryptorUtil.

For more information about EncryptorUtil, refer to *AccessMatrix Common Administration Guide: Utility Tools: Encryptor Utility*.

Apache ActiveMQ Server

Some common objects in AccessMatrix such as policies, configurations and login modules that are often accessed but are not constantly updated, are stored in Ehcache (in-memory cache inside AccessMatrix)

for performance and efficiency reasons. AccessMatrix uses Apache ActiveMQ Classic for Ehcache replication using Java Message Service (JMS).

Apache ActiveMQ is deployed as a container inside the OpenShift cluster. Therefore, it must be prepared just like the AccessMatrix Docker image and pushed to the OpenShift Internal Container Registry.

> **Important Note**: AccessMatrix currently supports ActiveMQ Classic 5 release.

Build ActiveMQ Docker Image

An ActiveMQ Docker image must be built locally before pushing to the container registry.

A sample ActiveMQ Dockerfile looks like below:

Dockerfile

Code

```
FROM adoptopenjdk/openjdk8:alpine-jre

ENV ACTIVEMQ_VERSION 5.17.6
ENV ACTIVEMQ apache-activemq-$ACTIVEMQ_VERSION
ENV ACTIVEMQ_HOME /opt/activemq

RUN set -x &&\
mkdir -p /opt &&\
       mv /usr/glibc-compat/lib/ld-linux-x86-64.so.2 /usr/glibc-
compat/lib/ld-linux-x86-64.so &&\
       ln -s /usr/glibc-compat/lib/ld-linux-x86-64.so /usr/glibc-
compat/lib/ld-linux-x86-64.so.2 &&\
apk --update add --virtual build-dependencies curl &&\
curl
https://archive.apache.org/dist/activemq/$ACTIVEMQ_VERSION/$ACTIVEMQ
-bin.tar.gz -o $ACTIVEMQ-bin.tar.gz

RUN tar xzf $ACTIVEMQ-bin.tar.gz -C  /opt &&\
   ln -s /opt/$ACTIVEMQ$ACTIVEMQ_HOME&&\
   addgroup -S activemq && adduser -S -H -G activemq -h
$ACTIVEMQ_HOME activemq &&\
   chown -R activemq:activemq /opt/$ACTIVEMQ&&\
   chown -h activemq:activemq $ACTIVEMQ_HOME&&\
   apk del build-dependencies &&\
   rm -rf /var/cache/apk/*

#Allow root access for OpenShift Arbitrary UID
RUN chgrp -R 0 $ACTIVEMQ_HOME&&\
   chgrp -R 0 /opt/$ACTIVEMQ&&\
   chmod -R g=u $ACTIVEMQ_HOME&&\
   chmod -R g=u /opt/$ACTIVEMQ
```

Dockerfile

```
USER activemq

WORKDIR $ACTIVEMQ_HOME
EXPOSE 8161 61616

CMD ["/bin/sh", "-c", "bin/activemq console"]
```

In the Dockerfile above, ActiveMQ started on a lightweight alpine image, and the following ports are exposed:

Port	Description
8161	ActiveMQ Dashboard
61616	Openwire/JMS

Additionally, in OpenShift, Docker images/containers are not allowed to be run as Root. It also does not allow images to run with an unidentifiable user name such as "activemq". It will not be able to identify if the user is using any reserved user ID. To allow for the usage of an arbitrary user ID, some directories will require root access. For more information about this behaviour, refer to the Openshift Guidelines.

Use `docker build` to build image:

Command

```
$ docker build . -f Dockerfile -t activemq:5.17.6
```

Response

```
Sending build context to Docker daemon 295.3MB
Step 1/11: FROM openjdk8:alpine-jre
  ---> f7a292bbb70c
Step 2/11: ENV ACTIVEMQ_VERSION 5.17.6
  ---> Using cache
  --->27f09332fd42
Step 3/11: ENV ACTIVEMQ apache-activemq-$ACTIVEMQ_VERSION
  ---> Using cache
  --->658e2fabfb9d
Step 4/11: ENV ACTIVEMQ_HOME /opt/activemq
  ---> Using cache
  --->10dda8e72182
...
Step 11/11: CMD ["/bin/sh", "-c", "bin/activemq console"]
  ---> Using cache
```

Command
``   --->36652b81ef18 Successfully built 36652b81ef18 Successfully tagged activemq:5.17.6 ```

Push ActiveMQ Docker Image to Registry

Use `docker tag` and `docker push` to tag the docker image and then push it to the OpenShift container registry:

Command
`$docker image tag activemq:5.17.6 default-route-openshift-image-registry.apps.vmc1bjob.southeastasia.aroapp.io/am/activemq:5.17.6` `$ docker image ls`
Response
`` REPOSITORY TAG IMAGE ID CREATED SIZE default-route-openshift-image- registry.apps.vmc1bjob.southeastasia.aroapp.io/am/activemq 5.17.6 36652b81ef18 4 weeks ago 282MB activemq 5.17.6 36652b81ef18 4 weeks ago 282MB## ```

Use `docker login` to log the authenticated AAD user into the OpenShift Internal Container Registry.

In the previous section, an AAD user is authenticated against the OpenShift Internal Container Registry. You can refer to it here: Prepare Container/Image Registry. This user is required to log into the OpenShift CLI before proceeding with the following steps.

For ease of logging in, the AAD-user can first log in to the OpenShift Dashboard web console, click on the top right-hand context menu and select "Copy Login Command". It will look similar to the following: `$ oc login --token=Uz10ufh3FuguRAotrrl6gwhQglttCFwOXnOJk3nQ_Ec \ --server=https://api.vmc1bjob.southeastasia.aroapp.io:6443`

Enter it into the CLI:

Command
`$ oc login --token=Uz10ufh3FuguRAotrrl6gwhQglttCFwOXnOJk3nQ_Ec \` `--server=https://api.vmc1bjob.southeastasia.aroapp.io:6443`
Response

Command
Logged into "https://api.vmc1bjob.southeastasia.aroapp.io:6443" as "aad_user001@i-sprint.com" using the token provided. You have access to the following projects and can switch between them with 'oc project <projectname>': * openshift-image-registry am Using project "openshift-image-registry".

You can retrieve the login credentials when the user is successfully authenticated in the CLI:

Command
`$ oc whoami` `$ oc whoami -t`
Response
aad_user001@i-sprint.com Uz10ufh3FuguRAotrrl6gwhQglttCFwOXnOJk3nQ_Ec

Log into OpenShift Internal Container Registry using Docker login:

Example: `docker login -u <AAD user>-p <Login token><Registry URL>`

Parameters
`#Obtain and store the OpenShift registry name so it can be easily referenced.` `$ OpenshiftRegistry=$(oc get route default-route --template='{{ .spec.host }}')`
Command
`$ docker login -u aad_user001@i-sprint.com \` `-p Uz10ufh3FuguRAotrrl6gwhQglttCFwOXnOJk3nQ \` `$OpenshiftRegistry`
Response
WARNING! Using --password via the CLI is insecure. Use --password-stdin. Login Succeeded

After successfully logging in, the image can be pushed to the internal container registry:

Command

```
$ docker push default-route-openshift-image-
registry.apps.vmc1bjob.southeastasia.aroapp.io/am/activemq:5.17.6
```

Response

```
The push refers to repository [default-route-openshift-image-
registry.apps.vmc1bjob.southeastasia.aroapp.io/am/activemq]
c5f8f59fff70: Pushed
5ba9c921e10c: Pushed
161272d64886: Pushed
edd61588d126: Pushed
9b9b7f3d56a0: Pushed
f1b5933fe4b5: Pushed
5.17.6: digest: sha256:7483407e21facf99fa149eb8debf8f0342b607ed86c888fe88cbe4d22cce463c
size: 1583
```

Deploy ActiveMQ

ActiveMQ requires a deployment to represent how it will run in the cluster. Below is a sample of an ActiveMQ deployment.

In the deployment file, we can specify the Service Account created in the previous section, as well as the imagePullSecret. The image registry url is as follows:

image-registry.openshift-image-registry.svc:5000/<Project Name>/<ImageName>:<Version>

activemqdeployment.yaml

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: activemq
  labels:
    app: activemq
spec:
  replicas: 1
  selector:
    matchLabels:
      app: activemq
  template:
    metadata:
      annotations:
        pod.beta.kubernetes.io/hostname: activemq
      labels:
        app: activemq
    spec:
      serviceAccountName: runasanyuid
```

activemqdeployment.yaml

```
      imagePullSecrets:
        - name: runasuid15000-dockercfg-61q8k
      containers:
      - name: activemq
        image:  image-registry.openshift-image-
registry.svc:5000/am/activemq:5.17.6
        imagePullPolicy: Always
        resources:
            requests:
              memory: 500Mi
              cpu: 200m
            limits:
              memory: 1000Mi
              cpu: 400m
```

Use `oc apply` to apply the changes:

Command

```
$ oc apply -f activemqdeployment.yaml
```

Expose ActiveMQ Service

For AccessMatrix to access and use ActiveMQ as a message broker, the deployment must be exposed via a ClusterIP service. Below is a sample of an ActiveMQ service.

activemqservice.yaml

YAML

```
apiVersion: apps/v1
kind: Service
metadata:
  name: active-mq
  namespace: default
  labels:
    app: active-mq
spec:
  selector:
    app: activemq
  ports:
  - name: dashboard
    port: 8161
    targetPort: 8161
    protocol: TCP
  - name: jms
    port: 61616
```

activemqservice.yaml

```
    targetPort: 61616
  type: ClusterIP
```

Use `oc apply` to apply the changes:

Command

```
$ oc apply -f activemqservice.yaml
```

To check that the service is running, use `$ oc get svc`

Command

```
$ oc get svc
```

Response

```
NAME            TYPE            CLUSTER-IP       EXTERNAL-IP       PORT(S)
                        AGE
active-mq       ClusterIP       10.70.9.45       <none>
          8161:35614/TCP,61616:31580/TCP    5m
kuberentes      ClusterIP       10.70.0.1        <none>            443/TCP
                        3d18h
```

Configure ActiveMQ Connection in AccessMatrix

After you set up ActiveMQ container correctly, you then update related configurations in AccessMatrix point to the ActiveMQ container. All we need the ActiveMQ service name (which is "active-mq" as above) that can be resolved to its ClusterIP automatically, and port (61616).

The IP address can be used directly in the configuration below, or you can use a domain name, e.g. www.example.com. Refer to Domain name and TLS Connection.

You need to configure amsystem.properties using the above information in AccessMatrix package:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Set the following properties (find and uncomment if it is already present):

   Properties (.properties files)

```
# Cache Replicator Implementation: RMI,JMS
am.ehcache.replicator.factory=JMS
...
# EhCache JMS Cache Peer Provider
am.ehcache.jms.initialContextFactoryName=com.isprint.am.core.cache.
ehcache.AtiveMQInitialContextFactory
am.ehcache.jms.providerURL=failover:tcp://localhost:61616
am.ehcache.jms.userName=amquser1
am.ehcache.jms.password=Password.1234
am.ehcache.jms.replicationTopicBindingName=amtopic
am.ehcache.jms.getQueueBindingName=amqueue
#am.ehcache.jms.acknowledgementMode
#       : (Optional) The mode used to acknowledge messages after
they are consumed.
#    Configure with one of the following values:
#        - AUTO_ACKNOWLEDGE
#        - DUPS_OK_ACKNOWLEDGE (Default)
#        - SESSION_TRANSACTED
#am.ehcache.jms.acknowledgementMode=AUTO_ACKNOWLEDGE
```

Notes:

- Replace am.ehcache.jms.providerURL=failover:tcp://localhost:61616 with your domain name or IP. For example: failover:(ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-1.amazonaws.com:61617,ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-southeast-1.amazonaws.com:61617).
- The "Ative" in am.ehcache.jms.initialContextFactoryName=com.isprint.am.core.cache.ehcache.AtiveMQInitialContextFactory is not a misspelling.

4. Save the changes.

Next, configure the ActiveMQ trusted package:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\bin*.
2. Open the **setenv.sh** file.
3. Search for JAVA_OPTS and add -Dorg.apache.activemq.SERIALIZABLE_PACKAGES=* under it:

Bash (Unix Shell)

```
JAVA_OPTS="$JAVA_OPTS $JAVA_GC_OPTS -Xms400m -Xmx1g -
Dderby.system.home=db
-Dam.ehcache.set_rmi_ip_addr=true -
XX:MaxMetaspaceSize=256M -XX:NewSize=128M
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=logs -
Dhost.name=$(cat /etc/hostname)
-Djavax.net.ssl.trustStore=conf/amtrust.keystore -
Djavax.net.ssl.trustStorePassword=passphrase
```

```
-Djava.awt.headless=true" -
Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*
```

4. Save the changes.

Lastly, add the ActiveMQ libraries in the *WEB-INF\lib* folder:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\tomcat\am5\WEB-INF\lib*.
2. Obtain and copy the ActiveMQ libraries listed below into the directory:
    - **activemq-broker-5.17.6.jar**
    - **activemq-client-5.17.6.jar**
    - **hawtbuf-1.11.jar**

Domain name and TLS Connection

A domain name must be prepared (e.g. *am.example.com*), to be used in TLS certificate and set in a proper DNS server. Please request your network administrator to add DNS record to the cluster's the external IP that can be accessed from the Internet. To retrieve the external IP, using the command below:

Command
`oc describe cluster | grep "Ingress IP"`
Response
`Ingress IP:  20.212.31.106`

The terminology TLS and SSL are used interchangeably. SSL is deprecated, replaced with TLS. However, SSL terminology is still being used by Kubernetes itself.

There are two possibilities of TLS connection endpoint, i.e.

1. TLS Connection Ends in OpenShift Route
2. TLS Connection Ends in AccessMatrix

Which configuration to choose, subject to customer choice.

TLS Connection Ends in OpenShift Route

Referring to the architecture diagram, this configuration means, secure TLS connection from the user's endpoint (browser or mobile device) ends.

TLS Connection Ends in AccessMatrix

Referring to the architecture diagram, this configuration means, secure TLS connection from the user's endpoint (browser or mobile device) ends in AccessMatrix instances/pods. Route is configured as SSL-Passthrough, that means, any TLS connection will only pass through the Route.

1. Navigate to the directory <*ACMATRIX_ROOT*>*tomcat**conf*\.
2. Open the **server.xml** file.
3. Check Connector element for https port (which default to 8443).
    a. Inject your TLS certificate with the proper domain name to a keystore file; for example, **testserver.keystore**.
    b. Set the keystore file name to keystoreFile parameter of the Connector element in **server.xml**.

4.1.3. Create AccessMatrix Docker Image

At this stage, all of the services that AccessMatrix will use are ready, and related AccessMatrix base package configurations have been updated. So now you can proceed to build AccessMatrix Docker image. An AccessMatrix Docker image must be built locally before being pushed to the registry.

Repack AccessMatrix Base Package

By now, you have prepared database service, Redis cache service, and ActiveMQ. You have also updated all related configurations in AccessMatrix. Now you need to repack it back to its respective name (**AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**), for example, **AccessMatrix-5.7.5.7535-GA-tomcat-only.tar.gz**.

Pre-image Build Checklist

Before building the AccessMatrix Docker Container Image, ensure the following are done:

Checklist	Description
AccessMatrix container mode	AccessMatrix container mode has been enabled in **amsystem.properties**. Refer to Enable AM Container Mode for more information.
Database service	Database service is ready and its connection information is configured in **am5.xml**. Refer to Database Service for more information.
Redis Cache service	Redis Cache service is ready and its connection information is configured in **amsystem.properties**. Refer to Redis Cache Server for more information.
Apache ActiveMQ container	Apache ActiveMQ container is up and running and its connection is configured in **am5-ehcache.xml** and **am5.xml**. Refer to ActiveMQ am5-ehcache.xml configuration for more information.
TLS Connection Ends in AM	If the TLS connection were to end in AccessMatrix instance (Ingress controller passthrough), the certificate and port are configured in **server.xml**. Refer to TLS Connection Ends in AM for more information.
AM Base Package	AM base package that was previously unpacked must be repacked back to its original name.
Dependent libraries	Any additional libraries or drivers such as JDBC, or any external drivers, must be placed in the lib folder in the same directory as the Dockerfile.

Checklist	Description
Openshift Container Registry	The OpenShift internal container registry can be accessed, and a valid AAD user is authenticated against it and can push/pull images. Refer to OpenShift Container Registry

Build AccessMatrix Docker Image

A sample AccessMatrix Dockerfile is shown as part of the steps below. Arrange the files as follows:

1. Place the Dockerfile in any working directory that you choose. Change the environment variable AM_VERSION to the respective AccessMatrix version (e.g. 5.7.5.7535-GA).

Dockerfile

Code

```
# Certification Requirement: Build image based on UBI
# Use Red Hat Universal Base Image so that it is compatibile
with OpenShift
# Best practices: Use image hash instead of tag to prevent
sudden changes, i.e. tag might be reassigned to another
build
# The below hash is equivalent to the following amd64
architecture released on 04-April-2023 with the tag
8.ad01d53d5b69bca0f6b6cec1

# Certification Requirement: Image metadata requirements
# Must include these labels: name, vendor, version, release,
summary, description
# Best practices: Label the image

LABEL name="5.7.5.7535-GA ubi8 with Temurin OpenJDK 17" \
      vendor="i-Sprint" \
      maintainer="PE-SG <pe-sg@i-sprint.com>"

# Best practices: Consolidate configuration at the top, so
it is easy to find and change.
# ENV PATH has to be on a new line because it uses
"JAVA_HOME" and cannot be on the same line as "JAVA_HOME"
declaration

ENV AM_VERSION="5.7.5.7535-GA" \
    JAVA_HOME="/opt/jdk/jdk-17.0.6+10"
ENV PATH="$PATH:${JAVA_HOME}/bin"

RUN mkdir -p /opt/jdk/ \
    && mkdir -p /opt/am/${AM_VERSION}

ADD repo/OpenJDK17U-jdk_x64_linux_hotspot_17.0.6_10.tar.gz
/opt/jdk/
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
```

Dockerfile

```
/opt/am/${AM_VERSION}

# Certification Requirement: must contain a "licenses"
directory
# Best practices: Set group ownership and file permissions
# In the Dockerfile, set permissions on the directories and
files that the process uses.
# The root group must own those files and be able to read
and write them as needed.
# For compatibility with Kubernetes, the Dockerfile should
specify a non-root user ID, then set file ownership to that
user ID and the root group

RUN mv /opt/am/${AM_VERSION}/licenses /licenses \
    && groupadd -g 2020 am \
    && useradd -r -g am -u 2021 -s /bin/bash am \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:0 /opt/am/${AM_VERSION} \
    && chown -R am:0 /opt/am/am \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/db-init/*.sh \
    && chmod -R g=u /opt/am/am

COPY ["lib", "/opt/am/am/tomcat/lib"]

EXPOSE 8080 8443 8161 61616

# Certification Requirement: Run as non-root user
# Best practices: Run as non-root user
# If a process running as root breaks out of the container,
it gains root (privileged) access to the host machine.
Therefore, run the container as a non-root user so that, if
the process breaks out of the container, its access to the
host machine is much more limited.

USER am

CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]

# Other Certification Requirements:
# - must not change content provided by Red Hat packages or
layers
# - must have less than 40 layers
# - must not include RHEL kernel packages
# - must not contain Red hat components with identified
important or critical vulnerabilities
```

2. Create a subdirectory, *lib,* to place all dependent libraries, e.g. JDBC drivers.
3. Create a subdirectory, *repo,* to place the AccessMatrix base package **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz**.
4. Download and place the Eclipse Temurin Prebuilt OpenJDK Binaries **OpenJDK17U-jdk_x64_linux_hotspot_17.0.6_10.tar.gz** in the *repo* subdirectory.

> **Note**: The Eclipse Temurin Prebuilt OpenJDK Binaries can be downloaded here.

5. Use `docker build` to build image:

Command

```
$ docker build . -f Dockerfile -t am5:5.7.5.7535-GA
```

Response

```
=> [internal] load build definition from Dockerfile
                                    0.0s
=> => transferring dockerfile: 32B
                                    0.0s
=> [internal] load .dockerignore
                                    0.0s
=> => transferring context: 2B
                                    0.0s
=> [internal] load metadata for
registry.access.redhat.com/ubi8@sha256:e3311058176628ad7f0f288f894ed2afef61be77a
0.0s
=> [internal] load build context
                                    0.0s
=> => transferring context: 198B
                                    0.0s
=> [1/6] FROM
registry.access.redhat.com/ubi8@sha256:e3311058176628ad7f0f288f894ed2afef61be77ad
01d53d5b69bca0f6b  0.0s
=> CACHED [2/6] RUN mkdir -p /opt/jdk/ && mkdir -p /opt/am/5.7.5.7535-GA
                                    0.0s
=> CACHED [3/6] ADD repo/OpenJDK17U-jdk_x64_linux_hotspot_17.0.6_10.tar.gz
/opt/jdk/                                  0.0s
=> [4/6] ADD repo/AccessMatrix-5.7.5.7535-GA-tomcat-only.tar.gz
/opt/am/5.7.5.7535-GA                              1.8s
=> [5/6] RUN mv /opt/am/5.7.5.7535-GA/licenses /licenses && groupadd -g 2020 am
&& useradd -r -g am -u            6.7s
=> [6/6] COPY [lib, /opt/am/am/tomcat/lib]
                                    0.1s
=> exporting to image
                                     1.5s
=> => exporting layers
                                    1.5s
=> => writing image
sha256:8ea0a84447f5fef8e8cba578793c11c620804bb179593b8ab1e3d9deb907bbf4
              0.0s
=> => naming to docker.io/library/am5:5.7.5.7535-GA
                                     0.0s
```

> **Note**: The above container image has undergone the Red Hat Container Certification process with the following results:
>
> - **Certification Test:** "Passed"
> - **Health Index:** "A"
>
> This certification serves as an indicator that a Red Hat partner's product meets the Red Hat standards of interoperability, security, and life cycle

management when deployed on Red Hat OpenShift. It is intended to ensure that customers benefit from a trusted application and infrastructure stack, tested and jointly supported by Red Hat and its partner.

New container images must meet the highest health index grade “A” to be certified.

For more information on Red Hat OpenShift certification, see the following references:

- Red Hat OpenShift certification requirements
- Red Hat OpenShift certification policies
- Red Hat OpenShift certification best practices
- Red Hat Container Health Index grades

Push AccessMatrix Docker Image to Registry

Use `docker tag` and `docker push` to tag the docker image then push to the OpenShift internal container registry:

Important Note: For OpenShift, the image tagging syntax is as follows:
`<Container_Registry_URL>/<Project_name>/<Image_name>:<Version>`
Example:
`default-route-openshift-image-registry.apps.vmc1bjob.southeastasia.aroapp.io/am/am5:5.7.5.7535-GA`

Command
`$ docker image tag am5:5.7.5.7535-GA default-route-openshift-image-registry.apps.vmc1bjob.southeastasia.aroapp.io/am/am5:5.7.5.7535-GA` `$ docker image ls`
Response
`default-route-openshift-image-registry.apps.vmc1bjob.southeastasia.aroapp.io/am/am5 5.7.5.7535-GA afc1178a5a33 2 weeks ago 755MB` `am5 5.7.5.7535-GA afc1178a5a33 2 weeks ago 755MB`

Use `docker login` to log the authenticated AAD user into the OpenShift Internal Container Registry. Ensure that the user is logged into the Openshift internal container registry. Refer to Push ActiveMQ Docker Image to Registry for more information about logging in via the authenticated AAD user.

After successfully logging in, the image can be pushed to the internal container registry:

Command
`$ docker image push default-route-openshift-image-registry.apps.vmc1bjob.southeastasia.aroapp.io/am/am5:5.7.5.7535-GA`

Response
The push refers to repository [default-route-openshift-image-registry.apps.vmc1bjob.southeastasia.aroapp.io/am/am5:5.7.5.7535-GA] 247946ea3c85: Pushed ... 5.7.5.7535-GA: digest: sha256:a977f44de80368bd83511456f8facd2375da70691ac62dd23d49afade326ac2f size: 3672

!	**Important Note**: Version tagging of images is important as tagging with the "latest" tag can lead to problems in a managed environment. According to Kubernetes docs, labelling all images with latest makes it difficult to track the current version of the image running and makes it difficult to roll back when errors occur.

4.1.4. Create and Deploy OpenShift Objects

At this stage, AccessMatrix Docker image has been configured properly and pushed to the container registry. Now you are ready to create/deploy OpenShift objects.

OpenShift objects are persistent entities in the OpenShift system. OpenShift uses these entities to represent the state of your cluster. We describe the Kubernetes object using YAML files. For any YAML files listed in this chapter, use command `kubectl apply -f <file-name>.yaml` to create OpenShift objects in the cluster. For example:

Command
`$ oc apply -f am-k8s.yaml`
Response
`deployment.apps/am-k8s created`

Note: DeploymentConfigs (DC) is a native solution provided by RedHat. It is not being used in this deployment for now.

Persistent Volumes and Claims

A Persistent Volume (PV) is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes. It is a resource in the cluster, just like a node is a cluster resource. PVs are volume plugins similar to Docker Volumes but have a lifecycle independent of any individual pod that uses the PV. This API object captures the details of the implementation of the storage, be that NFS, iSCSI, or a cloud-provider-specific storage system.

A Persistent Volume Claim (PVC) is a request for storage by a user (refer to architecture diagram). It is similar to a Pod in a way that Pods consume node resources, and PVCs consume PV resources. Pods can request specific levels of resources (CPU and Memory). PVC can request specific size and access modes (e.g., they can be mounted once read/write or many times read-only).

Usage of PV (Persistent Volume) and PVC (Persistent Volume Claim) for AccessMatrix deployed in Kubernetes is to have persistent centralized storage of log files generated in multiple AccessMatrix instances/Pods.

Dynamically provisioning PVC from Azure

In OpenShift, there are additional steps to be taken before PVCs can be dynamically provisioned like in the standard Kubernetes environment. To do so, you need to create a service account and grant it privileges to provision the PVC. The following steps are an example:

1. Create a role:

PVCRole.yaml

YAML

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: system:controller:persistent-volume-binder
  namespace: am
rules:
  - apiGroups: [""]
    resources: ["secrets"]
    verbs: ["create", "get", "delete"]
```

2. Create a role bind in the kube-system project.

PVCRoleBinding.yaml

YAML

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: system:controller:persistent-volume-binder
  namespace: am
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: system:controller:persistent-volume-binder
Subjects:
- kind: ServiceAccount
  name: persistent-volume-binder
namespace: kube-system
```

3. Add the service account as an admin to the user's project

Command

```
$ oc policy add-role-to-user admin system:serviceaccount:kube-
system:persistent-volume-binder \
-n am
```

Command

Response

```
clusterrole.rbac.authorization.k8s.io/admin added: "system:serviceaccount:kube-system:persistent-volume-binder"
```

Azure has various storage class options. The PV and PVC yaml below shows an example of Azure File to be claimed as the logs folder:

am-pv-azure.yaml

YAML

```yaml
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: azurefile
provisioner: kubernetes.io/azure-file
mountOptions:
  - dir_mode=0777
  - file_mode=0777
  - uid=2021
  - gid=2020
  - mfsymlinks
  - cache=strict
parameters:
  skuName: Standard_LRS
```

Important Note:

- While there are multiple storage access modes, take note that when using ReadWriteMany means the volume can be mounted as read-write many nodes.
- The default reclaim policy is "Delete", if an old Persistent Volume is deleted, and then recreated with the same Persistent Volume Claim, the old contents will be overwritten.
- The uid and gid value must be same as the uid and gid of am user in the Dockerfile as described in Build AccessMatrix Docker Image.

In the case where there are multiple deployments, it might not be desirable for all the deployments to share the same volume. To separate the logs based on deployment, you can create multiple Persistent Volume Claims (PVC) such as:

am-deployment-1-pvc.yaml

am-deployment-1-pvc.yaml

YAML

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: deploymentlogs
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: azurefile
  resources:
    requests:
      storage: 1Gi
```

am-deployment-2-pvc.yaml

YAML

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: sandboxlogs
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: azurefile
  resources:
    requests:
      storage: 2Gi
```

There might be a need to register the storage provider if provisioning a PVC fails. To do so, register the storage provider with the following. For more information about registration of providers, refer to the official Azure documentation.

Command

```
$ az provider register --namespace Microsoft.Storage
```

Temporary Report Directory

When the AccessMatrix system receives a request to generate a report, the system will generate a report resulting in any of these report files: CSV, PDF, or HTML. The generated report file(s) will be stored temporarily in a reports folder that is automatically created when the AccessMatrix system generates a

report for the first time. The reports folder will be mounted under this directory: *<ACMATRIX_HOME>/work/*.

Example: *opt/am/am/tomcat/work/reports/.*

Generally, the generated report file(s) in this directory will be removed after AccessMatrix has completely streamed the report to the user. However, for whatever reason that the generated report will not be deleted (i.e. report generation is aborted halfway by the user or an unexpected error occurred), a daily report cleaner (ReportCleanerSchedJob) is scheduled to run at 4:00 AM every day.

Below is a sample YAML file for creating PVC for the reports:

am-report-pvc.yaml

YAML

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: reports
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: azurefile
  resources:
    requests:
      storage: 1Gi
```

Note: Refer to TLS Connection Ends in Ingress Controller and TLS Connection Ends in AccessMatrix sections for the sample Deployment YAMLs.

Deployment

Deployment is the main Kubernetes Object that represents which and how an application will run in the cluster. The concept is the same for OpenShift Container Platform.

Each OpenShift Pod is meant to run a single instance of the application; thus, we deploy only one AccessMatrix container per Pod. In this case, Pod is a wrapper around a single container while Kubernetes manages the Pods rather than the containers directly.

As explained in Domain name and TLS Connection section, there are two possibility of TLS connection endpoint, i.e.

1. TLS Connection Ends in Route

2. TLS Connection Ends in AccessMatrix

Deployment YAML files for both scenarios will be the same. Both will have:

- Two Pod replicas.
- Request for 100m CPU, which is equivalent to 10% of 1vCPU. For further information, read Kubernetes Resource Units.

OpenShift Secrets

Secrets can be used like ConfigMap and are used to store sensitive information such as passwords. Pods in a deployment can refer to the secret as environment variables as well. For example, use OpenShift Secrets to reference credentials:

Command
`$ oc create secret generic am-credentials --from-file=DBCRED`
Response
`secret/am-credentials created`

By default, OpenShift uses the filename of the text file as the key. In this case, DBCRED is the key and the value will be the file's contents. The user salt is used to encrypt the database passphrase.

Database Initialization

To avoid the concurrent initialization of the AccessMatrix database because of multiple replicas, a database administrator must initialize the AccessMatrix database with a pod. The pod will run a script to initialize the server and close, and it must be run before deployment of AccessMatrix.

Below is a sample YAML for a Database Initialization pod.

Image pull secret can be obtained from the Managing Security Context Constraints section, from the details of the service account.

am-dbinit.yaml
YAML

am-dbinit.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: dbinit
spec:
  serviceAccountName: runasanyuid
  imagePullSecrets:
    - name: runasanyuid-dockercfg-rpx0z
  containers:
  - image: image-registry.openshift-image-
registry.svc:5000/am/am5:5.7.2
    name: dbinit
    env:
    - name: DBCRED
      valueFrom:
        secretKeyRef:
          key: DBCRED
          name: am-credentials
    command: [ "/bin/sh", "-c"]
    args: ["cd opt/am/am/tomcat/db-init; ./dbUtil.sh;"]
    volumeMounts:
    - mountPath: opt/am/am/tomcat/logs
      name: <pvc_name>
  volumes:
  - name: <pvc_name>
    persistentVolumeClaim:
      claimName: <pvc_name>
  restartPolicy: Never
```

Note: Replace <pvc_name> with the name of the PVC created in the previous section, eg. (deploymentlogs, sandboxlogs)

Create AccessMatrix Deployment

Below is a sample Deployment YAML for AccessMatrix where the TLS connection ends in Ingress controller (which means, the connection from the Ingress controller to AccessMatrix Pods will unencrypted):

am-k8s.yaml

YAML

```
apiVersion: "apps/v1"
kind: "Deployment"
metadata:
  name: "accessmatrix"
  namespace: "am"
  labels:
    app: "accessmatrix"
```

am-k8s.yaml

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: "accessmatrix"
  template:
    metadata:
      labels:
        app: "accessmatrix"
    spec:
      serviceAccountName: runasanyuid
      imagePullSecrets:
        - name: runasanyuid-dockercfg-rpx0z
      containers:
      - name: am5
        image: image-registry.openshift-image-
registry.svc:5000/am/am5:5.7.2
        env:
        - name: DBCRED
          valueFrom:
            secretKeyRef:
              key: DBCRED
              name: am-credentials
        readinessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 240
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 220
        livenessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 240
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 220
        ports:
        - containerPort: 8080
          protocol: TCP
        volumeMounts:
        - mountPath: opt/am/am/tomcat/logs
          name: <pvc_name>
        - mountPath: opt/am/am/tomcat/work/reports
          name: reports
        securityContext:
          privileged: true
        resources:
          requests:
            cpu: 100m
      volumes:
      - name: <pvc_name>
        persistentVolumeClaim:
```

am-k8s.yaml

```
          claimName: <pvc_name>
      - name: reports
        persistentVolumeClaim:
          claimName: reports
```

Note: Replace <pvc_name> with the name of the PVC created in the previous section, e.g. (deploymentlogs, sandboxlogs, reports).

Create and Expose Service

Once the deployment has been created in the cluster, it cannot be accessed yet. We will need to create a network service that enables its network access internally, and an OpenShift Route that will expose AccessMatrix to the outside of the cluster.

The **am-service.yaml** below sets up the service by exposing both 80 and 443 port that maps to the Tomcat port 8080 and 8443 in the Pods.

am-service.yaml

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: am-service
  namespace: am
spec:
  selector:
    app: accessmatrix
  ports:
    - name: am-port-ssl
      protocol: TCP
      port: 443
      targetPort: 8443
    - name: am-port-nonssl
      protocol: TCP
      port: 80
      targetPort: 8080
```

Option 1: Create Route with TLS Passthrough Enabled

The **am-route-passthrough.yaml** below defines an OpenShift Route, that binds AccessMatrix to the host name *am.exmaple.com*, let all the TLS (redirect if non-TLS) traffic pass through the Route to Tomcat directly.

am-route-passthrough.yaml

YAML

```yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: am-route-passthrough
spec:
  host: am.example.com
  port:
    targetPort: am-port-ssl
  tls:
    termination: passthrough
    insecureEdgeTerminationPolicy: Redirect
  to:
    kind: Service
    name: am-service
```

Option 2: Create Route with ACME supported TLS ends in Route

Alternatively, we can configure TLS with Automated Certificate Management Environment (ACME) enabled. The certificate will be automatically issued and renewed by the OpenShift-ACME service deployed inside the cluster. The TLS encryption will terminate inside the Route before sending traffic to AccessMatrix Pods.

The OpenShift-ACME can be installed under the current project namespace using the command below:

Command

```
$ oc apply -fhttps://raw.githubusercontent.com/tnozicka/openshift-acme/master/deploy/single-namespace/{role,serviceaccount,issuer-letsencrypt-live,deployment}.yaml
```

Response

```
role.rbac.authorization.k8s.io/openshift-acme created
serviceaccount/openshift-acme created
configmap/letsencrypt-live created
deployment.apps/openshift-acme created
```

We need to bind the service account to the `openshift-acme` role that is just created:

Command

```
$ oc create rolebinding openshift-acme --role=openshift-acme --serviceaccount="$( oc project -q ):openshift-acme" --dry-run -o yaml | oc apply -f -
```

Command
Response
`rolebinding.rbac.authorization.k8s.io/openshift-acme created`

Now the OpenShift-ACME has been installed under your current project namespace. If you would prefer to install it cluster wide, please refer to the documentation on GitHub.

Once the OpenShift-ACME is installed, we can create the Route with tls-acme:"true" annotated:

am-route-acme.yaml

YAML

```yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: am-route-acme
  annotations:
    kubernetes.io/tls-acme: "true"
spec:
  host: am.example.com
  path: /
  port:
    targetPort: am-port-nonssl
  to:
    kind: Service
    name: am-service
```

Now, you will be able to access AccessMatrix Admin Console via *https://am.example.com/am5* in your browser.

Horizontal Pod Autoscaler

The Horizontal Pod Autoscaler automatically scales the number of pods in a replication controller, deployment, replica set or stateful set based on observed CPU utilization (or, with custom metrics support, on some other application-provided metrics)

The sample **am-hpa.yaml** below defines an Horizontal Autoscaler that will scale to four Pod replicas when CPU utilization hits 80% on current Pods:

am-hpa.yaml

am-hpa.yaml

YAML

```yaml
apiVersion: "autoscaling/v2beta1"
kind: "HorizontalPodAutoscaler"
metadata:
  name: "am-hpa"
  namespace: "default"
  labels:
    app: "am-hpa"
spec:
  scaleTargetRef:
    kind: "Deployment"
    name: "accessmatrix"
    apiVersion: "apps/v1"
  minReplicas: 1
  maxReplicas: 4
  metrics:
  - type: "Resource"
    resource:
      name: "cpu"
      targetAverageUtilization: 80
```

Use `oc get hpa` to view current status of the scaler:

Command

```
$ oc get hpa
```

Response

```
NAME      REFERENCE                 TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
am-hpa    Deployment/accessmatrix   8%/80%    1         4         1          14d
```

Verify OpenShift Deployment

- Use `$ oc get pods` to verify all the Pods are running.
- Visit AccessMatrix Admin Console via your domain name (e.g. *k8s.i-sprint.com*) to verify it is successfully deployed.

4.2. On-Premise OpenShift Deployment

This guide contains information on deploying AccessMatrix in an on-premise OpenShift cluster. The following sections detail the various steps required for the deployment:

- Prepare Database/Cache Services and Configure AccessMatrix
- Create AccessMatrix Docker Image
- Create and Deploy OpenShift Objects
- Set up OpenShift Logging
- Enable HSM Connectivity

OpenShift Internal Container Registry

This guide uses the i-Sprint OpenShift internal container registry. The container registry is currently configured as such:

- **Internal address**: image-registry.openshift-image-registry.svc:5000
- **External address** (hereafter assigned to the variable REGEXTURL): default-route-openshift-image-registry.apps.openshift.i-sprint.io

Before proceeding with the steps detailed in the next pages, first do the following:

1. Obtain the OpenShift login command by visiting https://oauth-openshift.apps.openshift.i-sprint.io/oauth/token/request; the command will look something like this:
   `oc login --token=<sha256~token> --server=https://api.openshift.i-sprint.io:6443`
2. Log in to Docker by running the following command:
   `docker login -u $(oc whoami) -p $(oc whoami -t) ${REGEXTURL}`

4.2.1. Prepare Database/Cache Services and Configure AccessMatrix

Redis Cache Server

AccessMatrix utilizes Redis Cache as an in-memory database for storing transient data, which includes:

- E2EE sessions
- SMS-OTP tokens
- SAML response IDs (to prevent replay attacks)
- CLP login sessions for AM applications like UAS TAP and USO SSF
- OIDC Relying Party Lookup Module nonces (to prevent request forgery or replay)

For detailed information on Redis, please consult the official Redis documentation at https://redis.io/documentation.

Deploy Redis

Before deploying the Red Hat-certified Redis 6 image in the OpenShift cluster, the image should first be pulled from the Red Hat registry and then pushed to the internal registry.

1. Authenticate to the Red Hat registry by running the command:
   `docker login registry.redhat.io`
2. Log in with your Red Hat portal credentials when prompted to do so.
3. Pull the image by running the command:
   `docker pull registry.redhat.io/rhel8/redis-6:1-125`
4. Tag the image by running the command:
   `docker tag registry.redhat.io/rhel8/redis-6:1-125 ${REGEXTURL}/accessmatrix/rh8redis6:1-125`
5. Push the image by running the command:
   `docker push ${REGEXTURL}/accessmatrix/rh8redis6:1-125`

The following YAML file serves as a sample for deploying the image in the OpenShift cluster:

redis-k8s.yaml

redis-k8s.yaml

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: accessmatrix
  labels:
    app: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: image-registry.openshift-image-
registry.svc:5000/accessmatrix/rh8redis6:1-125
        imagePullPolicy: Always
        env:
        - name: REDIS_PASSWORD
          value: password1
        ports:
          - containerPort: 6379
        securityContext:
          privileged: false
          allowPrivilegeEscalation: false
          capabilities:
            drop: ["ALL"]
          runAsNonRoot: true
          seccompProfile:
            type: RuntimeDefault
```

Apply this YAML by running the command:

```
oc apply -f redis-k8s.yaml
```

Configure the Redis Service

The following YAML serves as a sample for configuring a service for Redis:

redis-service.yaml

YAML

redis-service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: redis-service
  labels:
    app: redis
spec:
  ports:
    - port: 6379
      targetPort: 6379
  selector:
    app: redis
```

Apply this YAML by running the command:

`oc apply -f redis-service.yaml`

Configure Redis Connection in AccessMatrix

The final step is to update the relevant configurations in AccessMatrix to point to the Redis service. Open *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes/amsystem.properties* and make the following changes (annotated for clarity):

Properties (.properties files)

```properties
am.cache.vendor=redis

# Name of the service exposed in the section 3.2 YAML
am.redis.host=redis-service

# Service port exposed in the section 3.2 YAML
am.redis.port=6379

# Password as defined in the section 3.1 YAML
# Setting the password is optional
am.redis.password=password1
```

Apache ActiveMQ Server

Certain objects in AccessMatrix, such as policies, configurations, and login modules, are frequently accessed but not frequently updated. To enhance performance and efficiency, these objects are stored in Ehcache, an in-memory cache within AccessMatrix. For Ehcache replication using the Java Message Service (JMS), AccessMatrix utilizes Apache ActiveMQ Classic.

In the OpenShift cluster, Apache ActiveMQ is deployed as a container. Hence, it needs to be prepared and pushed to the OpenShift Internal Container Registry, similar to the AccessMatrix Docker image.

> **Important Note:** AccessMatrix currently supports the ActiveMQ Classic 5 release.

Build ActiveMQ Docker Image

The following Dockerfile serves as a sample for ActiveMQ 5.17.6. Assign the version number to the variable ACTIVEMQ_VERSION.

Dockerfile

YAML

```
FROM library/eclipse-temurin:17.0.7_7-jre-alpine

ENV ACTIVEMQ_VERSION 5.17.6
ENV ACTIVEMQ apache-activemq-${ACTIVEMQ_VERSION}
ENV ACTIVEMQ_HOME /opt/activemq

ADD repo/${ACTIVEMQ}-bin.tar.gz /opt

RUN ln -s /opt/${ACTIVEMQ} ${ACTIVEMQ_HOME} && \
  addgroup -S activemq && \
  adduser -S -H -G activemq -h ${ACTIVEMQ_HOME} activemq &&
\
  chown -R activemq:activemq /opt/${ACTIVEMQ} && \
  chown -h activemq:activemq ${ACTIVEMQ_HOME}

# Copy modified configurations
COPY repo/env ${ACTIVEMQ_HOME}/bin/env

#Allow root access for OpenShift Arbitrary UID
RUN chgrp -R 0 ${ACTIVEMQ_HOME} && \
  chgrp -R 0 /opt/${ACTIVEMQ} && \
  chmod -R g=u ${ACTIVEMQ_HOME} && \
  chmod -R g=u /opt/${ACTIVEMQ}

USER activemq

WORKDIR ${ACTIVEMQ_HOME}
EXPOSE 8161 61616

CMD ["/bin/sh", "-c", "bin/activemq console"]
```

In this Dockerfile, ActiveMQ runs on a lightweight Alpine image, and the following ports are exposed:

Port	Description
8161	ActiveMQ Dashboard
61616	Openwire/JMS

Moreover, in OpenShift, running Docker images/containers as the root user is not allowed. Additionally, images cannot use an unidentified username like "activemq" to prevent the use of reserved user IDs. To enable the usage of an arbitrary user ID, certain directories require root access. For more information on this behavior, please consult the OpenShift guidelines.

To push an ActiveMQ Docker image to the container registry, it must be built locally. Begin by placing the ActiveMQ package **apache-activemq-${ACTIVEMQ_VERSION}-bin.tar.gz** in the repo subdirectory.

Customize Configurations for ActiveMQ

ObjectMessage messages are utilized in the JMS replication mechanism of AccessMatrix. However, it is important to note that ObjectMessage objects rely on Java serialization for marshaling and un-marshaling the payload, which is generally considered unsafe due to the potential for malicious payloads to exploit the host system. To address this concern, ActiveMQ now enforces users to explicitly whitelist packages that can be exchanged using ObjectMessage.

For more detailed information about ObjectMessage, please refer to https://activemq.apache.org/objectmessage. However, for internal usage within AM, the whitelist mechanism is bypassed by customizing the configuration to trust all packages.

Follow these steps to configure and build the ActiveMQ image:

1. Extract the file *bin/env* from the tar file archived in **apache-activemq-${ACTIVEMQ_VERSION}-bin.tar.gz** and place it in the *repo* subdirectory.
2. Modify *repo/env* by appending the same JVM argument to ACTIVEMQ_OPTS:

   Code

   ```
   ACTIVEMQ_OPTS="$ACTIVEMQ_OPTS -
   Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*"
   ```

3. Build the image by running the following command:

   `docker build . -f Dockerfile -t activemq:${ACTIVEMQ_VERSION}`

Push ActiveMQ Docker Image to Registry

Once the Docker image has been built, you can proceed to tag the image and push it to the OpenShift container registry using the docker tag and docker push commands:

```
docker tag activemq:${ACTIVEMQ_VERSION}
${REGEXTURL}/accessmatrix/activemq:${ACTIVEMQ_VERSION}

docker push ${REGEXTURL}/accessmatrix/activemq:${ACTIVEMQ_VERSION}
```

Deploy ActiveMQ

To deploy ActiveMQ in the cluster, you can use the provided sample ActiveMQ deployment.

Here is the sample ActiveMQ deployment YAML:

activemq-deploy.yaml

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: activemq
  namespace: accessmatrix
  labels:
    app: activemq
spec:
  replicas: 1
  selector:
    matchLabels:
      app: activemq
  template:
    metadata:
      annotations:
        pod.beta.kubernetes.io/hostname: activemq
      labels:
        app: activemq
    spec:
      containers:
      - name: activemq
        image: image-registry.openshift-image-
registry.svc:5000/accessmatrix/activemq:5.17.6
        imagePullPolicy: Always
        resources:
          requests:
            memory: 500Mi
            cpu: 200m
          limits:
            memory: 1000Mi
```

activemq-deploy.yaml

```
            cpu: 400m
        securityContext:
          privileged: false
          allowPrivilegeEscalation: false
          capabilities:
            drop: ["ALL"]
          runAsNonRoot: true
          seccompProfile:
            type: RuntimeDefault
```

Apply the changes by running the following command:

`oc apply -f activemq-deploy.yaml`

Running the `oc apply` command with the provided YAML file will apply the changes and deploy the ActiveMQ instance in the accessmatrix namespace.

Configure and Expose ActiveMQ Service

To expose the ActiveMQ deployment as a ClusterIP service for AccessMatrix to access and use it as a message broker, you can utilize the provided sample ActiveMQ service.

Here is the sample ActiveMQ service YAML:

activemq-service.yaml

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: activemq-svc
  namespace: accessmatrix
  labels:
    app: activemq-svc
spec:
  selector:
    app: activemq
  ports:
  - name: dashboard
    port: 8161
    targetPort: 8161
    protocol: TCP
  - name: jms
    port: 61616
    targetPort: 61616
  type: ClusterIP
```

Apply the changes by running the following command:

`oc apply -f activemq-service.yaml`

Running the `oc apply` command with the provided YAML file will apply the changes and create the ActiveMQ service with a ClusterIP.

You can verify the status of the service by running the command `oc get svc`. This command will display a list of services, including the activemq-svc service. Check the "CLUSTER-IP" column to confirm that the service has been successfully created and is running.

(Optional) Once this has been verified, proceed to expose the service by running the command `oc expose service activemq-svc`. Exposing the service allows for access to the ActiveMQ web console, which may be useful for troubleshooting.

Configure ActiveMQ Connection in AccessMatrix

The final step is to update the relevant configurations in AccessMatrix to point to the ActiveMQ container. Follow these steps:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes*.
2. Open the **amsystem.properties** file.
3. Set the following properties (find and uncomment if it is already present):

Properties (.properties files)

```
# Cache Replicator Implementation: RMI,JMS
am.ehcache.replicator.factory=JMS
...
# EhCache JMS Cache Peer Provider
am.ehcache.jms.initialContextFactoryName=com.isprint.am.core.cache.
ehcache.AtiveMQInitialContextFactory
am.ehcache.jms.providerURL=failover:tcp://localhost:61616
am.ehcache.jms.userName=amquser1
am.ehcache.jms.password=Password.1234
am.ehcache.jms.replicationTopicBindingName=amtopic
am.ehcache.jms.getQueueBindingName=amqueue
#am.ehcache.jms.acknowledgementMode
#       : (Optional) The mode used to acknowledge messages after
they are consumed.
#    Configure with one of the following values:
#        - AUTO_ACKNOWLEDGE
#        - DUPS_OK_ACKNOWLEDGE (Default)
#        - SESSION_TRANSACTED
#am.ehcache.jms.acknowledgementMode=AUTO_ACKNOWLEDGE
```

Notes:

- Replace am.ehcache.jms.providerURL=failover:tcp://localhost:61616 with your domain name or IP. For example: failover:(ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-1.mq.ap-southeast-1.amazonaws.com:61617,ssl://b-5f01b4f3-9d49-4c8b-8ad6-276e6dfce7e2-2.mq.ap-southeast-1.amazonaws.com:61617).
- The "Ative" in am.ehcache.jms.initialContextFactoryName=com.isprint.am.core.cache.ehcache.AtiveMQInitialContextFactory is not a misspelling.

4. Save the changes.

Next, configure the ActiveMQ trusted package:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\bin*.
2. Open the **setenv.sh** file.
3. Search for JAVA_OPTS and add -Dorg.apache.activemq.SERIALIZABLE_PACKAGES=* under it:

Bash (Unix Shell)

```
JAVA_OPTS="$JAVA_OPTS $JAVA_GC_OPTS -Xms400m -Xmx1g -
Dderby.system.home=db
-Dam.ehcache.set_rmi_ip_addr=true -
XX:MaxMetaspaceSize=256M -XX:NewSize=128M
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=logs -
Dhost.name=$(cat /etc/hostname)
-Djavax.net.ssl.trustStore=conf/amtrust.keystore -
Djavax.net.ssl.trustStorePassword=passphrase
-Djava.awt.headless=true" -
Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*
```

4. Save the changes.

Lastly, add the ActiveMQ libraries in the *WEB-INF\lib* folder:

1. Navigate to the directory *<ACMATRIX_ROOT>\tomcat\webapps\tomcat\am5\WEB-INF\lib*.
2. Obtain and copy the ActiveMQ libraries listed below into the directory:
    - **activemq-broker-5.17.6.jar**
    - **activemq-client-5.17.6.jar**
    - **hawtbuf-1.11.jar**

4.2.2. Create AccessMatrix Docker Image

With ActiveMQ and Redis deployed, we can now configure AccessMatrix to point to these services.

Build AccessMatrix Docker Image

A sample AccessMatrix Dockerfile (annotated with comments explaining how to meet the requirements of the Red Hat Container Certification grade "A") is shown as part of the steps below. Arrange the files as follows:

1. Place the Dockerfile in any working directory that you choose. Change the environment variable AM_VERSION to the respective AccessMatrix version (e.g. 5.7.5.7535-GA) and assign this value to the variable TAG as well (TAG=5.7.5.7535-GA).

Dockerfile

Code

```
# Certification Requirement: Build image based on UBI
# Use Red Hat Universal Base Image so that it is compatibile with
OpenShift
# Best practices: Use image hash instead of tag to prevent sudden
changes, i.e. tag might be reassigned to another build
# The below hash is equivalent to the following amd64 architecture
released on 04-April-2023 with the tag 8.7-1112

FROM
registry.access.redhat.com/ubi8@sha256:e3311058176628ad7f0f288f894ed2a
fef61be77ad01d53d5b69bca0f6b6cec1

# Certification Requirement: Image metadata requirements
# Must include these labels: name, vendor, version, release, summary,
description
# Best practices: Label the image

LABEL name="5.7.5.7535-GA ubi8 with Temurin OpenJDK 17" \
      vendor="i-Sprint" \
      maintainer="PE-SG <pe-sg@i-sprint.com>"

# Best practices: Consolidate configuration at the top, so it is easy
to find and change.
# ENV PATH has to be on a new line because it uses "JAVA_HOME" and
cannot be on the same line as "JAVA_HOME" declaration

ENV AM_VERSION="5.7.5.7535-GA" \
    JAVA_HOME="/opt/jdk/jdk-17.0.7+7"
ENV PATH="$PATH:${JAVA_HOME}/bin"

RUN mkdir -p /opt/jdk/ \
    && mkdir -p /opt/am/${AM_VERSION}
```

Dockerfile

```
ADD repo/OpenJDK17U-jdk_x64_linux_hotspot_17.0.7_7.tar.gz /opt/jdk/
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}

# Certification Requirement: must contain a "licenses" directory
# Best practices: Set group ownership and file permissions
# In the Dockerfile, set permissions on the directories and files that
the process uses.
# The root group must own those files and be able to read and write
them as needed.
# For compatibility with Kubernetes, the Dockerfile should specify a
non-root user ID, then set file ownership to that user ID and the root
group

RUN mv /opt/am/${AM_VERSION}/licenses /licenses \
    && groupadd -g 2020 am \
    && useradd -r -g am -u 2021 -s /bin/bash am \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:0 /opt/am/${AM_VERSION} \
    && chown -R am:0 /opt/am/am \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/db-init/*.sh \
    && chmod -R g=u /opt/am/am

COPY ["lib", "/opt/am/am/tomcat/lib"]

EXPOSE 8080 8443 8161 61616

# Certification Requirement: Run as non-root user
# Best practices: Run as non-root user
# If a process running as root breaks out of the container, it gains
root (privileged) access to the host machine. Therefore, run the
container as a non-root user so that, if the process breaks out of the
container, its access to the host machine is much more limited.

USER am

CMD ["/opt/am/am/tomcat/bin/catalina.sh", "run"]

# Other Certification Requirements:
# - must not change content provided by Red Hat packages or layers
# - must have less than 40 layers
# - must not include RHEL kernel packages
# - must not contain Red hat components with identified important or
critical vulnerabilities
```

2. Create a subdirectory *lib* and place all dependent libraries, such as JDBC drivers, in this directory.
3. Create another subdirectory *repo* and place the AccessMatrix base package **AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz** inside it.
4. Download the Eclipse Temurin Prebuilt OpenJDK Binaries **OpenJDK17U-jdk_x64_linux_hotspot_17.0.7_7.tar.gz** and put it in the *repo* subdirectory.

5. To build the image, run the following command:

   ```
   docker build . -f Dockerfile -t am5:${TAG}
   ```

> **Note:** For more information on Red Hat certification process, refer to the notes under OpenShift Deployment with Azure - Build AccessMatrix Docker Image.

Push AccessMatrix Docker Image to Registry

Use `docker tag` and `docker push` to tag the docker image and push it to the OpenShift internal container registry:

1. Tag the Docker image using the following command:

   ```
   docker tag am5:${TAG} ${REGEXTURL}/accessmatrix/am5:${TAG}
   ```

2. Push the Docker image by running the following command:

   ```
   docker push ${REGEXTURL}/accessmatrix/am5:${TAG}
   ```

4.2.3. Create and Deploy OpenShift Objects

To deploy AccessMatrix, you should ensure that the following requirements are met:

1. The PVC(s) for the AM deployment is/are deployed
2. The secret(s) is/are configured

The steps to do so are detailed in the sections below.

Persistent Volumes and Claims

Deploy PVC for AM

Below are the deployment YAML templates for deploying a PVC in the OpenShift platform; use them to create the two PVCs required in the AM deployment YAML to come:

- deploymentlogs for log files like the AM log.
- reports for AM-generated reports.

am-deployment-pvc.yaml

YAML

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: deploymentlogs
  namespace: accessmatrix
spec:
  accessModes:
    - ReadWriteMany
  # This storage class has already been configured in the
cluster
  storageClassName: ocs-storagecluster-cephfs
  resources:
    requests:
      storage: 1Gi
```

am-report-pvc.yaml

YAML

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: reports
```

am-report-pvc.yaml

```
  namespace: accessmatrix
spec:
  accessModes:
    - ReadWriteMany
  # This storage class has already been configured in the
cluster
  storageClassName: ocs-storagecluster-cephfs
  resources:
    requests:
      storage: 1Gi
```

Deployment

Configure OpenShift Secrets for AM

In the AM deployment, the environment variable DBCRED is used for passphrase decryption. As such, its value should be securely passed via the Secret mechanism.

1. Prepare a YAML to create an opaque Secret with the key DBCRED and the desired value. The following is a sample YAML for this purpose:

   am-secret.yaml

   YAML

   ```
   apiVersion: v1
   kind: Secret
   metadata:
     name: secret-ingredient
   type: Opaque
   stringData:
     DBCRED: am5salt
   ```

2. Apply the YAML file using the command:

   `oc apply -f am-secret.yaml`

3. (Optional) Configure the ServiceAccount used to deploy AM; the default ServiceAccount used is default.

   Note: For more information, refer to the Red Hat OCP documentation at https://docs.openshift.com/container-platform/4.11/authentication/understanding-and-creating-service-accounts.html.

4. Add the Secret to the ServiceAccount by applying the following YAML with `oc apply -f am-service-account.yaml`:

am-service-account.yaml

YAML

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  # Replace this with the SA configured in step 2, if any
  name: default
secrets:
# Secret name from step 1
- name: secret-ingredient
```

5. Verify that the Secret has been added successfully by running the command:

   `oc get sa/<ServiceAccount name> -o json`

   The following is a sample of the output that you should see:

```
$ oc get sa/default -o json
{
    "apiVersion": "v1",
    "imagePullSecrets": [
        {
            "name": "default-dockercfg-zg9pl"
        }
    ],
    "kind": "ServiceAccount",
    "metadata": {
        "annotations": {
            "kubectl.kubernetes.io/last-applied-configuration":
"{\"apiVersion\":\"v1\",\"kind\":\"ServiceAccount\",\"metadata\":{\"annotations\"
:{},\"name\":\"default\",\"namespace\":\"accessmatrix\"},\"secrets\":[{\"name\":\
"secret-ingredient\"}]}\n"
        },
        "creationTimestamp": "2023-02-20T06:16:09Z",
        "name": "default",
        "namespace": "accessmatrix",
        "resourceVersion": "181694165",
        "uid": "1a7a2db3-d83d-4946-924b-99f85c2b3e5e"
    },
    "secrets": [
        {
            "name": "secret-ingredient"
        },
        {
            "name": "default-dockercfg-zg9pl"
        }
    ]
}
```

Check the value of "name" under the "secrets" object, which should be the same as the name of the Secret created in step 1.

Create AccessMatrix Deployment

The following deployment YAML serves as a sample for deploying AccessMatrix with ActiveMQ and Redis.

am-k8s.yaml

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: am575
  namespace: accessmatrix
  labels:
    app: accessmatrix
spec:
  replicas: 3
  selector:
    matchLabels:
      app: am575
  template:
    metadata:
      labels:
        app: am575
    spec:
      containers:
      - name: am5
        # Replace TAG with the appropriate version
        image: image-registry.openshift-image-
registry.svc:5000/accessmatrix/am5:TAG
        imagePullPolicy: Always
        env:
        # Secret created in section 4.2.1
        - name: DBCRED
          valueFrom:
            secretKeyRef:
              key: DBCRED
              name: am-credentials
        # JVM args for ActiveMQ, see section 2.5
        - name: JAVA_OPTS
          value: "-
Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*"
        readinessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 240
          successThreshold: 1
```

am-k8s.yaml

```
          failureThreshold: 3
          initialDelaySeconds: 220
        livenessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 240
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 220
        ports:
        - containerPort: 8080
          protocol: TCP
        volumeMounts:
        - mountPath: opt/am/am/tomcat/logs
          name: deploymentlogs
        - mountPath: opt/am/am/tomcat/work/reports
          name: reports
        securityContext:
          privileged: false
          allowPrivilegeEscalation: false
          capabilities:
            drop: ["ALL"]
          runAsNonRoot: true
          seccompProfile:
            type: RuntimeDefault
        resources:
          requests:
            cpu: 100m
      # PVCs created in section 4.2.1
      volumes:
      - name: deploymentlogs
        persistentVolumeClaim:
          claimName: deploymentlogs
      - name: reports
        persistentVolumeClaim:
          claimName: reports
```

Create and Expose AM Service

The following deployment YAML is a sample for exposing AM as a service:

am-service.yaml

YAML

```
apiVersion: v1
kind: Service
metadata:
  name: am575-svc
```

am-service.yaml

```
  namespace: accessmatrix
spec:
  selector:
    app: am575
  ports:
    - name: am-port-ssl
      protocol: TCP
      port: 443
      targetPort: 8443
    - name: am-port-nonssl
      protocol: TCP
      port: 80
      targetPort: 8080
```

The next deployment YAML is a sample for using a Route to expose AM.

am-route-passthrough.yaml

YAML

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: am-route-passthrough
spec:
  host: am575-accessmatrix.apps.openshift.i-sprint.io
  port:
    targetPort: am-port-ssl
  tls:
    # TLS traffic is allowed to pass through to the AM pods
    termination: passthrough
    # Insecure traffic is redirected to HTTPS
    insecureEdgeTerminationPolicy: Redirect
  to:
    kind: Service
    name: am575-svc
```

For more information on exposing the AM Route, refer to the Red Hat OCP documentation at https://docs.openshift.com/container-platform/4.11/networking/routes/secured-routes.html.

4.2.4. Set Up OpenShift Logging

Once AccessMatrix has been successfully deployed, the next step is to set up its logging. Depending on your requirements, you should choose either one of the following approaches:

- Namespace-specific ELK stack
- Cluster-wide logging

Namespace-Specific ELK Stack

This section details the setup steps required to integrate a namespace-specific ELK stack with the AccessMatrix OpenShift deployment. ELK is an open-source stack (comprising the tools Elasticsearch, Logstash, and Kibana) used for log management and analysis.

Before deploying the ELK stack into OpenShift, it is important to first pull the images from the official Elastic registry, and then push them to our internal registry. These images include:

- elastic/elasticsearch:${ELK_VERSION}
- elastic/kibana:${ELK_VERSION}
- elastic/logstash:${ELK_VERSION}
- elastic/filebeat:${ELK_VERSION}

> **Notes:**
>
> - The ELK_VERSION used in this guide is 8.8.1.
> - Filebeat is a lightweight shipper for logs; here it is installed as a sidecar alongside AM.

To do so:

1. Pull the image from Elastic by running the docker pull command. For example:
   `docker pull elastic/elasticsearch:${ELK_VERSION}`
2. Tag the image by running the docker tag command. For example:
   `docker tag elastic/elasticsearch:${ELK_VERSION} ${REGEXTURL}/accessmatrix/elasticsearch:${ELK_VERSION}`
3. Push the image to our internal registry by running the docker push command. For example:
   `docker push ${REGEXTURL}/accessmatrix/elasticsearch:${ELK_VERSION}`

It is recommended that the components of the ELK stack be installed in this order:

1. ElasticSearch
2. Kibana
3. Logstash

ElasticSearch

The following is a sample deployment YAML for ElasticSearch:

am-elasticsearch.yaml

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: elasticsearch
spec:
  selector:
    matchLabels:
      app: elasticsearch
  replicas: 1
  template:
    metadata:
      labels:
        app: elasticsearch
    spec:
      containers:
        - name: elasticsearch
          image: image-registry.openshift-image-
registry.svc:5000/accessmatrix/elasticsearch:8.8.1
          resources:
            limits:
              memory: 2Gi
            requests:
              memory: 1Gi
          env:
            # Configures Elasticsearch to run in single-node
mode
            - name: discovery.type
              value: single-node
            - name: xpack.security.enabled
              value: "false"
            - name: xpack.security.enrollment.enabled
              value: "false"
            - name: ELASTIC_PASSWORD
              value: password1
          ports:
            # Port on which ES will listen for HTTP traffic
            - containerPort: 9200
              name: http
            # Port used by ES for internal cluster
communication
```

am-elasticsearch.yaml

```
            - containerPort: 9300
              name: transport
          securityContext:
            privileged: false
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
            runAsNonRoot: true
            seccompProfile:
              type: RuntimeDefault
          resources:
            requests:
              cpu: 100m
```

The following is a sample service definition YAML for ElasticSearch:

am-elasticsearch-service.yaml

YAML

```
apiVersion: v1
kind: Service
metadata:
  name: elasticsearch
spec:
  selector:
    app: elasticsearch
  ports:
    - protocol: TCP
      port: 9200
      targetPort: 9200
```

Since ElasticSearch itself is not being accessed by the user, there is no need to expose the service.

For more information on configuring ElasticSearch, refer to the Elastic documentation at https://www.elastic.co/guide/en/elasticsearch/reference/current/settings.html.

Kibana

The following is a sample deployment YAML for Kibana, referencing the ElasticSearch deployment in the previous section.

am-kibana.yaml

am-kibana.yaml

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kibana
spec:
  selector:
    matchLabels:
      app: kibana
  replicas: 1
  template:
    metadata:
      labels:
        app: kibana
    spec:
      containers:
        - name: kibana
          image: image-registry.openshift-image-
registry.svc:5000/accessmatrix/kibana:8.8.1
          resources:
            # Maximum memory limit for the Kibana pod
            limits:
              memory: 1Gi
            # Minimum memory requirement for the Kibana pod
            requests:
              memory: 512Mi
          env:
            - name: ELASTICSEARCH_HOSTS
              value: "http://elasticsearch:9200"
          ports:
            - containerPort: 5601
              name: http
          securityContext:
            privileged: false
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
            runAsNonRoot: true
            seccompProfile:
              type: RuntimeDefault
          resources:
            requests:
              cpu: 100m
```

The following is the corresponding service definition for Kibana:

am-kibana-service.yaml

YAML

am-kibana-service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: kibana
spec:
  selector:
    app: kibana
  ports:
    - protocol: TCP
      port: 5601
      targetPort: 5601
```

The Kibana interface is designed to allow users to access logs that are sent to ElasticSearch. Therefore, it should be exposed using a Route. You can expose it either by running the `oc expose service kibana` command, or by deploying a Route object.

For more information on configuring Kibana, refer to the Elastic documentation at https://www.elastic.co/guide/en/kibana/current/settings.html.

Logstash

Once the ElasticSearch and Kibana services have been set up, the next step is to establish the Logstash service.

The following YAML sample demonstrates this:

am-logstash.yaml

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: logstash
spec:
  selector:
    matchLabels:
      app: logstash
  replicas: 1
  template:
    metadata:
      labels:
        app: logstash
    spec:
      containers:
```

am-logstash.yaml

```
      - name: logstash
        image: image-registry.openshift-image-
registry.svc:5000/accessmatrix/logstash:8.8.1
        command:
        - "bin/logstash"
        - "--config.reload.automatic"
        volumeMounts:
        - mountPath: /usr/share/logstash/pipeline
          name: logstash-pipeline-vol
        resources:
          limits:
            memory: 1Gi
          requests:
            memory: 512Mi
        env:
        - name: ELASTICSEARCH_HOST
          value: "elasticsearch:9200"
        ports:
        - containerPort: 5044
          name: beats
        securityContext:
          privileged: false
          allowPrivilegeEscalation: false
          capabilities:
            drop: ["ALL"]
          runAsNonRoot: true
          seccompProfile:
            type: RuntimeDefault
        resources:
          requests:
            cpu: 100m
      volumes:
      - name: logstash-pipeline-vol
        configMap:
          name: logstash-pipeline
```

Notice that the Logstash pipeline(s) are referenced using a ConfigMap, and then mounted onto the container as a volume without subpath. This works in tandem with the command-line argument "`--config.reload.automatic`" to allow pipeline changes to be "hot-swappable" by simply updating the ConfigMap, instead of having to restart Logstash.

The following sample shows what a pipeline file **logstash.conf** might look like:

logstash.conf

Code

```
input {
  beats {
```

logstash.conf

```
    port => 5044
  }
}

filter {
  mutate {
    remove_field => ["host"]
  }
  grok {
    match => {
      "message" => [
        "%{TIMESTAMP_ISO8601:timestamp} \[%{DATA:thread}\]
%{LOGLEVEL:loglevel} - \(%{DATA:class}\:%{INT:line}\)
%{GREEDYDATA:message}",

"%{TIMESTAMP_ISO8601:timestamp}\s+\[%{DATA:thread}\]\s+%{LOGLEVEL:loglevel}\
s+-\s+\(%{DATA:class}\.java:%{INT:line}\)\s+%{GREEDYDATA:message}",

"%{TIMESTAMP_ISO8601:timestamp}\s+\[%{DATA:thread}\]\s+%{LOGLEVEL:loglevel}\
s+-\s+%{GREEDYDATA:message}",
        "%{MONTHDAY:day}-%{MONTH:month}-%{YEAR:year} %{TIME:time}
%{WORD:am_pm} %{GREEDYDATA:log_message}"
      ]
    }
  }
}

output {
  elasticsearch {
    hosts => "http://elasticsearch:9200"
    ilm_rollover_alias => "logs"
    ilm_pattern => "000001"
    ilm_policy => "logstash_policy"
  }
  #stdout {
  #  codec => rubydebug
  #}
}
```

In this sample, the filter processes the vast majority of the log output that can be seen in AM. If a new category of log output is introduced in AM, then the pipelines should be updated to capture it; for this, you can either create a new conf file, or update the existing conf file.

To create the logstash-pipeline ConfigMap to hold these pipeline files, run the following command (append more "`--from-file=<filename>.conf`" if you have multiple pipeline files):

```
oc create configmap logstash-pipeline --from-file=logstash.conf
```

For more information on Logstash pipelines, refer to the Elastic documentation at https://www.elastic.co/guide/en/logstash/current/config-examples.html.

Filebeat

From the Elastic documentation, Filebeat is a lightweight shipper for forwarding and centralizing log data. It monitors the log files or locations that are specified, collects log events, and forwards them either to Elasticsearch or Logstash for indexing. The most straightforward way to channel logs from AM to the ELK stack is to deploy Filebeat running as a sidecar alongside AM.

The following deployment YAML for AM is based on the sample AM deployment YAML in the Create AccessMatrix Deployment section. The sample specifically highlights changes that should be made for Filebeat, which are also flagged out via the comments for clarity:

am-k8s.yaml

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  # See earlier section for full listing of metadata
spec:
  replicas: 3
  selector:
    matchLabels:
      app: am575
  template:
    metadata:
      labels:
        app: am575
    spec:
      containers:
      # See earlier section for full listing of AM container
      # Filebeat sidecar container
      - name: filebeat
        image: image-registry.openshift-image-
registry.svc:5000/accessmatrix/filebeat:8.8.1
        volumeMounts:
        # Mount onto Filebeat the same volume that holds the
AM logs
        - mountPath: /opt/am/am/tomcat/logs
          name: deploymentlogs
        # Location of default Filebeat configuration
        - mountPath: /usr/share/filebeat/filebeat.yml
          subPath: filebeat.yml
          readOnly: true
          name: filebeat-config
        - mountPath: /usr/share/filebeat/data
          name: filebeat-data
        - mountPath: /usr/share/filebeat/logs
          name: filebeat-logs
        securityContext:
          privileged: false
```

am-k8s.yaml

```
          allowPrivilegeEscalation: false
          capabilities:
            drop: ["ALL"]
          runAsNonRoot: true
          seccompProfile:
            type: RuntimeDefault
        resources:
          limits:
            memory: 200Mi
          requests:
            cpu: 100m
            memory: 100Mi
      # PVCs created in earlier section
      volumes:
      - name: deploymentlogs
        persistentVolumeClaim:
          claimName: deploymentlogs
      - name: reports
        persistentVolumeClaim:
          claimName: reports
      # Folder for Filebeat internal configuration
      - name: filebeat-config
        configMap:
          name: filebeat-config
      # Folder for Filebeat internal data
      - name: filebeat-data
        emptyDir: {}
      # Folder for Filebeat internal logs
      - name: filebeat-logs
        emptyDir: {}
```

Note:

The objects referenced in the deployment file comments can be found in the following sections respectively:

- metadata - Create and Deploy OpenShift Objects - Create AccessMatrix Deployment
- containers - Create and Deploy OpenShift Objects - Create AccessMatrix Deployment
- PVCs - Create and Deploy OpenShift Objects - Deploy PVC for AM

The following is a sample for **filebeat.yml**, the Filebeat internal configuration file. It should be put into a ConfigMap and mounted to a volume, similar to how the Logstash configuration is mounted.

filebeat.yml

YAML

filebeat.yml

```
logging:
  level: debug   # Default is info
  to_files: true
  files:
    path: /usr/share/filebeat/logs
    name: filebeat
    keepfiles: 7
    permission: 0640

filebeat:
  inputs:
  # This filestream looks specifically at the AM log
  - type: filestream
    id: am-log-stream
    paths:
    - "/opt/am/am/tomcat/logs/*_am.log"
    multiline:
      pattern: '^[[:space:]]'
      negate: false
      match: after
  # This one looks at all the other logs from AM
  - type: filestream
    id: other-logs-stream
    paths:
    - "/opt/am/am/tomcat/logs/*.log"
    multiline:
      pattern: '^[[:space:]]'
      negate: false
      match: after

output:
  logstash:
    hosts: ["logstash:5044"]

setup:
  kibana:
    host: "http://kibana:5601"
```

For more information on Filebeat, refer to the Elastic documentation at https://www.elastic.co/guide/en/beats/filebeat/current/configuring-howto-filebeat.html.

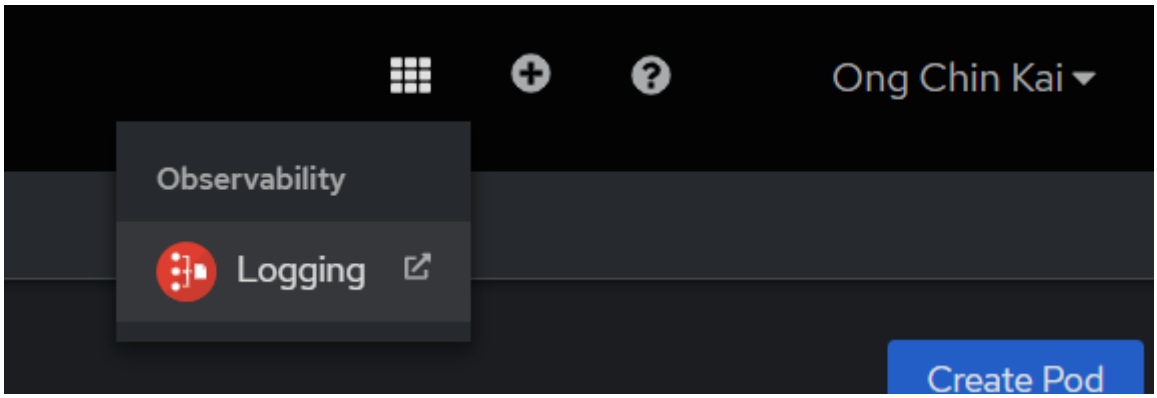
Cluster-wide Logging

Another approach to observe the AM logs is to make use of the cluster-wide Logging feature. This feature, when enabled by the cluster administrator, allows you to observe all projects to which you have been granted access.

In terms of architecture, the key difference between the Logging feature differs and the namespace-specific ELK stack is that the former uses Fluentd as the log collector, while the latter uses Logstash. For

more information on how to set up the Logging feature, refer to the OpenShift documentation at https://docs.openshift.com/container-platform/4.11/logging/cluster-logging-deploying.html.

To access the Logging feature, click on the Application Launcher icon and select “Logging”. Refer to the following image:



If you are accessing this for the first time, you will need to set up your Kibana index patterns before you can visualize your data. Details can be found in the OpenShift documentation at https://docs.openshift.com/container-platform/4.11/logging/cluster-logging-deploying.html#cluster-logging-visualizer-indices_cluster-logging-deploying.

However, in order for the Logging feature to capture the AM logs, changes have to be made to the AM image such that any log output is channeled not just to the respective log files, but also to the console to be visible in the output of the command `oc logs <pod> -c <container>`.

Modify the properties file **amlog4j2.properties** which can be found at *<ACMATRIX_ROOT>/tomcat/webapps/am5/WEB-INF/classes/amlog4j2.properties*. The changes involve referencing an additional console appender in all loggers that currently reference LOG_ROOT, the appender responsible for handling **am.log**.

1. Check that the following console appender exists in the properties file (and add it if it does not already exist):

Properties (.properties files)

```
# print to console
appender.CONSOLE.type=Console
appender.CONSOLE.name=CONSOLE
appender.CONSOLE.layout.type = PatternLayout
appender.CONSOLE.layout.charset = UTF-8
```

```
appender.CONSOLE.layout.pattern = %d{yyyy-MM-dd
HH:mm:ss.SSS} [%t] %-5p - (%F:%L) %m%n
```

2. Identify all existing loggers (including RootLogger) that reference LOG_ROOT. For instance,

Properties (.properties files)

```
logger.COM_ISPRINT.name = com.isprint
logger.COM_ISPRINT.level = INFO
logger.COM_ISPRINT.additivity = false
logger.COM_ISPRINT.appenderRef.startup.ref =
START_UP_LOG
logger.COM_ISPRINT.appenderRef.handler.ref =
HANDLER_LOG
logger.COM_ISPRINT.appenderRef.rolling.ref = LOG_ROOT
```

3. To each of these loggers, add the following line (using the above example):

Properties (.properties files)

```
logger.COM_ISPRINT.appenderRef.console.ref = CONSOLE
```

4. Rebuild the AM image.

4.2.5. Enable HSM Connectivity

> **Note:** This guide assumes that the HSM has already been set up and is ready to use; the setup of the HSM is beyond the scope of this guide.

The sections below detail how to enable HSM connectivity with a Luna Network HSM. Before proceeding, complete the steps detailed in Kubernetes Deployment with Azure and On-Premise HSM - Configure HSM.

Prerequisites

Ensure that the following files/binaries are on hand (e.g. in the *repo* folder):

- The tar file for the minimal version of LunaClient (e.g **LunaClient-Minimal-v7.4.0-226.x86_64.tar**).
- The binaries **lunacm** and **vtl** - these cannot be found in the above tar file, and must be copied from a machine that has the full LunaClient installed (installation of the full LunaClient is beyond the scope of this guide).
- The certificate files generated during the process of creating an NTLS connection to the HSM.
- The LunaClient configuration file **Chrystoki.conf.**
- The PKCS configuration file **pkcs11.config** with the following contents:

  Code

  ```
  # cat config/pkcs11.config
  library=/usr/local/luna/libs/64/libCryptoki2.so
  slot=0
  name=LunaSA
  ```

For more information about the certificate and configuration files, refer to Kubernetes Deployment with Azure and On-Premise HSM - Configure HSM.

Make Changes to AccessMatrix Dockerfile

The following sample is an excerpt from the AccessMatrix Dockerfile presented in earlier sections of this guide, with the addition of instructions necessary to process the files mentioned in the previous section:

YAML

```
RUN mkdir -p /opt/jdk/ \
    && mkdir -p /opt/am/${AM_VERSION}

ADD repo/OpenJDK17U-
jdk_x64_linux_hotspot_17.0.7_7.tar.gz /opt/jdk/
ADD repo/AccessMatrix-${AM_VERSION}-tomcat-only.tar.gz
/opt/am/${AM_VERSION}

# Setup for Luna HSM
ENV ChrystokiConfigurationPath=/usr/local/luna/config
COPY repo/LunaClient-Minimal-v7.4.0-226.x86_64.tar /tmp
COPY repo/luna/lunacm /usr/local/bin
COPY repo/luna/vtl /usr/local/bin
RUN mkdir -p /usr/local/luna
RUN tar xvf /tmp/LunaClient-Minimal-v7.4.0-
226.x86_64.tar --strip 1 -C /usr/local/luna

# Certification Requirement: must contain a "licenses"
directory
# …(so on)

RUN mv /opt/am/${AM_VERSION}/licenses /licenses \
    && groupadd -g 2020 am \
    && useradd -r -g am -u 2021 -s /bin/bash am \
    && ln -s /opt/am/${AM_VERSION} /opt/am/am \
    && chown -R am:0 /opt/am/${AM_VERSION} \
    && chown -R am:0 /opt/am/am \
    && chown -R am:0 /usr/local/bin \
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/bin/*.sh
\
    && chmod 754 /opt/am/${AM_VERSION}/tomcat/db-
init/*.sh \
    && chmod -R g=u /opt/am/am \
    && chmod 754 /usr/local/bin/* \
    && cp /usr/local/luna/jsp/LunaProvider.jar
/opt/am/am/tomcat/webapps/am5/WEB-INF/lib/
```

In this Dockerfile excerpt,

- The path where the LunaClient configuration can be found is defined by the environment variable ChrystokiConfigurationPath. The folder contents will be mounted by the OpenShift deployment YAML file.
- The minimal LunaClient tar file is copied from *repo* in the host to a temporary directory in the image, and then unzipped to the directory */usr/local/luna/*.
- The LunaClient binaries are copied from */repo/luna/* in the host to */usr/local/bin/* in the image. Ownership of these binaries must be transferred to the user am, and permission to run must be granted.

- AM requires the library file **LunaProvider.jar**, which comes with the minimal LunaClient tar file; it is alright to copy it from */usr/local/luna*.

Make the changes above before updating and rebuilding the Docker image.

Make Changes to OpenShift Deployment

Once the Docker image has been updated and rebuilt, the next step is to modify the deployment YAML **am-k8s.yaml** (previously created in this section).

1. Modify the environment variable JAVA_OPTS as follows:

   YAML

   ```
   - name: JAVA_OPTS
     value: "-Dorg.apache.activemq.SERIALIZABLE_PACKAGES=*
   --add-
   exports=jdk.crypto.cryptoki/sun.security.pkcs11=ALL-
   UNNAMED"
   ```

   This is to address an error caused by the PKCS package **sun.security.pkcs11** no longer being visible in Java 17.

2. Create OpenShift secrets for the LunaClient certificate and configuration files by running the following commands:

   ```
   oc create secret generic luna-server-auth --from-
   file=./repo/luna/certs/server/111.223.87.201Cert.pem --from-
   file=./repo/luna/certs/server/CAFile.pem
   oc create secret generic luna-client-auth --from-
   file=./repo/luna/certs/client/clientDocker.pem --from-
   file=./repo/luna/certs/client/clientDockerKey.pem
   oc create secret generic luna-config --from-file=./repo/luna/Chrystoki.conf --
   from-file=./repo/luna/pkcs11.config
   ```

3. Add the volumes to **am-k8s.yaml** as shown below:

   YAML

   ```
   volumes:
   - name: deploymentlogs
     persistentVolumeClaim:
       claimName: deploymentlogs
   - name: reports
   ```

```
  persistentVolumeClaim:
    claimName: reports
- name: lunaconf
  secret:
    secretName: luna-config
- name: servercerts
  secret:
    secretName: luna-server-auth
- name: clientcerts
  secret:
    secretName: luna-client-auth
```

4. Mount the volumes as shown:

YAML

```
volumeMounts:
- mountPath: opt/am/am/tomcat/logs
  name: deploymentlogs
- mountPath: opt/am/am/tomcat/work/reports
  name: reports
- name: lunaconf
  mountPath: /usr/local/luna/config
  readOnly: true
- name: servercerts
  mountPath: /usr/local/luna/config/certs/server
  readOnly: true
- name: clientcerts
  mountPath: /usr/local/luna/config/certs/client
  readOnly: true
```

5. Appendices

> **Note**: The following sections assume Docker is the container tool for Kubernetes/OpenShift.

Terminology

The following section lists some of the common terms that are used in this document. The accompanying descriptions provide a brief overview for each item; they are not intended to be comprehensive explanations.

Kubernetes with Azure

Term	Description
Azure Container Registry (ACR)	To store and manage Docker images on Azure cloud.
Azure Kubernetes Service (AKS)	Microsoft offers a managed Kubernetes cluster in Azure, with other Azure Services like Microsoft Entra ID integrated. Its registration requires a credit card or bank account details.
Azure Command Line Interface (Azure CLI)	Command-line interface that can be used on the workstation to allow the user to access various Azure services, such as creating Kubernetes Clusters, provisioning SQL databases, etc.
Azure Cache for Redis	An in-memory data store based on Redis. Azure offers Redis as a managed service that provides caching services to AccessMatrix. Redis Cache is used to store transient data that are frequently updated, for example, E2EE session, SMS-OTP tokens, SAML Response Id (to prevent replay attack), CLP login sessions (for AccessMatrix Applications, such as UAS TAP, USO SSF), and OIDC Relying Party Lookup Module nonces (to prevent request forgery or replay).
Apache ActiveMQ (AMQ)	Apache ActiveMQ is one of the most commonly used message queue servers, which manages messages between multiple instances of AccessMatrix. For AccessMatrix deployment, AMQ will be used for Ehcache replication.
Audit Micro Service (AMS)	Audit Micro Service (AMS) is a web application developed by i-Sprint independent from AccessMatrix. It is used to greatly improve AccessMatrix's performance during authentication by separating out audits as different microservice. When AMS is deployed, instead of directly writing into the database, AccessMatrix will send audit write requests to AMS via REST API. It is also possible to integrate AMS with the Message Queue (MQ) service to further improve the load distribution on multiple AMS instances.

Term	Description
Azure Service Bus	Azure Service Bus is a messaging service on the cloud used to connect any applications, devices, and also services running in the cloud to any other applications or services. It acts as a messaging backbone for applications in the cloud or across devices.
Cloud service provider (CSP)	Cloud Service Providers are companies that provide IT environments like the public clouds or managed private clouds that abstract, pool, and share scalable resources across a network. For example, Amazon Web Service (AWS), Microsoft Azure cloud platform, Google Cloud Platform (GCP) and more.
Java Message Service (JMS)	JMS is a mechanism for interacting with message queues. It is used as the underlying mechanism for cache replication operations in Ehcache. While Redis Cache is used to store transient data that are frequently updated, AccessMatrix still has some common objects (such as policies, configurations and login modules) that are frequently accessed but are not frequently updated, and for performance and efficiency reasons are stored in Ehcache. In a cloud environment like Azure, Ehcache replication using multicast is not supported. Thus Ehcache replication is done by using JMS through a message queue server.
Kubernetes Node	Each Kubernetes node represents a virtual machine (VM) that runs Kubernetes node components and container runtimes like kublet. For example, in Azure Kubernetes Service (AKS), each node represents an Azure Virtual Machine (VM). Every Kubernetes cluster has at least one worker node.
Kubernetes Pod	Each Kubernetes pod represents a single instance of AccessMatrix. Pods are assigned to nodes and will remain there until they are scheduled to be deleted.
Persistent Volume (PV)	A Persistent Volume (PV) is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes. It is a resource in the cluster, just like a node is a cluster resource. PVs are volume plugins similar to Docker Volumes but have a lifecycle independent of any individual pod that uses the PV. This API object captures the details of the implementation of the storage, be that NFS, iSCSI, or a cloud-provider-specific storage system.
Persistent Volume Claim (PVC)	A Persistent Volume Claim (PVC) is a request for storage by a user (refer to architecture diagram). It is similar to a Pod in that Pods consume node resources, and PVCs consume PV resources. Pods can request specific levels of resources (CPU and Memory). PVCs can request specific size and access modes (e.g. they can be mounted once read/write or many times read-only).
Resource Group	Resource Group holds related resources for an Azure Solution. It stores the metadata about resources.

Kubernetes with AWS

Term	Description
Amazon Virtual Private Cloud (VPC)	Amazon Virtual Private Cloud is a commercial cloud computing service that provides you with a virtual private cloud dedicated to your AWS account. It enables you to launch resources into a virtual network that you have defined. This virtual network closely resembles a traditional network that you would operate in your own data centre, with the benefits of using the scalable infrastructure of AWS.
Amazon Elastic Kubernetes Service (EKS) cluster	Amazon Elastic Kubernetes Service cluster is a managed service that makes it easy for you to run Kubernetes on AWS without needing to stand up or maintain your own Kubernetes control plane. Applications running on Amazon EKS are fully compatible with applications running on any standard Kubernetes environment, whether running in on-premises data centres or public clouds. This means that you can easily migrate any standard Kubernetes application to Amazon EKS without any code modification required.
Amazon Elastic Container Registry (ECR)	Amazon Elastic Container Registry (ECR) is a fully managed container registry that makes it easy to store, manage, share, and deploy your container images and artifacts anywhere as it transfers your container images over HTTPS and automatically encrypts your images at rest.
Amazon Message Queue (MQ)	Amazon MQ in AccessMatrix deployment is a managed message broker service for the broker (based on Apache ActiveMQ) that makes it easy to set up and operate message brokers on AWS. MQ will be used for Ehcache replication.
Audit Micro Service (AMS)	Audit Micro Service (AMS) is a web application developed by i-Sprint that is independent from AccessMatrix. It is used to improve AccessMatrix performance during authentication tremendously by separating audit as different microservices. On the cloud, AccessMatrix will write the audits to the Message Queue (MQ) service whilst on-premises, AccessMatrix will write the audits to AMS by calling API. All historical reports that are currently read from audit tables in AccessMatrix will be moved to AMS.
Cloud service provider (CSP)	Cloud Service Providers are companies that provide IT environments like the public clouds or managed private clouds that abstract, pool, and share scalable resources across a network. For example, Amazon Web Service (AWS), Microsoft Azure cloud platform, Google Cloud Platform (GCP) and more.
Java Message Service (JMS)	JMS is a mechanism for interacting with message queues. It is used as the underlying mechanism for cache replication operations in Ehcache. While Redis Cache is used to store transient data that are frequently updated, AccessMatrix still has some common objects (such as policies, configurations and login modules) that are frequently accessed but are not

Term	Description
	frequently updated. Therefore, performance and efficiency reasons are stored in Ehcache. In a cloud environment like Azure, Ehcache replication using multicast is not supported. Thus Ehcache replication is done by using JMS through a message queue server.
Kubernetes Node	Each Kubernetes node represents a virtual machine (VM) that runs Kubernetes node components and container runtimes like kublet. For example, in Amazon Elastic Kubernetes Service (EKS), each node represents a Virtual Machine (VM). Every Kubernetes cluster has at least one worker node.
Kubernetes Pod	Each Kubernetes pod represents a single instance of AccessMatrix. Pods are assigned to nodes and will remain there until they are scheduled to be deleted.
Persistent Volume (PV)	A Persistent Volume (PV) is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes. It is a resource in the cluster, just like a node is a cluster resource. PVs are volume plugins similar to Docker Volumes but have a lifecycle independent of any individual pod that uses the PV. This API object captures the details of the implementation of the storage, be that NFS, iSCSI, or a cloud-provider-specific storage system.
Persistent Volume Claim (PVC)	A Persistent Volume Claim (PVC) is a request for storage by a user (refer to architecture diagram). It is similar to a Pod in that Pods consume node resources, and PVCs consume PV resources. Pods can request specific levels of resources (CPU and Memory). PVCs can request specific size and access modes (e.g. they can be mounted once read/write or many times read-only).
Route Table	A set of rules called routes used to determine where network traffic is directed.

OpenShift with Azure

Term	Description
Azure Container Registry (ACR)	To store and manage Docker images on Azure cloud.
Azure Kubernetes Service (AKS)	Microsoft offers a managed OpenShift cluster in Azure, with other Azure Services like Microsoft Entra ID (AAD) integrated. Its registration requires a credit card or bank account details.
Azure Command Line Interface (Azure CLI)	Command-line interface that can be used on the workstation to allow the user to access various Azure services, such as creating OpenShift Clusters, provisioning SQL databases, etc.

Term	Description
Azure Cache for Redis	An in-memory data store based on Redis. Azure offers Redis as a managed service that provides caching services to AccessMatrix. Redis Cache is used to store transient data that are frequently updated, for example, E2EE session, SMS-OTP tokens, SAML Response Id (to prevent replay attack), CLP login sessions (for AccessMatrix Applications, such as UAS TAP, USO SSF), and OIDC Relying Party Lookup Module nonces (to prevent request forgery or replay).
Apache ActiveMQ (AMQ)	Apache ActiveMQ is one of the most commonly used message queue servers, which manages messages between multiple instances of AccessMatrix. For AccessMatrix deployment, AMQ will be used for Ehcache replication.
Java Message Service (JMS)	JMS is a mechanism for interacting with message queues. It is used as the underlying mechanism for the replicated operations in Ehcache. While Redis Cache is used to store transient data that are frequently updated, AccessMatrix still has some common objects (such as policies, configurations and login modules) that are frequently accessed but are not frequently updated performance and efficiency are stored in Ehcache. In a cloud environment like Azure, Ehcache replication using multicast is not supported. Thus Ehcache replication is done by using JMS through a message queue server.
Microsoft Entra ID (AAD)	This is Azure's preferred multi-tenant cloud directory service, capable of authenticating security principals or federating with other identity providers, such as Microsoft's Active Directory. AAD allows various applications (web application, Windows desktop application, Universal applications, mobile applications, etc.) to authenticate and use Kusto services uniformly.
OpenShift Node	Each OpenShift node represents a virtual machine (VM) that runs OpenShift node components and container runtimes.
OpenShift Pod	Each OpenShift pod represents a single instance of AccessMatrix. Pods are assigned to nodes and will remain there until they are scheduled to be deleted.
OpenShift Container Registry (OCR)	This container registry is used as a publication target for images built on the cluster; a source of images for workloads running on the cluster.
OpenShift Command Line Interface (OpenShift CLI)	Command-line interface that can be used on the workstation to allow the user to access various OpenShift services, such as creating OpenShift Clusters, provisioning SQL databases, etc.
OpenShift Service Accounts	Service Accounts are OpenShift accounts that allow a component to access the API directly. Service Accounts can be granted policies that allow or deny access to the cluster.

Term	Description
Persistent Volume (PV)	A Persistent Volume (PV) is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using Storage Classes. It is a resource in the cluster, just like a node is a cluster resource. PVs are volume plugins similar to Docker Volumes but have a lifecycle independent of any individual pod that uses the PV. This API object captures the details of the implementation of the storage, be that NFS, iSCSI, or a cloud-provider-specific storage system.
Persistent Volume Claim (PVC)	A Persistent Volume Claim (PVC) is a request for storage by a user (refer to architecture diagram). It is similar to a Pod in that Pods consume node resources, and PVCs consume PV resources. Pods can request specific levels of resources (CPU and Memory). PVCs can request specific size and access modes (e.g., they can be mounted once read/write or many times read-only).
Resource Group	It holds related resources for an Azure solution and stores the metadata about resources.
Security Context Constraint(SCC)	An OpenShift solution, similar to how Role-Based Access Control (RBAC) works. These permissions include actions inside a pod or container, which can be executed or accessed. SCCs are the security feature that defines a set of conditions that a pod must run to be accepted in the OpenShift system.

Multiple Deployments

Kubernetes with Azure Deployment

In the case where there are multiple deployments (e.g. Production, UAT, SIT) of AccessMatrix, to differentiate the log files, create different databases and Persistent Volume Claims for each deployment.

For example, to create a UAT deployment PVC:

am-uat-logs-pvc.yaml

YAML

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: amuatlogs
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: azurefile
```

am-uat-logs-pvc.yaml

```
  resources:
    requests:
      storage: 1Gi
```

am-uat-report-pvc.yaml

YAML

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: amuatreport
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: azurefile
  resources:
    requests:
      storage: 1Gi
```

To reference it in a deployment file:

am-k8s.yaml

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: am-k8s
  namespace: default
  labels:
    app: am-k8s
spec:
  replicas: 2
  selector:
    matchLabels:
      app: am-k8s
  template:
    metadata:
      labels:
        app: am-k8s
    spec:
      containers:
      - name: am5
        image: <container-registry-server>/am5:<version>
        env:
        - name: DBCRED
```

am-k8s.yaml

```
          valueFrom:
            secretKeyRef:
              key: DBCRED
              name: am-credentials
        readinessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        livenessProbe:
          httpGet:
            path: /am5/
            port: 8080
          periodSeconds: 5
          timeoutSeconds: 60
          successThreshold: 1
          failureThreshold: 3
          initialDelaySeconds: 70
        ports:
        - containerPort: 8080
          protocol: TCP
        volumeMounts:
        - mountPath: /opt/am/am/tomcat/work/reports
          name: amuatreports
        - mountPath: /opt/am/am/tomcat/logs
          name: amuatlogs
        securityContext:
          privileged: true
        resources:
          requests:
            cpu: 100m
      volumes:
      - name: amuatreports
        persistentVolumeClaim:
          claimName: amuatreports
      - name: amuatlogs
        persistentVolumeClaim:
          claimName: amuatlogs
```

Troubleshooting Kubernetes with Azure Deployment

Viewing Log Files

When running AccessMatrix Server as a container, the log files are normally stored in a persistent volume whereas the standard output/error is normally viewable via the `kubectl logs` command.

Standard Output/Error

Checking standard output is useful, especially when AccessMatrix Server fails to start. It may show useful error messages.

Show standard out log

```
kubectl logs <pod_name>
```

Show standard out log for previous container that has exited

```
kubectl logs -p<pod_name>
```

For more information about `kubectl logs`, you may refer to the kubectl logs command reference.

AM Log Files (Using Debugging Pod)

In general, to read AM log files, you can start another pod for this purpose (e.g. debugging pod). You can start the debugging pod with the log persistent volume mounted so that you can read the content of the persistent volume.

Example of debug-pvc.yaml

YAML

```yaml
kind: Pod
apiVersion: v1
metadata:
  name: debug-pvc
spec:
  volumes:
    - name: logs
      persistentVolumeClaim:
        claimName: logs
  containers:
    - name: debugger
      image: busybox
      command: ['sleep', '3600']
      volumeMounts:
        - mountPath: "/data/logs"
          name: logs
```

You need to ensure that the persistent volume claim's claim name is configured to Persistent Volume Claim (PVC) for you to log persistent volume. You can then start the pod.

Start debug-pvc pod
`kubectl create -f debug-pvc.yaml`

The pod will start the `sleep` command as stated in the **debug-pvc.yaml** file. After the pod is started, you can then get shell access.

Get shell access of the debug-pvc
`kubectl exec -it debug-pvc sh`

In the shell, then you can access the log files in */data/logs* as specified in the mountPath.

Once you are done, you may delete the debug-pvc pod.

Delete debug-pvc pod
`kubectl delete pod debug-pvc`

AM Log Files (On Running AM Pod)

If AccessMatrix Server pod is still running, it is possible to get shell access or run the `tail` command directly to the running pod. This way, there is no need to start a separate debugging pod to access the log persistent volume.

Get shell access of AM pod
`kubectl exec -it <pod name> sh`

AccessMatrix Server Pod Failed to Start

When AccessMatrix (AM) Server Pod failed to start (e.g. `kubectl get pods` shows that the pod status is "Failed" or "CrashLoop..."), there are a few steps that can be used to troubleshoot. One way is to check the log file, especially the standard output/error (`kubectl logs`). This is discussed in the Viewing Log Files section.

However, there are times where this cannot be done when the failure happened very early. One way to troubleshoot is to start a single AccessMatrix Server pod instead of part of a deployment. This is to make it easier for troubleshooting. Then, you can also try to run a different command to start the pod. After

the pod is started, you can get shell access and run the AccessMatrix server manually. This is to see whether there is an issue with the start-up script.

This is an example of a pod YAML file that starts the AccessMatrix server pod. You can have your own based on your deployment YAML file but modified for pod deployment instead. What is important is to override the command like the example below.

AccessMatrix server pod and override command to run sleep command instead

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: ampod
spec:
  securityContext:
    runAsUser: 0
  containers:
  - name: am5
    image: <container-registry-server>/am5:<version>
    command: ["/bin/sh", "-c"]
    args:
      - echo starting;
        whoami;
        sleep 3600;
    ports:
    - containerPort: 8080
      protocol: TCP
    env:
    - name: DBCRED
      valueFrom:
        secretKeyRef:
          key: DBCRED
          name: am-credentials
    volumeMounts:
    - mountPath: /opt/am/am/tomcat/logs
      name: <pvc_name>
    securityContext:
      privileged: true
    resources:
      requests:
        cpu: 100m
  volumes:
  - name: <pvc_name>
    persistentVolumeClaim:
      claimName: <pvc_name>
```

Note: Replace <pvc_name> with the name of the PVC created in the previous section, e.g. (deploymentlogs, sandboxlogs).

Once the pod has been started, you can get Shell Access to start the AccessMatrix server manually.

Access shell

```
kubectl exec -it ampod sh
```

Troubleshooting Kubernetes with AWS Deployment

Viewing Log Files

When running AccessMatrix Server as a container, the log files are normally stored in a persistent volume, whereas the standard output/error is normally viewable via `kubectl logs` command.

Standard Output/Error

Checking standard output is useful, especially when the AccessMatrix Server fails to start. It may show useful error messages.

Show standard out log

```
kubectl logs <pod_name>
```

Show standard out log for previous container that has exited

```
kubectl logs -p<pod_name>
```

For more information about `kubectl logs`, you may refer to the kubectl logs command reference.

AM Log Files (Using Debugging Pod)

In general, to read AM log files, you can start another pod for this purpose (e.g. debugging pod). You can start the debugging pod with the log persistent volume mounted so that you can read the content of the persistent volume.

Example of debug-pvc.yaml

YAML

```yaml
kind: Pod
apiVersion: v1
metadata:
  name: debug-pvc
spec:
```

Example of debug-pvc.yaml

```
volumes:
  - name: logs
    persistentVolumeClaim:
      claimName: logs
containers:
  - name: debugger
    image: busybox
    command: ['sleep', '3600']
    volumeMounts:
      - mountPath: "/data/logs"
        name: logs
```

You need to ensure that the persistent volume claim's claim name is configured to Persistent Volume Claim (PVC) for you to log persistent volume. You can then start the pod.

Start debug-pvc pod

```
kubectl create -f debug-pvc.yaml
```

The pod will start the `sleep` command as stated in the **debug-pvc.yaml** file. After the pod is started, you can then get shell access.

Get shell access of the debug-pvc

```
kubectl exec -it debug-pvc sh
```

In the shell, then you can access the log files in */data/logs/* as specified in the mountPath.

Once you are done, you may delete the debug-pvc pod.

Delete debug-pvc pod

```
kubectl delete pod debug-pvc
```

AM Log Files (On Running AM Pod)

If AccessMatrix Server pod is still running, it is possible to get shell access or run `tail` command directly to the running pod. This way, there is no need to start a separate debugging pod to access the log persistent volume.

Get shell access of AM pod

```
kubectl exec -it <pod name> sh
```

AccessMatrix Server Pod Failed to Start

When AccessMatrix (AM) Server Pod failed to start (e.g. `kubectl get pods` shows that the pod status is "Failed" or "CrashLoop..."), there are a few steps that can be used to troubleshoot. One way is to check the log file, especially the standard output/error (`kubectl logs`). This is discussed in the Viewing Log Files section.

However, there are times where this cannot be done when the failure happened very early. One way to troubleshoot is to start a single AccessMatrix Server pod instead of part of a Deployment. This is to make it easier for troubleshooting. Then, you can also try to run a different command to start the pod. After the pod is started, you can get shell access and run the AccessMatrix server manually. This is to see whether there is an issue with the start-up script.

This is an example of a pod YAML file that starts the AccessMatrix server pod. You can have your own based on your deployment YAML file but modified for pod deployment instead. What is important is to override the command like the example below.

AccessMatrix server pod and override command to run sleep command instead

YAML

```yaml
apiVersion: "v1"
kind: "Pod"
metadata:
  name: "ampod"
spec:
  securityContext:
    runAsUser: 0
  containers:
  - name: am5
    image: "<container-registry-server>/am5:<version>"
    command: ["/bin/sh", "-c"]
    args:
      - echo starting;
        whoami;
        sleep 3600;
    ports:
    - containerPort: 8080
      protocol: TCP
    env:
    - name: "AMDB_URL"
```

AccessMatrix server pod and override command to run sleep command instead

```
      valueFrom:
        configMapKeyRef:
          key: "AMDB_URL"
          name: "am-config"
    - name: "AMDB_USERNAME"
      valueFrom:
        configMapKeyRef:
          key: "AMDB_USERNAME"
          name: "am-config"
    - name: "AMDB_PASSWORD"
      valueFrom:
        configMapKeyRef:
          key: "AMDB_PASSWORD"
          name: "am-config"
    volumeMounts:
    - mountPath: /opt/am/am/tomcat/logs
      name: logs
    securityContext:
      privileged: true
    resources:
      requests:
        cpu: 100m
  volumes:
  - name: logs
    persistentVolumeClaim:
      claimName: logs
```

Once the pod has been started, you can get Shell Access to start the AccessMatrix server manually.

Access shell

```
kubectl exec -it ampod sh
```

Troubleshooting OpenShift with Azure Deployment

Viewing Log Files

When running AccessMatrix Server as a container, the log files are normally stored in a persistent volume whereas the standard output/error is normally viewable via `oc logs` command.

Standard Output/Error

Checking standard output is useful, especially when AM Server fails to start. It may show useful error messages.

Show standard out log

```
$ oc logs <pod_name>
```

Show standard out log for previous container that has exited

```
$ oc logs -p<pod_name>
```

For more information about `oc logs`, you may refer to the oc logs command reference.

AccessMatrix Log Files (Using Debugging Pod)

In general, to read AM log files, you can start another pod for this purpose (e.g. debugging pod). You can start the debugging pod with the log persistent volume mounted so that you can read the content of the persistent volume.

Example of debug-pvc.yaml

YAML

```yaml
kind: Pod
apiVersion: v1
metadata:
  name: debug-pvc
spec:
  volumes:
    - name: logs
      persistentVolumeClaim:
        claimName: logs
  containers:
    - name: debugger
      image: busybox
      command: ['sleep', '3600']
      volumeMounts:
        - mountPath: "/data/logs"
          name: logs
```

You need to ensure that the persistent volume claim's claim name is configured to Persistent Volume Claim (PVC) for your log persistent volume. You can then start the pod.

Start debug-pvc pod

```
$oc create -f debug-pvc.yaml
```

The pod will start the `sleep` command as stated in the **debug-pvc.yaml** file. After the pod is started, you can then get shell access.

Get shell access of the debug-pvc
`$ oc exec -it debug-pvc sh`

In the shell, then you can access the log files in */data/logs/* as specified in the mountPath.

Once you are done, you may delete the debug-pvc pod.

Delete debug-pvc pod
`$ oc delete pod debug-pvc`

AccessMatrix Log Files (On Running AccessMatrix Pod)

If AM Server pod is still running, it is possible to get shell access or run the `tail` command directly to the running pod. This way, there is no need to start a separate debugging pod to access the log persistent volume.

Get shell access of AM pod
`$ oc exec -it <pod name> sh`

AccessMatrix Server Pod Failed to Start

When AM Server Pod failed to start (e.g. `kubectl get pods` shows that the pod status is "Failed" or "CrashLoop..."), there are a few steps that can be used to troubleshoot. One way is to check the log file, especially the standard output/error (`kubectl logs`). This is discussed in the Viewing Log Files section.

However, there are times where this cannot be done when the failure happened very early. One way to troubleshoot is to start a single AM Server pod instead of part of a Deployment. This is to make it easier for the troubleshooting. Then, you can also try to run a different command to start the pod. After the pod is started, you can get the shell access and run the AM server manually. This is to see whether there is an issue with the start-up script.

This is an example of a pod YAML file that starts the AM server pod. You can have your own based on your deployment YAML file but modified for pod deployment instead. What is important is to override the command like the example below.

AM server pod and override command to run sleep command instead

YAML

```yaml
apiVersion: "v1"
kind: "Pod"
metadata:
  name: "ampod"
spec:
  securityContext:
    runAsUser: 0
  containers:
  - name: am5
    image: "accessmatrix.azurecr.io/am5:latest"
    command: ["/bin/sh", "-c"]
    args:
      - echo starting;
        whoami;
        sleep 3600;
    ports:
    - containerPort: 8080
      protocol: TCP
    env:
    - name: "AMDB_URL"
      valueFrom:
        configMapKeyRef:
          key: "AMDB_URL"
          name: "am-config"
    - name: "AMDB_USERNAME"
      valueFrom:
        configMapKeyRef:
          key: "AMDB_USERNAME"
          name: "am-config"
    - name: "AMDB_PASSWORD"
      valueFrom:
        configMapKeyRef:
          key: "AMDB_PASSWORD"
          name: "am-config"
    volumeMounts:
    - mountPath: /opt/am/am/tomcat/logs
      name: logs
    securityContext:
      privileged: true
    resources:
      requests:
        cpu: 100m
  volumes:
  - name: logs
    persistentVolumeClaim:
      claimName: logs
```

Once the pod has been started, you can get Shell Access to start the AccessMatrix server manually.

Access shell
`kubectl exec -it ampod sh`